

SYSTEM THINKING

FOR CLOUD ENGINEERS

From Strategy to Architecture



CHRISTOS PANAGIOTIDIS

System Thinking for Cloud Engineers

From Strategy to Architecture

PUBLISHED By Christos Panagiotidis

© Copyright Christos Panagiotidis 2026 -All rights reserved.

The content contained within this book may not be reproduced, duplicated or transmitted without direct written permission from the author or the publisher.

Under no circumstances will any blame or legal responsibility be held against the publisher, or author, for any damages, reparation, or monetary loss due to the information contained within this book. Either directly or indirectly. You are responsible for your own choices, actions, and results.

Legal Notice:

This book is copyright protected. This book is only for personal use. You cannot amend, distribute, sell, use, quote or paraphrase any part, or the content within this book, without the consent of the author or publisher.

Disclaimer Notice:

Please note the information contained within this document is for educational and entertainment purposes only. All effort has been executed to present accurate, up to date, and reliable, complete information. No warranties of any kind are declared or implied. Readers acknowledge that the author is not engaging in the rendering of legal, financial, medical or professional advice. The content within this book has been derived from various sources. Please consult a licensed professional before attempting any techniques outlined in this book.

By reading this document, the reader agrees that under no circumstances is the author responsible for any losses, direct or indirect, which are incurred as a result of the use of the information contained within this document, including, but not limited to, — errors, omissions, or inaccuracies.

About the Author

Christos "Chris" Panagiotidis | Azure Cloud Engineer, MCT, Cloud Coach, and Lab Builder

Christos "Chris" Panagiotidis is a Germany-based Azure Cloud Engineer, Microsoft Certified Trainer, cloud coach, mentor, and hands-on lab builder. His public work sits at the intersection of Azure infrastructure, networking, automation, and practical education. He writes and builds for people who want to move beyond clicking through a portal and toward repeatable cloud systems that can be read, reviewed, tested, improved, and rebuilt with confidence.

Chris approaches cloud engineering as a craft that becomes stronger when it is practiced in the open. His GitHub profile describes his current role as Azure Cloud Engineer and MCT, and his focus areas include Azure infrastructure and networking, Terraform and Infrastructure as Code, cloud coaching and mentoring, and hands-on lab development. Those focus areas explain the structure of this book. The goal is not only to show that an Azure resource can be deployed. The goal is to teach the habit of turning a cloud idea into files, commands, review steps, validation checks, cleanup routines, and documentation that a learner or operator can trust.

His learning philosophy is direct: the best way to learn is by doing. That belief shapes his labs, his writing, and the way this book is organized. A reader should be able to see the architecture, inspect the Terraform, run the commands, validate the result, and understand why each piece exists. The important lesson is rarely a single resource. It is the relationship between naming, network boundaries, provider authentication, variables, state, outputs, monitoring, source control, and cleanup. Chris teaches those relationships by keeping examples concrete and by making the operational path visible.

Author Links

Bio: <https://www.beyondcloudwithchriz.com/about>

GitHub profile: <https://github.com/jamonygr>

LinkedIn: <https://www.linkedin.com/in/christospanagiotidis/>

Table of Contents

About the Author

Introduction: The Cloud Is a System, Not a Collection of Resources

Chapter 1: Seeing the Whole Azure System

Chapter 1.1: From Resources to Relationships

Chapter 1.2: Choose the Boundary Before Solving the Problem

Chapter 1.3: Stocks, Flows, and Cloud Queues

Chapter 1.4: Feedback, Delay, and Cloud System Archetypes

Chapter 1.5: Leverage Points and the CAF Learning Loop

Chapter 2: Strategy — Give the Cloud System a Purpose

Chapter 2.1: Assess the Current System Honestly

Chapter 2.2: Define Motivations, Mission, and Objectives

Chapter 2.3: Map Stakeholders, Incentives, and Constraints

Chapter 2.4: Measure Outcomes Without Creating Bad Incentives

Chapter 2.5: Run Strategy as a Testable Hypothesis

Chapter 3: Plan — Design the Operating Model

Chapter 3.1: See the Estate as a Portfolio

Chapter 3.2: Choose a Cloud Operating Model

Chapter 3.3: Design Responsibilities and Skills

Chapter 3.4: Sequence Work Around Dependencies

Chapter 3.5: Make the Plan a Living System

Chapter 4: Ready — Architect the Azure Landing Zone

Chapter 4.1: Platform and Application Landing Zones

Chapter 4.2: Tenant, Billing, Management Groups, and Subscriptions

Chapter 4.3: Identity as the Control-Plane Boundary

Chapter 4.4: Connectivity as a Shared System

Chapter 4.5: Integrate the Landing-Zone Design Areas

Chapter 5: Ready as Code — Building the Platform with Terraform

Chapter 5.1: Terraform as a System Controller

Chapter 5.2: Bootstrap Authentication and Remote State Safely

Chapter 5.3: Use Azure Verified Modules for the Platform Landing Zone

Chapter 5.4: Deliver Infrastructure Through a Safe Pipeline

Chapter 5.5: Control Drift, Upgrades, Imports, and Recovery

Chapter 6: Observe and Test Azure with CLI and PowerShell

Chapter 6.1: Create a Safe Azure CLI Session

Chapter 6.2: Query the Azure Resource Graph

Chapter 6.3: Verify Hierarchy, Access, and Inheritance

Chapter 6.4: Test Policy, Connectivity, and Monitoring

Chapter 6.5: Build a Reusable Evidence Pack

Chapter 7: Adopt — Build the Platform-to-Workload Interface

Chapter 7.1: Treat the Platform as a Product

Chapter 7.2: Design the Application Landing-Zone Contract

Chapter 7.3: Create Paved Roads Without Creating Bottlenecks

Chapter 7.4: Connect CAF to Well-Architected Workloads

Chapter 7.5: Measure Adoption Friction

Chapter 8: Govern — Turn Risk into Useful Guardrails

Chapter 8.1: Translate Risk into Cloud Policy

Chapter 8.2: Understand Azure Policy as a Hierarchical System

Chapter 8.3: Move from Monitoring to Enforcement Carefully

Chapter 8.4: Manage Policy as Code and Govern Exceptions

Chapter 8.5: Govern Cost and Resource Consistency

Chapter 9: Secure — Protect the Azure System

Chapter 9.1: Security Across Every CAF Methodology

Chapter 9.2: Protect the Identity and Privilege System

Chapter 9.3: Build the Platform Security Baseline

Chapter 9.4: Measure Posture and Detect Threats

Chapter 9.5: Secure the Delivery Chain and Learn from Incidents

Chapter 10: Manage — Operate and Improve the Platform

Chapter 10.1: Organize Operations with RAMP

Chapter 10.2: Administer Through Controlled Change

Chapter 10.3: Monitor the Platform as a System

Chapter 10.4: Protect and Recover the Platform

Chapter 10.5: Close the Improvement Loop

The Platform That Keeps Learning

Introduction: The Cloud Is a System, Not a Collection of Resources

A cloud incident arrives with a precise technical label. A route is wrong. A policy blocks a deployment. A role assignment is missing. A Terraform plan contains a change that nobody expected.

That label is useful because it points toward an immediate action. An engineer corrects the route, grants access, changes the policy assignment, or reconciles the configuration. The service recovers, the ticket closes, and attention moves to the next task. Then a similar incident appears in another subscription.

Repetition changes the engineering question. Even when the route is incorrect, why did the design allow several teams to create conflicting routes? Even when the policy is valid, why was it assigned at a scope where workload teams cannot comply? A portal change can explain the Terraform plan while exposing two change paths that both seemed legitimate.

The resource remains part of the answer. It is no longer the whole answer.

Azure estates behave through relationships. Resources sit inside resource groups, subscriptions, and management groups. Identities gain authority through roles and scope. Policies express risk decisions. Networks create paths and dependencies.

Pipelines convert reviewed configuration into platform change. Monitoring carries evidence back to the people expected to act. Neither path is useful when ownership is unclear.

Teams, budgets, deadlines, skills, and incentives are part of the same environment. A technically sound control can create a long approval queue. That queue can encourage manual work or unmanaged subscriptions. A cost target can reduce visible spending while weakening the telemetry needed to understand an outage. Each local choice can look rational while their interaction produces a poor result.

This book treats Azure as a sociotechnical system: a system in which technology, people, process, information, and incentives shape one another. System thinking is the discipline used to study that behavior. It examines components, but it pays special attention to relationships, boundaries, accumulated work, feedback, delay, rules, and purpose.

That wider view doesn't turn every failed command into an organization-wide investigation. Immediate restoration still matters. The wider view becomes valuable when the same problem returns, when one team's improvement creates trouble elsewhere, or when technical controls produce behavior nobody intended.

Several basic ideas make the lens practical. A stock is something that accumulates, such as unresolved incidents, policy exemptions, technical debt, platform requests, or standing

privilege. A flow changes that stock through new demand, delivery, remediation, review, or closure. A boundary defines which Azure scopes, teams, processes, and time horizons are included in an analysis.

Feedback exists when the result of an action eventually influences its cause. Delays separate an action from the evidence of its effect. A leverage point is a place where a focused change can produce a wider and more durable improvement. These terms are simple, but they explain cloud behavior that a resource list cannot.

Three habits keep the analysis grounded. First, name the result that matters. “The deployment failed” is an event. “Approved workloads cannot deploy within the promised lead time” describes a result that the service is meant to support.

Second, trace authority, dependency, information, and cost before proposing a fix. Record which team can decide, which service depends on another, where evidence stops, and who pays. The answers often show that the first visible resource isn’t the best place to act.

Third, compare the intended effect with the observed effect. A deny policy is meant to reduce risk. If it also drives unmanaged work, a complete review must address both results. This doesn’t weaken the control. It helps engineers keep the control while removing avoidable friction.

These habits don’t require a large workshop. A short incident review can use them. A design record can use them too. Their value comes from making hidden assumptions testable.

Each habit leads to a clear test. A goal should name a real result. A link should have an owner. A signal should reach a team that can act. A delay should have a start and an end.

The tests keep the work close to facts. They also make debate more useful. Teams can check a claim, learn from the result, and change the next step. The system view then guides work instead of adding vague theory to it.

Consider a platform team that receives application landing-zone requests faster than it can fulfill them. The backlog grows, delivery pressure rises, and engineers introduce manual shortcuts. Those shortcuts reduce lead time briefly. They also create variation, support demand, and rework, which consume the capacity needed to reduce the backlog.

The local question asks how to complete the next request faster. The system question asks which relationships among demand, capacity, variation, and rework keep the queue growing. Both questions matter. The second one helps engineers choose a correction that does more than move pressure from one part of the platform to another.

Microsoft’s Cloud Adoption Framework, or CAF, gives this engineering view a practical lifecycle. CAF organizes cloud adoption through seven methodologies: Strategy, Plan, Ready,

Adopt, Govern, Secure, and Manage. Strategy connects cloud work to business outcomes. Plan turns intent into an operating model, skills, responsibilities, and sequenced work. Ready prepares the Azure foundation.

Adopt guides workload migration, modernization, and cloud-native development. Govern turns risk and business tolerance into guardrails. Secure protects the environment throughout its lifecycle. Manage covers administration, monitoring, protection, and continuous improvement. The methodologies have a useful order, but they shouldn't become isolated phases. Strategy without operational evidence becomes wishful thinking. A plan without capacity feedback becomes a static schedule. A landing zone without an adoption interface becomes central infrastructure that workload teams struggle to use. Governance, security, and management continue after deployment and return evidence to earlier decisions.

CAF therefore acts as a learning cycle. Incidents expose assumptions. Cost data reveals demand and consumption. Security findings reveal weak controls or unclear ownership.

Policy exemptions reveal friction. Onboarding delays reveal missing platform capabilities. Evidence creates value only when it reaches the people who can revise strategy, plans, designs, and priorities.

The running example in this book is an Azure landing-zone platform. A platform landing zone supplies shared capabilities such as identity, connectivity, governance, security, and management. An application landing zone provides a governed environment in which a workload team can deploy and operate an application. The two are different, but neither succeeds in isolation.

The capstone concentrates on shared Azure capabilities. It assumes one Microsoft Entra tenant and pre-existing Identity, Connectivity, and Management subscriptions. It establishes hierarchy, access, connectivity, policy, telemetry, automation, and the interface offered to workload teams. It stops before deploying an application workload.

That boundary keeps the technical argument clear. A platform must offer a repeatable service, not merely a collection of shared resources. Workload teams need to know what they receive, which controls they inherit, what information they must provide, and where their responsibility begins. Platform engineers need evidence that the service is secure, usable, supportable, and aligned with its purpose.

Terraform is the source of truth for persistent platform configuration in the capstone. It expresses desired state: the reviewed configuration Azure is expected to have. Terraform links configuration to Azure objects, calculates proposed changes, and applies approved changes through a controlled delivery process.

Azure CLI has a different responsibility. Running inside PowerShell 7, it observes actual state: the configuration that exists in Azure at the time of inspection. It checks context, hierarchy, role assignments, policy compliance, networking, diagnostic settings, and other platform evidence. Carefully marked bootstrap or recovery steps can use imperative commands, but routine configuration returns to Terraform.

Separating those responsibilities avoids competing control paths. If Terraform and ad hoc commands both own persistent configuration, a quick repair can leave code and state behind. The next plan becomes surprising. Repeated surprise weakens confidence in automation, and engineers begin treating plans as obstacles rather than evidence.

The book follows a control loop that begins with purpose. CAF Strategy and Plan define the outcomes, constraints, and measures that should guide the platform. Landing-zone design then turns those decisions into clear responsibilities, boundaries, and technical tradeoffs.

Terraform expresses those design decisions as desired state. Reviews and tests check the proposed configuration before it reaches Azure. Azure Resource Manager then holds the resources, hierarchy, access, and settings that were deployed.

Deployment is not the end of the loop. Azure CLI, running through PowerShell, observes what actually exists and how the platform behaves. Inventory, compliance, connectivity, cost, and telemetry provide evidence rather than assurance based on intent alone.

Engineers compare that evidence with the expected outcome. The gaps become work for review, repair, or redesign. CAF reviews and engineering backlogs carry the learning into new priorities and controls. The loop then returns to purpose.

That return is essential. A technically correct deployment can support the wrong goal.

Accurate evidence can be ignored. A useful policy can become harmful at the wrong scope. Automation can repeat a weak decision faster than a person could.

System thinking is therefore not introductory decoration before the Azure material begins. It changes how the material is understood.

Management groups become structures for scope, authority, and inheritance. Terraform state becomes part of a control mechanism. Azure Policy becomes a relationship among risk decisions, technical effects, evidence, and human response. Monitoring becomes an information flow rather than a collection of dashboards.

The aim isn't a perfect platform. Azure services change, organizations reorganize, threats evolve, budgets tighten, and workloads create new demands. Even a strong original design becomes fragile when it cannot learn.

An adaptive platform can explain its purpose, express its approved state, observe its actual state, identify meaningful differences, and improve without losing control. It still needs precise resources, commands, and configurations. It also needs engineers who can see the relationships that make an event recur, the delays that hide consequences, and the goals that shape behavior. That wider engineering view is where system thinking for cloud engineers begins.

Chapter 1: Seeing the Whole Azure System

Chapter 1.1: From Resources to Relationships

Azure portals, APIs, and Terraform plans present the cloud as a set of technical objects. That view is necessary, but it is incomplete. Seeing the Whole Azure System requires a second view: the relationships that allow resources, teams, rules, money, and information to influence one another.

An inventory answers what exists. It can list subscriptions, virtual networks, private endpoints, policies, role assignments, workspaces, and storage accounts. It can't explain why a safe deployment takes weeks, why one policy produces many exemptions, or why engineers spend most of their capacity on support.

Those outcomes are created between the items on the list. A virtual network has address spaces and routes, but it also has an owner, consumers, a funding source, and a change process. A policy has a definition, assignment, scope, and effect. That policy represents a risk decision and creates work for the teams that must comply.

A delivery pipeline runs code, yet its behavior depends on its identity, permissions, agent, approval gates, reviewers, and failure messages. Monitoring collects records, but those records matter only when they reach someone with enough context and authority to act. Technical configuration and organizational design are already connected.

This makes Azure a sociotechnical system. A sociotechnical system joins technology, people, process, incentives, and information. The term doesn't mean adding team names beside resource icons. It means accepting that technical choices shape human behavior and that organizational choices become technical conditions.

Azure Resource Manager provides a clear example. Its hierarchy organizes resources beneath resource groups, subscriptions, and management groups. Controls assigned at higher scopes can affect lower scopes. That inheritance makes large Azure estates manageable, but it also lets one decision travel across many teams and workloads.

Hierarchy is only one kind of relationship. Identity connects people and automation to permitted actions. Networking connects services to consumers, shared platforms, the internet, and on-premises environments. Azure Policy connects risk decisions to deployment behavior. Terraform connects reviewed configuration to changes in Azure.

Economic relationships are just as real. Subscriptions sit within billing arrangements. Resources create usage, usage creates cost, and that cost must reach an owner. Tags and allocation rules

influence which platform capabilities appear expensive and which teams feel pressure to change. Two resources without a technical dependency can still be connected by one budget.

Work also crosses organizational lines. A workload team requests an application landing zone from a platform team. Platform engineers depend on identity, networking, governance, security, operations, and finance information before the request can be completed safely. Each group sees part of delivery, but no group sees the whole through its own tools alone.

Consider a request that reads, “Create a subscription.” A usable result needs an accountable owner, a cost center, correct management-group placement, inherited policy, and appropriate Azure RBAC assignments. The workload team needs enough authority to operate without gaining broad platform rights.

The subscription also needs suitable connectivity choices, name resolution, controlled ingress and egress, and access to approved shared services. Diagnostic settings must send useful records to the expected destination. A Terraform pipeline needs an identity that can deploy only the approved platform configuration. Completion depends on all of these relationships, not on the existence of the subscription alone.

If those connections are missing, the Azure object exists while delivery remains incomplete. Engineers fill gaps by hand, request policy exemptions, or open support tickets. The workload team experiences an unfulfilled promise. The central group experiences unexpected demand and growing operational load.

That result is emergent behavior. Emergent behavior is an outcome created by interactions among parts rather than by one part acting alone. No single Azure resource contains the entire delay. No single team owns every cause. The delay emerges because information, authority, dependencies, capacity, and incentives do not align.

A component inventory can’t reveal that alignment. It can prove that a subscription, virtual network, and role assignment exist. A relationship record asks who requested them, who approved them, which automation created them, which rules apply, what proves completion, and who carries the cost or risk. Those questions describe behavior rather than contents.

Start the record with one result, not with every known object. Suppose subscription delivery took twelve days. Note when the request arrived and when it became usable. Then find the waits, handoffs, and rework between those points.

Where did the request wait? The answer needs evidence, not a guess. Approval timestamps, pipeline runs, policy results, support tickets, and change records can show how work moved. A team label such as “slow” can’t explain the path.

Next, connect each wait to authority and dependency. One group may need data that another group owns. An engineer may see the missing field but lack the right to correct it. A pipeline may have permission to deploy yet lack an approved network choice.

Which relationship made the delay hard to detect? A request can move through several tools with no shared status. A technical step can finish while the result remains unfit for use. If completion means something different to each team, the metric won't reveal the gap.

Finally, name an owner and a review point for each weak connection. The owner doesn't have to perform all the work. That person or team does need enough authority to improve the path. Without that link, the record becomes another report that nobody can act on.

A useful record stays small. It names one result. It uses facts that teams can check. It shows the few links that shape that result.

The record marks what is not yet known. A blank is more honest than a false link. New facts can change it. That change is a sign of learning, not a flaw.

The record should lead to a next step. One team owns the step. One date sets the review. New evidence then shows whether the link became stronger.

Four relationship types provide a practical starting point. Authority relationships show who can decide, approve, configure, or block. Azure RBAC at management-group and subscription scope makes some of that authority visible. Excess privilege, stalled approval, and unclear decision rights reveal weak authority design.

Dependency relationships show what must exist or happen first. DNS, routing, a deployment identity, or a policy prerequisite can all sit on a delivery path. Waiting, rework, manual sequencing, and partial delivery are common signs of a missing or poorly owned dependency. Information relationships show what evidence moves, where it goes, and when it arrives. Terraform plans, Activity Logs, alerts, compliance results, and service requests all carry information. Surprises, unactioned alerts, and conflicting records show that the information path is weak.

Economic relationships connect funding, use, and cost. Subscription ownership, budgets, tags, and shared-service allocation make these links visible. Orphaned cost, disputed ownership, and distorted optimization show that technical consumption has lost its business context.

The categories are analytical tools, not separate layers. One Azure DevOps pipeline can participate in all four. It depends on an agent and provider endpoints.

Its deployment identity carries authority. Its plan transfers information to a reviewer. Its agents and deployed resources create cost.

Treating that pipeline as one box hides the conditions that make it safe, slow, trusted, or fragile. A relationship view keeps the technical object visible while exposing the forces around it. The aim is not to make a bigger inventory. The aim is to explain why the object behaves as it does. Azure Policy provides another example. A policy definition is a technical object. An assignment determines where it applies, an effect changes resource behavior, and a compliance result carries information. Governance chooses the rule, platform engineers deliver it, and workload teams experience its consequences.

An exemption therefore carries more than configuration. It is also a risk decision, a transfer of authority, and possible evidence that the service contract is difficult to use. Counting exemptions measures volume. Connecting each exemption to its owner, reason, expiration, affected team, and remediation path makes the evidence useful.

A relationship model should remain focused on the behavior under study. For application landing-zone onboarding, the working context needs business demand, workload teams, platform engineering, security, governance, finance, identity, Azure hierarchy, connectivity, monitoring, and delivery automation. Each element must contribute or receive something meaningful.

Business stakeholders provide outcomes, priorities, and constraints. They need evidence that platform work supports those goals. When demand cannot be traced to an agreed outcome, the work can become busy without becoming valuable.

Workload teams provide demand, required metadata, incident reports, and adoption feedback. They receive governed application landing zones. A healthy relationship lets requests complete at a predictable pace without hidden manual work.

Finance supplies budgets and allocation rules. In return, Azure usage and cost data must reach an accountable owner. Cost without ownership invites disputes, while ownership without useful data blocks sound choices.

Security and governance turn risk decisions into guardrails. They need compliance and security evidence from Azure. Findings become useful when they lead to owned action with a clear due date.

Platform engineers connect these groups through design, automation, support, and service ownership. They receive priorities, demand, and operating evidence. A healthy engineering group spends more capacity improving a repeatable service than repairing one-off gaps.

Microsoft Entra ID and the Azure hierarchy provide identity, scope, and inheritance. Approved access and placement decisions give those structures purpose. Authority should remain limited, explainable, and open to review.

Connectivity and DNS provide shared paths and name resolution. Workload requirements and operating signals show whether those paths work. A healthy relationship supports required communication without unmanaged bypasses.

Azure Monitor and related evidence stores hold logs, metrics, alerts, and history. Diagnostic settings and ownership data give those signals a destination. Evidence has value when it reaches people who understand it and can act.

Terraform delivery carries reviewed desired state into Azure. Plans, test results, deployment records, and drift observations return evidence to the engineers who own change. The relationship is healthy when persistent configuration remains reproducible and unexpected changes are investigated.

Each link needs a simple test. A team should know what it gives. It should know what it gets. It should know how success is shown.

If one answer is vague, the link needs work. This test is quick. It can fit into a design or service review. The test keeps a large Azure estate from hiding one weak link. The link can now be checked.

This record exposes missing connections. A Log Analytics workspace can collect data without routing an actionable signal to an owner. A policy report can exist without a review cadence. A subscription request can contain technical fields but omit cost ownership. A pipeline can deploy successfully without returning evidence to the approver.

More Azure resources don't repair those gaps by themselves. Another dashboard cannot create accountability. Another policy cannot settle an unclear decision right. Another automation step cannot define success. Technology can support a relationship, but it cannot replace one the organization never designed.

The relationship view also prevents misleading local optimization. Central engineers can shorten subscription delivery by removing checks, while security and operations inherit greater risk. Security can reduce one exposure by adding manual approval, while workload teams inherit delay. Finance can lower a shared-service budget, while every workload team inherits weaker capability.

None of those decisions can be judged from one resource or metric. The effect depends on who gains, who waits, who pays, who receives evidence, and who can correct the result. Those relationships turn isolated Azure changes into system behavior.

Component and relationship views therefore serve different purposes. The component view tells an engineer where a configuration lives and how to change it precisely. The relationship

view shows whether that change supports its purpose, shifts a problem, creates a delay, or weakens another team's ability to act.

Both views are necessary. Azure engineering loses precision without components. It loses judgment without relationships.

A cloud platform becomes understandable when resources are placed back among the teams, rules, information, and goals that give them meaning. An inventory shows what Azure contains. Relationships explain what the Azure system does.

Chapter 1: Seeing the Whole Azure System

Chapter 1.2: Choose the Boundary Before Solving the Problem

No cloud engineer investigates an entire system. The engineer selects part of it and gathers evidence. The evidence then supports a judgment. That selection creates a system boundary: the chosen limit around the elements, causes, and effects included in an analysis.

The boundary is a reasoning choice, not another Azure object. Microsoft Entra tenants, management groups, subscriptions, resource groups, and resources provide real technical scopes. A review can follow one of those scopes. It can also cross several or include only the part that shapes the behavior under study.

Every boundary makes something visible and leaves something out. A narrow one can support fast action. It can also hide a condition. That hidden condition can keep producing the problem. A broad boundary can expose causes that no single team owns. It can also bury the immediate fault beneath irrelevant history. Extra context has a cost. Good analysis therefore begins by choosing a boundary, not by expanding it without limit.

Consider a production deployment denied by Azure Policy. The same Terraform change worked in development. The pipeline identifies a rejected resource property. The first review therefore includes the failed run, the proposed setting, and the policy result.

That incident boundary can be correct. Engineers compare the setting with the rule and fix an invalid value. Delivery resumes. The evidence supports one limited conclusion: this deployment did not satisfy this control.

It doesn't explain why development accepted the configuration. It doesn't show why several teams encounter the same denial. It also can't explain why an approved module still creates the rejected value.

Those questions require a wider technical boundary. The review now includes the production subscription and its management-group path. It covers policy assignments, initiative parameters, exemptions, and the development subscription. It also includes the approved module version that produced the plan.

Azure Resource Manager exposes four management scopes. They are management groups, subscriptions, resource groups, and resources. Settings applied at higher scopes can affect lower scopes through inheritance. A denial on one resource can therefore express a decision made several levels above it.

Suppose development and production sit beneath different management groups. Production inherits a stricter initiative. The development run proves that Azure can process the setup. It doesn't prove that the setup meets production rules.

The word scope carries two meanings in this discussion. Azure management scope describes where a setting, role, or deployment applies. Analytical scope describes the territory an investigation covers.

The two overlap, but they aren't interchangeable. The failed resource lives inside one subscription. The useful analysis also needs a module repository and a policy rollout. It needs an exception process that no subscription contains.

Expanding the Azure scopes can reveal the immediate cause. It doesn't always explain recurrence. A supported module can lag behind a new policy parameter. A rollout can also omit tests against the supported path.

The useful boundary must then cross team lines. It includes the workload engineer, module maintainer, platform service owner, and governance owner. A person able to accept temporary risk also belongs. These roles enter because their facts or decisions can change the result.

Perspective is the viewpoint from which someone observes the bounded system. A workload engineer sees an interrupted release. A governance owner sees a preventive control working as intended.

A platform engineer sees a supported path that no longer reaches production. All three views can rest on facts. All three can remain incomplete. The disagreement comes from different purposes and success measures, not always from a misunderstanding of Azure.

The workload perspective values a safe release completed when promised. The governance perspective values prevention of an unapproved configuration. The platform perspective values a compliant path that works without hidden repair.

A sound review records those success conditions before selecting a response. Removing the denial can increase risk. Leaving it unchanged can preserve the failure. A manual approval can merely move the wait elsewhere.

Time creates another boundary. The failed pipeline runs for twelve minutes. The conditions behind it developed over twelve weeks. Looking only at execution time favors a property change because earlier decisions remain invisible.

A time horizon is the period treated as relevant to cause and effect. A longer horizon can include policy design, testing, rollout, module releases, and notices. It can also include earlier denials, exceptions, and repair work. That view separates one event from a repeating pattern.

The horizon should begin before the latest change that can explain the behavior. Suppose failures started after a policy revision. Evidence from both sides of that revision belongs inside. An older denial calls for evidence about demand, enforcement, or module use.

The horizon must also extend forward. A temporary exemption can restore delivery today. The control gap can remain for months. A module fix can help new consumers while older pinned versions continue to fail.

Environment names create another tempting boundary. Development, test, and production look like separate systems. They are stages in one delivery path. A production-only control makes evidence from earlier stages less useful.

The answer isn't to make every environment identical. Production can require stronger controls. Some differences can change a release. The delivery boundary must expose them early enough for action.

A Microsoft Entra tenant forms a major identity and admin boundary. Azure subscriptions rely on one tenant for authentication. Management-group structures also operate within a tenant. A single-tenant platform review can remain inside that limit.

Some organizations operate multiple tenants for isolation, regulation, mergers, or other needs. A service that spans them can have separate policy, identity, operating, and evidence paths. Those paths don't all belong in every review. The system boundary crosses tenants only where a difference affects the observed behavior.

This restraint matters because broader is not better by default. A review can include every tenant, vendor, budget, policy, team, and past decision. It then becomes a research program. A useful boundary includes what can change a decision and states the rest as context or exclusion. Four checks provide a practical inclusion test. An element belongs inside if it can change the suspected cause. The same is true if it changes the next owner, available action, or meaning of success. One such change is enough.

An element can remain outside when none of those conditions apply. It stays outside for now. Those last two words keep the choice open. A boundary is a claim that evidence can challenge. New facts can widen it. A policy result can lead from a resource to an assignment. The assignment can lead to a management group and then to a rollout decision. Deliberate expansion is a normal part of learning.

A team can also make the boundary smaller. It can begin with too much context. Evidence can then isolate the behavior. The team can narrow the review around the scopes, roles, and period that determine the result.

A boundary worksheet makes these choices reviewable. It starts with one observed behavior. The statement doesn't include a preferred cause. In this case, production deployments using the approved Terraform module failed three times in two weeks, while development deployments succeeded.

The next entry states the decision the review must support. One option updates the module or a policy parameter. Another grants a temporary exemption or changes promotion checks. A named decision prevents an unlimited search for context.

The worksheet then records the initial technical boundary. It includes the failed resource, Terraform plan, production subscription, and management-group path. Policy assignments, initiative parameters, exemption state, module version, and the development comparison also belong. Unrelated workloads and controls stay outside unless evidence connects them.

The team boundary names the roles whose facts or authority can alter the result. Here they are the workload team and module owner. The platform service owner and governance owner also belong. Security or risk staff enter when they own the need or can approve temporary treatment. The time horizon begins before the relevant change. It continues through the expected effects of a fix. It covers the policy rollout, module versions on both sides, three failures, active exceptions, and the planned repair date. That period can test recurrence without becoming a full history.

Each perspective receives a separate success condition. Workload delivery and platform use must be visible. Governance results and security risk must be visible too. Agreement can come later, but confident language can't repair an omission.

Evidence comes next. Pipeline logs and policy results show what Azure reported. Repository history, assignment changes, and release notes show how conditions developed. Service requests and decision records add the team context.

Conversations explain intent and local context. Observed records test whether the system behaved as intended. The two sources can disagree. That mismatch is evidence, not an inconvenience.

The worksheet also names exclusions. Application code can remain outside because the denial occurs before deployment. Cost can remain outside too. No clear cost path changes the cause or decision in this case.

Each exclusion needs a reentry trigger. Application code enters if the required property depends on workload design. Cost enters if the compliant option changes the service tier. It also enters when the option creates a material shared cost.

The final entry sets a review point. Before approving a solution, the team compares new evidence with the chosen boundary. A fix that works only inside the incident boundary is containment. It is not resolution.

Containment restores service or delivery within a narrow horizon. Resolution changes the conditions that keep producing the failure. Both can be necessary. Calling one the other hides remaining work.

For this case, containment grants a time-limited exemption with an owner. Resolution aligns the module and policy. It adds a compatibility test and exposes production differences earlier. The actions use different boundaries and need different proof of success.

Boundary discipline also reduces misplaced blame. A narrow team view points at the engineer nearest the failure. A wider view can show that the engineer followed an approved path. Its testing and governance limits did not align.

Accountability still matters. The resource author can correct a property. The module owner can repair reuse, and the governance owner can improve rollout checks. The platform owner can measure whether the whole supported service works.

Different decisions require different boundaries. Restoring one deployment can require minutes of logs. One resource definition can be enough. Preventing the next ten failures can require several Azure scopes, several views, and months of history.

A good boundary is small enough to support a decision. It is broad enough to preserve the causes and effects that matter. The boundary is a working claim, not the whole truth about the platform. Before the next Azure problem is solved, what becomes visible when its boundary moves one level outward?

Chapter 1: Seeing the Whole Azure System

Chapter 1.3: Stocks, Flows, and Cloud Queues

Cloud dashboards favor snapshots. They show resource counts, open incidents, policy exemptions, pending requests, and accumulated cost. A snapshot describes now, but it cannot explain how the number arrived there.

System behavior becomes clearer when the count is paired with motion. A stock is something that accumulates or decreases over time. A flow is a rate that adds to or removes from that stock during a period.

Suppose a platform has twenty open subscription requests. The backlog is a stock. Five accepted requests per week enter, while four completed requests per week leave.

If those rates persist, the stock grows by one request each week. The rule is simple: change in stock equals inflow minus outflow. The current total alone cannot show that direction.

A queue is a stock of work waiting for attention or completion. Some cloud queues appear in service-management tools. Others hide in email, chat, approval steps, and personal task lists.

Hidden work still waits. It still consumes attention and carries risk. Without a recorded state, the platform cannot measure its real demand.

Application landing-zone delivery offers a practical model. An application team supplies ownership, budget, network, and classification details. The platform team validates the request, configures a subscription, and hands it over.

Accepted requests increase the open-request stock. Completed application landing zones reduce it when they meet the handoff criteria. Returned requests either remain open or enter a separate validation queue.

That accounting rule must be explicit. A returned request can disappear from the main backlog while its customer still waits. The visible queue shrinks, but unfinished work does not.

Flows always need a time unit. “Seven requests” is an amount. “Seven requests per week” is a rate.

The observation intervals must match. Daily arrivals and monthly completions cannot be subtracted until they share a period. Mixing them creates precision without meaning.

Capacity limits the possible outflow. Platform capacity is the rate at which people and automation produce safe, accepted results. It is not the headcount on an organization chart.

Engineers spend time on incidents, support, reviews, maintenance, and improvement. Automation can configure Azure quickly yet wait for missing input. Effective capacity is measured by accepted completion, not available hours.

A team can remain fully busy while its outflow falls. Rework, blocked approvals, and task switching consume effort without draining the queue. Utilization cannot stand in for delivery. When demand stays above effective capacity, backlog accumulates. The first week can look harmless because the starting queue is small. Later, each request finds more work ahead of it. No task needs to become harder for the service to become slower. More unfinished work is enough. Accumulation changes the user experience.

An improvement reverses the arithmetic. If automation raises accepted completions above arrivals, the queue begins to shrink. Users still experience the old stock until the service works through it.

One strong week does not erase months of imbalance. The stock carries the result of earlier flows. A review based on one snapshot can miss real progress.

The model also clarifies why a target is not capacity. A goal of ten completions per week expresses intent. Only observed, accepted outflow shows what the service can currently deliver. Active Azure Policy exemptions form another cloud queue. New approvals add to the stock. Remediation, retirement, and expiration remove items from active status.

The exemption object can remain after its expiration. Active exemptions and stored exemption objects are different stocks. Clear labels keep those measures honest.

The current total, new approvals, and closures answer different questions. Age adds essential context. A stable total can hide old exemptions that never move while newer items close quickly.

Two estates can each hold forty exemptions and follow different paths. One creates ten and closes ten each month. The other creates two and closes none, so its stock keeps growing.

The first estate has high movement and a stable stock. The second has low movement and steady accumulation. A single count makes them look the same.

Open incidents share this structure. New incidents enter, resolved incidents leave, and reopened incidents return. A falling stock can reflect stronger reliability or faster accepted resolution.

It can also reflect weaker closure rules. If unresolved conditions move into another queue, the service hasn't removed the work. It has changed the label.

Technical debt accumulates through shortcuts, unsupported versions, manual steps, and deferred fixes. Refactoring, upgrades, automation, and retirement remove it. The measures are imperfect, but ignoring the stock does not remove it.

Not all debt belongs in one bucket. A critical unsupported provider and a naming cleanup carry different risk. Separate stocks are clearer when items have different owners, urgency, or removal work.

Cost requires careful classification. Daily spending is a flow. Accumulated cost for a month is a stock built by daily charges.

A monthly total can remain within budget while the daily rate rises. The current rate can fall after a spike while accumulated cost remains high. Both measures are valid, but they answer different questions.

A remaining budget can also act as a stock. Approved funding establishes the amount, and spending draws it down. A revised allocation can add to what remains.

The cost basis must stay consistent. Actual, amortized, and forecast figures are not interchangeable flows. Clear labels protect the arithmetic from a false balance.

Resource counts are stocks too. Creation and deployment add resources, while deletion and retirement remove them. Growth is not automatically failure because demand and value can grow.

Purpose determines the useful balance. Sandbox resources need a healthy retirement flow when experiments end. Production subscriptions require slower removal and stronger evidence.

An unresolved change backlog offers another example. Proposed platform changes enter through demand, maintenance, security findings, and service updates. Released or rejected changes leave only when the decision is recorded.

Unclear ownership reduces the outflow. So do incomplete acceptance criteria and large batches. The backlog then grows even if its arrival rate remains stable.

A practical stock-and-flow model begins with one outcome: predictable application landing-zone delivery. Its main stock is accepted requests not yet handed off. The unit is requests.

The inflow is accepted requests per week. The outflow is completed and accepted application landing zones per week. Matching the unit and interval makes comparison possible.

Next, name the work that changes either rate. Missing data can reduce accepted inflow while building a validation queue. Policy conflicts, address-space choices, and approval waits can reduce completion.

Automation, clear product lines, and useful defaults can raise effective capacity. They count as improvements only when accepted outflow rises. Automation that creates rework has not improved the service.

Record the stock and both flows at a consistent interval. Add age bands so an unchanged total cannot hide stalled work. Track returned and reopened items rather than erasing them.

Then compare observed demand with observed capacity. Eight accepted requests and six accepted completions per week increase the backlog by two. The same service needs either less inflow, more outflow, or both to stop growth.

That statement does not prescribe the intervention. Reducing useful demand can damage adoption. Increasing outflow by weakening handoff quality can move work downstream.

The model exposes the arithmetic before a team chooses an action. It does not decide which goal matters most. That judgment still belongs to the people responsible for the service.

The model should never treat people as inventory. Engineers contribute judgment, skill, and learning that a throughput rate cannot represent. Their busyness is not the platform's product.

Completion needs a stable definition too. A subscription object is not a finished application landing zone. Handoff requires agreed ownership, placement, access, governance, connectivity choices, and evidence.

Weakening that definition can make the queue shrink on paper. Support demand and rework then grow outside the model. Accepted completion protects the outflow from that distortion.

A useful review therefore asks for five measures. It needs the current stock, inflow, outflow, item age, and effective capacity. Together, they explain load, direction, stagnation, and the service's present limit.

Snapshots still matter because they show current load. Their meaning grows when arrivals and departures reveal motion. Count the stock, measure the flows, and manage the gap.

Chapter 1: Seeing the Whole Azure System

Chapter 1.4: Feedback, Delay, and Cloud System Archetypes

A cloud incident does not end when service returns. The response changes configuration, team habits, and future work. Those effects can return as conditions for another incident.

A feedback loop is a chain of cause and effect in which a change eventually influences its source. Feedback is not a survey here. An effect travels through the system and comes back.

Reinforcing feedback amplifies the direction of change. More of a condition produces still more, or less produces still less. Reinforcement can drive growth or decline.

Balancing feedback resists change or moves a system toward a target. Azure Policy can block a state that differs from an approved rule. Monitoring and response can move service health toward an expected condition.

Neither loop type is good by definition. Reinforcing feedback can spread a useful platform path. Balancing control can defend a poor target or create friction.

Portal-first firefighting shows reinforcement. An incident creates urgency, an engineer changes a resource in the Azure portal, and service returns. The fast result rewards the manual action.

That reward makes the response more likely next time. Manual work consumes time that could improve Terraform, tests, and diagnostics. The approved path stays weak, so later incidents create more urgency.

The chain is easy to state. More urgency produces more manual intervention. More manual intervention reduces improvement capacity, and weak automation produces more urgent work.

The direction can reverse. Engineers capture the repair in code, test it, review it, and improve diagnostics. Stronger automation reduces manual repair and releases capacity for improvement.

A balancing loop appears in incident response. Monitoring observes a condition, an alert reaches an owner, and an action restores service. The loop needs a useful signal, valid target, authority, and effective action.

An ownerless alert cannot balance the system. A target without a measure cannot guide action.

A response that removes only the symptom does not change recurrence.

Delay makes loops hard to read. A delay is the time between an action and an observable effect.

Evaluation, information movement, approval, adoption, and learning all take time.

Azure Policy compliance evidence is not instant for every change. An Azure Monitor alert can fire before a human investigation finishes. A platform release can precede adoption across many team cycles.

Ignoring delay encourages repeated action. Teams add another alert or tighten another rule before judging the first change. Overlapping responses make later behavior difficult to explain. Delay can create false confidence too. A risky change can look safe before enough use reaches it. Support demand or exemption volume can rise after success is declared.

Platform and consumer clocks differ. A central team measures from a control's release. A workload team experiences the control during a later deployment.

The two dates can be weeks apart. One marks technical availability, while the other marks practical exposure. A useful review records both.

Nonlinear behavior means equal input changes do not always create equal output changes. A small demand increase can create a large wait after capacity saturates. The system crosses a condition instead of moving smoothly.

Policies and quotas contain thresholds. A deployment proceeds until one property violates a rule. Capacity appears available until the next request cannot be served.

Alert volume is nonlinear. Ten useful alerts can prompt action. Hundreds of similar alerts can overwhelm responders, so more notification creates less response.

Trust can change sharply too. Teams can accept several useful controls and reject a platform after one high-impact failure. Repairing trust requires more than reversing one setting.

Averages can hide these changes. An average response time can look stable while one class of requests becomes unusable. Distribution and context reveal the break point.

Recurring structures are called system archetypes. A system archetype is a common causal arrangement that produces recognizable behavior. It guides investigation without replacing evidence.

Fixes that fail begins with a real problem and a useful short-term response. The response reduces the symptom while creating a consequence that restores it. Portal-first repair fits this pattern.

The manual change restores service. If approved configuration remains unchanged, a later deployment can reverse the repair. The fix succeeds now and fails over time.

Shifting the burden adds dependence. Expert rescue becomes easier than documentation, self-service, or automated recovery. The deeper capability receives less investment.

Each rescue proves the expert's value. It can also keep knowledge concentrated. The organization becomes dependent on the rescue it celebrates.

Access work provides an example. A privileged engineer repairs one assignment because the normal request path is unclear. The case closes while the supported path stays weak.

Limits to growth begins with success. A subscription-vending service attracts demand because it works. Validation, network planning, or support capacity then fails to grow with adoption. Delivery slows and confidence falls. Teams return to older routes or create alternatives. The constrained capability, not the value of self-service, limits growth.

More promotion can worsen the problem. Demand grows while the constraint remains. Growth resumes only when the relevant capacity or design changes.

The constraint can move after improvement. Automation can remove configuration work and expose slow approval. The service must observe the whole path instead of declaring the limit solved.

Local optimization improves one measure while harming the whole. Governance can reduce visible policy risk through broad deny controls. Workload teams can restore delivery through repeated exemptions.

Both local dashboards improve. The platform gains exception work, inconsistent treatment, and lower trust. Local success creates system failure.

Security can optimize for fewer actions while operations optimizes for recovery speed. Finance can reduce shared cost while platform quality falls. A system measure is needed to expose the trade.

Policy friction can form a reinforcing loop. More denials create more delivery pain. Pain encourages unmanaged workarounds or alternative subscription paths, reducing visibility.

Lower visibility raises perceived risk. Governance responds with stricter controls, creating more denials. Slow and complex subscription access can contribute to shadow IT.

The intended balancing loop joins control with usability. Policy identifies an unacceptable state. A useful message and supported path guide the team toward compliance.

Evidence confirms the result. Governance learns whether the control reduces risk without blocking legitimate work. Correction without observation is an incomplete loop.

The two policy loops can exist together. The control balances unsafe configuration while poor usability reinforces bypass behavior. Looking at only compliance hides the second result.

Analyze a loop from behavior over time. Name what rose, fell, recovered, or repeated. Do not start with an archetype label.

Write the causal chain in plain sentences. More urgent incidents create more manual changes. More manual changes reduce automation time, and weaker automation creates more incidents. Test every link. Incident records show urgency, Azure activity records show changes, and repositories show whether repairs reached code. Capacity records show which improvement work was displaced.

Sequence matters. A change that occurs after the outcome cannot explain that outcome. Dates protect a persuasive story from becoming a false cause.

Mark the important delays. Record when a control, alert, automation change, or operating decision began. Compare that point with later behavior.

Find the target in balancing loops. Compliance, service health, delivery time, or budget can provide one. Confirm that the measure represents the intended result.

Check nonlinear conditions. Look for thresholds, saturation, large batches, alert floods, and sharp changes in trust. An average can hide the point where behavior changed.

Search for a repeated short-term response. Then ask which deeper capability receives less attention after each use. That test often exposes fixes that fail or shifting the burden.

Compare local and system measures. A team can meet its target while moving work, risk, or cost elsewhere. The receiving queue is part of the result.

Then test how an intervention changes the structure. Another manual repair can restore service while strengthening dependence. A tested code change, usable runbook, and actionable signal can improve the path.

Feedback thinking does not reject urgent repair. It separates containment from the conditions that recreate urgency. Service returns when the incident closes; resilience grows when the loop changes.

Chapter 1: Seeing the Whole Azure System

Chapter 1.5: Leverage Points and the CAF Learning Loop

Cloud teams often change the nearest resource when behavior is poor. They resize a service, add an alert, grant access, or edit policy. The action can help without changing what makes the problem recur.

A leverage point is a place where change can significantly affect system behavior. Leverage does not mean a large change. It means changing a condition that shapes many later decisions. Resource settings are visible but often shallow. A corrected setting repairs one state. A reusable module, validation rule, or ownership decision can influence every later use.

Deeper is not always better. An incident can require immediate resource repair. The response should also test the condition that made repair necessary.

Parameters form one intervention level. Teams change budgets, thresholds, quotas, timeouts, staffing, or service sizes. The surrounding process can absorb those changes without changing its behavior.

Added capacity can move work faster. If intake remains unclear, the capacity fills with rework. The parameter changed while the service design stayed weak.

Information flows reach deeper. Workload teams can see policy requirements before deployment. Platform teams can see request age, return reasons, and adoption friction.

More dashboards do not guarantee leverage. Information must arrive in time and reach someone able to act. Meaning, ownership, and authority complete the path.

Rules shape available choices. Azure Policy, Azure RBAC, pipeline checks, exception criteria, and architecture decisions all influence action. A rule can remove unsafe options or create workaround pressure.

The supported path determines rule quality. A deny result without guidance only blocks action. A compliant module, clear evidence, and named owner make the rule usable.

Incentives shape preferred choices. Teams respond to delivery targets, cost allocation, audit findings, support load, recognition, and career signals. Actual rewards can contradict written policy.

Metrics participate in that system. Closed-ticket counts can reward premature closure. Accepted-outcome measures reward work that remains useful after handoff.

Ownership is another leverage point. Work persists when no role owns the whole result. Accountability without decision rights changes only a label.

Effective ownership joins authority, evidence, and review. A platform service owner can improve intake, automation, support, and quality together. Distributed tasks still produce one owned result.

Goals reach deeper because they define what the system produces. Deploying more resources differs from enabling safe business outcomes. Volume and value create different decisions.

Goals must be testable. “Be cloud first” cannot resolve a tradeoff. A measurable outcome can be compared with evidence and revised.

Learning is the deepest practical leverage used here. A learning system changes information, rules, ownership, and goals when assumptions fail. It does not protect the first design from evidence.

Depth has a cost. A rule change can affect many teams. A goal change can reshape far more work.

The team must match reach to proof. It can start small when risk is high. It can widen the change as proof grows.

Some fixes work at more than one level. A new form can add data and enforce a rule. A named owner can use that data to improve the service.

The levels are not a strict rank. They widen the set of choices. The best mix depends on the result and the risk.

The Microsoft Cloud Adoption Framework, or CAF, organizes guidance into seven methodologies. They are Strategy, Plan, Ready, Adopt, Govern, Secure, and Manage. Together, they can link purpose, delivery, operation, and learning.

CAF's foundational methodologies are Strategy, Plan, Ready, and Adopt. They provide a sequence from business goals to prepared environments and workload adoption. Govern, Secure, and Manage shape ongoing cloud operations.

That structure is not a one-way project plan. Organizations revisit foundational choices as adoption matures. Operational evidence can expose an assumption that needs revision.

Strategy defines why cloud adoption matters. It connects motivations, mission, objectives, and measurable outcomes. In system terms, it sets the purpose used to judge evidence.

Strategy is a hypothesis rather than a slogan. It predicts that cloud capabilities will create value. Later operation can support or challenge that prediction.

Plan turns purpose into an operating approach. It addresses the estate, operating model, responsibilities, skills, dependencies, sequence, and cost. Plan changes who can act and how work moves.

A plan has leverage when it resolves decisions and constraints. A document without owners or capacity is only information. A living plan changes with demand and evidence.

Ready establishes the Azure foundation. Tenant, subscription, management-group, identity, connectivity, governance, management, security, and automation choices become shared conditions. They shape every application landing zone.

Ready applies defaults and rules at scale. A platform landing zone, Terraform module, or management-group placement can affect many deployments. Broad reach makes design quality important.

Adopt is where prepared environments support workloads and business outcomes. This book stops its capstone at the platform-to-workload interface. The interface still carries demand, requirements, acceptance, and adoption evidence.

Adoption friction is useful information. Repeated exceptions, abandoned requests, long handoffs, and unsupported paths expose weak assumptions. That evidence should influence Ready, Plan, and Strategy.

Govern translates risk into policies, controls, and accountable decisions. Its leverage comes from rules, information, and incentives. Good governance makes safe action easier to repeat.

Govern learns from exceptions and compliance. A repeated exemption can reveal a poor rule, poor guidance, or missing platform capability. Weaker enforcement is not the only answer.

Secure applies protection across the journey. Security affects strategy, planning, readiness, adoption, governance, and operations. A final security gate discovers risk after change becomes expensive.

Secure creates leverage through identity, least privilege, protective defaults, detection, response, and learning. Exposure, incidents, and recovery evidence must change earlier decisions. Static protection loses value as threats and platforms change.

Manage operates the cloud estate and produces evidence about actual behavior. Administration, monitoring, protection, cost, incidents, change, and recovery test the design. Operations turn assumptions into observable results.

Manage is not CAF's endpoint. Its evidence returns to every earlier methodology. That return forms the CAF learning loop used in this book.

The loop needs a path for proof. Logs, cost, wait time, policy results, and user reports must reach a review. A review with no right to act cannot close the loop.

The loop also needs pace. Some facts need a fast response. Goals need a slower and broader review.

Teams should record what they thought would happen. They should state what they saw. The gap is the raw material for learning.

Not all proof points the same way. Cost can improve while wait time grows. A sound review keeps both facts in view.

Suppose application landing-zone delivery keeps missing its target. Manage records request age, failures, support demand, and completion quality. Govern adds denial and exemption patterns, while Secure adds access and protection findings.

Adopt contributes workload-team experience. Ready identifies platform and automation constraints. Plan examines ownership and capacity, while Strategy tests the intended outcome. The response can use several leverage points. A parameter adds temporary capacity. Better information exposes return reasons, while a rule validates required data at intake.

Ownership can give one service owner end-to-end authority. A goal can replace subscription-object delivery with accepted landing-zone handoff. A learning change makes evidence review and revision routine.

Aligned interventions reinforce one another. A metric without ownership becomes another report. Ownership without authority becomes blame, and a rule without support becomes friction.

Misaligned changes can cancel out. A fast path can clash with a slow approval rule. A cost goal can clash with paid tools enabled by default.

The receiving team can reveal that clash. Work that leaves one queue may enter another. The whole path must count in the result.

A leverage review starts with observed behavior. It uses the boundary, stocks, flows, loops, and delays already developed. Those models show where behavior is produced.

List interventions across parameters, information, rules, incentives, ownership, goals, and learning. Ask which choice changes repeated decisions. Do not select depth only because it sounds ambitious.

Test reach and risk. A management-group change can affect many subscriptions. A changed goal can alter budgets, teams, and controls.

Broader reach requires stronger evidence and safer rollout. Set a measure and review date before acting. Observe both intended results and displaced work.

Use a stop rule as well. A broad change should pause when harm grows past an agreed point. Safe rollback keeps learning from becoming a bet.

The review should name the assumption under test. It should record the expected behavior and the evidence source. A result that contradicts the expectation becomes input, not failure to hide.

CAF provides the organizational path. Strategy supplies purpose, Plan organizes action, Ready builds the foundation, and Adopt connects demand. Govern, Secure, and Manage shape and observe operation.

Evidence travels back through the path. Chapter 2 returns to Strategy because a platform should not merely keep running. It should keep learning what it is for.

Chapter 2: Strategy — Give the Cloud System a Purpose

Chapter 2.1: Assess the Current System Honestly

Strategy — Give the Cloud System a Purpose begins with an honest account of the present. A strategy built on declared maturity inherits the gaps in that declaration. The organization needs evidence of how its Azure system behaves now.

Maturity labels can summarize a finding. They cannot replace one. Teams need to know which outcomes the system produces, under which conditions, and with what effort.

A platform can own management groups, policies, pipelines, and dashboards while delivering poor service. Those items prove that components exist. They do not prove that safe application landing zones arrive predictably.

Observable behavior is stronger evidence. Request age, deployment results, exemptions, recurring incidents, recovery tests, access reviews, and cost ownership show what the system does. Interviews explain the local context.

Azure records and interviews need each other. A denied deployment does not explain why a supported path failed a legitimate need. A workshop can describe a smooth process while timestamps show repeated waits.

Honest assessment compares claims with results. Agreement raises confidence. A mismatch becomes a finding that needs explanation.

Missing proof is also a result. It shows where the system cannot yet see itself. The gap should be named without blame.

No team needs perfect data to start. It needs enough sound facts for the next choice. More proof can follow when the choice has more risk.

Begin with the outcomes people expect today. The platform might promise safe subscription delivery, reliable connectivity, clear ownership, controlled access, useful evidence, and understandable cost. Write each promise in observable terms.

Then test actual outcomes. Record elapsed time, repair, incomplete handoffs, variation, and hidden work. Real cases add meaning that an average cannot provide.

This work does not define the future mission. Chapter 2.2 owns that task. The assessment records today's promise and how well the system meets it.

Architecture is the next area. Inventory the tenant, management-group hierarchy, subscriptions, shared services, identity, connectivity, governance, management, security, and automation. Include important external and on-premises dependencies.

Move beyond inventory. Record unsupported versions, manual settings, single points of knowledge, address-space limits, quota pressure, inherited controls, and recovery dependencies. These conditions shape possible strategies.

Sample depth as well as breadth. A tenant-wide query can reveal distribution. One traced service can reveal cause.

Keep the raw record behind the claim. A short summary helps readers, while the source lets them test it. Both forms have value.

Choose samples by risk and variation. Include a common path, a difficult path, and a recent failure. This avoids assessing only the polished case.

An architectural constraint makes some choices costly, slow, risky, or unavailable for now. Naming it supports deliberate sequence. Hiding it makes the future plan fictional.

Some constraints are required. Regulation can shape data location or separation, and commercial terms can shape subscription creation. Approved network design can limit placement.

Other constraints are historical habits. An undocumented script, broad role, or manual route can persist without a current need. Separate required limits from untested assumptions.

Organization is part of current state. Identify who performs platform, workload, governance, security, finance, and operations work. Compare decision rights, named accountability, and actual action.

An organization chart shows formal structure. Approvals, incidents, repositories, and service requests show the operating structure. Repeated unblockers can hold more practical authority than named owners.

Organizational readiness is the present ability to carry change. It includes skill, time, authority, collaboration, leadership support, and partner access. A training plan alone is not readiness.

Load matters as much as skill. A capable team with no improvement capacity cannot deliver large change. A security group without a review path cannot make controls usable.

Partners can hold key knowledge. Record the service, contract, access, and exit constraints around that knowledge. External help is capacity only when it is available when needed.

Check the resilience of ownership. A process that works only when one person is present has a continuity limit. Backup ownership needs evidence from real use.

Delivery evidence shows how approved change reaches Azure. Record Terraform and other infrastructure-as-code paths, portal use, reviews, tests, approvals, deployment, rollback, and reconciliation. Mark which routes teams actually use.

A pipeline does not prove controlled delivery. Check whether failures are clear and emergency changes return to code. History provides stronger evidence than screenshots.

Record waits around delivery. Missing inputs, unclear ownership, approval waits, and rework can dominate elapsed time. Describe those conditions without redesigning the workflow.

Governance evidence links risk decisions with control. Review policy assignments, initiatives, exemptions, compliance results, remediation, and ownership. Compare actual scope with team expectations.

A high compliance percentage can coexist with important gaps. Exclusions, inapplicable resources, old evidence, and repeated exemptions change its meaning. Trace critical controls from intent to result.

Governance readiness includes safe control change. Record who proposes, tests, approves, deploys, explains, and retires a rule. A control lifecycle without an owner is a constraint.

Security assessment examines exposure and capability, not product presence. Review privileged access, identity lifecycle, baselines, telemetry, incident readiness, response authority, and recovery learning. Tie findings to scope and time.

A license does not prove protection. Confirm that capabilities are enabled, signals reach responders, and actions are practiced. Unknown coverage should remain unknown.

An unknown is not a pass or a fail. It is a task with scope and an owner. This keeps doubt from turning into false safety.

Operations evidence shows whether the platform can be administered, monitored, and protected. Review incidents, changes, health, alert ownership, recovery tests, support demand, and manual work. Separate platform duties from workload duties.

No recorded incident can mean reliability or weak detection. Compare incidents with alerts, user reports, availability, and change history. Several sources reduce false confidence.

Recovery evidence deserves its own test. A written plan shows intent, while an exercise shows capability. Record the last test, scope, result, and unfinished action.

Support demand can expose hidden operations. Chat messages and direct requests may never enter the service system. Estimate their volume and mark the low confidence.

Cost assessment connects spend with value and ownership. Record actual and amortized cost where relevant, forecasts, budgets, shared allocation, commitments, and unowned resources. Keep scope and period consistent.

Low spend does not prove efficiency. The platform can defer resilience, security, or automation.

High spend is not waste when it supports explicit outcomes.

Mark data quality. Tags can be stale, cost centers can be missing, and dashboards can omit subscriptions. Confidence belongs beside each finding.

The current-state canvas joins these areas in prose. It starts with an outcome and the people who experience it. It records evidence, observed behavior, constraints, confidence, and the next needed fact.

For architecture, record the estate shape and limiting conditions. For organization, record real authority, skill, load, and ownership. For delivery, record paths, controls, waits, and repair.

For governance and security, record intent, scope, operation, exceptions, and response. For operations and cost, record health, recovery, demand, spend, and ownership. Keep every area tied to the chosen outcome.

Every claim names a source. Useful sources include Azure Resource Graph, policy compliance, role assignments, activity records, pipelines, repositories, incidents, requests, cost details, interviews, and decisions. Sensitive data stays protected.

Evidence has a date and scope. One old subscription report cannot establish tenant-wide current state. The canvas states both coverage and exclusions.

Confidence should reflect source quality. High confidence needs consistent sources or one direct authoritative measure. Low confidence creates a discovery task.

Each discovery task needs an owner and limit. Endless assessment can delay action as effectively as weak evidence. Gather enough proof to support the next strategic choice.

Set a stop point for the first review. New facts can be added after that point. The strategy does not need to wait for a flawless map.

Some risks cannot wait for the full review. Mark them for prompt containment. Keep that action separate from the future design.

Review the canvas with the people who supplied the facts. They can correct scope and context. The record should change when better proof appears.

Record contradictions rather than smoothing them away. Strong enforcement claims and frequent bypass reports can both be sincere. Scope, exemptions, and deployment evidence can resolve the difference.

Do not turn the canvas into a score contest. A maturity score compresses context. Evidence and constraints must remain visible behind it.

External benchmarks can reveal a capability gap. They cannot define the right state without purpose, risk, estate, and operating model. Comparison is a prompt, not a verdict.

End with constraints, not a preferred design. Name facts that limit strategy, gaps that need evidence, and risks that need containment. Do not let the desired future rewrite the present.

An honest current state can look less mature than the declared one. That discomfort replaces performance with a basis for choice. Strategy begins when the system no longer needs to pretend about where it stands.

Chapter 2: Strategy — Give the Cloud System a Purpose

Chapter 2.2: Define Motivations, Mission, and Objectives

Cloud adoption needs a clear and measurable reason for change. That reason must connect business purpose to the behavior expected from the cloud system. Without it, Azure adoption becomes a list of unrelated projects.

A datacenter lease can expire. A vendor can end support for a product. An audit can expose weak control evidence.

These events create urgency, but urgency is not purpose. An event explains why action must begin now. A motivation explains which lasting business condition should improve.

That distinction prevents a common mistake. “Leave the datacenter by December” is a deadline. It does not explain which value the move must preserve or create.

The deadline still matters as a constraint. It shapes sequence and available choices. The motivation can be lower continuity risk, faster market response, or more transparent cost.

One event can reveal several motivations. A merger can create pressure to integrate teams, standardize control, remove duplicate services, and report cost by business unit. Those motives rarely have equal priority.

The reverse also occurs. One motivation can produce several change efforts. Better resilience can require changes to the platform, workloads, processes, and supplier agreements.

A motivation is the business reason for changing the cloud system. It identifies value to gain, harm to reduce, or an ability the organization needs. It should remain meaningful after the trigger passes.

The Microsoft Cloud Adoption Framework groups common motivations into three broad areas. Organizations seek to reduce business risk, accelerate innovation, or enhance agility and efficiency. These categories prompt discussion rather than supply the answer.

“Reduce business risk” is too broad alone. The organization must say which exposure matters and why. It can seek reliable recovery for a regulated service or stronger proof of access review.

“Accelerate innovation” also needs a boundary. Faster experiments should create faster learning, not uncontrolled service use. The platform must support useful speed while preserving required control.

“Improve efficiency” can hide several aims. It can mean less manual platform work, shorter waits, clear cost ownership, or consistent operations. Each result needs different capabilities.

Start with evidence from the current-state assessment. Use recurring waits, incident patterns, audit findings, cost ambiguity, missed opportunities, and architectural limits. Do not repeat the full assessment.

Write the observed condition for each candidate motivation. Then state the business consequence and desired change. This connects aspiration to evidence.

Consider a company where application teams wait weeks for Azure subscriptions. Requests arrive through email, required inputs vary, and policy failures appear late. Product launches absorb the resulting delay and rework.

The immediate event is a planned datacenter exit. The enduring motivation is greater delivery agility without weaker control. Earlier evidence gives that motivation a real starting point.

The wording should survive a product substitution. “Adopt Azure Policy” names a possible means. “Reduce late discovery of noncompliant designs” states a reason that can guide several means.

Motivations need both priority and urgency. Priority expresses relative business importance. Urgency expresses how soon delay creates an unacceptable result.

The two are not identical. A high-priority resilience improvement can require careful preparation. A smaller contract issue can still require immediate action.

Classifying both qualities exposes sequence choices. It also keeps the loudest event from becoming the entire strategy. Leaders can decide which pressure warrants investment now.

Too many top priorities erase choice. A short ranked set is more useful than a complete catalog. Deferred motivations should remain visible with a reason for deferral.

Motivations provide direction, but they do not define the platform’s role. That role belongs in a mission. A mission is a stable statement of what the Azure platform exists to enable.

The mission is not a service inventory. It should not promise management groups, virtual networks, dashboards, or pipelines as ends in themselves. Those are possible implementation parts.

A useful platform mission answers four questions. Who does the platform serve? Which business value does it enable?

It also explains the means at a high level and preserves a boundary. The answer must guide tradeoffs. It must also survive technology change.

The example platform mission is to enable workload teams to obtain and operate governed application landing zones. Teams use supported paths to consume the service. The platform retains shared identity, connectivity, governance, and management boundaries.

This mission identifies the consumer and enabling value. It names the interface and retained platform responsibility. It does not claim ownership of application delivery.

The boundary matters. A mission to “deliver all cloud solutions” creates unclear accountability.

The capstone platform provides a foundation and contract, not an application workload.

A mission should also help reject work. A request can be technically interesting and still fall outside the declared purpose. Rejection protects capacity for capabilities that support the mission.

Test the mission against real decisions. It should guide a choice between portal fulfillment and a supported automated path. It should clarify whether workload monitoring belongs to the platform team.

If the mission cannot guide those choices, it is ceremonial. If it decides every implementation detail, it is too narrow. A useful mission sets direction without becoming a design.

The mission then needs objectives. An objective is a defined change in outcome that advances the mission. It makes the desired future open to evidence.

An objective is not an activity. “Build a subscription pipeline” describes work. “Reduce the effort and variation in governed landing-zone delivery” describes an intended change.

An objective is not merely an output. A policy initiative, reference module, or dashboard can exist without improving the outcome. Strategy should not mistake delivered components for realized value.

An objective also differs from a platform capability. A capability is an ability the system must provide. Subscription vending, policy testing, ownership registration, and evidence collection are examples.

These distinctions create a strategy chain. Motivation, mission, objective, capability, and evidence form its five links. Each link answers a different question.

Motivation explains why change matters. Mission states the platform’s enduring role. An objective defines the outcome that should change.

A capability names what the platform must be able to do. Evidence shows whether the expected change occurred. The chain connects intent to observable behavior.

Write every link as a short statement. Record the motivation and its present evidence first. Place the platform mission after it.

Add a small set of objectives. Under each objective, name the abilities the platform needs. Finish with the evidence that could confirm or challenge the result.

The first example objective is to provide governed application landing zones through one supported request path and an agreed service window. The current wait supplies a baseline. Request and deployment records can show change.

A second objective is to expose control failures before production deployment. The objective concerns earlier feedback, not the number of policy definitions. Pipeline results and rework records can supply evidence.

A third objective is to establish ownership and cost context when each application landing zone is created. The intended result is accountable operation. A tag cannot prove that an owner accepts responsibility.

Good objectives name an outcome, scope, and time horizon. They also identify evidence that can confirm progress. Accountability must be clear enough for review.

The full measurement system comes later. Chapter 2.4 will balance outcomes, drivers, system health, and counter-metrics. At this stage, an objective needs testability rather than elaborate numbers.

Testability means the objective can be challenged. Reviewers can ask what changed, for whom, over which period, and against which starting condition. A claim that cannot fail cannot support learning.

Absolute language deserves caution. “Achieve complete compliance” can ignore exemptions, inapplicable resources, and evidence gaps. “Move everything to Azure” can reward migration without value.

A testable objective can remain ambitious. It should expose tradeoffs instead of hiding them. Speed, risk, cost, and experience need joint consideration.

Avoid writing an objective for every concern. Several related outcomes can support one mission. Too many objectives divide attention and create competing queues.

Rank objectives by contribution to the mission. Name important dependencies between them.

A self-service path can depend on policy feedback and ownership data being ready first.

Constraints from Chapter 2.1 remain active. Team capacity, required separation, contract dates, or address-space pressure can shape the first objective. They should affect sequence, not silently replace purpose.

Assumptions remain visible as well. Leaders can believe that slow platform fulfillment delays product releases. The chain should show the supporting evidence and its confidence.

Chapter 2.5 will manage such claims as strategy hypotheses. For now, do not present an assumption as a proven cause. The mission can remain stable while evidence changes the path.

Review the strategy chain in both directions. Read forward from motivation to evidence and ask whether each link follows. Read backward from a capability and ask which objective requires it.

A capability without an objective can be solution enthusiasm. An objective without a motivation can be local optimization. Evidence that cannot relate to an objective cannot test the strategy. The chain also exposes missing capabilities. A mission can promise supported landing-zone delivery while every route depends on manual coordination. That gap belongs in later planning. Do not design the final architecture while creating the chain. Name the platform's required ability before choosing its implementation. This preserves room for Plan and Ready decisions. Keep the chain readable. Link each motivation to a small number of objectives. Link each objective to the minimum abilities needed for the outcome.

Place present evidence beside the motivation and future evidence beside the objective. Present evidence explains why change matters. Future evidence will show whether the change helped. Review the wording with business and technical leaders. Business review protects the value. Technical review protects feasibility and platform boundaries.

Workshop agreement does not prove wider alignment. Groups can gain or lose autonomy, budget, or control. Chapter 2.3 will make those incentives and conflicts explicit.

Here, the goal is shared strategic meaning. Words such as "secure," "fast," and "efficient" need context. Otherwise, each group can endorse a different strategy with the same terms.

Several warning signs expose a weak chain. The motivation repeats a deadline, the mission lists products, or the objectives list projects. Evidence appears only after implementation begins.

Circular reasoning is another warning. The organization adopts Azure because it has a cloud strategy, while its strategy is to adopt Azure. No business condition tests that logic.

Traceability provides the remedy. Start with evidence of a condition that matters. State the lasting reason to change it.

Give the platform a bounded mission. Define outcome-based objectives. Name the capabilities and evidence that connect intent to action.

The chain does not promise that every assumption is correct. It provides a purpose that can govern choice. Motivation explains why change matters, mission states what the platform is for, and objectives make that purpose testable.

Chapter 2: Strategy — Give the Cloud System a Purpose

Chapter 2.3: Map Stakeholders, Incentives, and Constraints

A strategy chain can be logical and still fail in practice. Each link passes through people. They control money, risk, knowledge, labor, or permission.

Those people experience the Azure system from different perspectives. One group sees delayed product delivery. Another sees unmanaged exposure.

A third group sees support work that its budget never recognized. These views are part of the technical system. Their interaction shapes approvals, architecture, automation, and daily behavior.

A stakeholder is a person or group that affects a decision or lives with its result. Job title alone does not prove that link. Practical authority can differ from formal position.

The organization chart is therefore a weak starting map. It shows reporting lines, not who stops a release or approves a contract. It also misses who accepts risk or restores service.

Begin with the objectives from Chapter 2.2. For each objective, find who benefits, supplies work, carries risk, pays, and approves change. This keeps the map tied to purpose.

The cloud strategy team provides a core forum for this work. Microsoft guidance includes business, technology, finance, security, compliance, architecture, and executive views. A small core can seek wider input when needed.

Representation is necessary, but attendance is not alignment. A person can join every workshop without authority to commit a team. An absent person can still hold an effective veto.

Map the decision before assigning the meeting. Give the decision a subject, scope, and time horizon. Give it an escalation route as well.

A decision right is the recognized authority to choose within a set boundary. It differs from responsibility for execution. It also differs from giving advice.

Several teams can contribute to one outcome. Many can be asked or told. Each decision still needs a clear accountable role.

Ask five plain questions for each choice. Who can say yes? Who can say no?

Who must act? Who must be heard? Who needs to know the result?

A right needs a clear scope. It needs a useful response time. It also needs a fallback when the main owner is absent.

Vague rights create hidden queues. A design waits because nobody knows which group can approve it. Each group protects itself by asking for one more review.

Overlapping rights can be just as harmful. Two groups can believe they own network standards or policy exceptions. The clash looks technical even when power is the cause.

Missing rights also shape architecture. If nobody can accept an exception, engineers may encode the strictest rule. The Azure control then replaces an absent decision.

Record who proposes, decides, performs, verifies, and advises. Include the role that grants exceptions. Name a backup role too.

The map must cover routine and urgent paths. A right that works only at a monthly council cannot support an incident. Time is part of decision design.

Decision rights explain authority. Incentives explain likely behavior. An incentive is a gain, pressure, avoided loss, or local signal that favors one act.

Some incentives are formal. Budgets, delivery goals, audit results, support terms, and reviews all guide behavior. Informal incentives can be just as strong.

A team can guard its reputation by avoiding ownership. An engineer can save scarce time by adding a manual gate. A manager can favor visible delivery over unseen care.

Motivation and incentive are different. A motivation states why the organization wants the cloud system to change. An incentive shapes how one group acts inside that change.

The organization can value speed while its approval path rewards caution. It can value cost ownership while shared charges remain someone else's problem. Declared purpose does not erase local pressure.

A dependency is another group's required decision, fact, work, or service. A dependency is not always a conflict. It becomes a constraint when the input is late or weak.

An organizational constraint is a limiting condition in authority, capacity, process, contract, or incentive. It can make a path slow, risky, costly, or closed. It can later take a technical form.

Suppose a network team must review each new subscription link. The team has less review capacity than demand. Subscription delivery then slows.

Engineers may respond by fixing one route in code. That route avoids future review but removes design choice. The architecture now reflects a team constraint.

This test works in reverse. Find a rigid technical rule. Ask which risk, right, skill, wait, or reward helped create it.

Leadership connects strategy to resources and authority. Executives can set priorities, resolve conflict, and accept major tradeoffs. Their concerns include growth, reputation, risk, and investment value.

Executive sponsorship is weak when it is just a name. A sponsor without authority to settle a funding dispute adds little capacity. The map must state which decisions the sponsor owns.

Finance brings budget rules, forecasts, allocation, and investment review. It often values predictable spend and clear ownership. These needs can expose waste that technical teams miss. Finance can also face pressure for near-term savings. That pressure can conflict with resilience, training, or platform care. The map should name who judges the tradeoff and its time span.

Procurement controls contract routes, supplier terms, purchase lead time, and approved deals. Its work can decide when a service becomes usable. Late contact can turn a design choice into delay.

Procurement can be judged by price and contract fit. Engineering can be judged by speed and choice. Neither view is complete on its own.

Security and compliance teams carry exposure and evidence duties. They define or interpret required controls and help set exception rules. Their authority must match their accountability. A security group judged only by incident absence can favor denial. A workload team judged only by release dates can favor broad access. The resulting conflict is easy to predict.

Early security feedback can change that link. Supported patterns and clear exception routes reveal limits before release. A late veto becomes an early design input.

The platform team wants shared services that stay coherent and supportable. It controls parts of identity, connectivity, governance, management, and automation. It also absorbs demand from many workload teams.

Standards can reduce variation and repair. They can also become central control that ignores a valid need. The platform needs a line between a supported path and a forced path.

Local platform incentives deserve review. Closing tickets can matter more than removing request causes. Shipping new code can matter more than caring for an existing service.

Operations carries service continuity, incident response, recovery, and support load. It sees effects that project plans leave out. Its knowledge should enter before a service goes live.

Operations can still resist useful change when stability is judged too narrowly. Avoiding each change can preserve calm now. It can also raise risk over time.

Workload teams are consumers and contributors. They need timely environments, clear controls, and room to meet product needs. They also own workload choices and operating duties.

A workload team facing a fixed launch date will route around a slow platform path. That choice can look careless while being rational for the team. Friction changes which path looks safe.

The answer is not unlimited freedom. It is clear freedom within guardrails and a usable route outside them. The boundary must fit both platform and workload duties.

Partners and managed service providers can hold key skill or knowledge. Contracts define only part of their role. Access, response time, handoff quality, and knowledge transfer also matter. A supplier can gain from continued dependence or manual work. That does not prove bad intent. Strategy should still inspect the commercial pull and exit terms.

Architecture, identity, networking, data, legal, privacy, and support groups can also matter. Include them when they affect an objective or bear a cost. Do not invite every group to every choice.

The stakeholder and incentive map should stay compact. For each group, record its stake, hoped-for result, feared result, rights, responsibilities, dependencies, and incentives. Add evidence for each claim.

Finish each entry with a feedback channel. Service reviews, exception records, incidents, cost talks, and user interviews can return facts to strategy. Each channel needs an owner and cadence.

Do not turn a feedback channel into another gate. Its purpose is to reveal how the system behaves for that group. Chapter 2.4 will decide how measures support that learning.

Apply the map to the running platform objective. Workload teams want governed application landing zones through one supported path. They provide ownership, data, region, and service context.

The platform team owns the service interface and shared base. Security and compliance define control intent and exception terms. Operations defines the support line and recovery duties. Finance defines cost ownership and reporting needs. Procurement clears required supplier and purchase paths. Leadership resolves conflicts that cross these lines.

This allocation does not design the landing zone. It names decisions and dependencies that later design must honor. It also finds missing authority before automation hardens it.

Now trace conflicts between entries. Platform consistency can conflict with workload freedom. Cost cuts can conflict with resilience.

Stable operations can conflict with rapid change. Contract discipline can conflict with fast experiments. Security assurance can conflict with broad self-service.

Do not settle these conflicts with slogans. “Security first” and “move fast” hide the choice. State the case, affected objective, decision owner, and acceptable boundary.

Some conflicts share a cause. Security and workload teams can both suffer from late control feedback. Fixing that delay can help both sides.

Other conflicts require a real tradeoff. Regional resilience can cost more. Leadership must decide whether less risk justifies that cost.

The map should mark dependencies as well as conflicts. Finance needs ownership data from the landing-zone request. The platform needs approved security intent before it can automate guardrails.

Operations needs design context before accepting support. Workload teams need clear platform promises before planning delivery. A missed dependency becomes waiting, rework, or bypass. Test the map with real cases. Walk through a normal request, an exception, an incident, a budget cut, and an absent owner. Watch where authority or information disappears.

Use recent records to challenge polite answers. Approval trails, incident timelines, contract waits, and service messages show actual behavior. Workshop agreement cannot prove the map. Name shadow decision makers. A senior engineer, supplier contact, or assistant can control access without formal accountability. Ignoring that role leaves the practical system unchanged. Also name people who bear costs but lack a formal voice. Service desk staff, application operators, and internal users may carry new work. Their lack of authority does not remove their stake.

Avoid committee ownership. A committee can consult, compare, and recommend. Accountability still needs a person or defined role with authority.

Avoid universal consultation. When each choice needs every group, consultation becomes a delay mechanism. Match participation to impact and skill.

Avoid treating conflict as a personality defect. Repeated tension often follows opposed incentives or unclear rights. Changing people without changing structure repeats the result.

Publish the map where affected teams can challenge it. Use plain descriptions of authority and responsibility. Correct it when real decisions expose a mismatch.

The map is not permanent. Goals, suppliers, rules, team skill, and scale can change. Review entries when those conditions move.

Keep strategy distinct from operating-model design in Chapter 3. This section reveals forces that design must address. It does not select the final team shape.

Keep measure design in Chapter 2.4 as well. Here, incentives are observed as causes of behavior. The next section will test how measures can strengthen or distort them.

A sound strategy chain states what the platform should achieve. A stakeholder map shows who can make that chain real or block it. Together, they turn organizational ambiguity into a design concern.

Azure will encode these choices in roles, scopes, policies, pipelines, and service boundaries. Those choices will favor some views of success. Whose view will the cloud system enforce?

Chapter 2: Strategy — Give the Cloud System a Purpose

Chapter 2.4: Measure Outcomes Without Creating Bad Incentives

An objective becomes useful when evidence can show whether its outcome changed. A number can provide that evidence. The number can also act back on the system.

People notice what leaders count. Teams adapt work to targets, dashboards, and review questions. A poor measure can improve its own report while making the platform worse.

This is local optimization. One part improves its score by shifting cost, risk, delay, or repair elsewhere. The whole system pays for a narrow success.

Suppose the platform team is judged by subscriptions created each month. It can raise the count by splitting needs across more subscriptions. Demand, value, and support quality need not improve.

Suppose it is judged only by average delivery time. The team can favor simple requests and leave hard requests waiting. The average falls while the oldest work becomes older.

Suppose security is judged only by policy compliance. Teams can improve visible compliance through broad exemptions or reduced scope. The score rises while exposure remains.

Measurement must therefore do two jobs. It must show movement toward an objective. It must also reveal harm caused by that movement.

Start with the strategy chain from Chapter 2.2. Each objective needs evidence tied to its outcome, not merely the work performed. The stakeholder map shows who can react to that evidence.

A metric is a defined quantitative observation. A key performance indicator is a selected metric used to judge progress toward an important objective. Not every collected number is a KPI.

Give each metric a small contract. State the question it answers and the objective it supports. Then define its formula, unit, events, scope, exclusions, source, and owner.

Add collection frequency, data confidence, and the action a result can trigger. Record the definition version. A dashboard label cannot carry all this meaning.

“Provisioning time” sounds clear but hides choices. Does the clock begin when a request opens or when all inputs are valid? Does it end at subscription creation or usable handoff?

Pauses matter too. Waiting for workload data can be counted, removed, or reported separately. Each choice tells a different story about the service.

Write event boundaries before collecting the number. Name stopped-clock rules and exclusions. Keep the raw timestamps that support the result.

Scope also needs care. A tenant-wide figure can hide product lines, risk classes, or request types. A small segment can be mistaken for the whole estate.

Segment when a difference can guide action. Do not create so many cuts that each result becomes unique. The measure must remain comparable over time.

Define the denominator as carefully as the count. Ninety successful runs means little without all attempted runs. A high rate can hide discarded or missing records.

Every measure should show data confidence. Missing events, manual entries, changed definitions, and weak ownership reduce trust. Unknown quality should remain visible.

Do not wait for perfect data. Begin with a useful baseline and state its limits. Improve collection when the decision deserves stronger proof.

The metric tree starts with one strategy objective at its root. Its first layer holds outcomes. Those outcomes use five views: delivery speed, platform reliability, risk, cost, and adoption experience.

The second layer contains drivers that can influence those outcomes. The third layer shows system health. The fourth contains counter-metrics that expose unwanted effects.

These are layers, not five separate dashboards. A single outcome can connect to several drivers, health signals, and counter-metrics. Each link should have a plain-language reason.

A branch with no link to the objective is noise. A link with no evidence is an assumption. The tree makes both conditions visible.

An outcome measure shows a result that matters to a stakeholder. It often changes after the system has acted. This makes it a lagging measure.

Delivery lead time is one example. The share of valid requests completed within a service window is another. Successful use without repair adds the consumer's result.

Lagging outcomes confirm what happened. They often arrive too late to guide the work that caused them. The tree therefore needs leading indicators.

A leading indicator is an earlier signal linked to a future outcome. Complete request data can predict smoother delivery. Early control feedback can predict less late rework.

Leading does not mean causal. A rising automated-test pass rate can accompany better delivery without causing it. The link remains an assumption to test.

Chapter 2.5 will manage that assumption as a strategy hypothesis. This section only makes the link explicit. Evidence must be able to challenge it.

Driver measures describe conditions a team can influence. Request completeness, decision wait, automation coverage, and early failure discovery can act as drivers. None proves value alone.

A team can improve a driver and still miss the outcome. More automation can make a bad process fail faster. A longer form can increase completeness while driving users away.

System-health measures show whether the platform can sustain its service. They expose capacity, reliability, recovery, and operational load. They prevent temporary output from looking like durable performance.

Pipeline success rate can show delivery reliability. Repair time after a failed platform release can show recovery ability. Queue age and manual intervention can expose hidden load.

Health also includes evidence quality. Missing request events or stale ownership data weaken every related measure. A dashboard needs a view of its own blind spots.

A counter-metric watches for damage created by a goal or driver. It sits beside the primary measure. Its job is to make a tradeoff visible.

Pair lead time with rework, post-handoff defects, exceptions, and support demand. Pair automation coverage with failure clarity and manual recovery. Pair compliance with exception age and bypass activity.

Pair cost reduction with reliability, security, team effort, and deferred work. Pair platform adoption with user success and unsupported alternatives. Pair ticket closure with repeat contacts.

A counter-metric does not prevent every game. It makes one easy distortion harder to hide. Several measures can still fail together when their boundaries are weak.

Apply the tree to the running platform objective. The objective is timely delivery of governed application landing zones through the supported request path. The outcome layer begins with five views.

The delivery view tracks time from a valid request to a usable landing zone. It also tracks the share completed inside the service window. A high percentile can expose waits hidden by an average.

The reliability view asks whether the platform service works when needed. It can observe request-path availability, successful handoff, and first use without platform repair. It does not claim workload availability.

The risk view asks whether guardrails produce the intended protection. Critical control coverage, exception state, remediation age, and evidence freshness can help. Scope and materiality remain visible.

The cost view includes more than Azure spend. Platform labor, shared services, support, and rework can all matter. Cost must be compared with supported value and risk.

The adoption-experience view asks whether teams choose and succeed with the supported path. Completion, repeat use, abandonment, support demand, and direct feedback can help. Mandated use is not proof of value.

Microsoft platform engineering guidance emphasizes speed, quality, ease of use, and whether internal customers thrive. These ideas keep adoption from becoming a raw usage count. Qualitative feedback remains part of the evidence.

Now add drivers below those outcomes. Request completeness, decision wait, automated steps, and early policy feedback can influence several views. Give each driver a named outcome link. Then add health signals across the service. Pipeline availability, failed runs, change failure, repair time, queue age, and manual effort show service condition. Capacity belongs beside demand.

Finally, pair the branches with counter-metrics. Watch post-handoff incidents, control exceptions, ownership gaps, cost allocation gaps, abandoned requests, and shadow paths. User comments can reveal harm that events miss.

One measure can serve more than one view. A failed handoff affects speed, reliability, cost, and experience. Avoid copying the same number into several apparent facts.

Do not combine the whole tree into one weighted score. Weighting hides which tradeoff produced the total. A balanced view should preserve tension between results.

Reliability needs an explicit service boundary. Platform pipeline health differs from workload availability. The capstone platform cannot claim reliability for an application it does not operate.

Measure the service the platform owns. That includes request intake, shared controls, platform configuration, and handoff. Workload operations remain outside the claim.

Compare distributions, not just averages. A median can describe the common case while a high percentile exposes long waits. The oldest open item can reveal starvation.

Trends also matter. One good month can be noise or shifted work. Show enough history to see direction, variation, and step changes.

Targets need a baseline and a reason. A target chosen for appearance creates pressure without meaning. A target tied to an objective can guide a tradeoff.

Use ranges or thresholds when exact precision adds no value. A platform can be healthy within a band. Chasing one point can create needless work.

Name the action attached to each threshold. A red dashboard with no response path is decoration. The measure owner must know who investigates and who decides.

Ownership includes definition care. When a process changes, the metric can change even if behavior does not. Record definition versions and preserve comparison notes.

Do not change a formula to protect a target. If a change is valid, show the break in the series.

Rebuild the baseline when comparison is no longer sound.

Review measures with stakeholders who supply and experience the data. Finance can test cost scope. Security can test risk meaning.

Workload teams can test whether “ready” matches actual use. Operations can test support and recovery measures. The platform team can test whether signals are actionable.

No group should own every definition and every judgment about its performance. Independent challenge reduces self-protection. It does not remove the need for context.

Beware measure overload. A dashboard with hundreds of signals can hide the few that guide strategy. Each metric should justify the decision it supports.

Retire measures that no longer change a choice. Preserve history when it still supports learning.

Collection cost is part of the system.

Manual data can be useful during discovery. It becomes risky when a strategic review depends on it without controls. Note effort, error risk, and missingness.

Automation does not make data neutral. A query encodes scope, filters, joins, and time. Review those choices as carefully as a policy definition.

Do not reward a team for a measure it cannot influence. Do not hold it accountable for scope it does not own. Shared outcomes need shared review and clear decision rights.

Do not tie personal ratings to a new metric too early. Teams learn how the number behaves before leaders understand it. Early punishment suppresses honest data.

Use measures first for learning. Ask what changed, what caused surprise, and where data is weak. Action should follow interpretation rather than color alone.

The metric tree does not prove the strategy correct. It observes outcomes and side effects.

Chapter 2.5 will use that evidence to test strategic assumptions.

A single metric makes the platform easy to rank and easy to distort. A balanced tree makes the story harder to compress. That difficulty reflects the value the Azure system carries.

Good measurement does not remove judgment. It disciplines judgment with scope, history, and visible tradeoffs. The best measure shows both improvement and the cost of achieving it.

Chapter 2: Strategy — Give the Cloud System a Purpose

Chapter 2.5: Run Strategy as a Testable Hypothesis

Strategy describes a desired future, but it cannot guarantee that future. It contains beliefs about causes, choices, and results. Treating those beliefs as facts makes correction look like failure. Cloud conditions will change. Business priorities, regulation, demand, team capacity, cost, and Azure services can move. A strategy must learn without losing its purpose.

The Microsoft Cloud Adoption Framework treats strategy as recurring work rather than a one-pass document. Conditions and evidence should prompt reassessment. Maturity includes changing direction when the current direction no longer fits.

A strategy hypothesis is a testable claim about how a chosen change will influence an objective. It links an intervention to an expected outcome. It also states why that link seems credible.

An objective and a hypothesis have different roles. The objective says which result should improve. The hypothesis says why a proposed capability or action should improve it.

Consider the platform objective from this chapter. Workload teams should receive governed application landing zones through a supported path and service window. That desired result is the objective.

The hypothesis is more specific. If the platform provides one request path with complete inputs and early control feedback, delivery time and rework should fall. Variation and late discovery are the suspected causes.

A useful hypothesis has three parts. The if clause names the intervention. The then clause names the expected change.

The because clause states the causal reason. That part matters because one intervention can rest on several beliefs. Each belief can require different evidence.

A hypothesis also needs a boundary. State the users, request types, platform service, and period to which it applies. A claim without a boundary can survive any result.

Keep the claim small enough to test. “Automation will transform cloud adoption” is too broad. It mixes many interventions, users, and outcomes.

“Automated validation of required fields will reduce clarification waits for standard requests” is narrower. Request events can test the expected change. Nonstandard requests stay outside the first boundary.

The strategy hypothesis register stores these claims. Its first part identifies the entry, owner, status, and linked objective. It preserves the route back to motivation and mission.

The second part stores the if, then, and because statement. It adds scope, baseline, current evidence, and confidence. It also states the cost of being wrong.

The third part defines the evidence plan. Link the relevant outcome, driver, health, and counter-metrics from Chapter 2.4. State what would support or challenge the claim.

The final part controls review. Name affected stakeholders, decision authority, review date, and review triggers. Record the next action after each review.

Useful status values describe learning, not victory. A hypothesis can be proposed, active, supported, challenged, or retired. A supported claim has gained confidence only within its tested boundary.

Evidence can shift confidence without forcing a binary verdict. One result may support part of the claim and challenge another part. The register should preserve that nuance.

Confidence should reflect current evidence. High confidence can come from repeated direct observation. Low confidence can be appropriate when the organization must still act.

Impact and confidence belong together. A low-confidence claim with little consequence needs little attention. One that drives an expensive commitment needs an early test.

Assumptions sit inside every hypothesis. An assumption is a condition treated as true for the current decision despite incomplete proof. Naming it makes uncertainty manageable.

Separate assumptions from facts. A request log can show that approval waits occurred. It cannot prove that removing one approval will improve both control and speed.

Separate unknowns as well. An unknown has no supported answer yet. A discovery task is safer than a convenient guess.

For each important assumption, record the claim, evidence, confidence, and consequence if wrong. Add the next fact needed and an owner. Link it to the parent hypothesis.

Do not track every casual guess. Track assumptions that can change priority, architecture, cost, risk, or sequence. The purpose is better choice, not document growth.

Evidence can come from current behavior. Request timestamps, queue age, incidents, exceptions, costs, interviews, and support records can test a claim. Use the metric tree without rebuilding it.

Evidence can also come from a bounded trial. A synthetic landing-zone request can test required inputs, early policy feedback, and handoff behavior. The capstone does not need an application workload.

A thin slice can reveal integration and ownership gaps before a full service exists. A manual simulation can test logic before automation. A prototype can test an interface without becoming production.

Define the test before seeing the result. State what will change, what will remain fixed, and which evidence will be observed. Name the result that would weaken the claim.

Avoid success criteria written after the result. That practice turns learning into justification. It also moves the goal without admitting the move.

One trial rarely settles a broad claim. Exceptional support can make a pilot look faster than normal service. A narrow group can hide adoption or skill problems.

Record those limits. Ask whether the trial used realistic demand, control, staffing, and support. Scale assumptions need their own evidence.

Negative evidence has value. A challenged hypothesis can prevent months of investment. It can also reveal a better causal explanation.

Missing evidence has value too. It shows that the system cannot yet judge the claim. That gap can justify better observation before more change.

Confirmation bias is a constant risk. Teams can favor data that supports work they already want. Counter-metrics and independent review make selective evidence harder to hide.

Do not treat activity as evidence. A completed pipeline or workshop shows that work happened. It does not show that the intended outcome improved.

Do not change several causes at once when a smaller test is practical. A large bundle can produce a result without showing which change mattered. Later planning then rests on weak learning.

Some hypotheses lead to significant architecture choices. Those choices need a lightweight architecture decision record, often called an ADR. The record preserves why the current path was chosen.

Azure Well-Architected guidance applies ADRs to consequential workload architecture choices. The same concise pattern can preserve significant platform decisions. The scope changes, but the need for context remains.

The hypothesis register and ADR are linked but not interchangeable. A hypothesis predicts behavior. An ADR records a decision made under current evidence and constraints.

Create an ADR for choices that affect system structure, important quality attributes, or costly reversal. Do not record every small setting. A crowded log hides the choices that matter.

Begin the ADR with the problem and context. Link the relevant objective, hypothesis, constraints, and stakeholders. State the decision that must be made.

Record the options considered, including the option to defer. State the selected option and rationale. Name important tradeoffs and consequences.

Add status, owner, date, and confidence. Include the evidence used and the signals that should cause reconsideration. Keep the record brief enough for a later review.

For example, the platform can choose one supported request interface before selecting its final automation tool. The ADR explains why one front door matters and which alternatives were considered. It also records what remains deferred.

An accepted ADR should not be rewritten when direction changes. Create a new record that supersedes it and link both records. History shows how evidence changed the decision.

This append-only approach prevents silent revision. It also stops old debates from returning without new facts. A superseded choice remains part of the system's memory.

The hypothesis register manages uncertainty about expected behavior. The ADR manages memory about consequential choice. Review links let evidence move between them.

The third control is a scheduled review. A review cadence is the planned rhythm for evaluating strategy evidence and choices. Different signals arrive at different speeds.

Operational signals can be reviewed often. Business outcomes may require a longer window. Judging a lagging outcome too early can stop a useful change.

Waiting too long creates the opposite risk. A weak assumption can harden into process and architecture. Review frequency should match impact, reversibility, and feedback delay.

Microsoft guidance recommends regular review of objectives and KPIs and presents quarterly review as a useful preference. That rhythm can anchor strategy without becoming a universal rule. The organization should choose a fitting cadence.

The fourth control is trigger-based review. An invalidated assumption, material cost change, control failure, demand threshold, major incident, or service retirement can trigger it. High-risk evidence should not wait for the calendar.

Record triggers when the claim or decision is created. Assign someone to watch each trigger. A trigger without observation cannot protect the strategy.

Each review should answer a small set of questions. What new evidence arrived? Which assumptions gained or lost support?

Which decisions still fit their context? Which outcome or counter-metric changed? What must continue, change, pause, or stop?

Bring the right decision authority into the review. A meeting that can observe but not decide creates another queue. Chapter 2.3 already identified those rights.

Review the evidence before the narrative. Teams naturally explain surprising results. Starting with definitions, scope, and data quality keeps the explanation grounded.

Then examine the causal link. A better outcome can occur for reasons outside the intervention. A worse outcome can occur despite a sound change when demand shifts.

Record the review decision. Continue the hypothesis, adjust its boundary, run another test, change the intervention, or retire the claim. Do not leave the result as discussion.

Changing strategy is not the same as changing the mission each month. Motivations and mission should be more stable than hypotheses and implementation choices. Different layers change at different rates.

Objectives can change when business priorities or evidence move. Hypotheses can change when causal beliefs weaken. Capabilities and decisions can change as the solution space becomes clearer.

Version those changes. Preserve the old statement, reason, evidence, and decision. Traceability prevents learning from looking like inconsistency.

Avoid using iteration to escape accountability. A target should not move merely because it was missed. The review must state what changed in context or evidence.

Avoid freezing strategy to protect authority. A challenged assumption is not an attack on its owner. It is information about the system.

The strategy team owns the integrity of this learning process. Metric owners provide evidence. Decision owners act within their boundaries.

The register owner keeps links and status current. The ADR owner records significant choices. Shared access lets affected teams challenge the record.

Keep both artifacts light. A hypothesis entry should help select a test or investment. An ADR should help a future reader understand a consequential choice.

If neither artifact changes a decision, it has become ceremony. Retire stale entries and supersede outdated choices. Preserve history without preserving clutter.

Chapter 2 began with an honest account of current behavior. It then defined motivations, mission, objectives, stakeholders, incentives, and balanced evidence. The hypothesis register now connects those elements to learning.

Strategy is ready for planning when purpose is clear, uncertainty is visible, and decisions can be revised. It does not need certainty. It needs enough evidence for the next responsible step.

Chapter 3 will turn that direction into a cloud adoption plan. The estate becomes a portfolio, while current hypotheses inform priority, sequence, skills, cost, and ownership. Strategy has set the purpose; Plan must now organize the work.

Chapter 3: Plan — Design the Operating Model

Chapter 3.1: See the Estate as a Portfolio

Plan — Design the Operating Model begins with a workload portfolio. The portfolio shows which systems, owners, dependencies, and uncertainties the organization must support. It turns a scattered Azure estate into an explicit field of planning.

An asset inventory records what discovery tools can see. It may list servers, databases, subscriptions, and applications. It rarely shows what must change together.

A portfolio organizes the estate around workloads and relationships. A workload is a set of technical and organizational elements that supports one or more business processes. Its boundary follows the outcome it provides.

That boundary can cross Azure scopes and ownership records. One workload may span resource groups, subscriptions, repositories, and external services. One resource group can also serve several workloads.

An Azure resource group is therefore not a workload by default. Scope remains important for access, policy, deployment, and cost. It does not prove business purpose or end-to-end ownership.

This distinction changes the planning conversation. A resource list invites isolated estimates and product choices. A portfolio reveals shared dependencies, competing needs, missing owners, and unresolved facts.

Begin each record with a stable workload identifier and a plain-language name. Add the business process, users, and outcome it supports. Those fields explain why the workload belongs in the plan.

Link its purpose to the strategy chain from Chapter 2. Some workloads advance a current objective directly. Others meet a legal duty or enable another service.

Do not assign a migration strategy during initial discovery. Retire, retain, rehost, and modernization choices require evidence. Premature labels can turn an assumption into a commitment.

Business criticality is part of that evidence. It describes the impact of unacceptable workload behavior. A high, medium, or low label cannot carry the full meaning.

Write the impact statement before selecting a class. State who is affected, how harm appears, and how quickly it develops. Include customer, revenue, safety, regulatory, and service effects where relevant.

Context can change the result. Payroll processing has sharp time boundaries. An ordering service may become critical during a seasonal sales event.

Support the classification with observable evidence. Incidents, service commitments, demand data, contracts, and continuity exercises can help. When support is weak, keep the classification provisional.

Ownership requires the same discipline. Record a business owner who can judge value and consequence. Record a technical owner who can explain and authorize change.

Add a service contact for operational discovery. These fields do not define the future team design. Chapter 3.3 will assign detailed responsibilities, backup ownership, and skills.

A name in a configuration database is only a claim. The named person must accept ownership or identify the right path. A stale record is a discovery finding, not accountability.

Technical constraints narrow the available choices. Examples include unsupported runtimes, license terms, hardware dependencies, latency limits, fixed IP allowlists, data residency, and restricted maintenance windows. Record the constraint without solving it yet.

Some constraints are inherited from external providers. A vendor might support only one operating system. A partner might permit interface changes during a fixed contract window.

Every constraint needs provenance, an owner, and a review date. Treat it as current evidence rather than permanent law. A familiar rule can still be obsolete.

The estate rarely has one reliable source at the start. A configuration database supplies the declared view. Billing, repositories, telemetry, incidents, network observations, and interviews reveal other views.

Use these sources together. Automated discovery can find a connection without explaining its business meaning. An interview can explain intent while missing actual traffic.

Azure Resource Graph can query Azure Resource Manager resource data across subscriptions the caller can read. It supports broad filtering, grouping, and inventory checks at scale. Its results reflect both available resource properties and the caller's access.

That boundary matters. A partial result can look like a complete estate when subscription access is incomplete. Resource Graph also cannot infer business purpose or an undocumented manual handoff.

Azure Migrate can support discovery of servers, applications, and interdependencies in supported source estates. Manual discovery remains necessary where tools lack reach or context. Tool output contributes evidence; it does not become the portfolio by itself.

Attach provenance to every material fact. Record its source, collection date, and evidence status. Provenance says where the claim came from, while status says how strongly it is supported.

Use four evidence states: confirmed, reported, inferred, and unknown. Confirmed means a repeatable observation or accountable owner has validated the claim. Reported means a source stated it but validation is pending.

Inferred means related evidence suggests the claim. Unknown means no supported answer exists yet. Both states should lead to a discovery task when a decision depends on them.

Avoid false precision. A percentage can make subjective confidence look measured. A clear state, source, and next validation action are more useful.

The discovery-data schema defines the minimum portfolio record. It requires identity, purpose, ownership status, criticality, lifecycle, provenance, and dependencies. Optional detail can grow as later decisions require it.

The following JSON Schema implements that contract. A missing owner is represented explicitly instead of hidden in a blank string. Schema validation checks record shape, not the truth of its claims.

```
JSON
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://example.org/schemas/workload-portfolio-record.json",
  "title": "Workload portfolio record",
  "type": "object",
  "required": [
    "workloadId",
    "name",
    "businessPurpose",
    "businessOwner",
    "technicalOwner",
    "ownershipStatus",
    "criticality",
    "lifecycle",
    "provenance",
    "dependencies"
  ],
  "properties": {
    "workloadId": { "type": "string", "pattern": "^wl-[a-z0-9-]+$" },
    "name": { "type": "string", "minLength": 1 },
    "businessPurpose": { "type": "string", "minLength": 1 },
    "businessOwner": { "type": ["string", "null"], "minLength": 1 },
    "technicalOwner": { "type": ["string", "null"], "minLength": 1 },
    "ownershipStatus": { "enum": ["accepted", "unconfirmed", "missing"] },
    "criticality": {
      "type": "object",
      "required": ["class", "impactStatement"],
      "properties": {
        "class": { "enum": ["mission-critical", "business-critical", "business-operational", "noncritical", "provisional"] },
        "impactStatement": { "type": "string", "minLength": 1 }
      },
      "additionalProperties": false
    },
    "lifecycle": { "enum": ["proposed", "active", "declining", "retirement-candidate", "retired"] },
  }
}
```



```

    "provenance": {
      "type": "object",
      "required": ["source", "collectedOn", "evidenceStatus"],
      "properties": {
        "source": { "type": "string", "minLength": 1 },
        "collectedOn": { "type": "string", "format": "date" },
        "evidenceStatus": { "enum": ["confirmed", "reported", "inferred", "unknown"] }
      },
      "additionalProperties": false
    },
    "dependencies": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["providerId", "kind", "failureEffect", "evidenceStatus"],
        "properties": {
          "providerId": { "type": "string", "minLength": 1 },
          "kind": { "enum": ["identity", "network", "dns", "data", "api", "messaging",
"operations", "deployment", "business", "vendor"] },
          "failureEffect": { "type": "string", "minLength": 1 },
          "evidenceStatus": { "enum": ["confirmed", "reported", "inferred", "unknown"] }
        },
        "additionalProperties": false
      }
    },
    "additionalProperties": false
  }
}

```

The workload identifier should outlive an implementation. Resources can move or be replaced while the business service continues. Stable identity preserves decisions and evidence across that change.

The portfolio dependency map joins these records. Represent each workload or shared capability as a node. Represent each dependency as a directed edge from consumer to provider. Direction must be explicit. If an order service calls a stock API, the order service consumes it. The stock API provides the required result.

Consider a bounded portfolio example. The order service also consumes shared identity, DNS, and monitoring capabilities. An external carrier API supplies shipping data.

Record every relationship separately. Each edge states its kind, interface, failure effect, owner, evidence status, and validation date. Latency, data, or timing conditions can be added when they affect planning.

Suppose an old diagram is the only source for the carrier relationship. That edge remains reported rather than confirmed. A discovery task can compare network evidence with the current contract.

Suppose the order service has no accepted business owner. Its known relationships still belong on the map. The ownership gap becomes visible work with a resolver and due date.

Runtime dependencies include APIs, data stores, queues, identity, DNS, connectivity, certificates, and secrets. Delivery dependencies include repositories, package feeds, build agents, deployment identities, and approval services. Both sets can block safe change.

Operational dependencies count as well. Monitoring, backup, incident routing, and vendor support affect service behavior. A workload is not independent merely because it can start.

Business dependencies may exist without network traffic. A report can depend on a finance close. A service can depend on a signed partner notice.

Shared platform capabilities must appear as providers. Identity, connectivity, DNS, management, governance, and deployment paths may serve much of the portfolio. Hiding them conceals demand and concentrated impact.

Many incoming edges make a capability important to examine. They do not prove that sharing is unsafe. Resilient design and tested recovery may make a shared service the sound choice.

Sparse edges need caution too. They can describe a simple workload or weak discovery. Evidence status prevents absence of data from becoming evidence of independence.

Review the map for clusters, bridges, and unknown links. A cluster contains items with many mutual dependencies. A bridge connects groups that would otherwise appear separate.

These patterns frame later planning questions. They do not create migration waves in this section. Chapter 3.4 will own sequencing, capacity, and work-in-progress decisions.

The same boundary applies to organization design. Missing ownership reveals a gap but does not choose the team structure. Chapters 3.2 and 3.3 will place authority and responsibility.

Turn every material gap into a discovery action. Name the claim to test, the resolver, the due date, and the blocked decision. An unknown then becomes manageable work.

Discovery depth should follow consequence. Validate a mission-critical identity dependency before polishing a low-impact description. Equal detail across all records is rarely the best use of capacity.

Use progressive elaboration. The first pass needs purpose, owners, criticality, key constraints, major dependencies, and uncertainty. Deeper assessment can wait until a hard-to-reverse choice approaches.

Review records with business and technical perspectives together. Business owners can challenge purpose and impact. Engineers can challenge boundaries, constraints, and observed connections.

Preserve disagreement rather than averaging it into false consensus. Store each claim with its provenance. Assign the smallest validation step the next decision requires.

The portfolio must remain time-aware. Owners move, interfaces retire, critical periods shift, and new evidence appears. Each record needs a last-validated date and a review trigger.

The platform-only capstone can test the method with synthetic workload records. Those records can consume planned identity, connectivity, management, and governance capabilities. No application workload needs to be deployed.

That exercise validates the schema and relationship language. It does not replace discovery in a real estate. A real adoption plan still needs local evidence and accepted owners.

A resource list says what a tool can see. A portfolio says which workloads matter, what they depend on, who can decide, and what remains unknown. The operating model must stand on that evidence base.

Chapter 3: Plan — Design the Operating Model

Chapter 3.2: Choose a Cloud Operating Model

Cloud authority needs a deliberate home. A cloud operating model defines how platform and workload responsibilities are distributed. The choice must fit skill, risk, and scale.

A cloud operating model distributes work and authority. It covers responsibilities, decision rights, processes, and capabilities. It states who provides shared services and who operates workloads. It also defines the boundary between them.

The model is not an organization chart. Team names can change while decision paths remain stable. A useful model explains how work and authority move when demand arrives.

It is not an Azure architecture either. The choice influences the landing zone. Chapter 4 owns that design. This section defines the team boundary that architecture must support.

Three common models form the comparison: centralized, shared management, and decentralized. None is a universal maturity level. Each can fit a different estate, risk profile, and skill base.

Evidence must drive the choice. Use the Chapter 3.1 portfolio. Its workload count, criticality, variation, shared dependencies, ownership, and uncertainty all matter. Add the outcomes and constraints from Strategy.

Demand must also be compared with effective capacity. Headcount alone says little about service flow. A small expert group can still become a severe queue.

Authority and capability have to remain connected. A team cannot own an outcome if another group blocks each safe action. Authority should not move without skill. The team must be able to act safely.

In a centralized model, one team manages the estate. Its scope includes governance, security, operations, and cloud change. Workload teams request service from that group. Most decision rights remain at the center.

This arrangement can fit a small or uniform portfolio. It can also pool scarce expertise under strict control needs. One team can apply consistent standards and maintain a clear support path. Centralization can stabilize early adoption. The central team sees much of the estate and can stop incompatible patterns. Workload teams do not need every cloud skill at once.

The main scaling limit is queue growth. New workloads, access changes, exceptions, incidents, and platform requests compete for the same capacity. Business and technical context must travel into that queue.

Handoffs multiply as demand becomes more varied. The central team spends time recovering local facts. Workload teams wait for people who may not understand the application.

Long waits can push work outside the approved path. Teams may copy old templates or create unmanaged subscriptions. Tight control can move risk instead of reducing it.

The transition evidence is observable. Rising queue age, repeated escalations, growing exception work, and varied requests indicate structural strain. Capable workload teams seeking routine authority add another signal.

One busy period does not require a new model. Automation, a smaller catalog, or clearer service levels may restore flow. Transition is warranted when the boundary itself causes persistent delay.

In a decentralized model, each team manages its platform needs and workloads independently. It carries governance, security, and operational duties within its scope. Decisions stay close to the workload.

This arrangement can fit highly skilled teams with few shared needs. It can also suit bounded innovation work. Fast feedback and local context are its main strengths.

Many central handoffs disappear. A team can design, change, and operate its own path. Production evidence reaches the people who own the product.

That autonomy requires broad capability. Each team needs enough platform, security, governance, operational, and workload skill. Scarce expertise can become thinly spread.

Unplanned variation creates another limit. Identity, networking, policy, logging, and incident evidence may diverge. Shared dependencies can expose conflicts between local designs.

Duplication consumes capacity as well. Several groups may build similar subscription, monitoring, or connectivity solutions. Local speed can hide a large estate-wide support cost.

Decentralization does not remove enterprise consequences. Minimum controls, evidence, and shared interfaces still cross team boundaries. Independence therefore needs a declared scope.

Repeated control gaps and cross-team incidents are transition signals. Duplicated platform work suggests a common service. A shortage of local specialists shows that authority has moved beyond capability.

Shared management divides the boundary. A platform team provides reusable services. It also maintains baseline guardrails. Workload teams operate their environments and applications within those limits.

A platform team maintains capabilities for many internal consumers. A workload team remains accountable for one workload. That scope covers design, delivery, and operation. Neither side owns the whole cloud system alone.

This model combines a common foundation with local context. The platform team takes work that repeats across the portfolio. Workload teams keep decisions that depend on the application and its users.

The platform boundary is the critical choice. Include capabilities that gain value from one implementation, one interface, or one risk decision. Exclude workload-specific design that the platform cannot judge well.

Typical shared capabilities include tenant foundations, subscription placement, identity integration, connectivity, and DNS. Management, governance, and deployment interfaces may also be shared. Chapter 4 will design their Azure form. The operating model assigns their service boundary.

Workload teams retain application code, data, and product decisions. They also own workload configuration and reliability. They consume platform capabilities through supported interfaces. They challenge guardrails that block valid needs.

Shared management often fits varied portfolios with recurring needs. It can serve larger, hybrid, or multicloud estates. It scales when platform capabilities behave as products instead of ticket queues.

The leverage can be substantial. One improvement to subscription vending or policy feedback can help many teams. Common services reduce repeated platform work.

Unclear boundaries create the central failure mode. Each side can assume the other owns a task. Gaps then appear in security, support, or operations.

Shared management can also conceal centralization. If every routine request needs manual platform approval, the queue still controls delivery. A catalog without practical self-service does not delegate authority.

The opposite failure provides too little. Guardrails without usable support shift integration work to each workload team. Local fixes spread while the platform reports standardization.

Platform products create real work of their own. They need roadmaps, support, versioning, and user feedback. Multiple platform teams also need stable interfaces between capabilities.

Transition signals adjust this boundary over time. Manual queue growth suggests automation or delegation. Repeated local implementations suggest a new shared capability.

Weak platform adoption may reveal poor usability or a false need. High exception volume can expose an overbroad guardrail. Repeated workload incidents may show that local capability is not ready.

One model does not need to cover every portfolio segment. A mixed model may centralize regulated systems, share management for the main estate, and decentralize a bounded lab. Each difference must be intentional.

A mixed model is not accidental inconsistency. Every segment needs entry criteria, minimum controls, and decision ownership. Movement between segments needs a review path.

This term also differs from hybrid cloud. Hybrid cloud describes technology across cloud and noncloud environments. A mixed operating model describes different distributions of authority. The operating-model decision matrix preserves the comparison. Give each candidate the same context and boundary fields. Add strengths, failure modes, and transition signals. Record evidence, decision status, owner, and review date.

Decision status keeps the outcome precise. A candidate can be selected, conditionally selected, rejected for current conditions, or deferred pending evidence. Rejection is not permanent when the context can change.

Populate context from the portfolio. State which workloads fit, which dependencies are shared, and where regulation changes the choice. Preserve important unknowns instead of filling the record with confidence.

Compare the same capability areas for every candidate. Start with governance, security, identity, connectivity, and subscription delivery. Add management, deployment, cost, and workload operations. A broad label may hide a mixed boundary.

For each area, state who sets the standard. Then identify who builds, operates, funds, supports, and changes the capability. Chapter 3.3 will convert the selected boundary into detailed assignments.

Decision latency provides one test. How far must routine and urgent choices travel? How much context must cross a team boundary before the decision becomes safe?

Control quality provides another. Can the model produce consistent evidence without routing every action through one person? Can it detect delegated authority used outside the boundary?

Scale must be tested with real demand. Estimate request volume, exception rate, support load, and workload variation. Compare those needs with effective capacity.

Organizational resilience matters too. One specialist creates a fragile boundary. A model that needs unavailable skills in every team is not viable.

Partners belong in the matrix. A partner can supply capability. The organization still retains accountability. Contract scope, evidence access, escalation rights, and exit conditions affect the choice.

Choose the narrowest common boundary that protects shared outcomes. Centralize a decision when variation creates broad harm or common scale provides leverage. Delegate it when local context and feedback speed dominate.

A common standard does not require central execution. A platform team can define a tested interface that teams use without approval. Policy and automation can preserve a boundary without a manual gate.

For the capstone, shared management is conditionally selected. The platform team owns the platform landing zone. It also owns common identity, connectivity, management, governance, and automation capabilities. Synthetic workload consumers retain their side of the contract.

No application workload will be deployed. Synthetic inputs will test how a workload requests and consumes platform service. This keeps the capstone within its approved boundary.

The selection depends on explicit assumptions. The estate has shared platform subscriptions and varied workload needs. It also needs repeatable application landing zones. Workload teams can act within clear guardrails.

The other candidates remain visible. Centralization is rejected for routine workload decisions. The intended portfolio requires delegated flow. Decentralization is rejected because shared dependencies create common risk and leverage.

Those outcomes apply only to current capstone conditions. Some operations may stay central while workload skills grow. More platform teams may form when one group cannot support the capability range.

Record these transition signals beside the selection. Name the decision owner and review date. The model should change before its failure mode becomes ordinary work.

Selection is sufficient when later design can follow the boundary. Teams must know which decisions remain common and which move toward workloads. Detailed role and skill mapping comes next.

Control and autonomy are not opposing goals. The reflective question is where each decision can produce a safe result without avoidable delay. The portfolio turns that question into evidence.

Chapter 3: Plan — Design the Operating Model

Chapter 3.3: Design Responsibilities and Skills

Responsibility needs more than a boundary. Each recurring cloud outcome needs ownership, skill, authority, and a supported interface. Otherwise, a sensible operating model remains only an organizational claim.

Begin with responsibility, not tooling. A responsibility is an ongoing obligation to produce or protect an outcome. It is broader than a task or permission.

“Own Azure Policy” is too vague. The phrase can include risk intent, policy code, assignments, exemptions, evidence, and remediation. Several teams may take part.

Tool-first planning hides those differences. Portal access does not create accountability. Terraform skill does not grant authority to define governance intent.

Start from the shared-management boundary chosen in Chapter 3.2. The platform side provides common services and guardrails. The workload side retains application and product decisions.

Decompose that boundary into service outcomes. Give each outcome a scope, primary owner, practiced backup, decision right, and evidence. Add the interface used by other teams.

The primary owner is accountable for the outcome. That owner need not perform every task. Accountability means ensuring that work, proof, and review occur.

Prefer a durable team or role for primary ownership. A personal name can show who fills the role now. It should not be the only structure holding the responsibility.

A backup owner needs more than a listing. The backup requires access, current context, and practiced skill. Otherwise one person's absence can stop the service.

Test backup coverage through action. Can the backup approve normal change, interpret an alert, and lead recovery? A name without practice is only reported coverage.

RACI terms can clarify participation. One party remains accountable while responsible parties perform the work. Other groups can be consulted or informed.

The letters do not replace an outcome description. Several accountable teams create doubt. Consulting everyone can also increase decision latency.

Build the responsibility map around capabilities. Begin with governance, security, platform services, operations, and workload delivery. Add business and partner roles where they hold required authority.

Governance turns business risk into control intent. It sets policy purpose, risk tolerance, exception rules, and evidence requirements. It is not merely the group that edits Azure Policy.

Security sets protection needs across the lifecycle. It owns security requirements, threat judgment, incident authority, and security evidence. Platform and workload teams can share implementation.

The platform function provides common cloud capabilities. It owns supported interfaces, automation, shared-service reliability, versioning, and platform support. It does not own every Azure resource.

Operations maintains service health through monitoring, administration, incident response, recovery, and controlled change. Platform operations and workload operations require distinct scopes. One broad operations label hides that boundary.

Workload teams own application code, data, product behavior, workload reliability, and workload cost. They act within shared controls. They also supply business context that common teams cannot infer.

These functions overlap by design. Security can define a control while the platform encodes a reusable guardrail. A workload can add a local control and operations can observe its result.

Overlap becomes unsafe when the interface is unclear. The map must state who decides, who acts, and who verifies. It must also show where evidence returns.

An interface links two responsibilities. Record its trigger, required inputs, expected output, service expectation, escalation path, and evidence. State the decision right for exceptions.

Subscription vending provides a useful example. A workload team supplies purpose, ownership, criticality, data needs, and connectivity needs. The platform returns a governed application landing zone.

Governance supplies baseline control intent. Security supplies protection requirements. Operations supplies management and support needs for the shared foundation.

The interface must explain failure as well. Missing inputs return with a clear reason. A nonstandard need follows a review path with named authority.

Without this contract, teams rely on memory and direct messages. The platform becomes a translation service for every request. Workload teams cannot see why work waits.

Attach service measures to the interface. Request age, clarification cycles, failed validations, exception volume, and support demand reveal friction. Counter-metrics can expose work shifted elsewhere.

Do not promise more than current capacity can deliver. Record the service gap instead. A false target only hides uncertainty in the queue.

Map required skills after outcomes and interfaces are clear. A skill is a demonstrated ability to perform work at the needed level. Course completion alone does not prove that ability.

Use practical skill levels. Awareness means someone can recognize the topic and route a question. Working skill supports routine activity with guidance.

Independent skill supports normal work and common failure modes. Expert skill supports novel or high-risk cases and teaching. Not every responsibility requires an expert.

Map skills to responsibility rather than title. Platform automation may need Terraform, source control, review, testing, and Azure delivery. Platform ownership also needs product, support, and stakeholder skills.

Governance needs risk analysis, policy design, exception judgment, evidence review, and clear communication. Security needs identity, protection, detection, and incident skills. Operations needs monitoring, diagnosis, change, and recovery.

Workload teams need enough Azure, security, reliability, cost, and delivery skill to use delegated authority safely. The platform should remove needless complexity. It cannot remove workload accountability.

Cognitive load is the mental effort required to understand and perform work. Some load belongs to the business problem. Some is created by cloud and platform interfaces.

The platform provides value when it removes avoidable load. Useful defaults, reusable patterns, documentation, and fast feedback reduce repeated research. A hidden abstraction that fails without explanation adds load.

Observe cognitive load through behavior. Repeated support questions, copied workarounds, long onboarding, and dependence on one expert are signals. Interviews can explain which interface creates the burden.

Do not hide every choice to reduce load. Workload teams still judge data, reliability, cost, and risk. The goal is less accidental complexity, not less accountability.

Record current skill coverage in the map. For each skill, state the required level, current holders, backup coverage, evidence, and confidence. Add the next learning or sourcing action.

Practical evidence is strongest. A person may have shipped a reviewed change, diagnosed an incident, led an exercise, or taught the process. A certificate can support but not replace that proof.

Keep gap types separate. A skill gap means the team cannot yet perform the work. A capacity gap means it lacks enough time.

An authority gap blocks a skilled person from deciding. An information gap hides facts needed for safe action. Training cannot solve all four conditions.

A capability gap is the difference between the required outcome and current organizational ability. It can include one or more gap types. Naming them guides the response.

A capability bottleneck is a constrained skill, authority, or service on which many outcomes depend. Demand accumulates faster than the constraint can respond. The resulting queue can exist across several teams.

Mark every single-person dependency. Note where one person holds the knowledge, access, or decision right. Record what stops during that absence.

Then add a real control. Pairing, runbooks, shared access, rotation, exercises, and succession can distribute capability. A backup name alone cannot.

Practice matters for rare work. Recovery, incident command, privileged access, and policy exceptions may occur infrequently. Exercises keep access and judgment current.

The bottleneck is not always the busiest person. It can be the only person with authority or context. Queue data and dependency evidence help expose it.

Avoid loading every skill into one platform team. Identity, networking, governance, security, automation, operations, and product work create a wide span. Large estates may need several capability-aligned teams.

Splitting the team also creates interfaces. Specialist groups need shared standards, integration checks, escalation, and joint planning. More teams can increase coordination load.

A partner can close a gap for a time. Record the supplied work, internal accountable owner, evidence access, escalation, knowledge transfer, and exit conditions. Business accountability remains inside the organization.

Use time in the responsibility map. Some capability is required before the next adoption step. Other skill can grow while a partner or central team supplies temporary support.

Mark temporary assignments with an end condition. Add a review date and transition owner. A short-term solution should not become the operating model by accident.

The complete map follows a consistent record. State the outcome, scope, primary owner, practiced backup, decision right, and interface. Add service evidence, required skills, current coverage, gap type, confidence, review date, and trigger.

This structure connects accountability with actual ability. It also separates a weak interface from a missing skill. Each finding can then become a precise planning item.

The capstone map follows its shared-management choice. Governance owns control intent and exception decisions. Security owns protection requirements and security escalation.

The platform team owns Terraform-based platform configuration, supported request inputs, common service interfaces, and platform evidence. Platform operations owns shared-service health, incidents, and recovery. One small group may fill both roles while the duties remain distinct.

Synthetic workload consumers own request quality and workload-side decisions. They validate whether the contract is understandable. They do not deploy an application workload.

Each capstone responsibility needs a primary and practiced backup role. Every interface needs inputs, outputs, evidence, escalation, and authority. Every required skill needs coverage and confidence.

Turn capability gaps into named plan items. Record the blocked outcome, gap type, consequence, and next action. Chapter 3.4 will decide when that work enters the backlog.

Review the map when a team, partner, portfolio, service, or operating model changes. Review it after incidents and repeated handoff failures. The map must describe current ability rather than historical intent.

The responsibility map cannot remove ambiguity forever. It makes ambiguity visible enough to test. A working boundary must survive absence, pressure, and change.

Who can decide when the primary owner is unavailable? Which interface fails as demand rises? Which required skill still exists in one person's head?

Those questions reveal more possible work than any team can perform at once. The map leaves the next choice unresolved. Which dependency or capability gap should move first?

Chapter 3: Plan — Design the Operating Model

Chapter 3.4: Sequence Work Around Dependencies

Cloud adoption produces more work than an organization can perform at once. Independent schedules hide that shared limit. Each local plan can still look sound while the whole adoption system stalls.

Several projects may reserve the same identity specialist. Another may need network work absent from every schedule. Starting them all creates queues rather than capacity.

Partial work then spreads across teams. Context switching grows, and escalation competes with delivery. Status can remain green while no outcome becomes usable.

An adoption backlog places demand in one ordered view. It is the maintained set of work that advances cloud adoption outcomes. It contains more than workloads.

Platform capabilities, decisions, discovery, skill development, risk reduction, and validation all belong. Workload adoption items belong too. One backlog reveals shared constraints.

The backlog is not a fixed calendar. Sequence describes the order in which work should be pulled. Chapter 3.5 will maintain dates, estimates, and cost.

Begin with the dependency map from Chapter 3.1. A dependency states what a consumer needs from a provider. Sequencing decides whether that need controls order or coordination.

Use three relationship types. A prerequisite must be satisfied before the consumer can produce a safe outcome. An approved identity model can be a prerequisite for access automation.

A coordination need allows work to overlap but requires aligned timing or interfaces. A priority preference makes one order safer or cheaper without blocking the result. These relationships should not be treated as equal.

Record the type and rationale. Attach source, evidence status, and confidence. A suspected relationship should not become an unquestioned gate.

Technical links are only one source. Decisions, missing owners, scarce skills, contracts, business windows, and evidence gaps can also constrain sequence. Represent each material condition.

An unknown dependency needs a discovery item. State the question, owner, required evidence, and blocked decision. A risk note without action cannot release work.

Identify dependency clusters next. Workloads or capabilities that must change together form a coordinated group. Shared data, APIs, identity, DNS, and business cycles can create one.

A group should preserve the relationships required for safety. It should not absorb an entire estate. If the group exceeds capacity, find a smaller slice or stable interface.

Bridges deserve separate review. One shared capability may connect several groups. Its change can unlock broad value or create broad harm.

Dependency reach can justify foundational work. Subscription vending can enable many later requests. Reach alone does not make every platform idea a prerequisite.

Avoid building a complete foundation for imagined needs. Tie each enabler to a near-term consumer and observable result. Otherwise preparation can postpone all value.

Avoid a stream of easy wins as well. Moving simple items while leaving identity, governance, or operations risk untouched preserves the system constraint. Early value and foundational risk reduction must inform each other.

Compare business value, risk reduction, enabling value, and learning value. Business value serves an outcome. Risk reduction removes or contains harm.

Enabling value releases later work. Learning value reduces uncertainty for a decision. Record the reason instead of only a score.

Add urgency, reversibility, and effort. A fixed contract date may control sequence. A reversible experiment may create evidence before a large commitment.

Do not hide tradeoffs in one weighted number. Equal totals can represent different risk. Preserve both the evidence and the judgment.

Use thin slices where possible. A slice should cross enough roles and interfaces to test a real path. It should remain small enough to finish and learn from.

One platform slice can validate a synthetic application landing-zone request. It can test inputs, control feedback, ownership, and evidence. No application workload is required.

That slice still has prerequisites. The request schema, control intent, decision authority, and platform boundary must be clear. Missing facts become entry conditions or discovery items.

Use one standard backlog record. Start with identifier, outcome, type, owner, consumers, prerequisites, coordination needs, and evidence. Add value, risk, learning, urgency, effort, and confidence.

Then state required skills, capacity, decision rights, planned slice, and workflow stage. Link capability gaps from Chapter 3.3. Finish with entry criteria, exit criteria, blocker, and review trigger.

Entry criteria state what must be true before active work begins. The outcome and scope must be clear. The primary owner and decision authority must accept the item.

Material dependencies must be known or bounded. Required inputs, skills, and capacity must be available. The team must understand the main risk and validation method.

Exit criteria state the observable result. “Implementation complete” is not enough. Accepted evidence must show that the outcome works inside its boundary.

Useful exit conditions include validated behavior, stored evidence, owner acceptance, support readiness, and updated records. Remaining gaps return to the backlog. Dependent items receive an explicit release signal.

The release signal names what became safe to start. It links evidence from the completed prerequisite to each consumer. Completion without that handoff can leave downstream work blocked.

Entry and exit criteria differ from task lists. They govern the boundaries of work. Tasks can change as the team learns.

Use readiness with care. Planning readiness means enough evidence, ownership, skill, and capacity exist to pull the item. It does not mean the Azure landing zone is deployed.

Execution readiness later may require a verified platform capability, tested recovery, or approved change window. Name the required form in the item. CAF Plan should not claim CAF Ready is complete.

Keep workflow stages few and clear. Candidates hold unrefined demand. Ready holds items that pass entry criteria.

Active holds work being performed. Verify holds work awaiting evidence and acceptance. Done means every exit condition passed.

Blocked is a condition, not forgotten storage. Record the cause, owner, next action, and review time. An active blocked item normally remains within work in progress.

Work in progress, or WIP, is started but unfinished work. A WIP limit restricts the number of items in a stage. Its purpose is to improve finishing and expose constraints.

Set limits at meaningful service boundaries. One portfolio-wide limit can hide overload in security review. A per-person limit can hide shared service queues.

Start with a low limit based on effective capacity. Treat the number as a hypothesis. Lead time, blocked age, quality, and demand should guide adjustment.

A WIP limit is not a target to fill. Empty space is valid when no item meets entry criteria. Pulling weak work to keep people busy increases delay.

When a stage reaches its limit, finish or unblock before starting more. People can help downstream or repair the interface. The limit directs effort toward flow.

Do not raise the limit merely because it feels restrictive. Repeated breaches are evidence. Large items, weak entry criteria, missing skills, or overloaded approval may be the cause.

Define an emergency policy. Name who can expedite, which risk qualifies, and what active item will pause. An unlimited fast lane makes the WIP limit meaningless.

Sequence around the bottleneck. More upstream design only enlarges a constrained security queue. Improve or protect the constraint before adding demand.

The same applies to scarce platform skills. Pairing within a thin slice can deliver value and grow backup capability. Broad parallel work can deepen a single-person dependency.

Dependency-aware order is not a simple graph sort. Business deadlines, criticality, learning, reversibility, and capacity still matter. The sequence is a decision over changing evidence.

Challenge priority with portfolio facts. A high-value workload with an unknown data link may need discovery first. A lower-value item may safely test a common path.

Early work should produce reusable learning. It can validate an interface, exercise a backup owner, or test an operational handoff. That learning must influence a later choice.

Do not avoid representative complexity forever. Include a bounded case that crosses a common hard dependency. Otherwise early success proves only that easy work is easy.

Plan near-term work in detail. Leave later candidates at lower fidelity. Completed slices will reveal new dependencies and capacity.

Review the backlog after each slice. Update relationship evidence, readiness, and effort assumptions. Reorder future work when the case changes.

The capstone begins with decision-ready platform inputs. Scope, control intent, responsibility interfaces, and required skills must be accepted. Blocking gaps become backlog items.

Architecture work then supplies approved platform decisions. Terraform codification depends on those decisions. Independent observation depends on a deployed platform state.

The synthetic request contract depends on a clear platform boundary and supported inputs. Its validation depends on representative synthetic requests and expected feedback. No workload deployment enters the plan.

This remains a high-level sequence. Chapters 4 through 6 own design, implementation, and platform observation. Chapter 7 owns the detailed workload-facing contract.

The artifact is one dependency-based adoption backlog. Every item records why it occupies that position. Every active item has passed entry criteria.

Every completed item has passed exit criteria and released evidence. WIP limits protect constrained stages. Blocked work remains visible.

The anchor is simple: start only what the system can support, then finish enough to release the next safe dependency. Busyness is not progress. Verified flow is progress.

Chapter 3: Plan — Design the Operating Model

Chapter 3.5: Make the Plan a Living System

An approved adoption plan can become wrong while everyone follows it. Evidence changes, forecasts shift, and new risks appear. A frozen document conceals those signals.

Teams soon create shadow plans in messages and spreadsheets. Leaders still see the formal dates. One system now carries several versions of truth.

A living plan repairs that split. It is the maintained source for adoption intent, evidence, choices, forecasts, costs, risks, and commitments. Learning changes it through explicit control.

That last condition matters. Revision isn't permission for random motion. A valid change has a cause, owner, authority, and effective date.

The adoption backlog from Chapter 3.4 remains the source of delivery order. The plan links to its items, dependency states, and release evidence. Duplicating the sequence would create a rival backlog.

The plan governs the conditions around that order. It joins work to outcomes, usable capacity, estimates, funding, exposure, decisions, and reviews. A change in one record can reveal effects elsewhere.

Begin the template with plan identity. Record scope, purpose, accountable owner, version, status, approver, effective date, and latest review. Those fields identify the trusted state.

Exclusions belong beside the scope. The capstone covers a platform landing zone and synthetic contract validation. No workloads are deployed.

Next, link the plan to strategy. Name the mission, objectives, required platform capabilities, and measures that justify the effort. Unsupported work should return for review.

Outputs and outcomes must remain separate. A deployed service is an output. Safer access or shorter delivery delay is an outcome.

For each outcome, record a baseline, target, source, formula, owner, and measurement rhythm. Add a counter-metric where progress could shift harm. Speed without quality is not improvement.

The plan also separates four statement classes. Evidence describes what was observed, assumptions support action before full proof, decisions choose direction, and commitments bind authority. Mixing them manufactures certainty.

An evidence record carries provenance, collection time, scope, and confidence. Old evidence can remain accurate or become stale. Its review trigger should show when age matters.

An assumption record names the claim, current support, confidence, owner, affected records, test, and review date. An expiry condition prevents quiet permanence. Weak assumptions should look weak.

Impact guides test order. Expected traffic may influence service tiers, regional design, operating effort, and consumption. Testing that assumption can improve several forecasts at once.

A decision entry stays short. It records status, selected option, approver, effective date, affected records, and a link to the full decision record. Rationale does not need duplication.

Replaced decisions remain visible. Each old entry points to its successor and keeps its original context. That chain explains later movement in cost or time.

A commitment is more binding than a target. It states the promised outcome, accountable party, accepted conditions, and approval authority. Changed conditions can then trigger honest review.

Dates require plain labels. A target expresses intent, a forecast reflects present evidence, and a committed date carries formal accountability. Calling every date a deadline erases useful meaning.

A forecast record uses a range rather than false precision. It includes the method, estimate date, owner, confidence, and governing conditions. The range should narrow only when evidence improves.

Methods deserve names. Historical flow, assessment data, expert judgment, or a tested slice support different confidence. Later comparison can show which method performed well.

Conditions make the forecast usable. Approval time, specialist access, freeze periods, prerequisites, and platform readiness all affect elapsed delivery. A change in any condition prompts reassessment.

Capacity means available effort, not names on an organization chart. Support, incidents, training, leave, and existing obligations consume attention. Record effective capacity at a stated date.

Scarce skills and decision rights need separate visibility. Extra engineers cannot substitute for missing security approval or identity expertise. Capability constraints shape credible delivery.

Allocation shows how capacity is protected. Operations, adoption work, platform improvement, learning, and risk reduction all need space. Hidden allocations become hidden queues.

Each review compares demand with effective capacity. A mismatch requires a choice about scope, sequence, staffing, or forecast. Pulling more work only spreads delay.

Financial estimates need clear categories. Separate one-time transition cost, recurring Azure use, operations, training, tools, partners, and temporary parallel running. Their drivers and owners differ.

Shared platform spend also needs an allocation rule. The plan distinguishes common capability cost from workload-specific choices. A visible rule supports fairer tradeoffs.

Architecture supplies the model inputs. Services, tiers, regions, traffic, retention, and resilience decisions change consumption. Open design choices remain linked assumptions.

Chapter 3 therefore avoids invented precision. The plan records provisional ranges and model inputs before Chapter 4 settles the landing-zone architecture. Later decisions refine the financial baseline.

Operational effort belongs beside service charges. Lower Azure consumption can demand more engineering or recovery work. Total cost should expose that exchange.

People investment belongs there too. Training consumes time before it increases capability. Omitting that interval makes budget and capacity look healthier than they are.

An approved estimate becomes a baseline. Later forecasts and observed values are compared with it. The difference becomes a named variance, not a quiet edit.

Variance carries evidence. Usage, design, approval delay, market price, or incomplete discovery may explain the gap. The cause should update every related record.

Keep the estimate source with the baseline. A pricing model, historical bill, assessment result, or test deployment has a date and scope. Future reviewers need that context.

Risk entries must direct action. Record cause, uncertain event, possible effect, exposure, owner, response, trigger, due date, and residual risk. A color cannot do this work.

Responses may avoid, reduce, transfer, accept, or investigate exposure. Acceptance requires the proper authority. Every other response needs an owned action.

When uncertainty becomes fact, the risk becomes an issue. That change should reach dates, cost, dependencies, and affected decisions. Otherwise the plan describes a vanished condition.

Opportunity deserves equal discipline. A reusable capability could release unexpected value or capacity. It still needs an owner, evidence test, and decision trigger.

Dependency details stay in the backlog. The plan records the linked identifier, forecast effect, owner, confidence, and review condition. It never invents a second sequence.

Completion alone does not release dependent work. The prerequisite must produce its agreed release signal. New evidence can then revise downstream exposure and forecasts.

Stable identifiers connect the model. Every assumption, decision, estimate, risk, measure, commitment, and change receives one. Links should not rely on informal titles.

A change entry preserves shared memory. It records the reason, affected identifiers, approver, effective date, and required communication. Material changes also create a new accepted version.

Draft edits are not yet trusted states. Approval publishes a version with a clear owner and date. Superseded versions stay available but lose current status.

History supports learning. Reviewers can compare past evidence with later outcomes and inspect why judgment changed. The aim is better calibration, not blame.

Cadence creates the regular control pulse. Participants follow the decisions under review, including business, platform, finance, security, operations, and workload owners. Attendance should serve authority, not ceremony.

Short flow reviews address blockers, capacity, dependencies, and near-term forecasts. Wider reviews examine outcomes, cost, risk, and strategic fit. One meeting need not carry every question.

Calendar rhythm is only one control. A failed assumption, large variance, regulation, supplier change, security finding, or major decision can require immediate review. Evidence should not wait for a meeting.

Each event trigger names an owner and response window. That owner decides whether records need revision. An unowned alert is merely noise.

Review starts with reality. What changed, and what evidence supports the claim? The answer becomes a new or revised evidence record.

Impact analysis then follows links. Reviewers identify affected assumptions, forecasts, costs, risks, measures, decisions, and commitments. This is where a living plan earns its name.

Proposals follow declared decision rights. Routine forecast updates may belong to the plan owner. Scope, funding, strategic outcomes, or risk acceptance require matching authority.

Approved changes should remain internally consistent. A revised cost also updates its basis and assumptions. A moved forecast also exposes the changed capacity or dependency.

Consumers receive a concise change notice. It identifies the new version, material differences, and required action. Nobody should hunt through a whole document for one altered promise.

Several tools can hold these records. A repository, planning platform, document system, or linked combination can work. The governance model matters more than the product.

One canonical location remains essential. Exports display version, status, and effective date. Links lead to current records while preserving history.

The complete artifact is therefore a connected plan template. Identity and outcomes frame assumptions, decisions, backlog links, capacity, estimates, cost, risk, measures, commitments, changes, and reviews. Every record has ownership and time context.

For the platform capstone, early financial and delivery values remain ranges. Identity, connectivity, management, and governance choices will revise their inputs. The plan makes that uncertainty visible.

Its first gate asks whether architecture has decision-ready evidence. Missing facts become discovery items, assumptions, or risks. Accepted inputs release Ready work without designing the answer here.

The Plan phase now closes without becoming static. Strategy, responsibility, sequence, and evidence have a controlled way to change. Architecture can advance against a visible state.

A living plan cannot make prediction perfect. It can make learning traceable and response deliberate. That discipline points the organization toward an Azure foundation it can defend and revise.

Chapter 4: Ready — Architect the Azure Landing Zone

Chapter 4.1: Platform and Application Landing Zones

Ready — Architect the Azure Landing Zone asks a direct question: which duties belong to a shared platform, and which belong to a workload? That split makes later technical choices easier to test. A useful answer must give both sides clear authority and clear limits.

An Azure landing zone is a standardized Azure foundation for hosting and governing workloads at scale. It isn't a product, a lone subscription, or a finished design. It is a repeatable set of decisions, controls, and services.

An Azure landing zone contains two related parts. A platform landing zone provides common capabilities. Application landing zones give individual workloads governed environments.

The terms prevent a common ownership failure. Calling every subscription a landing zone hides who supports what. Calling shared services mere dependencies hides their service boundaries. The platform landing zone is the common Azure foundation. It provides identity, connectivity, management, governance, and other cross-workload capabilities. Central teams maintain those capabilities for their consumers.

Shared does not mean unmanaged. Each shared capability needs an owner, a supported interface, and entry conditions. It also needs known limits, change rules, and service evidence. Identity makes the boundary clear. The platform provides common directory capabilities and access controls. Workload teams own application roles, user journeys, and application authorization.

Connectivity follows the same division. The platform can offer routing, name resolution, ingress, egress, and hybrid connection services. Workload teams decide how their components use those services.

Management is also shared. Central destinations and baseline diagnostics reduce repeated setup. Workload teams still monitor their application behavior and respond to application failures.

Governance acts through this boundary. The platform establishes controls that apply across a defined scope. Workload teams choose within that scope and own their choices.

An application landing zone hosts one workload or application. A workload normally needs separate landing zones for material environments, such as development, test, and production. It can use one subscription or more when isolation or scale requires it.

Application does not mean a small web service. A workload can include services, data, automation, integrations, owners, and commitments. Its landing zone bounds the Azure environment for those components.

Application landing zones inherit required controls from their parent scope. Those controls create a common floor for security, governance, and operations. Workload designs can still differ above that floor.

This is how the platform supports autonomy. A workload team receives an environment with understood conditions. Routine work no longer needs central approval one resource at a time. Autonomy is not rule-free access. It is the ability to act quickly inside trusted boundaries. Clear guardrails reduce negotiation and protect shared concerns.

The two sides remain connected. Platform capabilities and guardrails flow toward application landing zones. Demand, usage, incidents, cost, and adoption feedback flow back.

That exchange is a platform-to-workload interface. Inputs include the workload need, environment, risk conditions, and supported connectivity choice. Outputs include the governed landing zone, inherited controls, service interfaces, and escalation route.

An interface needs more than a ticket. It needs a named consumer, stable contract, ownership, and a change process. A request for a subscription is incomplete until its conditions are visible. For example, a request can state ownership, criticality, environment, data classification, region need, and connectivity need. The response can state scope, controls, shared services, and open conditions. Chapter 7 later represents this exchange with synthetic contract data.

The planned artifact is conceptual and appears in prose. One side holds identity, connectivity, and management capabilities. The other holds workload resources, with policy, access, network, and telemetry interfaces between them.

The model does not impose one operating model. Some organizations centrally run application landing zones. Others delegate most operation to application teams.

A shared-management model often fits the middle. Platform teams provide guardrails and reusable services. Workload teams operate their applications inside published boundaries.

Responsibility must match authority. A team cannot own workload availability without authority over workload configuration. A platform team cannot promise a shared service without capacity to run it.

Poor boundaries produce queues. Platform engineers can become an approval desk for ordinary application changes. Workload engineers then bypass the intended path, and useful feedback disappears.

The opposite error also harms the platform. A workload team can change a shared route, policy, or logging destination to solve a local problem. That local change can damage many consumers or hide evidence.

The key question is not whether a resource is shared. Does a change need common policy, specialist capability, broad coordination, or a cross-workload commitment? If so, the platform boundary needs explicit ownership and authority.

Subscription placement matters, but it is not the whole design. A subscription separates management, billing, quotas, and access. It does not define a complete service contract or operating model.

Chapter 4.2 gives these boundaries enforceable Azure scopes. It designs the tenant, billing, management-group, and subscription hierarchy. This section establishes why that hierarchy must honor responsibility boundaries.

The platform commonly uses separate Identity, Connectivity, and Management subscriptions. They hold capabilities that many application landing zones consume. Separation helps limit accidental coupling and clarify operational duties.

The application side also needs repeatability. Subscription vending can allocate governed subscriptions through code and place them under expected controls. It should remove routine friction without concealing risk, cost, or operational conditions.

Vending is a controlled handoff, not merely account creation. The platform supplies a known environment to the workload team. The resulting landing zone must make its controls and services understandable.

A fast path can still fail its users. Unexplained policy may look like a random deployment failure. The platform should expose the reason, owner, documentation, and exception route.

Standardization has a limit. Standardize interfaces, guardrails, evidence, and common services first. Do not standardize every workload implementation detail.

Workload teams own workload architecture. They choose and configure permitted application resources, manage application data and behavior, and meet operating commitments. Platform ownership does not absorb those duties.

Platform teams own platform architecture. They maintain shared services, guardrails, automation, and consumption conditions. They must change them predictably because many workloads can be affected.

Some duties remain shared. Workload teams state their needs and validate the service they consume. Platform teams provide the supported service and publish its limits.

Shared responsibility cannot mean shared ambiguity. Every interaction needs an accountable owner, a backup, handoff evidence, and escalation. Chapter 3 provided those operating-model disciplines.

The capstone keeps this design narrow. The existing Identity, Connectivity, and Management subscriptions remain in one Microsoft Entra tenant. Those subscriptions represent the platform landing zone under design.

No application workload is deployed. No application database, virtual machine, web service, or workload network is created. Synthetic request data later represents the application side.

That limit keeps the work honest. The capstone tests a reusable platform foundation, not an application tutorial. A platform can be evaluated through its controls and interface before a workload consumes it.

The capstone does not purchase subscriptions either. It treats the three existing platform subscriptions as design inputs. Subscription vending remains a later contract, not a billing automation exercise.

Terraform will later express approved persistent configuration. Azure CLI running in PowerShell will later observe and validate actual Azure state. This section gives neither tool separate architecture authority.

Architecture needs evidence before code. Teams need to know the capability, consumer, boundary, and failure path first. Terraform can then preserve the decision, and observation can test it.

westeurope is only a replaceable sample region in later examples. It is not a universal default. Availability, compliance, and cost decide the appropriate region.

A boundary is useful only when people can use it. It must help a team choose a path before a change starts. It must not appear only during a review.

This test is simple. Can the workload team see the rule, the service, and its owner? If not, the boundary is still too vague.

Good rules do not need a long wait. Most work should move on a known path. A rare case should have a named way to ask for help.

Data must cross the boundary with care. Logs, tags, use data, and alerts can help both sides learn. They should meet a clear need, not just a wish for more data.

A change in the shared base can have a wide reach. The team that owns it must test its effect and give fair notice. That is part of the service.

No fixed split suits all firms. A small team may hold more work in one place at first. Growth may later make a clearer split wise.

The right split is one people can run well. It must fit skill, risk, and the pace of change. It should be checked as needs shift.

A new need is a signal, not a flaw. It may point to a gap in the shared path. The platform team can then add, change, or end a service.

This is why demand data matters. It shows which steps take too long or fail too often. It also shows where a paved road may help.

The goal is not to make all work the same. The goal is to make safe work easy and rare risk clear. That gives both sides room to learn.

The architecture is compact. Shared capabilities serve future application landing zones through defined interfaces. Inherited controls set a common floor without erasing workload ownership. The boundary is a beginning, not an end state. It makes the next questions about scope, hierarchy, and subscription roles precise. A sound platform begins when teams can state what they own, consume, and must not silently change.

That statement turns subscriptions into a platform. Without it, central services become opaque dependencies and autonomy becomes unmanaged variation. The next section gives the boundary an enforceable Azure hierarchy.

Chapter 4: Ready — Architect the Azure Landing Zone

Chapter 4.2: Tenant, Billing, Management Groups, and Subscriptions

Azure scope needs a clear shape. It makes a control's reach, a subscription's placement, and a team's responsibility visible before implementation begins. The hierarchy should answer these questions without relying on a diagram or an organizational chart.

Scope must be visible. Its purpose must be clear. Its owner must be known.

Azure has several connected hierarchies. A Microsoft Entra tenant is the identity and access boundary associated with Azure. A billing agreement establishes the commercial relationship and price basis. A management group organizes subscriptions for governance. A subscription is the management, billing, quota, and access boundary that contains resource groups and resources.

The terms are close enough to invite confusion. A billing owner is not necessarily a platform owner. A management group is not a cost-center label. A subscription alone neither makes a landing zone nor proves that a workload team has a supported environment. Good resource organization keeps the questions separate.

The tenant is the broad identity context for this book's capstone. Every capstone subscription has its trust relationship with that one Microsoft Entra tenant. The tenant also contains the root management group, the top Azure governance scope. The access design comes in Chapter 4.3. This section defines where that design will later take effect.

The root scope is special. It should stay narrow. It is not an everyday deployment scope.

Billing deserves equal care, but it answers a different question. A subscription has one billing relationship to one contract. A tenant can manage subscriptions linked to more than one contract. That separation can matter during an acquisition, an agreement transition, or a review of shared-service costs. It does not tell the platform where a policy should apply.

Billing needs a named owner. It should not direct policy. That is a separate design choice.

Financial reporting often needs its own dimensions. A finance team might use agreements, billing profiles, invoice sections, legal entities, or cost centers. Those records can cut across the platform's engineering boundary. Keeping them distinct preserves useful cost allocation. It stops the governance hierarchy from mirroring every financial structure.

A management group is an Azure governance scope containing subscriptions or child management groups. A control assigned at a parent can reach its descendants through inheritance. That makes the scope useful when several subscriptions need the same condition.

The condition must be durable. It is not a container for a temporary program or a name that changes with a reorganization.

Inheritance is a force multiplier, so the scope needs a clear purpose. A common rule at a stable parent can remove repeated work from each subscription. The same rule at an overly broad parent can surprise every team below it. Before a policy or role is considered, state where the rule begins. State which subscriptions inherit it.

Each rule should name its scope. That scope should have a reason. This makes an unexpected result easier to trace.

The hierarchy should remain shallow and intentional. A new branch needs a durable reason. It might have a distinct rule set, a stable responsibility boundary, or a workload type with different needs. A business-unit chart, project portfolio, Azure region, or delivery stage usually is not enough. Those facts may still matter, but they should not automatically become a governance path.

Few levels help review. A person can read the path. They can see where a difference begins.

The capstone uses one Microsoft Entra tenant. Directly beneath its tenant root, an intermediate management group represents the platform foundation. That group has four direct branches: Platform, Landing Zones, Sandbox, and Decommissioned. The design is a target architecture. It does not direct anyone to deploy an application workload or buy a subscription.

This is a control point. It is not a resource collection. Later configuration will make the chosen shape real.

The Platform branch contains the pre-existing subscriptions that provide common capabilities. Identity, Connectivity, and Management sit within this part of the structure. Their placement says that a change to shared platform services has a common operational audience. Separation helps bound quotas, costs, access, and failure effects. It does not require one subscription for every platform component.

The Identity subscription is the resource boundary for platform identity-related capabilities. The Connectivity subscription is the boundary for common network capabilities. The Management subscription is the boundary for shared management and telemetry capabilities. This section assigns their place. Chapters 4.3 and 4.4 make the identity and connectivity choices in depth.

The Landing Zones branch is reserved for governed application landing-zone subscriptions. It supplies the common parent for controls that should apply to all such workload environments. A later design can add child groups for genuine workload types. They might need different connectivity or compliance controls. It should not create a management group for each workload, team, or delivery environment.

That restraint makes promoted application code more predictable. Development, test, and production environments often need the same base controls even when they use separate subscriptions. Separate environment branches can surprise a team. The team might meet a new rule only after promotion. Subscription isolation can provide meaningful separation without making every environment a new governance hierarchy.

The Sandbox branch offers a visible, accountable path for exploration. It may carry lighter controls. It still needs ownership, spending boundaries, and a route into a governed landing zone. Sandbox is not a back door to production. It is where a team can test an idea before it requests a supported workload environment.

The Decommissioned branch gives retirement a place in the system. A subscription moved there is no longer treated as an active landing zone. It can receive controls that limit new activity. It can preserve needed proof and support an owned retirement decision. That is safer than letting canceled subscriptions disappear among active work.

The planned hierarchy can be reconstructed without a diagram. Start at the tenant root and place the intermediate platform-foundation group immediately below it. Under that group, place Platform, Landing Zones, Sandbox, and Decommissioned as peers. Place the Identity, Connectivity, and Management subscriptions inside Platform. Leave the other branches without deployed workload subscriptions in this platform-only capstone.

The shape is easy to state. That is a feature. It makes scope visible during a review.

This structure separates placement from billing. A subscription can sit in Platform because it provides common services while its billing information records a different legal or financial responsibility. Conversely, a group of subscriptions can share a cost center without needing the same inherited technical controls. Scope should follow the decision that Azure must enforce, not the column easiest to export from a finance system.

A subscription also needs more than a name and a parent. Its record should identify a durable owner, a purpose, a life stage, a billing relationship, and a management-group placement. It should name the normal request path, the exception path, and the retirement path. These facts turn subscription placement from an administrative act into a decision a team can review.

Subscription boundaries are useful when access, quotas, billing, policy, or failure containment must differ. They also add work across those boundaries. A small component does not deserve its own subscription merely because it has a separate deployment. The question is whether a different responsibility or control condition requires a different scope.

Three checks keep the structure honest. First, every branch must correspond to a lasting difference in inherited conditions or platform responsibility. Second, every subscription must

have an owner who can explain its placement. Third, a routine workload request must follow a known route without asking for a custom hierarchy. A branch that fails these checks is probably complexity, not control.

Hierarchy changes need the same discipline as other shared-platform changes. Moving a subscription can change the controls it inherits. Adding a branch can change how future subscriptions are grouped. A change record should state the reason and affected scopes. It should also name the inherited settings, owner, proof to review, and correction path before the hierarchy changes.

The capstone deliberately remains narrow. Identity, Connectivity, and Management are pre-existing subscriptions used as design inputs. This chapter does not buy or vend subscriptions. It deploys no application resources. The later application contract will use synthetic request data to test the platform-to-workload interface before a workload consumes it.

The hierarchy also protects tool boundaries. Terraform later encodes approved durable configuration. Azure CLI running in PowerShell independently observes the configuration that exists. Neither tool decides whether a branch has a valid purpose or a subscription belongs at a scope. Architecture supplies that decision; implementation and observation make it durable and testable.

The outcome is deliberately modest: one tenant, commercial links kept distinct, stable governance scopes, and subscriptions placed for a reason. That sequence anchors later decisions about access, connectivity, policy, and automation. It makes the next design choices easier to test. A hierarchy becomes a platform only when its scopes express responsibility that people can understand, operate, and verify.

Chapter 4: Ready — Architect the Azure Landing Zone

Chapter 4.3: Identity as the Control-Plane Boundary

Every Azure action starts with an identity. Identity is the control-plane boundary because it determines the authorized action at a specific scope. That decision must be clear before a person, service, pipeline, or workload changes Azure.

Authentication answers who or what is presenting a credential. Authorization answers what that identity may do. Microsoft Entra ID supplies the directory context for authentication and identity management. Azure role-based access control, or Azure RBAC, grants actions on Azure resources at management-group, subscription, resource-group, or resource scopes.

The two authorization systems also need a clean distinction. Microsoft Entra roles manage directory resources such as users, groups, and applications. Azure roles govern Azure Resource Manager actions on Azure resources. A person can need one without the other. A task that crosses the boundary must name both kinds of authority.

Authorization always has a target. It needs a role and a scope. Both need an owner.

Least privilege gives this distinction a practical rule. Grant only the actions required for an approved task. Use the narrowest useful scope. Keep the grant only as long as the task needs it. A role can be excessive because its scope is broad, its duration is long, or its purpose is vague. Trust is not a substitute for design. A trusted person can still make a wide change by mistake. Small, time-bound grants reduce that reach.

The capstone access model has five actor classes. Human administrators operate shared platform services. Platform automation applies approved changes and checks. Security teams investigate controls, auditors inspect evidence, and workload teams run approved application environments. One broad administrator group would hide these different duties.

Human administrators need enough authority to maintain the platform. They do not need standing control of every workload resource. A platform operator might need elevated access to repair a shared management service for a short time. The access design should make that task possible without turning an ordinary account into a permanent cross-estate administrator.

Privileged Identity Management, or PIM, supports this design. It can make a privileged role eligible rather than always active. It also supports time-based or approval-based activation. The platform can require conditions such as justification, multifactor authentication, a maximum duration, approval, and notifications when a risk level calls for them.

Activation and eligibility are not the same. One asks whether privilege is needed now. The other asks who may request it.

Eligibility is still access. It needs an accountable owner and a review date. A person should not remain eligible because they once helped with an incident. The record should identify the role, scope, expected task, activation rule, and reviewer. That record makes privilege an operational decision rather than a memory held by one administrator.

Owned groups are the normal unit for human assignments. A group can show its purpose, membership rule, owners, role, and scope. It is easier to review than a long set of direct user assignments. A direct assignment may be justified, but it should be treated as a bounded exception with a reason and a removal plan.

Group ownership is part of the control plane. An owner decides whether a new member still needs the group's authority. A group with no accountable owner has no reliable decision path. A familiar team name is not enough evidence that every person in the group needs every permission it grants.

The owner must be able to remove access. A backup owner should be known. An escalation path should exist.

Platform automation needs a distinct nonhuman identity. It should not operate as the engineer who wrote its configuration. It should not rely on a copied long-lived secret. Its role and scope should match one approved deployment or validation purpose. Chapter 5 later defines the detailed Terraform and workload identity federation design.

Automation needs separation of duties too. A deployment identity should not grant itself new access. A validation identity should observe without becoming a second controller.

Managed identities are useful for Azure-hosted workloads that need Microsoft Entra tokens. They let applications obtain tokens without developers managing traditional credentials. The identity still needs authorization at the target resource. Removing a secret does not remove the need for a small role, a narrow scope, an owner, and a review path.

The platform-only capstone deploys no workload, so it creates no workload-managed identity. It establishes a rule instead: every machine identity has one named purpose, a durable owner, documented scope, and retirement path. Automation must not silently inherit the privileges of the person who created it. A nonhuman identity is a platform component with a lifecycle.

Security teams need a controlled path to investigate incidents and improve controls. Auditors need read access to inspect assignments, configuration, and evidence without routine authority to change them. Workload teams need delegated access inside their own application landing zones. These are different responsibilities, not different spellings of administrator.

Management-group scope is especially powerful because it can reach many subscriptions. Reserve it for a small number of controlled platform responsibilities. Most workload access

belongs at the subscription or resource-group scope that owns the work. That keeps a local application need from becoming inherited authority over unrelated environments.

The planned artifact is a reconstructable access model in prose. For each actor class, record purpose, identity type, assignment method, normal scope, allowed role, activation condition, review owner, and evidence source. Human administrators use owned groups and eligible privilege. Platform automation uses a dedicated nonhuman identity. Security, audit, and workload paths remain separate.

An assignment is not complete when a role appears in a portal. It is complete when its owner can explain the task, scope, duration, approval path, and review date. Evidence should show which identity was active and which role it used. This connects a permission to an accountable decision.

The normal path should be easy to find. A clear request avoids borrowed accounts. It also gives reviewers the facts they need.

One record cannot answer every question. Approval shows why access was requested. Assignment shows what was granted. Activation shows when high privilege was used.

Access reviews provide the second half of the lifecycle. They can recertify group membership, application access, and role assignments at a chosen rhythm or after a material change. PIM limits active time for high privilege. A review tests whether eligibility or membership remains needed at all. Both controls require a reliable removal path.

Review rhythm should match risk and change. Broad, unused access deserves close attention. Stable access still needs an owner and proof.

Removal must be routine. It should not wait for an audit. Stale access is a design failure.

Emergency access is a real requirement. It is not an excuse for uncontrolled accounts. The platform needs a narrow, documented recovery route with strong protection, named ownership, monitored use, and a prompt review after activation. Chapter 9 will set the wider security controls for exceptional access.

The capstone stays at the architecture level. It creates no users, groups, PIM assignments, role assignments, managed identities, or workload access. Terraform later represents the configuration that has been accepted. Azure CLI running in PowerShell independently inspects assignments and scopes. This section decides what those future artifacts must represent.

This limit is deliberate. It tests the model before identities are created. It keeps the capstone focused on the shared platform.

The access model can be tested with three questions. Can every high-impact action be traced to an identity, role, scope, owner, duration, and review path? Can a workload team work without

platform authority? Can an auditor inspect the answer without changing it? A model that cannot answer these questions has not yet defined a control-plane boundary.

The rule must work in practice. It must be clear. It must be used.

Identity is more than a sign-in screen. It is the point where a request becomes permitted Azure action. A sound design makes normal work clear, privilege temporary, machine access purposeful, and review continuous. The next section applies the same discipline to connectivity, where one shared path can affect every workload.

Chapter 4: Ready — Architect the Azure Landing Zone

Chapter 4.4: Connectivity as a Shared System

Network paths are shared system behavior. A route, name-resolution rule, egress decision, or ingress path can help one workload while changing conditions for many others. Connectivity therefore belongs to the platform design, with clear ownership and a published interface for workload teams.

The Connectivity subscription is the home for common network capabilities in the capstone. It does not own every virtual network or every application rule. It owns the shared services and decisions that need cross-workload coordination, such as common routing, shared name resolution, hybrid links, and approved ingress or egress patterns.

A hub-and-spoke design provides a small, understandable baseline. The hub holds shared connectivity services. A spoke is a workload network that connects to the hub under a defined contract. The hub is not a place where all application resources belong. It is a point for shared paths and shared controls.

The hub has a narrow purpose. It serves shared paths. It does not absorb workload design.

The capstone chooses this logical model because it can explain ownership without deploying a workload. It does not claim that hub-and-spoke is universal. Azure Virtual WAN can be a better fit when a large, distributed network needs managed global interconnection or extensive branch connectivity. A topology choice must follow traffic, scale, operations, and hybrid requirements. DNS is part of the platform interface, not a finishing task. A workload needs to know which names it can resolve, which zone owns those names, how private names link to its network, and where a failure is investigated. A name that resolves in one environment and not another is an application outage from the user's perspective.

Names need owners. Zones need clear scope. Resolution needs a test.

Routing needs the same clarity. Each path needs a declared source, destination, next hop, and owner. The platform should publish whether workload traffic stays local, passes through a shared hub, reaches an on-premises network, or exits to the internet. Hidden routes make an incident harder to diagnose and make a local change risky.

A route expresses intent. Its next hop must be known. Its effect must be observed.

Ingress and egress are separate decisions. Ingress defines how approved external traffic reaches a workload. Egress defines how workload traffic reaches approved external destinations. A public endpoint, reverse proxy, private endpoint, network virtual appliance, or firewall can be

part of a solution, but the choice must match a documented traffic need. A product name is not an architecture decision.

The capstone's baseline is intentionally small. It reserves the Connectivity subscription for shared network capabilities and describes a logical hub, shared DNS, documented routes, and explicit ingress and egress ownership. It does not deploy an application spoke, a virtual machine, a database, or a public endpoint. It also does not require paid connectivity or security services.

Paid services can be valuable when the risk and traffic case justify them. Azure Firewall, network virtual appliances, distributed denial-of-service protection, and large-scale hybrid connectivity can add capability and cost. In this capstone, they remain optional and disabled by default. A later decision must state the threat, traffic pattern, owner, operating cost, and evidence that justify enabling one.

Cost is part of the choice. Risk is part of the choice. Neither should remain implicit.

Hybrid connectivity is an interface between two operated networks. It needs named owners on both sides, agreed address ranges, route exchange rules, DNS behavior, change windows, and a fault path. A circuit or tunnel alone does not provide a service. The connection is useful only when teams can state what traffic may cross it and how they will see a failure.

Both sides must agree. Both sides must observe. Both sides must know how to escalate.

Network segmentation protects more than confidentiality. It limits blast radius and makes traffic intent visible. The platform should provide shared boundaries, while workload teams own the fine-grained paths inside their application networks. This keeps a local application requirement from changing the connectivity system for every consumer.

Address space is an early design input. Overlapping ranges can block peering, hybrid routing, and future growth. The platform should maintain an allocation decision and a change path before a workload creates a virtual network. A range is not just an address choice; it is a long-lived dependency.

Private connectivity also needs a service boundary. A private endpoint can reduce public exposure, but it changes DNS, routing, and consumer expectations. The workload team owns its use of the endpoint. The platform owns the shared resolution and network conditions that make the pattern reliable.

Observability starts with expected paths. The platform should define which route, name-resolution result, and egress decision can be tested after a change. Evidence should identify the time, scope, and result. A configuration that exists but carries no expected traffic is not proof that the service works.

The planned artifact is a logical hub-and-spoke model in prose. The Connectivity subscription hosts the shared hub capabilities. Future workload spokes connect through documented DNS, routing, ingress, egress, and hybrid interfaces. The model names the platform team as owner of the common services and the workload team as owner of its application network and traffic requirements.

An interface needs entry conditions. A workload request should state environment, address-space need, DNS names, inbound exposure, outbound dependencies, hybrid need, data sensitivity, and expected traffic. The platform response should state the assigned network boundary, supported routes, DNS integration, approved ingress or egress path, limits, monitoring evidence, and escalation route.

This is not a ticket template for its own sake. It prevents a workload from discovering a network constraint only after deployment. It also shows the platform which demand is common enough to become a supported service. Repeated exceptions are feedback that the platform interface may be incomplete.

The platform must state its limits as well as its services. A workload may need a connection that the baseline does not support. That request should become a design decision with owner, cost, risk, and review point. It should not become an undocumented rule in one network configuration.

Availability and recovery also cross the boundary. A shared DNS or routing failure can affect many workload teams at once. The connectivity owner needs an incident path, a service record, and evidence for recovery decisions. Workload teams need to know when to escalate rather than changing a shared path themselves.

The design must make change ownership visible. A platform owner approves changes to shared connectivity services and communicates their effects. A workload owner validates that its application still reaches its required dependencies. Neither side can safely declare a shared path healthy without evidence from the other.

Regional choice is also conditional. The capstone uses westeurope only as a replaceable example region in later implementation. A real network design must consider service availability, data obligations, latency, cost, and the location of connected networks. Region is not a label to copy into a hierarchy.

Connectivity changes deserve broad impact review. Before changing a route, DNS zone, peering, or shared egress path, the owner should identify consumers, intended and unintended traffic effects, rollback conditions, evidence, and notification needs. The change is complete

when observed behavior matches the intended path, not when a configuration command succeeds.

Change needs a forecast. Recovery needs a trigger. Completion needs proof.

Later Terraform configuration will preserve accepted network decisions. Azure CLI running in PowerShell will inspect the network state that exists. Neither tool selects a topology or decides whether a traffic path is justified. The architecture supplies that decision and gives future code and evidence a stable target.

Tools preserve the model. Tests challenge the model. Owners decide when it must change.

The test is direct. Can a workload team explain how its names resolve, where its traffic goes, which path reaches users, and who owns a fault at each boundary? Can the platform team change a shared path with evidence of affected consumers? If not, the connectivity design is still a collection of resources rather than a shared system.

A useful network contract can be explained before deployment. It can also be tested after every shared change. Those two properties turn connectivity from hidden plumbing into a platform service.

Connectivity makes the platform real to workload teams. It turns the abstract boundary of a landing zone into reachable services, controlled exits, and known failure paths. The next section integrates this design with governance, security, management, and automation so that no area is treated as an isolated checklist.

Chapter 4: Ready — Architect the Azure Landing Zone

Chapter 4.5: Integrate the Landing-Zone Design Areas

A landing-zone decision rarely stays in one design area. Management-group placement changes policy and access scope. Private access changes DNS, routing, and diagnostic checks. Integration makes those effects visible before implementation makes them costly.

The design areas are views of one platform. Tenant and billing decisions set trust and commercial boundaries. Resource organization defines where controls can act. Identity defines who may act there.

Other areas shape behavior inside those boundaries. Connectivity design creates shared paths. Governance, security, and management set controls and checks. Platform automation controls how approved change reaches Azure.

Separate reviews can still produce a broken architecture. Each checklist may pass while two decisions conflict. Integration finds the conflict and forces a clear choice. The output is one target, not a stack of plans.

A regional control shows the risk. It may meet a data-location need yet block recovery. The final decision must name the priority and conditions. An owner must accept the risk that remains.

Private access has the same wide reach. A private endpoint can reduce public exposure. It also needs DNS, routes, authorization rules, and diagnostic evidence. Approval is incomplete until those dependencies agree.

Shared telemetry crosses even more boundaries. Central collection affects identity, routes, retention, access, region, and cost. The Management subscription can host this shared capability. Its location alone does not define its operation.

The first artifact is a landing-zone decision register. It is the source of accepted architecture decisions. It is not a meeting log or product inventory. It is also not a backlog in disguise.

Each record starts with a stable ID and design area. It states the decision, rationale, scope, and accountable owner. It names consumers, dependencies, assumptions, costs, evidence, and status. A date or event starts the next review.

Status makes uncertainty visible. A proposed decision cannot guide implementation yet. An accepted decision can enter the implementation plan. Rejected and superseded options stay linked to their reasons.

Decision dependencies must work both ways. Private access may rely on shared DNS. The DNS record must also name its private consumers. Reciprocal links expose impact when either side changes.

Conditions keep the architecture honest. A paid service can stay disabled until risk or traffic supports it. A second region can wait for a recovery requirement. The trigger belongs inside the decision.

A product name is not a full decision. “Use Azure Firewall” leaves key facts open. It omits the traffic, threat, owner, cost, and evidence. Required behavior must come before the service selection.

The register also separates platform and workload authority. The platform owns shared boundaries, guardrails, network paths, and access patterns. Workload teams own their applications, data, dependencies, and local controls. Each responsibility must come with enough authority to perform it.

The second artifact is the integrated target architecture. Accepted records and their relationships form this target. The capstone starts with one Microsoft Entra tenant. It also assumes an existing billing relationship.

Both items are context for later implementation. Terraform must not create them by accident. Their ownership and lifecycle sit outside the capstone. The register still records their effect on scope and trust.

An intermediate platform-foundation management group sits below the tenant root. It keeps the book's platform scope apart from other tenant content. Platform, Landing Zones, Sandbox, and Decommissioned form its main branches. This shallow hierarchy supports clear control inheritance.

The hierarchy does not copy the company chart. Teams and reporting lines may change often. Stable control needs should drive the branches. A new management group therefore needs a clear control reason.

The existing Identity, Connectivity, and Management subscriptions sit below Platform. Identity can hold shared identity support. Connectivity owns common network services. Management owns shared telemetry and management capabilities.

Landing Zones is the future home for application subscriptions. The capstone creates none. Sandbox supports controlled experiments with a different risk level. Decommissioned holds subscriptions during safe retirement.

Access uses groups and workload identities. Direct standing grants to individuals are not the norm. Privileged human access is eligible and time-bound where supported. Emergency access remains separate from daily work.

Automation receives its own nonhuman identity. Its permissions match the task and scope. A pipeline cannot wait for an interactive sign-in. Deployment, emergency, routine, and read-only paths stay distinct.

The connectivity model uses a logical hub and spokes. Shared DNS, routes, ingress, egress, and hybrid links are platform interfaces. Future application spokes can use them. No workload network is built in this capstone.

Paid connectivity and security services stay optional. They are disabled by default here. Risk, traffic, regulation, or scale may justify a change. That case needs an owner, cost analysis, and test.

Governance starts with a small inherited baseline. Each control sits at the highest safe scope. It can require key metadata, approved placement, and diagnostic settings. Chapter 8 owns the full policy lifecycle.

This architecture records the intended result and scope. It does not prewrite every policy definition. That limit prevents an outline choice from becoming false detail. It also keeps later enforcement tied to observed impact.

Security is not a final gate or separate box. It appears in access, hierarchy, exposure, telemetry, and delivery. Chapter 9 will deepen posture and threat work. This foundation makes those controls possible.

Management places shared platform telemetry in the Management subscription. Records name who may send, read, retain, and act on it. No workload monitoring is promised before workloads exist. Chapter 10 will build the wider operations loop.

Platform automation preserves accepted decisions. Terraform will own persistent Azure configuration and versioned inputs. Azure CLI running in PowerShell will inspect deployed state. Neither tool may settle an open architecture choice by default.

The capstone uses westeurope as a replaceable sample region. A real decision tests service availability, data obligations, latency, connectivity, recovery, support, and cost. Those conditions belong in the register. A copied region name is not an assessment.

An application subscription request can test the integration points. It should reach the right branch and inherit the baseline. Its team should receive approved access and network paths. Platform evidence should show that each result matches the records.

This test uses a synthetic request later in the book. It does not deploy an application now. The request reveals weak platform interfaces without widening scope. It also gives future checks a stable input.

Record quality is the first exit test. Each accepted decision needs scope, rationale, owner, and expected evidence. Every dependency must appear from both sides. Each optional cost needs a clear activation condition.

The next test compares operating assumptions. Hierarchy, identity, connectivity, governance, security, management, and automation must tell one story. Central control cannot rely on unnamed local work. Authority and responsibility must meet at each shared boundary.

Failure tests expose weak integration points. A DNS failure must lead to an owner and known consumers. A control that blocks recovery needs an escalation path. Loss of deployment access must not erase independent checks.

Change review begins with consumers and dependencies. The owner finds affected records, teams, expected effects, and rollback needs. New evidence must show that the integrated architecture still works. Approval means the records agree again.

Regulation, service limits, threats, costs, and team changes can start review. Time alone can also expose stale assumptions. The register names these triggers before they occur. That makes review part of architecture, not cleanup.

Exceptions are decisions, not silent configuration edits. Each needs a rationale, scope, owner, end point, and evidence. A repeated exception is useful feedback. It may reveal a weak baseline or a missing platform service.

The capstone is ready when the records tell one clear story. A reviewer can trace each boundary to an owner and matching authority. Shared services name consumers, while controls name evidence. Open decisions remain proposed instead of entering code as defaults.

The target stops at the workload boundary by design. It defines the platform landing zone and its future consumption paths. The decision register holds intent, scope, ownership, dependencies, and conditions. The integrated architecture shows what implementation must preserve.

Architecture now has a firm handoff. Chapter 5 turns accepted decisions into Terraform desired state. Code will make the design repeatable and testable. It will express the architecture, not invent it.

Chapter 5: Ready as Code — Building the Platform with Terraform

Chapter 5.1: Terraform as a System Controller

Terraform becomes predictable when its authority has a visible edge. It can reach broad Azure APIs, yet it should change only assigned platform objects. That limit turns a deployment tool into a system controller.

Ready as Code — Building the Platform with Terraform starts at that edge. Accepted architecture becomes a versioned desired state. Ownership becomes visible in configuration, state, identity permissions, and plan review.

The control loop joins configuration, providers, state, and remote systems. Configuration states what managed objects should become. Providers read Azure and translate planned work into API operations. State links Terraform resource addresses to remote objects.

State is not Azure itself. It contains bindings, cached attributes, and dependency metadata. A normal plan refreshes managed objects before calculating changes. That refresh gives the plan a newer remote view.

The view can still be incomplete. Azure may contain objects that no Terraform state tracks. Access limits or failed API calls can also reduce what a provider reads. Independent Azure checks remain part of the system.

A plan compares configuration, refreshed data, and prior state. Its output is a proposal, not proof that the final result will work. Time, permissions, policy, limits, or another actor can affect a later apply. The concrete plan still needs review near deployment time.

A no-change plan also has a narrow meaning. It says Terraform found no managed difference that requires action. It does not prove the platform works or that unmanaged objects are absent. Operational evidence must answer those wider questions.

An apply executes approved actions against Azure. It can create, update, replace, or remove objects inside the configuration and state boundary. It cannot safely mean “make the tenant match an undocumented ideal.” The controller needs a declared scope.

That scope begins with an ownership model. The model separates objects managed by this root, existing objects only referenced, and elements outside the root. An object with no agreed class stays outside the apply boundary.

The capstone root will manage the hierarchy below the platform-foundation boundary. Later code adds selected role assignments, governance assignments, shared connectivity, and shared management settings. Resource addresses turn those broad claims into exact ownership.

The root does not own the Microsoft Entra tenant or billing relationship. It also does not create the existing Identity, Connectivity, and Management subscriptions. Their identifiers arrive as inputs. Reading an object does not transfer its lifecycle to Terraform.

Application workloads remain outside this state. A platform module can publish a future consumption interface without absorbing workload resources. The state boundary should match change authority, release pace, and failure impact. A module name alone cannot create that alignment.

Modules organize configuration, but a child module does not get separate state by default. The calling root and its children share one state boundary. A separate root is needed when ownership, release pace, privilege, or failure risk requires isolation.

Azure CLI follows the same ownership rule. Read-only queries can test remote behavior without becoming another controller. Imperative changes to managed routes, roles, policies, or diagnostics create a second configuration path. The conflict exists even when the command succeeds.

Suppose an operator changes a managed route with Azure CLI. Azure now differs from configuration. The next normal plan can propose restoring the coded value. The team must restore Azure or accept the new value in code.

Ignoring the difference leaves two stories. Configuration describes one route, while Azure serves another. A later apply may erase the manual fix. No operator should have to guess which story owns the object.

Emergency change remains possible. Its record needs an owner, time, reason, resource, effect, and recovery condition. After the event, the team updates Terraform or restores the approved configuration. Reconciliation closes the exception.

Bootstrap is a second narrow case. Remote state and deployment identity must exist before the main root can use them. Chapter 5.2 gives that work a separate path and control boundary. The platform root should not hide bootstrap commands inside routine deployment.

Providers connect Terraform to remote APIs. They define resource types, read objects, check argument types, and perform planned operations. Provider configuration belongs in the root module. That location keeps the target subscription and execution context visible.

Child modules declare provider requirements but do not configure credentials. The root passes a provider configuration into the child. The child receives only the context its resources need. This code boundary also guides privilege design.

Version constraints make execution assumptions explicit. The root constrains Terraform CLI and the AzureRM provider to tested ranges. Initialization selects an allowed provider version. The dependency lock file records the exact package and checksums.

The capstone uses Terraform 1.15 and AzureRM 4.81. The root accepts patch releases within those minor lines. The validated lock file selects AzureRM 4.81.0. Upgrades require a deliberate change to constraints and the lock file.

The first file declares the CLI and provider requirements. Explicit source addresses remove registry ambiguity. The provider block then selects the existing platform subscription through an input.

```
HCL / TERRAFORM
# terraform.tf
terraform {
  required_version = "~> 1.15.0"

  required_providers {
    azurearm = {
      source = "hashicorp/azurearm"
      version = "~> 4.81.0"
    }
  }
}

# providers.tf
provider "azurearm" {
  features {}

  subscription_id = var.platform_subscription_id
}
```

No credential appears in the HCL. Authentication comes from the controlled run context. That context can change without rewriting the provider block. Chapter 5.2 defines and protects that path.

The subscription input selects an existing boundary. It is not a request to create a subscription. The parent management-group input sets the highest scope this root may organize. The location remains a replaceable sample region.

```
HCL / TERRAFORM
# variables.tf
variable "location" {
  description = "Replaceable sample region for platform resources."
  type        = string
  default     = "westeurope"
}

variable "parent_management_group_id" {
  description = "Existing parent management group for the platform hierarchy."
  type        = string
}

variable "platform_subscription_id" {
  description = "Existing subscription used by the platform root provider."
  type        = string
}
```

Types catch some caller errors before a plan. Descriptions state why each value exists. Neither feature proves that a supplied identifier is correct. Context checks and review must supply that evidence.

The ownership map places the controller decision beside the root. Its three sets make managed, referenced, and excluded objects distinct. The map is useful review data, not a security control. Terraform does not enforce ownership from a string label.

```
HCL / TERRAFORM
# ownership.tf
locals {
  ownership = {
    managed_by_platform_root = toset([
      "management_group_hierarchy",
      "platform_governance_assignments",
      "platform_role_assignments",
      "shared_connectivity_configuration",
      "shared_management_configuration",
    ])

    referenced_not_managed = toset([
      "billing_relationship",
      "existing_platform_subscriptions",
      "microsoft_entra_tenant",
    ])

    outside_platform_root = toset([
      "application_workloads",
      "deployment_identity",
      "remote_state_foundation",
    ])
  }
}

output "ownership" {
  description = "Reviewed ownership boundary for the platform root."
  value       = local.ownership
}
```

Later resources must match the map. Permissions should let the deployment identity change the managed set and no more. State should bind the same objects. Plan review should flag any address outside the claim.

The root module is the directory where Terraform runs. Its files form one configuration even when they are split for readability. The root composes provider settings, inputs, outputs, and child modules. It also defines the state boundary used by those children.

The local platform child is only a contract skeleton here. It declares minimum tool needs and accepts two decisions from the root. It contains no provider block or Azure resource. Chapter 5.3 will fill the empty body with a selected implementation.

```
HCL / TERRAFORM
# main.tf
module "platform" {
  source = "../modules/platform"

  providers = {
    azurerm = azurerm
  }
}
```

```

    }

    location                = var.location
    parent_management_group_id = var.parent_management_group_id
  }

# modules/platform/terraform.tf
terraform {
  required_version = ">= 1.15.0"

  required_providers {
    azurerm = {
      source = "hashicorp/azurerm"
      version = ">= 4.81.0"
    }
  }
}

# modules/platform/variables.tf
variable "location" {
  description = "Region selected by the platform root."
  type        = string
}

variable "parent_management_group_id" {
  description = "Parent scope for the managed platform hierarchy."
  type        = string
}

# modules/platform/outputs.tf
output "configuration" {
  description = "Values accepted by the empty platform module skeleton."
  value = {
    location                = var.location
    parent_management_group_id = var.parent_management_group_id
  }
}

```

The root uses narrow upper bounds. The reusable child states only the minimum versions it needs. Terraform resolves one compatible provider version for the full configuration. Explicit provider passing shows which Azure context reaches the child.

The configuration output keeps the empty skeleton connected and testable. It is not a deployed platform result. Real outputs should expose only stable values that another system needs. A module should not leak its entire internal structure.

This skeleton can be checked without an Azure login. It has no backend block, resource block, or apply step. Formatting, backend-free initialization, validation, and linting inspect local structure. None of those checks changes Azure.

Local validation has strict limits. It confirms syntax, types, provider requirements, and module wiring. It cannot prove least privilege, correct target scope, policy acceptance, or safe Azure effects. A valid file can still encode the wrong decision.

The first plan with resources adds another layer. It should show only addresses claimed by the ownership model. Replacement or deletion deserves focused review. A plan that crosses the declared boundary fails even when its syntax is perfect.

Identity rights provide a second test. The deployment identity needs enough access for the planned set. Broad rights outside that set are not convenience; they are unused change paths. Least privilege should mirror Terraform ownership.

State review provides a third test. An object in state can change during apply even when the ownership map omits it. Such an orphan needs a named owner and a disposition. Routine release should not discover it by accident.

Independent checks complete the loop. They compare important Azure behavior with the accepted design after apply. Terraform reports what it attempted and recorded. Observation tests whether the remote system behaves as required.

One persistent object should have one declared controller and one accountable owner. Configuration, state, permissions, and evidence must agree on that claim. That ownership boundary is Terraform's first control.

Chapter 5: Ready as Code — Building the Platform with Terraform

Chapter 5.2: Bootstrap Authentication and Remote State Safely

A Terraform root cannot create the remote dependency it needs before initialization. A smaller authority must first create the Azure storage, protection, access, and network path. That exceptional step is bootstrap.

Bootstrap is not a hidden preface to routine deployment. It has a different owner and permission set. Its frequency and failure impact differ too. A separate boundary prevents the platform root from claiming that it created its own foundation.

The capstone places that foundation in the existing Management subscription. A dedicated resource group contains one general-purpose storage account and one private blob container. The platform root only references them; their lifecycle stays outside this root. That split is deliberate.

This separation creates a deliberate dependency. Bootstrap must finish before the root can initialize. Routine delivery must then work with less power. It should not need rights to create the account, protection settings, role assignment, or lock.

Remote state is a coordination service, not just a different file location. The AzureRM backend stores state as a blob. It uses Azure Blob Storage for locking and consistency checks. Terraform acquires the lock automatically before operations that could write state.

The lock prevents competing Terraform writers. It does not approve a plan or block a privileged Azure operator. It also provides no recovery copy. Concurrency control, access control, review, and recovery remain different protections.

A lock failure should stop the operation. Disabling locking with `-lock=false` creates a corruption risk and hides the signal. `force-unlock` is a recovery action, not a shortcut. Evidence must show that the named lock belongs to an abandoned Terraform operation.

State deserves the same care as a sensitive operational database. It can contain resource identifiers, outputs, dependency details, and provider values. Marking a Terraform value as `sensitive` can redact normal display. The value remains in state.

An unauthorized reader can learn about the platform. An unauthorized writer can damage Terraform's object bindings. Deletion can interrupt every later plan and apply. Confidentiality, integrity, and recovery all matter.

The capstone uses Microsoft Entra authorization for blob data. It disables Shared Key authorization. The deployment principal receives Storage Blob Data Contributor only on the state container. Its platform permissions will be granted separately at the scopes it manages.

Backend authentication and provider authentication solve different problems. The backend opens state before Terraform can load the provider configuration. The AzureRM provider later calls Azure management APIs for planned resources. A successful backend login says nothing about the provider's change authority.

One federated principal can keep the capstone understandable. Its two permission sets still need separate review. One covers container data actions; the other covers narrow management actions. Larger systems may split the paths when the isolation justifies the operating cost.

The account blocks anonymous blob access and requires HTTPS and TLS 1.2. It also denies network traffic unless a reviewed rule permits it. The example accepts a runner IPv4 address or CIDR. A production design can replace that rule with private connectivity and a controlled agent.

Blob versioning preserves earlier versions after writes. Blob and container soft delete add retention after deletion. A `CanNotDelete` Azure Resource Manager lock protects the storage account resource. These controls overlap because they cover different loss paths.

Version history can increase storage cost, although a state blob is normally small. A production retention and lifecycle policy should follow the recovery requirement. A generic cleanup rule is not enough. It must not delete the only usable recovery point.

Protection also needs a tested restoration path. Versioning makes previous data available. Selecting and promoting the correct version remains an operational decision. Chapter 5.5 places that choice inside a controlled recovery workflow.

The bootstrap begins with a fail-closed preflight. It checks the tested Terraform line and minimum Azure CLI version. It also checks the active tenant and subscription. The script reports intent and returns unless `-Apply` is present.

```
POWERSHELL
# bootstrap-state.ps1
[CmdletBinding()]
param(
    [Parameter(Mandatory)]
    [guid] $TenantId,

    [Parameter(Mandatory)]
    [guid] $StateSubscriptionId,

    [Parameter(Mandatory)]
    [guid] $DeploymentPrincipalObjectId,

    [Parameter(Mandatory)]
    [ValidatePattern('^[a-z0-9]{3,24}$')] $RunId
```

```

[string] $StorageAccountName,

[Parameter(Mandatory)]
[ValidatePattern('^(?:[0-9]{1,3}\.){3}[0-9]{1,3}(?:/[0-9]{1,2})?$',)]
[string] $AllowedIpCidr,

[string] $ResourceGroupName = 'rg-tfstate-platform-weu',

[ValidatePattern('^[a-z0-9](?:[a-z0-9-]{1,61}[a-z0-9])?$',)]
[string] $ContainerName = 'tfstate',

[string] $Location = 'westeurope',

[switch] $Apply
)

Set-StrictMode -Version Latest
$ErrorActionPreference = 'Stop'
$PSNativeCommandUseErrorActionPreference = $true

$null = Get-Command -Name terraform -ErrorAction Stop
$null = Get-Command -Name az -ErrorAction Stop

$terraformInfo = terraform version -json | ConvertFrom-Json
$terraformVersion = [version] $terraformInfo.terraform_version
if ($terraformVersion -lt [version] '1.15.0' -or
    $terraformVersion -ge [version] '1.16.0') {
    throw "Terraform 1.15.x is required; found $terraformVersion."
}

$azureCliInfo = az version | ConvertFrom-Json
$azureCliVersion = [version] $azureCliInfo.'azure-cli'
if ($azureCliVersion -lt [version] '2.88.0') {
    throw "Azure CLI 2.88.0 or later is required; found $azureCliVersion."
}

$context = az account show --only-show-errors --output json |
    ConvertFrom-Json
if ($context.tenantId -ne $TenantId.ToString() -or
    $context.id -ne $StateSubscriptionId.ToString() -or
    $context.state -ne 'Enabled') {
    throw 'Azure CLI context does not match the enabled bootstrap scope.'
}

$intent = [ordered]@{
    tenantId          = $TenantId
    subscriptionId    = $StateSubscriptionId
    resourceGroup     = $ResourceGroupName
    storageAccount    = $StorageAccountName
    container         = $ContainerName
    allowedIpCidr     = $AllowedIpCidr
    deploymentObjectId = $DeploymentPrincipalObjectId
    applyRequested    = $Apply.IsPresent
}
$intent | ConvertTo-Json

if (-not $Apply) {
    return
}

az group create `
    --name $ResourceGroupName `
    --location $Location `
    --output none

az storage account create `

```

```

--name $StorageAccountName `
--resource-group $ResourceGroupName `
--location $Location `
--kind StorageV2 `
--sku Standard_LRS `
--https-only true `
--min-tls-version TLS1_2 `
--allow-blob-public-access false `
--allow-shared-key-access false `
--public-network-access Enabled `
--default-action Deny `
--bypass None `
--output none

az storage account network-rule add `
--account-name $StorageAccountName `
--resource-group $ResourceGroupName `
--ip-address $AllowedIpCidr `
--output none

az storage account blob-service-properties update `
--account-name $StorageAccountName `
--resource-group $ResourceGroupName `
--enable-versioning true `
--enable-delete-retention true `
--delete-retention-days 30 `
--enable-container-delete-retention true `
--container-delete-retention-days 30 `
--output none

$storageId = az storage account show `
--name $StorageAccountName `
--resource-group $ResourceGroupName `
--query id `
--output tsv

az storage container-rm create `
--storage-account $storageId `
--name $ContainerName `
--public-access off `
--output none

$containerScope = "$storageId/blobServices/default/containers/$ContainerName"
az role assignment create `
--assignee-object-id $DeploymentPrincipalObjectId `
--assignee-principal-type ServicePrincipal `
--role 'Storage Blob Data Contributor' `
--scope $containerScope `
--output none

az lock create `
--name 'lock-state-foundation' `
--lock-type CanNotDelete `
--resource-group $ResourceGroupName `
--resource-name $StorageAccountName `
--resource-type 'Microsoft.Storage/storageAccounts' `
--output none

```

The script uses Azure CLI because remote state does not exist yet. Its commands make persistent changes. The exception is explicit, parameterized, and gated. The storage account name and network range must be reviewed before use.

The bootstrap operator needs rights to create these resources and assign the data role. The deployment principal needs none of those bootstrap rights. Role assignment propagation can

take several minutes. An initial 403 should trigger a wait and recheck, not a switch to an account key.

The script is not a general storage-account factory. An unchecked rerun could combine an old account with a new network rule. It could also change retention in an unexpected way. First compare actual evidence with the recorded foundation. Then apply only an approved correction. The first run should preserve its intent record and Azure activity evidence. Reviewers should confirm the account settings, container scope, network rule, and resource lock. They should also record the bootstrap operator and deployment principal. That evidence turns a one-time exception into an auditable foundation.

The main root declares an empty backend block. Storage coordinates arrive during initialization because backend blocks cannot use Terraform input variables. The file contains identifiers and behavior. It never contains an access key, SAS token, client secret, or OpenID Connect token.

```
HCL / TERRAFORM
# backend.tf
terraform {
  backend "azurerm" {}
}

# backend.hcl
use_azuread_auth      = true
storage_account_name = "replacewithgloballyunique"
container_name        = "tfstate"
key                   = "platform-foundation.tfstate"
```

The standard storage endpoint can be inferred from the account name. No management-plane lookup is needed. The backend also needs no Reader role on the storage account. Container-scoped data access is enough for normal state operations.

The state key is also a boundary. Different roots must not share one key merely because they use the same container. Separate keys prevent unrelated roots from competing for one lock or state lineage. High-impact roots may still deserve separate accounts or containers.

Local engineering uses a preauthenticated Azure CLI session. The exact tenant remains explicit, while the account key stays disabled. An active CLI identity may be allowed to change state. If so, it needs the same container data permission.

```
POWERSHELL
$env:ARM_USE_AZUREAD = 'true'
$env:ARM_USE_CLI = 'true'
$env:ARM_TENANT_ID = $TenantId
Remove-Item Env:ARM_USE_OIDC -ErrorAction SilentlyContinue
Remove-Item Env:ARM_CLIENT_SECRET -ErrorAction SilentlyContinue

terraform init -backend-config='backend.hcl' -reconfigure
```

Noninteractive delivery uses workload identity federation where the platform supports it. An Azure DevOps service connection lets the task request a short-lived token. The task supplies

the token endpoint context. The script maps only stable identifiers and authentication-mode flags.

Federation replaces a stored client secret with a trust rule. That rule covers issuer, subject, and audience. The workload presents an identity token for exchange by Microsoft Entra ID. Its claim set must match the trust rule. Short lifetime helps, but broad trust or Azure roles still create excess authority.

```
POWERSHELL
$required = @(
    'DEPLOYMENT_TENANT_ID',
    'DEPLOYMENT_CLIENT_ID',
    'AZURESUBSCRIPTION_SERVICE_CONNECTION_ID'
)
foreach ($name in $required) {
    $value = [Environment]::GetEnvironmentVariable($name)
    if ([string]::IsNullOrEmpty($value)) {
        throw "Required pipeline value is missing: $name"
    }
}

$env:ARM_USE_AZUREAD = 'true'
$env:ARM_USE_OIDC = 'true'
$env:ARM_TENANT_ID = $env:DEPLOYMENT_TENANT_ID
$env:ARM_CLIENT_ID = $env:DEPLOYMENT_CLIENT_ID
$env:ARM_OIDC_AZURE_SERVICE_CONNECTION_ID =
    $env:AZURESUBSCRIPTION_SERVICE_CONNECTION_ID
Remove-Item Env:ARM_USE_CLI -ErrorAction SilentlyContinue
Remove-Item Env:ARM_CLIENT_SECRET -ErrorAction SilentlyContinue

terraform init -backend-config='backend.hcl' -reconfigure
```

Tenant IDs, client IDs, and service-connection IDs describe trust; they are not client secrets.

The issued token is still a bearer credential. It must not appear in command tracing, pipeline logs, or backend arguments. Plan artifacts and committed files must not contain it either.

Backend credentials supplied through arguments can reach `.terraform` metadata and saved plans. Authentication therefore comes from the execution environment. The backend file carries only nonsecret coordinates. Its environment-specific copy stays untracked to reduce wrong-target initialization.

```
.GITIGNORE
.terraform/
*.tfstate
*.tfstate.*
*.tfplan
backend.hcl
crash.log
crash.*.log
```

The dependency lock file stays outside that ignore list. Terraform 1.15.x and AzureRM 4.81.x remain constrained in code. `.terraform.lock.hcl` records the selected provider package and checksums. The AzureRM backend ships with Terraform, so it has no provider version of its own.

Initialization should use the reviewed Terraform CLI line. Backend behavior comes from that executable. A provider lock file cannot constrain the backend implementation or Azure CLI. The preflight makes those version assumptions visible before any exceptional change.

Remote state can still leave local traces. A failed backend write can produce a local recovery state. A saved plan also contains backend configuration and proposed values. Both artifacts need restricted handling, deliberate retention, and timely deletion.

Initialization proves only that Terraform can configure and reach the backend. The identity's platform permissions remain unproven. It cannot show that a later apply is safe. Those controls belong to the plan and delivery path.

Safe bootstrap ends by removing creation power and recording who owns the foundation. Routine automation no longer needs a storage key or client secret. It has no broad state scope or authority to recreate its trust base. The safest foundation is defined as much by withdrawn privilege as by created resources.

Chapter 5: Ready as Code — Building the Platform with Terraform

Chapter 5.3: Use Azure Verified Modules for the Platform Landing Zone

An accepted architecture still leaves many Azure objects to describe. Management groups, policy assets, role assignments, and subscription placement use different APIs. Building every layer from scratch would make the capstone own those details.

Azure Verified Modules, or AVM, provide reviewed building blocks with published interfaces and versioned releases. The Azure landing-zone pattern module covers hierarchy, policy assets, policy role assignments, and subscription placement. It reduces custom code, but it cannot approve an architecture or its plan.

The verification label gives a team a maintained source, tests, release history, and a known interface. That shared base reduces local design and upkeep. It is most useful for common Azure plumbing that can change behind stable intent.

Verified modules are not universal. The module supports choices this capstone rejected as well as choices it accepted. The platform team still owns scope, inputs, permissions, cost, and upgrades.

Microsoft also publishes the Azure landing-zone infrastructure-as-code accelerator. It can bootstrap repositories, state, workload identities, pipelines, and starter code for Azure DevOps or GitHub. That wider scaffold is useful when a delivery foundation is absent.

This capstone composes the AVM pattern module directly. Chapter 5.2 already made bootstrap and authentication explicit. Chapter 5.4 will give the pipeline the same treatment.

Direct composition keeps those control boundaries visible. It also matches the accepted hierarchy without replacing the delivery model. A team may choose the accelerator when its generated assets and operating model fit better.

That choice changes ownership, not just setup time. Accelerator output becomes production code that someone must maintain. Direct composition leaves those delivery assets with their later owners.

Four release lines now interact. Terraform CLI reads the language, providers translate operations, AVM defines the module interface, and the ALZ library supplies policy assets. A provider lock file cannot pin the module or the policy library.

The root constrains Terraform 1.15, ALZ provider 0.21, AzAPI 2.11, and AzureRM 4.81. The child call pins AVM 0.21.0 exactly. Each line can then move through its own reviewed upgrade.

Initialization also selects transitive providers used inside AVM. The dependency lock file records their exact packages and checksums. Committing that file keeps another runner from making a different allowed selection.

The root `terraform.tf` file adds the providers used by the verified module. AzureRM stays for later platform resources. The ALZ module creates its hierarchy and policy objects through AzAPI.

```
HCL / TERRAFORM
terraform {
  required_version = "~> 1.15.0"

  required_providers {
    alz = {
      source = "Azure/alz"
      version = "~> 0.21.0"
    }
    azapi = {
      source = "Azure/azapi"
      version = "~> 2.11.0"
    }
    azurerm = {
      source = "hashicorp/azurerm"
      version = "~> 4.81.0"
    }
  }
}
```

The ALZ provider reads a library before Terraform can finish its plan. The first reference pins Microsoft's platform library to `2026.04.2`. The second overlays the capstone's custom files. Provider configuration remains in the root. Its custom path is known before planning. The existing Management subscription supplies the AzureRM context for later shared resources.

```
HCL / TERRAFORM
provider "alz" {
  library_references = [
    {
      path = "platform/alz"
      ref = "2026.04.2"
    },
    {
      custom_url = "${path.root}/modules/platform/lib"
    }
  ]
}

provider "azapi" {}

provider "azurerm" {
  features {}

  subscription_id = var.platform_subscription_ids.management
}
```

No credential or tenant ID appears in the provider file. Authentication still comes from the run context defined in Chapter 5.2. A valid provider setup cannot widen the controller's approved scope.

The provider downloads a local `.alzlib` cache. That generated directory belongs in `.gitignore`. The custom library under `modules/platform/lib` stays committed because it records design intent.

The three subscription IDs arrive as one typed object. They refer to existing Identity, Connectivity, and Management subscriptions. This root only references them; it never creates, renames, or cancels them.

UUID validation rejects malformed values before provider work. It cannot prove that an ID belongs to the right tenant. Target checks and plan review must still confirm that fact.

The location remains the replaceable `westeurope` sample. Here it supplies a region for policy identities when a policy needs one. Later Azure resources remain free to use an approved region.

```
HCL / TERRAFORM
variable "location" {
  description = "Replaceable sample region for policy assignment identities."
  type        = string
  default     = "westeurope"
}

variable "parent_management_group_id" {
  description = "Existing parent for the managed platform-foundation hierarchy."
  type        = string
}

variable "platform_subscription_ids" {
  description = "Existing subscriptions placed under the Platform management group."
  type = object({
    identity      = string
    connectivity  = string
    management    = string
  })

  validation {
    condition = alltrue([
      for id in values(var.platform_subscription_ids) :
      can(regex("^[0-9a-fA-F]{8}(?:-[0-9a-fA-F]{4}){3}-[0-9a-fA-F]{12}$", id))
    ])
    error_message = "Each platform subscription ID must be a UUID."
  }
}
```

The local platform child remains the root's composition boundary. Its provider map includes only ALZ and AzAPI. AzureRM stays at the root because this child has no use for it.

```
HCL / TERRAFORM
module "platform" {
  source = "../modules/platform"

  providers = {
    alz      = alz
    azapi    = azapi
  }

  location              = var.location
  parent_management_group_id = var.parent_management_group_id
  platform_subscription_ids = var.platform_subscription_ids
}
```

Inside the child, a fixed-key map sends all three subscriptions to Platform. Its keys and values are known from input before the plan. That timing matters to the ALZ data read.

The module warns against a generic `depends_on` on its call. Values known only after apply can also make planning unstable. Module-specific dependency inputs or provider functions should carry a real late dependency.

This constraint shapes future code. A fresh resource output may arrive too late for the library read. Known inputs and narrow dependency channels keep the result predictable.

The registry source pins AVM 0.21.0 exactly. Destroy behavior is also explicit. Removing placement moves the existing subscriptions back to the supplied existing parent.

```
HCL / TERRAFORM
locals {
  subscription_placement = {
    for name, subscription_id in var.platform_subscription_ids :
    name => {
      subscription_id      = subscription_id
      management_group_name = "platform"
    }
  }
}

module "management_groups_and_policy" {
  source      = "Azure/avm-ptn-alz/azurerem"
  version     = "0.21.0"

  providers = {
    alz    = alz
    azapi  = azapi
  }

  architecture_name = "platform_foundation"
  location           = var.location
  parent_resource_id = var.parent_management_group_id

  enable_telemetry = false

  subscription_placement = local.subscription_placement
  subscription_placement_destroy_behavior = "custom"
  subscription_placement_destroy_custom_target_management_group_id = var.parent_management_group_id
}
```

The custom target avoids an unknown tenant default. Destroy remains a high-risk action. Its plan would move subscriptions and remove governance, so it needs focused review.

Creation also needs strong rights. The deployment principal may need management-group, policy, role-assignment, and subscription-placement actions shown by the plan. The module neither grants those rights nor proves least privilege.

Review should start with planned addresses and API actions. Tenant-wide Owner is an unsafe default. The access boundary from Chapter 4.3 still limits this controller.

The default ALZ architecture stays out of this capstone. It adds levels and policy choices outside the accepted design. A custom library is required when the hierarchy changes.

The architecture file defines the full managed tree below the existing parent. Platform Foundation is the intermediate root. Platform, Landing Zones, Sandbox, and Decommissioned are its only children.

```
YAML
name: platform_foundation
management_groups:
- archetypes:
  - platform_foundation_root
  display_name: Platform Foundation
  exists: false
  id: platform-foundation
  parent_id: null
- archetypes: []
  display_name: Platform
  exists: false
  id: platform
  parent_id: platform-foundation
- archetypes: []
  display_name: Landing Zones
  exists: false
  id: landing-zones
  parent_id: platform-foundation
- archetypes: []
  display_name: Sandbox
  exists: false
  id: sandbox
  parent_id: platform-foundation
- archetypes: []
  display_name: Decommissioned
  exists: false
  id: decommissioned
  parent_id: platform-foundation
```

The `id` values are Azure resource names, while display names are reader-facing labels. Changing an ID after deployment can alter remote identity and Terraform's intended object. A rename therefore needs migration review, even when a diagram looks unchanged.

The design creates no Identity, Connectivity, or Management child group. Their subscriptions sit directly under Platform. It also creates no workload subscription.

Nothing enters Landing Zones, Sandbox, or Decommissioned in this section. Those empty branches reserve the accepted shape. A branch name alone provides no admission control.

Empty archetype lists still inherit policy from Platform Foundation. Stronger branch rules must be approved before a subscription enters Sandbox or Decommissioned. Until then, those paths remain unused.

Inheritance lets one assignment reach several branches without duplicate code. It also increases the impact of a bad root choice. Root policy changes need evidence from every child scope they can affect.

The custom root archetype selects only three policy assignments from the pinned library. One audits resource and resource-group location mismatch. Two deny classic resources and virtual machines with unmanaged disks.

YAML

```
name: platform_foundation_root
policy_assignments:
  - Audit-ResourceRGLocation
  - Deny-Classic-Resources
  - Deny-UnmanagedDisk
```

This set creates a small inherited baseline. It deploys no monitoring agent, Defender plan, DDoS protection, network hub, or paid service. It is a capstone starting point, not a full enterprise baseline.

The local archetype is intentionally small. New controls should enter through a reviewed library change, not an ad hoc portal assignment. Policy selection then stays inside the same desired-state boundary.

Policy mode still matters. Audit records a mismatch, while Deny can block a new request. The first Azure plan must expose each definition, effect, scope, and inherited reach.

Existing objects need the same care. A matching management-group ID may already exist outside this state. The team must reconcile that object before it accepts a planned create.

Using library assets avoids copying policy JSON into the repository. The library pin freezes the reviewed content for repeatable plans. An update must compare definitions, parameters, effects, and assignments before the reference changes.

Local checks can format and validate HCL, resolve the module, inspect provider schemas, and parse the custom library. They cannot prove permissions, policy behavior, placement, or the Azure plan. Those claims need an authenticated nonproduction plan and later observation.

A useful plan review groups changes by management groups, policy assets, role assignments, and subscription moves. Reviewers should treat replacements and scope changes as separate risks. A raw resource count offers too little evidence.

Only stable management-group resource IDs should leave the child. Its internal resource graph remains private. That small interface limits upgrade coupling.

Every accepted platform decision now has a module input or committed library asset. The configuration knows what hierarchy, policy base, and subscription placement to propose. It has not yet earned a controlled path to apply.

Chapter 5: Ready as Code — Building the Platform with Terraform

Chapter 5.4: Deliver Infrastructure Through a Safe Pipeline

Terraform can pass every local check and still reach Azure through an unsafe path. A developer might target the wrong tenant, approve one result, and apply another. Delivery must keep intent, evidence, approval, and execution joined.

The pipeline is a controller around the Terraform controller. A commit enters as proposed change, stages produce evidence, and a reviewer decides whether it may continue. Azure and state provide the result for later checks.

Pull requests cross the first boundary without cloud access. Formatting, language validation, and configuration tests run without a backend or Azure identity. Untrusted code needs no token to prove that its local files are sound.

The jobs use a fixed `ubuntu-24.04` image and a clean, shallow checkout. They do not keep repository credentials. A shared template installs Terraform 1.15.8 and checks its SHA-256 digest against HashiCorp's release file.

That pin prevents a hosted image from choosing a new CLI release. The dependency lock file fixes provider packages, while module declarations fix direct module releases. Each control protects a different link in the tool chain.

Early stages ask different questions. `terraform fmt -check` enforces canonical HCL. `terraform validate` checks references, types, and internal configuration rules.

Backend-free initialization installs the locked dependencies needed for validation. It neither opens remote state nor calls Azure for managed resources. Read-only lock mode fails if the committed selections cannot satisfy the configuration.

Every pull request and main-branch update runs these stages. Branch policy requires their result before main moves. Evidence from an older commit cannot stand in for the merged one.

Stage order saves time and keeps failures clear. Format precedes validation, and validation precedes tests. A later result never hides a cheaper earlier defect.

```
YAML
stages:
- stage: Format
  jobs:
  - job: Format
    pool:
      vmImage: ubuntu-24.04
    steps:
    - checkout: self
      clean: true
      fetchDepth: 1
      persistCredentials: false
```

```

- template: templates/terraform-setup.yml
- pwsh: |
    terraform "-chdir=$env:TERRAFORM_ROOT" `
      fmt -check -recursive -no-color
  env:
    TERRAFORM_ROOT: ${terraformRoot}

- stage: Validate
  dependsOn: Format
  jobs:
    - job: Validate
      pool:
        vmImage: ubuntu-24.04
      steps:
        - checkout: self
          clean: true
          fetchDepth: 1
          persistCredentials: false
        - template: templates/terraform-setup.yml
        - pwsh: |
            terraform "-chdir=$env:TERRAFORM_ROOT" `
              init -backend=false -input=false -lockfile=readonly -no-color
            terraform "-chdir=$env:TERRAFORM_ROOT" validate -no-color
          env:
            TERRAFORM_ROOT: ${terraformRoot}

```

Validation does not test design intent. It can accept a well-typed expression that sends subscriptions to the wrong group. A test supplies a known case and asserts the required relationship.

Terraform tests can use real providers and create real resources. This test mocks ALZ and AzAPI, then overrides the AVM call. It checks local mapping logic without credentials or provider API calls.

```

HCL / TERRAFORM
mock_provider "alz" {}
mock_provider "azapi" {}

override_module {
  target = module.management_groups_and_policy
  outputs = {
    management_group_resource_ids = {}
  }
}

run "maps_existing_platform_subscriptions" {
  command = plan

  variables {
    location = "westeurope"
    parent_management_group_id = "tenant-root"
    platform_subscription_ids = {
      identity = "identity-test"
      connectivity = "connectivity-test"
      management = "management-test"
    }
  }
}

assert {
  condition = alltrue([
    for placement in values(local.subscription_placement) :
    placement.management_group_name == "platform"
  ])
  error_message = "Every existing platform subscription must map to Platform."
}

```

```

assert {
  condition = (
    local.subscription_placement.identity.subscription_id ==
    "identity-test"
  )
  error_message = "The Identity subscription mapping must preserve its input."
}
}

```

This is a configuration test, not an Azure integration test. It proves the mapping for controlled inputs while replacing remote behavior. The canary run must still resolve real providers, state, permissions, library data, and Azure objects.

Tests should state approved behavior. Other cases could verify a destroy reparent target or reject invalid input. Repeating the implementation inside an assertion would add little evidence.

The PowerShell tests cover failure paths too. They remove the token and alter the tenant, commit, working path, or archive digest. Every case confirms that failed checks stop before Azure CLI or Terraform can run.

Each evidence level has a limit. Static checks inspect files, and mocked tests inspect local behavior. Only the canary observes the real integration.

Only main can create an authenticated canary plan. Branch policy must make main a reviewed branch, not just a name. Pull-request jobs receive no deployment variable group, service connection, or backend coordinates.

Each service connection is authorized only for this pipeline. The canary variable group uses the same narrow resource permissions. Repository write access alone cannot request an Azure deployment token.

The planning job uses workload identity federation. [AzureCLI@2](#) exposes a short-lived `idToken`, client ID, and tenant ID to its PowerShell process. No client secret enters the repository, YAML, or command arguments.

Planning and apply have separate service connections. The first identity needs state data rights and Azure read access. The second carries approved write rights.

Both identities still need the same target boundary. The script compares the injected tenant with the expected tenant and reads the active Azure CLI subscription. A valid login to the wrong canary target fails before initialization.

Separate identities do not create safety by themselves. Their Azure roles must match their jobs, and the apply role must stay narrow. The proposed operations show whether those scopes are sufficient.

```

POWERSHELL
if ($env:tenantId -ne $env:EXPECTED_TENANT_ID) {
  throw 'Federated identity tenant does not match the canary target.'
}
$subscriptionId = az account show --query id --output tsv

```



```

if ($LASTEXITCODE -ne 0 -or
    $subscriptionId -ne $env:EXPECTED_SUBSCRIPTION_ID) {
    throw 'Azure CLI subscription does not match the canary target.'
}

$env:ARM_USE_AZUREAD = 'true'
$env:ARM_USE_OIDC = 'true'
$env:ARM_USE_CLI = 'false'
$env:ARM_CLIENT_ID = $env:servicePrincipalId
$env:ARM_TENANT_ID = $env:tenantId
$env:ARM_SUBSCRIPTION_ID = $subscriptionId
$env:ARM_OIDC_TOKEN = $env:idToken
$env:TF_IN_AUTOMATION = 'true'
Remove-Item Env:ARM_CLIENT_SECRET -ErrorAction SilentlyContinue

```

The saved plan is the center of review. It is non-speculative because Terraform can later execute its recorded operations. Apply never creates a replacement result behind the approval gate.

`-detailed-exitcode` separates error, empty result, and changes. Code 2 means a successful run with changes. The script publishes that fact as a stage output.

The binary file is executable evidence, not readable evidence. `terraform show` creates the review copy. Reviewers check addresses, scopes, replacement, deletion, access, policy effects, and subscription moves.

```

POWERSHELL
terraform plan -input=false -lock-timeout=5m -no-color `
    -detailed-exitcode -out=$plan
$planExitCode = $LASTEXITCODE
if ($planExitCode -notin 0, 2) {
    throw 'Terraform plan failed.'
}

terraform show -no-color $plan |
    Set-Content -LiteralPath (Join-Path $ArtifactDirectory 'plan.txt')
if ($LASTEXITCODE -ne 0) {
    throw 'Terraform show failed.'
}

$hasChanges = ($planExitCode -eq 2).ToString().ToLowerInvariant()
Write-Output `
    "##vso[task.setvariable variable=hasChanges;isOutput=true]$hasChanges"

```

The reviewer also checks provenance. The run, commit, backend key, target IDs, and tool line must agree. A clear diff against the prior accepted result helps expose surprise.

This evidence ages. Terraform rejects a saved file when its state lineage has advanced, but an outside Azure change might not update state. Long delays call for a fresh run and decision.

The binary can hold configuration, state details, variables, and sensitive values. Terminal redaction does not clean the file. Artifact readers and retention must therefore be restricted.

The text form also reveals platform detail. Its audience must be wide enough for independent judgment but not open to every build reader. Hashes protect identity inside the run, not confidentiality.

The commit alone is not the whole planning environment. Saved files can depend on absolute paths, plugins, module caches, and generated files. Execution must reproduce those inputs too.

The planning job records the commit and absolute path. It hashes the executable file, then archives and hashes the initialized working directory. Apply rejects a mismatch and restores the archive at that same path.

```
POWERSHELL
$plan = Join-Path $ArtifactDirectory 'tfplan'
$workingDirectoryArchive = Join-Path $ArtifactDirectory `
    'working-directory.tgz'
$plannedVersion = Get-Content -Raw -LiteralPath `
    (Join-Path $ArtifactDirectory 'source-version.txt')
if ($plannedVersion.Trim() -ne $env:BUILD_SOURCEVERSION) {
    throw 'The saved plan was created from a different commit.'
}
$plannedDirectory = Get-Content -Raw -LiteralPath `
    (Join-Path $ArtifactDirectory 'working-directory.txt')
if ($plannedDirectory.Trim() -ne (Get-Location).Path) {
    throw 'The saved plan requires the original working directory.'
}
$expectedPlanHash = Get-Content -Raw -LiteralPath `
    (Join-Path $ArtifactDirectory 'plan-sha256.txt')
$actualPlanHash = (Get-FileHash -Algorithm SHA256 $plan).Hash
if ($expectedPlanHash.Trim() -ne $actualPlanHash) {
    throw 'The saved plan checksum does not match the reviewed artifact.'
}
$expectedArchiveHash = Get-Content -Raw -LiteralPath `
    (Join-Path $ArtifactDirectory 'working-directory-sha256.txt')
$actualArchiveHash = (Get-FileHash -Algorithm SHA256 `
    $workingDirectoryArchive).Hash
if ($expectedArchiveHash.Trim() -ne $actualArchiveHash) {
    throw 'The working-directory checksum does not match the plan run.'
}

tar -xzf $workingDirectoryArchive -C (Get-Location).Path
if ($LASTEXITCODE -ne 0) {
    throw 'The planned working directory could not be restored.'
}
```

The restored root initializes with its lock file read-only. This opens the backend without choosing new dependencies. Plan and apply share the operating system, architecture, Terraform release, source, and path.

Independent approval names a person who did not author the run. `ManualValidation@1` sits in an agentless job, blocks self-approval, and rejects on timeout. Its instructions require review of the named artifact and a change reference.

A checklist supports judgment but cannot replace it. Reviewers need room to reject unclear intent or missing evidence. Approval should record why high-impact actions are acceptable.

Repository YAML cannot be the final gate because its author can edit it. The canary environment also needs owner-managed approval and an exclusive lock. Those checks live on the Azure DevOps resource, outside the repository.

The exclusive lock and `lockBehavior: sequential` serialize pipeline runs. The backend lock serializes Terraform state writers. These controls solve concurrency, while approval solves intent.

An empty result skips approval and apply. It still records that refreshed state matched the reviewed commit at planning time. Changes continue to the guarded deployment job.

```
YAML
- stage: Apply
  dependsOn:
    - Plan
    - Approval
  condition: >-
    and(succeeded(), eq(variables['Build.SourceBranch'], 'refs/heads/main'),
    eq(dependencies.Plan.outputs['Plan.PlanResult.hasChanges'], 'true'))
  lockBehavior: sequential
  jobs:
    - deployment: ApplyCanary
      environment: platform-foundation-canary
      strategy:
        runOnce:
          deploy:
            steps:
              - task: DownloadPipelineArtifact@2
                inputs:
                  buildType: current
                  artifactName: terraform-plan
                  targetPath: $(Pipeline.Workspace)/terraform-plan
              - task: AzureCLI@2
                inputs:
                  azureSubscription: sc-platform-canary-apply
                  scriptType: pscore
                  scriptLocation: scriptPath
                  scriptPath: >-
                    $(Build.SourcesDirectory)/pipelines/scripts/terraform-run.ps1
                  arguments: >-
                    -Action Apply
                    -ArtifactDirectory "$(Pipeline.Workspace)/terraform-plan"
                  addSpnToEnvironment: true
                  visibleAzLogin: false
                  workingDirectory: $(terraformRoot)
```

The deployment uses the saved file, so no interactive prompt follows. Approval of that file is the authorization event. The apply identity cannot widen the recorded actions.

The canary is an isolated platform foundation with no workloads. It tests permissions, management groups, policy assignments, and subscription placement against Azure. A later environment gets a new result from its own state and inputs.

Isolation must exist beyond its YAML name. The parent, subscriptions, state key, service connections, and variables all need verified canary identifiers. A production subscription in those inputs would erase the safety boundary.

Canary scope reduces impact but cannot excuse destructive change. Review rejects unexplained deletion, replacement, access expansion, or movement. Post-apply checks compare Azure with the approved result before delivery expands.

Observation closes this delivery loop. A successful task is execution evidence, not outcome evidence. The next level of trust comes from actual Azure state.

Local validation parsed the YAML and PowerShell, linted the script, passed five fail-closed Pester cases, and passed the mocked Terraform test. It did not run Azure DevOps or authenticate

to Azure. Resource checks, service-connection scope, and the real plan still need canary evidence.

The path from commit to platform state is narrow. One reviewed commit produces one identified artifact, one independent decision, and one serialized apply. Only that saved plan may cross the delivery boundary.

Chapter 5: Ready as Code — Building the Platform with Terraform

Chapter 5.5: Control Drift, Upgrades, Imports, and Recovery

Terraform is reliable only while its ownership bindings survive change. Operators respond to incidents, Azure services normalize values, and dependencies release new versions. The controller must absorb that movement without treating every difference as an emergency.

Classification comes first. Drift is a difference between Terraform's recorded model and a remote object. It is not proof of an unauthorized act.

The source may be a portal edit, an approved emergency action, provider behavior, or service normalization. Detected drift is evidence about that difference. Accepted change is a decision about desired state and ownership.

A normal plan refreshes remote objects in memory. It then proposes how to make them match configuration. A refresh-only plan proposes how the recorded model and outputs could match remote changes.

Planning alone changes no Azure resource. Applying a normal plan changes remote objects, while applying a refresh-only result changes Terraform's record. Those effects need different decisions.

The capstone uses refresh-only mode only to produce a drift report. The script saves no executable plan and never runs apply. Exit code 2 means a nonempty difference was detected.

```
POWERSHELL
[CmdletBinding()]
param(
    [Parameter(Mandatory)]
    [string] $TerraformRoot,

    [Parameter(Mandatory)]
    [string] $ReportFile
)

Set-StrictMode -Version Latest
$ErrorActionPreference = 'Stop'
$PSNativeCommandUseErrorActionPreference = $false

$null = Get-Command -Name terraform -ErrorAction Stop

$versionInfo = terraform version -json | ConvertFrom-Json
$terraformVersion = [version] $versionInfo.terraform_version
if ($terraformVersion -lt [version] '1.15.0' -or
    $terraformVersion -ge [version] '1.16.0') {
    throw "Terraform 1.15.x is required; found $terraformVersion."
}

$reportDirectory = Split-Path -Parent $ReportFile
if (-not [string]::IsNullOrEmpty($reportDirectory)) {
    $null = New-Item -ItemType Directory -Force -Path $reportDirectory
}
```

```

$env:TF_IN_AUTOMATION = 'true'
$planOutput = terraform "-chdir=$TerraformRoot" plan `
    -refresh-only -detailed-exitcode -input=false `
    -lock-timeout=5m -no-color 2>&1
$planExitCode = $LASTEXITCODE
$planOutput | Set-Content -LiteralPath $ReportFile -Encoding utf8

switch ($planExitCode) {
    0 { $status = 'no-detected-drift' }
    2 { $status = 'detected-drift' }
    default { throw "Refresh-only plan failed with exit code $planExitCode." }
}

[pscustomobject]@{
    status      = $status
    report_file = (Resolve-Path -LiteralPath $ReportFile).Path
}

```

The report can expose platform structure and needs protected storage. Its identity needs backend access and enough Azure rights to read managed objects. It does not need apply permissions. A scheduled scan uses the same root, backend key, variables, and dependency locks as delivery. A green result outside that context proves little. A failed refresh is unknown, not evidence of no drift.

A difference opens triage. The record names the object, attribute, remote value, configured value, time, source, owner, and impact. It also links the incident or approved change when one exists.

Unauthorized drift normally returns to committed intent through a new reviewed plan. A valid emergency action normally becomes code and enters the same pipeline. Provider noise may require a configuration or provider fix first.

Triage ends with a named response. Revert restores committed intent, codify changes intent, and refresh-only accepts remote facts into the record. Deferral leaves an owned risk with a review time.

Applying refresh-only can be correct after a deliberate external action. It is never an automatic cleanup step. Updating Terraform's record belongs behind the state-change gate.

Existing resources create another ownership question. Import does not create a resource or discover its correct configuration. It binds one remote object to one Terraform address.

Configuration comes first. Its provider, subscription, name, location, tags, and lifecycle express the target. Planning then reveals the import and any changes that ownership would introduce.

```

HCL / TERRAFORM
resource "azurerm_resource_group" "platform_operations" {
    name     = var.platform_operations_resource_group_name
    location = var.location

    tags = {
        managed_by = "terraform"
        purpose    = "platform-operations"
    }
}

```

```

    lifecycle {
      prevent_destroy = true
    }
  }

import {
  to = azurerm_resource_group.platform_operations
  id =
"/subscriptions/${var.management_subscription_id}/resourceGroups/${var.platform_operations_resource_group_name}"
}

```

This example adopts an existing operations resource group in the Management subscription. Its ID must be known while planning, and AzureRM must support importing the type. The same remote object must not enter two addresses.

Discovery evidence precedes the import block. The operator confirms subscription, resource ID, tags, locks, access, and dependencies. A similar name does not establish identity.

Import does not authorize an overwrite. Unexpected tag deletion, replacement, or access change stops the adoption. The team corrects configuration, approves the effect, or abandons the import.

Configuration-driven import uses the usual plan and apply path. The block may remain as history or leave after the binding exists. Configuration stays authoritative for desired attributes. Refactoring creates a different identity problem. Moving a root resource into a child module changes its Terraform address. The Azure object may remain exactly where it is.

Without an address map, Terraform can read the change as destroy and create. A `moved` block records the old and new locations. The next plan should show only an address move.

```

HCL / TERRAFORM
module "operations" {
  source = "../modules/operations"

  location          = var.location
  resource_group_name = var.platform_operations_resource_group_name
}

moved {
  from = azurerm_resource_group.platform_operations
  to   = module.operations.azure_rm_resource_group.this
}

```

The map belongs with the refactor and its release. Removing it early breaks snapshots that still contain the earlier address. Shared modules should retain move history.

Review looks for the word moved and the absence of remote replacement. Any delete and create pair needs investigation. A matching name does not prove that Azure identity survived.

`terraform state mv` can alter the same binding imperatively. It remains useful for an exceptional migration. Committed movement is easier to review, test, and repeat.

Ownership may also leave this root. Deleting resource configuration normally proposes remote destruction. A `removed` block can instead forget the binding and preserve Azure.

HCL / TERRAFORM

```
removed {  
  from = azurerem_resource_group.platform_operations  
  
  lifecycle {  
    destroy = false  
  }  
}
```

This is a controller handoff, not a drift eraser. The new owner, access, code, and evidence path must already exist. Otherwise the resource becomes unmanaged.

Dependency upgrades create change without a portal edit. Terraform CLI, providers, AVM, and the ALZ library have separate release lines. Moving them together hides the source of surprise.

One review moves one release line. `terraform init -upgrade` selects allowed provider releases and updates the lock file. Remote modules require an explicit version change because that file does not lock them.

The review starts with release notes and compatibility rules. Format, validate, tests, and a speculative plan follow. The authenticated canary path supplies the integration evidence.

JSON

```
{  
  "change_id": "CHG-REPLACE-ME",  
  "one_release_line_per_change": true,  
  "component": {  
    "type": "provider",  
    "name": "hashicorp/azurerem",  
    "from": "4.81.0",  
    "to": "REPLACE_ME"  
  },  
  "evidence": {  
    "release_notes_reviewed": false,  
    "version_constraint_updated": false,  
    "dependency_lock_diff_reviewed": false,  
    "terraform_fmt_passed": false,  
    "terraform_validate_passed": false,  
    "terraform_test_passed": false,  
    "speculative_plan_reviewed": false,  
    "canary_plan_reviewed": false,  
    "canary_apply_observed": false,  
    "rollback_or_forward_fix_recorded": false  
  },  
  "approval": {  
    "owner": "REPLACE_ME",  
    "reviewer": "REPLACE_ME",  
    "decision": "pending"  
  }  
}
```

The checklist separates a requested version from proof that it works here. Providers can change defaults, validation, or planned actions. Modules can change internal resources and their public contract.

The lock-file diff should name only the intended provider and its checksums. A module diff should name only the approved release and related configuration. Unrelated dependency movement sends the upgrade back for repair.

A no-change plan is useful but not conclusive. New paths may appear only under later inputs or provider responses. Canary observation still matters.

Failed applies start a new control loop. Terraform reports the error, records completed operations, unlocks when it can, and exits. It does not roll back every remote effect.

Partial completion is not total failure. Some objects may match configuration, while others may be absent or retain older values. A fresh plan is the safest summary of that mixed result.

The team preserves logs, the approved artifact, current state evidence, and the upstream error. It stops competing writers and identifies completed operations. The repaired system then receives a new review.

The old saved plan is no longer enough. Some actions may already have happened, and remote facts may have moved. Reusing it can deepen the fault.

A stuck lock also needs proof. `force-unlock` is only for the team's own abandoned operation and its named lock. Unlocking an active writer can corrupt the record.

Backend write failure is rarer and more serious. Terraform may leave a local recovery file when it cannot persist to the backend. That sensitive artifact stays outside source and logs.

Capture current state before any repair. `terraform state pull` emits UTF-8 JSON from the configured backend. It converts the copy for the installed Terraform release.

That conversion cannot prove which CLI last wrote the remote snapshot. Lineage and serial still identify its family and sequence. Both values belong in the recovery record.

The script stores the document beside a manifest. SHA-256 detects changed bytes, while lineage, serial, and Terraform version add identity context. Verification cannot prove that the snapshot matches real Azure.

POWERSHELL

```
$stateOutput = terraform "-chdir=$TerraformRoot" state pull
if ($LASTEXITCODE -ne 0) {
    throw 'Terraform state pull failed.'
}

$stateText = $stateOutput -join [Environment]::NewLine
$document = Read-StateDocument -Text $stateText
$null = New-Item -ItemType Directory -Force -Path $BackupDirectory

$timestamp = (Get-Date).ToUniversalTime().ToString('yyyyMMddTHHmssfffZ')
$statePath = Join-Path $BackupDirectory "terraform-state-$timestamp.json"
$stateText | Set-Content -LiteralPath $statePath -Encoding utf8 -NoNewline

$manifestPath = "$statePath.manifest.json"
$manifest = [ordered]@{
    captured_utc      = (Get-Date).ToUniversalTime().ToString('o')
    state_file        = (Split-Path -Leaf $statePath)
    sha256            = (Get-FileHash -Algorithm SHA256 $statePath).Hash
    lineage           = [string] $document.lineage
    serial            = [long] $document.serial
    terraform_version = [string] $document.terraform_version
}
$manifest | ConvertTo-Json | Set-Content -LiteralPath $manifestPath `
```

State data can contain secrets even when normal output hides them. Backup access, storage, transfer, retention, and deletion need explicit controls. A checksum provides neither encryption nor authorization.

The current backup is required even when Blob versioning holds older copies. A previous version is only a candidate recovery point. Selection also needs the backend key, timeline, apply logs, and observed resources.

`terraform state push` overwrites remote state and is a last resort. Terraform normally blocks different lineage or an older serial. `-force` disables both protections and never belongs in a reusable script.

Backend-native version promotion also replaces the current blob. It needs stopped writers, independent approval, a protected current backup, and an isolated rehearsal. The next plan must explain every resulting action.

Recovery follows a strict arc: freeze, capture, compare, rehearse, restore, replan, and observe. It completes only after normal delivery resumes. Temporary files then follow the secure-deletion path.

Manual exceptions use the same discipline. Their records name scope, owner, reason, expiry, and the route back to code. An expired exception without reconciliation is unresolved drift.

The platform can now change without losing its ownership story. Drift becomes classified evidence, migrations preserve identity, upgrades isolate causality, and recovery protects history. Chapter 6 can test Azure without trusting the controller's account of itself.

Chapter 6: Observe and Test Azure with CLI and PowerShell

Chapter 6.1: Create a Safe Azure CLI Session

Observe and Test Azure with CLI and PowerShell begins with distrust. A deployment report cannot prove that Azure still matches its claims. Independent observation needs a controlled point of view.

The command session creates that view. Azure CLI holds cached authentication, a selected cloud, and a default subscription. A valid query in the wrong context can yield believable but false evidence.

Local tools shape the answer too. Azure CLI releases can change commands and output. Extensions follow separate release lines, while PowerShell handles arguments, streams, JSON, and failures.

A safe session states these inputs and tests them. Uncertainty stops the run before the first estate query. Nothing repairs a mismatch behind the operator's back.

Verification is stricter than convenience. A helper might select the first available subscription and continue. A guard compares facts with approved expectations and refuses every substitute. This chapter uses Azure CLI to communicate with Azure. PowerShell 7 supplies flow control, validation, and serialization. It does not turn these commands into Azure PowerShell or the `Az` module.

That distinction controls failure handling. PowerShell errors and Azure CLI exit codes are different signals. Confusing them can make a failed native command appear successful.

Versions are dependencies, not workstation trivia. The capstone requires PowerShell 7.4 or later and Azure CLI 2.88.0 or later. Those floors match its tested error model and command surface.

A minimum version is not an open upgrade policy. The companion environment should pin one reviewed release. The lower bound gives older operator machines a clear failure.

Newer tools still need review. They may satisfy the bound while changing behavior. Each evidence record names the versions that shaped it.

Authentication happens before preflight. A person can sign in interactively and complete multifactor authentication. Unattended work uses a managed identity, service principal, or federated workload identity.

User-password login is not an automation design. Current multifactor requirements interrupt unattended user sessions by intent. Workload identities keep that boundary explicit without weakening human controls.

Required privilege activation also comes first. Its scope and time window are separate session facts. Evidence can link the approved event without storing an access token.

The preflight never runs `az login`. It also avoids `az account set`. Either action would hide a material input from the observations that follow.

Approved inputs provide the expected cloud, tenant, subscription, and identity. The active account must match all four. A colorful prompt or familiar subscription name is not proof.

Persisted defaults are weak evidence. An old profile may retain a group, location, or subscription from other work. Later scripts still pass every material scope directly.

`az account show` reads the default subscription when none is supplied. That result matters only after comparison with the approved target. The subscription must also report an enabled state.

Cloud is part of identity. `AzureCloud` is wrong for a sovereign environment. A mismatch changes endpoints and the estate being examined.

Names alone are also unsafe selectors. Subscription names can repeat across tenants, while display labels can change. Immutable tenant and subscription identifiers decide the comparison.

The expected identity name supplies another exact check. It may represent a person or an automation principal. The returned identity type stays in the session record for review.

Every Azure CLI call passes through one JSON wrapper. The wrapper fixes output, checks the native exit code, and parses the response. Empty, null, or malformed JSON stops execution.

```
POWERSHELL
Set-StrictMode -Version Latest

function Invoke-AzJson {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [string[]] $Arguments
    )

    $ErrorActionPreference = 'Stop'
    $PSNativeCommandUseErrorActionPreference = $false

    $output = & az @Arguments --only-show-errors --output json 2>&1
    $exitCode = $LASTEXITCODE
    if ($exitCode -ne 0) {
        $commandName = ($Arguments | Select-Object -First 2) -join ' '
        throw "Azure CLI command 'az $commandName' failed with exit code $exitCode."
    }

    $json = $output -join [Environment]::NewLine
    if ([string]::IsNullOrEmpty($json)) {
        throw 'Azure CLI returned an empty response where JSON was required.'
    }
}
```

```

try {
    $document = $json | ConvertFrom-Json -Depth 100
}
catch {
    throw 'Azure CLI returned invalid JSON.'
}

if ($null -eq $document) {
    throw 'Azure CLI returned null where an object or array was required.'
}

return $document
}

```

PowerShell stores a native program's status in `$LASTEXITCODE`. A nonzero value does not raise a catchable exception by default. The wrapper reads that value at once and throws its own error. Native error conversion stays under explicit control. The wrapper disables `$PSNativeCommandUseErrorActionPreference` inside its scope. Automatic conversion cannot interrupt the direct status check.

Standard error joins standard output before parsing. Unexpected diagnostic text then breaks the JSON contract. It cannot slip quietly into an evidence file.

`--only-show-errors` lowers that risk but proves nothing. It changes neither the exit code nor the response schema. The wrapper remains responsible for both.

JMESPath selection occurs inside Azure CLI after it receives JSON. PowerShell then parses the smaller projection. Each layer gets one visible job, and less data needs protection.

Failure messages leave out the raw CLI response. Debug text can reveal endpoints, identifiers, and request details. Deeper troubleshooting belongs in a protected diagnostic run.

Normal evidence collection never adds `--debug`. That flag produces a far wider execution trace.

A failed run stays failed until controlled investigation resolves it.

The main function checks the ambient session. It changes no resource and discovers no estate data. Its only account operation reads the current subscription context.

```

POWERSHELL
function Test-AzureCliSession {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [guid] $ExpectedTenantId,

        [Parameter(Mandatory)]
        [guid] $ExpectedSubscriptionId,

        [Parameter(Mandatory)]
        [ValidateNotNullOrEmpty()]
        [string] $ExpectedIdentityName,

        [ValidateNotNullOrEmpty()]
        [string] $ExpectedCloudName = 'AzureCloud',

        [version] $MinimumAzureCliVersion = '2.88.0',

```

```

        [hashtable] $RequiredExtensions = @{
            'resource-graph' = '2.1.1'
        }
    )

    $ErrorActionPreference = 'Stop'

    if ($PSVersionTable.PSEdition -ne 'Core' -or
        $PSVersionTable.PSVersion -lt [version] '7.4.0') {
        throw 'PowerShell 7.4 or later is required.'
    }

    $null = Get-Command -Name az -ErrorAction Stop

    $env:AZURE_CORE_COLLECT_TELEMETRY = 'false'
    $env:AZURE_CORE_ONLY_SHOW_ERRORS = 'true'
    $env:AZURE_CORE_OUTPUT = 'json'
    $env:AZURE_EXTENSION_USE_DYNAMIC_INSTALL = 'no'

    $versionDocument = Invoke-AzJson -Arguments @('version')
    $versionText = [string] $versionDocument.'azure-cli'
    if ([string]::IsNullOrEmpty($versionText)) {
        throw 'Azure CLI did not report its version.'
    }

    try {
        $azureCliVersion = [version] $versionText
    }
    catch {
        throw "Azure CLI reported an invalid version: $versionText."
    }

    if ($azureCliVersion -lt $MinimumAzureCliVersion) {
        throw "Azure CLI $MinimumAzureCliVersion or later is required."
    }

    $context = Invoke-AzJson -Arguments @(
        'account', 'show',
        '--query',
        '{cloud:environmentName,subscription_id:id,subscription_name:name,' +
        'tenant_id:tenantId,state:state,identity_name:user.name,' +
        'identity_type:user.type}'
    )

    if ([string] $context.cloud -cne $ExpectedCloudName) {
        throw "Expected cloud '$ExpectedCloudName'; found '$($context.cloud)'."
    }

    if ([guid] $context.tenant_id -ne $ExpectedTenantId) {
        throw 'The active tenant does not match the expected tenant.'
    }

    if ([guid] $context.subscription_id -ne $ExpectedSubscriptionId) {
        throw 'The active subscription does not match the expected subscription.'
    }

    if ([string] $context.state -cne 'Enabled') {
        throw "The active subscription state is '$($context.state)'."
    }

    if ([string] $context.identity_name -cne $ExpectedIdentityName) {
        throw 'The signed-in identity does not match the expected identity.'
    }

    $extensionDocument = Invoke-AzJson -Arguments @(
        'extension', 'list',

```

```

        '--query', '{extensions: [].{name:name,version:version}}'
    )
    $installedExtensions = @($extensionDocument.extensions)
    $verifiedExtensions = [ordered]@{}

    foreach ($extensionName in ($RequiredExtensions.Keys | Sort-Object)) {
        $requiredVersion = [version] $RequiredExtensions[$extensionName]
        $installed = @(
            $installedExtensions |
            Where-Object { $_.name -ceq $extensionName }
        )

        if ($installed.Count -ne 1) {
            throw "Required extension '$extensionName' is missing or duplicated."
        }

        try {
            $installedVersion = [version] $installed[0].version
        }
        catch {
            throw "Extension '$extensionName' reported an invalid version."
        }

        if ($installedVersion -lt $requiredVersion) {
            throw "Extension '$extensionName' requires version $requiredVersion or later."
        }

        $verifiedExtensions[$extensionName] = $installedVersion.ToString()
    }

    [pscustomobject]@{
        status = 'ready'
        checked_utc = (Get-Date).ToUniversalTime().ToString('o')
        powershell_version = $PSVersionTable.PSVersion.ToString()
        azure_cli_version = $azureCliVersion.ToString()
        cloud = [string] $context.cloud
        tenant_id = ([guid] $context.tenant_id).ToString()
        subscription_id = ([guid] $context.subscription_id).ToString()
        subscription_name = [string] $context.subscription_name
        identity_name = [string] $context.identity_name
        identity_type = [string] $context.identity_type
        required_extensions = $verifiedExtensions
        dynamic_installation = 'disabled'
    }
}

```

Environment variables outrank persisted Azure CLI settings. The function uses them for JSON, quiet output, telemetry, and extension behavior. It does not edit the user's configuration file.

Dynamic extension installation helps exploration but harms repeatability. A command could otherwise add code during evidence collection. Approved extensions must exist before the run. Installing or updating an extension is a dependency change. The environment build records its source and version. Collection then keeps one tool context from start to finish.

Resource Graph is the first required extension. Version 2.1.1 matches the tested surface for Chapter 6.2. Other scripts can provide another approved requirement map.

The JMESPath projections return only needed fields. `az account show` contains more data than the check requires. A small contract eases parsing, review, and redaction.

JSON is the interface between Azure CLI and PowerShell. Table output serves people, not stable parsers. TSV needs an ordered projection whenever position matters.

The function returns an object only after all checks pass. It records tools, context, identity, and extensions. It never returns a token or secret.

Together, those fields form a session fingerprint. They do not authenticate a saved file by themselves. Integrity controls and a run identifier bind it to later evidence.

Expected values remain outside source. A controlled shell or protected pipeline task supplies them. Placeholder identifiers cannot become permissive defaults.

```
POWERSHELL
. ./Test-AzureCliSession.ps1

$result = Test-AzureCliSession `
  -ExpectedTenantId $env:EXPECTED_TENANT_ID `
  -ExpectedSubscriptionId $env:EXPECTED_SUBSCRIPTION_ID `
  -ExpectedIdentityName $env:EXPECTED_IDENTITY_NAME `
  -RequiredExtensions @{ 'resource-graph' = '2.1.1' }

$result | ConvertTo-Json -Depth 5
```

Missing inputs fail during binding. A mismatch fails before later queries begin. Success emits one JSON document for a protected evidence pack.

The timestamp uses Coordinated Universal Time. A run identifier links this record to query files and decisions. That link separates observations made under different conditions.

Tenant and subscription IDs are not secrets, but they expose estate structure. Identity names may be personal data. Session records still need access and retention controls.

Workstations and automation jobs can share this function. Their login methods differ, but the assertions stay constant. Automation isolates its Azure configuration directory before login.

Isolation prevents jobs from sharing tokens and defaults. The directory stays protected for the job's lifetime. Cleanup follows the organization's credential rules.

Long collections can outlive access or role activation. The preflight should start each bounded evidence batch. A later authorization failure ends that batch as incomplete.

Least privilege also applies to observers. Read access must cover every inspected scope. A safe context cannot create missing visibility.

Authorization failures remain visible as failures. Converting them into empty arrays would create false assurance. Later wrappers preserve the native error before reading any result count. This limit changes how empty results are read. Empty can mean absence, missing scope, a faulty filter, or service delay. Chapter 6.2 will preserve scope and counts before drawing conclusions.

Cross-platform scripts should pass argument arrays. String-built commands invite quoting and injection defects. Complex JSON arguments belong in files rather than nested escapes.

Every later command inherits the wrapper's contract. It checks status, parses a declared shape, and preserves scope. An exception, empty response, and verified empty result remain distinct.

The **ready** status proves only these session checks. It says nothing about drift, policy effect, or telemetry flow. Those claims require their own read-only tests.

The session is evidence, not background. It records who looked, where they looked, and which tools shaped the view. Before Azure can be observed with confidence, the observer's footing must be visible.

Chapter 6: Observe and Test Azure with CLI and PowerShell

Chapter 6.2: Query the Azure Resource Graph

One resource command answers one narrow question. It cannot explain an estate without many calls and local joins. Azure Resource Graph provides the wider read model.

Resource Graph indexes data supplied through Azure Resource Manager. It can query an explicit set of subscriptions at once. Estate patterns then emerge without reading each provider separately.

Its strength is control-plane breadth. It does not replace data-plane inspection, health probes, logs, or service metrics. A storage account record cannot prove that every blob operation works. The index is not the system of record. Resource Manager sends change notices, and Resource Graph scans for missed updates. A recent operation may take time to appear.

Its schema is also a projection. Provider API versions can expose different fields, while some properties may be absent. Queries must test what the index actually returns.

These limits shape the evidence claim. An inventory records what Resource Graph returned during one collection window. It is not a transactional Azure snapshot.

Nor is it a live operational view. Provisioning state and indexed configuration answer different questions from latency or availability. Later monitoring evidence must come from the services that own those signals.

Scope needs equal precision. Without a supplied scope, Azure CLI queries subscriptions available in its cached login. The result would then depend on the operator.

That default can change between people or days. Newly granted access may require a refreshed login before it appears. Approved scope and observed scope must therefore travel together.

The capstone supplies three approved platform subscription IDs. They represent Identity, Connectivity, and Management. Application subscriptions and unrelated access remain outside the run.

That list is an evidence input. Its owner, approval, and effective date belong with the collection. A complete query against an obsolete list is still incomplete evidence.

Explicit scope makes omission testable. The collector queries subscription containers and compares returned IDs with the request. A missing or extra ID stops export.

Subscription state remains in the record. A disabled subscription is not discarded for looking inconvenient. Review decides whether it reflects retirement, billing, access, or surprise.

Resource Graph follows Azure RBAC. A principal needs read access before an object can appear. Missing access can otherwise look like an empty estate.

The command omits `--allow-partial-scopes`. Partial success cannot satisfy a declared inventory boundary. An inaccessible requested scope must remain a failed collection.

Azure CLI forwards no more than 1,000 subscriptions per request. The function enforces that ceiling before execution. Larger estates require reviewed batches and coverage reconciliation.

Batch boundaries need unique membership and a final union check. Otherwise one subscription can appear twice while another disappears. Scale does not weaken the same exact-scope rule.

Two query languages act at separate layers. Resource Graph uses a Kusto Query Language dialect on the service. Azure CLI uses JMESPath on the command response.

KQL decides which estate rows exist. JMESPath narrows the local envelope to data, counts, totals, and continuation. Naming the correct layer makes failures easier to diagnose.

The first query proves subscription coverage. `ResourceContainers` holds subscriptions and resource groups. A case-insensitive type comparison avoids casing noise.

```
KQL
ResourceContainers
| where type =~ 'microsoft.resources/subscriptions'
| project id=tolower(id), subscription_id=tolower(subscriptionId),
    subscription_name=name,
    subscription_state=tostring(properties.state)
| order by id asc
```

The projection retains resource ID and subscription ID. The resource ID becomes a unique paging key. Subscription state remains observed evidence.

The second query collects Azure resources. It lowercases IDs, types, resource groups, and locations. This reduces false differences across Resource Graph tables.

```
KQL
Resources
| project id=tolower(id), name, type=tolower(type),
    subscription_id=tolower(subscriptionId),
    resource_group=tolower(resourceGroup),
    location=tolower(location), tag_keys=bag_keys(tags),
    owner=tostring(tags['owner']),
    managed_by=tostring(tags['managed_by']),
    missing_owner=isempty(tostring(tags['owner'])),
    missing_managed_by=isempty(tostring(tags['managed_by']))
| order by id asc
```

Normalization changes only evidence. It never renames an Azure object. Display names keep their original casing where people need it.

The tag contract stays small. Every key is visible, but only `owner` and `managed_by` values are collected. Other values may hold data the inventory does not need.

The selected keys are canonical contract names. Alternate casing becomes visible in `tag_keys` but does not satisfy the marker. That difference turns loose tagging practice into reviewable evidence.

Tags should not contain secrets, yet collectors cannot assume perfect practice. Leaving unrelated values behind reduces exposure. The key list still reveals what needs investigation. Missing ownership markers become query booleans. They identify review candidates without declaring policy failure. Chapter 8 retains the enforcement decision.

Location needs similar care. Global resources can return no common region. The normalizer labels that state `global-or-unspecified` instead of inventing a defect.

Each query ends with unique ordering. Type or location alone would leave ties unstable. Normalized resource ID gives static results a deterministic sequence.

One response cannot prove completeness. Resource Graph returns no more than 1,000 rows per page. A continuation token requests another page when supported.

The token is opaque and belongs to that query sequence. It is passed back unchanged and never parsed for meaning. Reusing it with another query or later run would break the collection boundary.

Queries containing `limit`, `take`, or `sample` may not produce a token. The inventory queries avoid those operators. A missing token before the reported total is therefore a hard failure.

Chapter 6.1 already checks native status and JSON. The paging function adds collection checks. It rejects malformed pages, moving totals, duplicate IDs, repeated tokens, and missing continuations.

POWERSHELL

```
function Invoke-AzGraphQuery {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [ValidateNotNullOrEmpty()]
        [string] $Query,

        [Parameter(Mandatory)]
        [ValidateCount(1, 1000)]
        [guid[]] $SubscriptionId,

        [ValidateRange(1, 1000)]
        [int] $PageSize = 1000
    )

    $ErrorActionPreference = 'Stop'
    $rows = [System.Collections.Generic.List[object]]::new()
    $seenIds = [System.Collections.Generic.HashSet[string]]::new(
        [System.StringComparer]::OrdinalIgnoreCase
    )
    $seenTokens = [System.Collections.Generic.HashSet[string]]::new(
        [System.StringComparer]::Ordinal
    )
    $expectedTotal = $null
    $skipToken = $null

    do {
        $arguments = @(
            'graph', 'query',
            '--graph-query', $Query,
```

```

        '--first', $PageSize.ToString(
            [System.Globalization.CultureInfo]::InvariantCulture
        ),
        '--query',
        '{data:data,count:count,total_records:total_records,' +
        'skip_token:skip_token}',
        '--subscriptions'
    )
    $arguments += @(
        $SubscriptionId |
        ForEach-Object { $_.ToString() }
    )

    if (-not [string]::IsNullOrEmpty($skipToken)) {
        $arguments += @('--skip-token', $skipToken)
    }

    $page = Invoke-AzJson -Arguments $arguments
    $properties = $page.PSObject.Properties.Name
    if ('data' -notin $properties -or
        'count' -notin $properties -or
        'total_records' -notin $properties -or
        'skip_token' -notin $properties) {
        throw 'Resource Graph returned an unexpected page shape.'
    }

    $pageRows = @($page.data)
    if ([int] $page.count -ne $pageRows.Count) {
        throw 'Resource Graph page count does not match its data.'
    }

    $pageTotal = [long] $page.total_records
    if ($null -eq $expectedTotal) {
        $expectedTotal = $pageTotal
    }
    elseif ($pageTotal -ne $expectedTotal) {
        throw 'Resource Graph total changed during pagination.'
    }

    foreach ($row in $pageRows) {
        if ('id' -notin $row.PSObject.Properties.Name -or
            [string]::IsNullOrEmpty([string] $row.id)) {
            throw 'Every paged Resource Graph row must contain an ID.'
        }

        if (-not $seenIds.Add([string] $row.id)) {
            throw "Resource Graph returned duplicate ID '$($row.id)'."
        }

        $rows.Add($row)
    }

    if ($rows.Count -gt $expectedTotal) {
        throw 'Resource Graph returned more rows than its total.'
    }

    $nextToken = [string] $page.skip_token
    if ($rows.Count -lt $expectedTotal -and
        [string]::IsNullOrEmpty($nextToken)) {
        throw 'Resource Graph truncated results without a continuation token.'
    }

    if (-not [string]::IsNullOrEmpty($nextToken) -and
        -not $seenTokens.Add($nextToken)) {
        throw 'Resource Graph repeated a continuation token.'
    }

```

```

        $skipToken = $nextToken
    } while (-not [string]::IsNullOrEmpty($skipToken))

    if ($rows.Count -ne $expectedTotal) {
        throw 'Resource Graph collection does not match its final total.'
    }

    return $rows.ToArray()
}

```

Every page must agree on the total. Its count must equal its data length. The final row count must also match the total.

These rules catch many unstable reads. They cannot freeze Azure between page requests. Balanced creation and deletion could preserve totals while changing membership.

Repeating the collection can add confidence without creating a transaction. Matching ID sets across quiet windows support stability. Differences show movement rather than proving corruption.

The evidence stores start and completion times. Its consistency label says pages passed checks but were not point-in-time. Critical work can repeat or partition collection by stable IDs.

Throttling is also an incomplete outcome. Resource Graph and Resource Manager both apply quotas. A throttled page fails instead of shortening the inventory.

Blind parallel retries can worsen pressure and hide timing. Scheduled jobs should stagger work and use bounded retry rules. This sample leaves the failure visible.

Raw rows still need a stable shape. Dynamic tags and optional locations require clear rules. The normalizer orders fields and tag keys.

It derives region and type counts locally. Those summaries expose estate shape without more service calls. Each count remains traceable to exported resources.

A summary is not a compliance verdict. An unfamiliar region or type opens a review question. Later control workflows decide whether it violates intent.

```

POWERSHELL
$normalizedResources = @(
    $resources | ForEach-Object {
        [pscustomobject][ordered]@{
            id                = [string] $_.id
            name              = [string] $_.name
            type              = [string] $_.type
            subscription_id   = [string] $_.subscription_id
            resource_group    = [string] $_.resource_group
            location          = if (
                [string]::IsNullOrEmpty([string] $_.location)
            ) { 'global-or-unspecified' } else { [string] $_.location }
            tag_keys          = @(
                $_.tag_keys |
                ForEach-Object { [string] $_ } |
                Sort-Object -CaseSensitive
            )
            owner             = [string] $_.owner
            managed_by       = [string] $_.managed_by
            missing_owner     = [bool] $_.missing_owner
            missing_managed_by = [bool] $_.missing_managed_by
        }
    }
)

```

```

    }
  }
)
$regionCounts = @(
  $normalizedResources |
  Group-Object -Property location |
  Sort-Object -Property Name |
  ForEach-Object {
    [ordered]@{ location = $_.Name; count = $_.Count }
  }
)
$typeCounts = @(
  $normalizedResources |
  Group-Object -Property type |
  Sort-Object -Property Name |
  ForEach-Object {
    [ordered]@{ type = $_.Name; count = $_.Count }
  }
)

$inventory = [ordered]@{
  schema_version = '1.0'
  started_utc    = $startedUtc
  completed_utc  = $completedUtc
  scope          = $scope
  consistency    = 'validated-across-pages-not-point-in-time'
  queries        = [ordered]@{
    subscriptions = $subscriptionQuery
    resources     = $resourceQuery
  }
  subscriptions  = @($subscriptions)
  resources      = $normalizedResources
}

$summary = [ordered]@{
  schema_version      = '1.0'
  completed_utc       = $completedUtc
  subscription_count   = $subscriptions.Count
  resource_count       = $normalizedResources.Count
  region_counts       = $regionCounts
  resource_type_counts = $typeCounts
  selected_gaps       = [ordered]@{
    missing_owner = @(
      $normalizedResources |
      Where-Object { $_.missing_owner }
    ).Count
    missing_managed_by = @(
      $normalizedResources |
      Where-Object { $_.missing_managed_by }
    ).Count
  }
}
}

```

The detailed file stores each query beside its results. Selection and normalization stay visible.

Query changes become reviewable changes.

Schema version belongs to the export, not Azure. New fields or gap rules require a schema decision. Consumers should reject unknown versions.

Baselines must name that version when comparing runs. A count change after query logic changed is not a pure estate change. Version and query text expose that break.

The summary never rewrites a tag. Azure CLI remains read-only here. Durable corrections return to Terraform or the platform backlog.

JSON preserves nested key arrays and named properties. The collector writes detailed and summary files. Chapter 6.5 will add package hashes and review presentation.

Summary comparison can reveal growth or unexpected spread. A difference remains a signal until resource records explain it. Counts without scope and query are easy to misuse.

The invocation begins with session preflight. Protected inputs form the three-subscription capstone scope. Missing or invalid GUIDs fail before collection.

```
POWERSHELL
. ../06_01/Test-AzureCliSession.ps1
. ./Get-AzureEstateInventory.ps1

$session = Test-AzureCliSession `
    -ExpectedTenantId $env:EXPECTED_TENANT_ID `
    -ExpectedSubscriptionId $env:MANAGEMENT_SUBSCRIPTION_ID `
    -ExpectedIdentityName $env:EXPECTED_IDENTITY_NAME `
    -RequiredExtensions @{ 'resource-graph' = '2.1.1' }

if ($session.status -cne 'ready') {
    throw 'The Azure CLI session preflight did not pass.'
}

$scope = @(
    $env:IDENTITY_SUBSCRIPTION_ID,
    $env:CONNECTIVITY_SUBSCRIPTION_ID,
    $env:MANAGEMENT_SUBSCRIPTION_ID
) | ForEach-Object { [guid] $_ }

$runStamp = (Get-Date).ToUniversalTime().ToString('yyyyMMddTHHmssfffZ')
$outputDirectory = Join-Path './evidence' "inventory-$runStamp"
Export-AzureEstateInventory -SubscriptionId $scope `
    -OutputDirectory $outputDirectory |
    ConvertTo-Json -Depth 5
```

The directory stores evidence, not configuration. Records expose resource IDs, names, markers, and platform structure. They require controlled access and deletion.

Each invocation creates a UTC-stamped directory. A later run cannot silently replace earlier files. Retention can remove an expired directory only after its review value ends.

The portal can help develop queries but should not own repeatability. Interactive exploration tests assumptions quickly. Committed KQL freezes the accepted question beside its answer.

An empty inventory passes only with valid scope, totals, and pages. Otherwise emptiness remains unknown or failed. Absence must never become automatic assurance.

This inventory describes estate shape, not governance ancestry or effective access. Chapter 6.3 will test those structures. First, the platform needs a counted and scoped field of view.

Chapter 6: Observe and Test Azure with CLI and PowerShell

Chapter 6.3: Verify Hierarchy, Access, and Inheritance

An inventory proves presence, not placement. It does not show whether resources sit beneath the intended governance boundary. It also cannot explain who can change them.

Hierarchy and access connect objects into a control structure. Management-group placement determines which parent assignments reach each subscription. Azure RBAC then links principals, roles, and scopes.

Verification starts with approved intent. That intent must exist before Azure is queried. Otherwise the current estate becomes its own standard.

The capstone intent names five managed groups. Platform Foundation contains Platform, Landing Zones, Sandbox, and Decommissioned. The external tenant root remains outside Terraform ownership.

Identity, Connectivity, and Management subscriptions belong under Platform. The other three branches remain empty in this platform-only capstone. A later application contract does not authorize an application deployment here.

Each relationship becomes a parent assertion. Child type and stable child ID form its key. Display names help readers but never drive comparison.

The intent also declares access-review scopes. It lists allowed assignment tuples and deployment principal object IDs. Privileged role definition IDs remain controlled inputs.

Role names can change while IDs remain stable. Principal names can also expose personal data or require Microsoft Graph. The comparison therefore uses object IDs and role definition IDs.

This baseline is an allowlist, not a discovery wish list. It must name every assignment permitted in the reviewed boundary. A vague baseline cannot reveal unexpected access.

Intent needs provenance as much as evidence does. Owner, approval, schema version, and effective date explain each allowed tuple. A later edit must preserve the baseline used by an earlier run.

The observer needs read visibility at every declared scope. A successful command still reflects that identity's authorized view. Expected objects turn a visibility gap into a finding.

The hierarchy query starts at the approved root management-group ID. `--expand` includes children, while `--recurse` walks every descendant. The command uses an ID, not a display name.

`--no-register` is a safety control. Without it, the command can register the management resource provider during a read workflow. Observation must not hide a setup mutation.

Azure Resource Manager can cache hierarchy changes. A recent move may take time to appear. Collection time and change time need review before declaring drift.

Each management group and subscription has one hierarchy parent. Ancestry is therefore a chain, not several competing paths. One wrong parent can change every inherited assignment below it.

The recursive response becomes one record per node. Each record keeps ID, name, type, display name, and parent name. Duplicate IDs stop collection because ancestry would be ambiguous. The first comparison finds missing children and wrong parents. A missing subscription can mean drift, limited visibility, or stale hierarchy data. The finding records the mismatch without choosing a cause.

The second comparison finds unexpected children inside the managed subtree. Unrelated siblings beside Platform Foundation are outside the capstone. Children beneath a managed group are not.

Finding categories preserve different repair questions. Missing, misplaced, and unexpected nodes do not share one cause. Review can route each result without losing the observed relationship.

```
POWERSHELL
$hierarchyDocument = Invoke-AzJson -Arguments @(
    'account', 'management-group', 'show',
    '--name', [string] $Intent.root_management_group_id,
    '--expand',
    '--recurse',
    '--no-register'
)
$seenIds = [System.Collections.Generic.HashSet[string]]::new(
    [System.StringComparer]::OrdinalIgnoreCase
)
$hierarchy = @(
    ConvertTo-HierarchyRecord -Node $hierarchyDocument `
        -ParentName $null -SeenId $seenIds
)

$actualByKey = @{}
foreach ($node in $hierarchy) {
    $actualByKey["$($node.node_type)|$($node.name)"] = $node
}

$hierarchyFindings = [System.Collections.Generic.List[object]]::new()
foreach ($relation in @($Intent.expected_parent)) {
    $key = @(
        ([string] $relation.child_type).ToLowerInvariant(),
        ([string] $relation.child_id).ToLowerInvariant()
    ) -join '|'
    $expectedParent = ([string] $relation.parent_id).ToLowerInvariant()
    if (-not $actualByKey.ContainsKey($key)) {
        $hierarchyFindings.Add([pscustomobject][ordered]@{
            category = 'missing_child'
            child = $key
        })
    }
}
```

```

        expected_parent = $expectedParent
        actual_parent = $null
    })
}
elseif ([string] $actualByKey[$key].parent_name -cne $expectedParent) {
    $hierarchyFindings.Add([pscustomobject][ordered]@{
        category = 'wrong_parent'
        child = $key
        expected_parent = $expectedParent
        actual_parent = [string] $actualByKey[$key].parent_name
    })
}
}
}

$managedParents = [System.Collections.Generic.HashSet[string]]::new(
    [System.StringComparer]::OrdinalIgnoreCase
)
foreach ($relation in @($Intent.expected_parent)) {
    if ([string] $relation.child_type -ceq 'management_group') {
        $null = $managedParents.Add([string] $relation.child_id)
    }
}

foreach ($node in $hierarchy) {
    if (-not [string]::IsNullOrEmpty([string] $node.parent_name) -and
        $managedParents.Contains([string] $node.parent_name)) {
        $key = "$($node.node_type)|$($node.name)"
        if (-not $validatedIntent.relation_keys.Contains($key)) {
            $hierarchyFindings.Add([pscustomobject][ordered]@{
                category = 'unexpected_child'
                child = $key
                expected_parent = $null
                actual_parent = [string] $node.parent_name
            })
        }
    }
}
}

```

The script does not move a child back. It does not rename a group or register a provider. Repair belongs to the approved Terraform workflow.

Hierarchy supplies the path for inherited access. Azure RBAC scopes run from management group down to resource. An assignment at a parent can apply below it.

A role assignment joins a security principal, role definition, and scope. Optional conditions can narrow parts of the grant. The collector retains the condition and version.

The review queries each declared scope. `--include-inherited` adds assignments applied above that scope. Source scope stays visible beside each observation point.

Direct and inherited describe where an assignment was created. They do not change its role definition. Comparing assignment scope with review scope keeps that distinction reconstructable.

The command omits `--all`. That option descends through a subscription and would mix child assignments into this check. Child access needs its own declared scope and intent.

`--fill-principal-name false` prevents a Microsoft Graph name lookup. Evidence keeps principal ID and type instead. Directory naming and access stay separate dependencies.

One inherited assignment can appear in several responses. The collector deduplicates by assignment ID. It records every scope where that assignment appeared.

An assignment that changes across calls is rejected. One ID cannot safely represent different tuples. A later run can collect a stable view.

The multi-scope read is not a transaction. An assignment can appear or vanish between calls without reusing an ID. Start and completion times define the evidence window.

The output envelope preserves an empty assignment array. This separates verified emptiness from null or malformed JSON. Chapter 6.1 already rejects native and parsing failures.

POWERSHELL

```
$assignmentById = @{}
foreach ($reviewScope in ($validatedIntent.review_scopes | Sort-Object)) {
    $assignmentDocument = Invoke-AzJson -Arguments @(
        'role', 'assignment', 'list',
        '--scope', $reviewScope,
        '--include-inherited',
        '--fill-principal-name', 'false',
        '--query',
        '{assignments: [].{id:id,principal_id:principalId,' +
        'principal_type:principalType,' +
        'role_definition_id:roleDefinitionId,' +
        'role_definition_name:roleDefinitionName,scope:scope,' +
        'condition:condition,condition_version:conditionVersion}}'
    )
    if ('assignments' -notin $assignmentDocument.PSObject.Properties.Name) {
        throw 'Azure CLI returned an unexpected role assignment shape.'
    }

    foreach ($item in @($assignmentDocument.assignments)) {
        foreach ($requiredProperty in @(
            'id',
            'principal_id',
            'principal_type',
            'role_definition_id',
            'role_definition_name',
            'scope',
            'condition',
            'condition_version'
        )) {
            if ($requiredProperty -notin $item.PSObject.Properties.Name) {
                throw "Observed role assignment is missing '$requiredProperty'."
            }
        }

        $assignmentId = ([string] $item.id).
            TrimEnd('/').ToLowerInvariant()
        if ([string]::IsNullOrEmpty($assignmentId)) {
            throw 'Observed role assignment ID is empty.'
        }

        $normalized = [pscustomobject][ordered]@{
            id = $assignmentId
            principal_id = ([guid] $item.principal_id).ToString()
            principal_type = [string] $item.principal_type
            role_definition_id = ([string] $item.role_definition_id).
                TrimEnd('/').ToLowerInvariant()
            role_definition_name = [string] $item.role_definition_name
            scope = ([string] $item.scope).
                TrimEnd('/').ToLowerInvariant()
            condition = [string] $item.condition
            condition_version = [string] $item.condition_version
            observed_at = @($reviewScope)
```

```

    }
    $key = Get-RoleAssignmentKey -Assignment $normalized

    if ($assignmentById.ContainsKey($assignmentId)) {
        $existing = $assignmentById[$assignmentId]
        if ((Get-RoleAssignmentKey -Assignment $existing) -cne $key -or
            [string] $existing.principal_type -cne
            [string] $normalized.principal_type) {
            throw "Role assignment '$assignmentId' changed across scopes."
        }

        $existing.observed_at = @(
            $existing.observed_at + $reviewScope |
            Sort-Object -Unique
        )
    }
    else {
        $assignmentById[$assignmentId] = $normalized
    }
}

$roleAssignments = @(
    $assignmentById.Values |
    Sort-Object -Property id
)
$actualKeys = [System.Collections.Generic.HashSet[string]]::new(
    [System.StringComparer]::Ordinal
)
$accessFindings = [System.Collections.Generic.List[object]]::new()
foreach ($assignment in $roleAssignments) {
    $key = Get-RoleAssignmentKey -Assignment $assignment
    $null = $actualKeys.Add($key)
    if (-not $validatedIntent.allowed_keys.Contains($key)) {
        $accessFindings.Add([pscustomobject][ordered]@{
            category = 'unexpected_assignment'
            assignment_id = [string] $assignment.id
            principal_id = [string] $assignment.principal_id
            role_definition_id = [string] $assignment.role_definition_id
            scope = [string] $assignment.scope
        })
    }

    if ([bool] $Intent.forbid_direct_user_assignments -and
        [string] $assignment.principal_type -ceq 'User') {
        $accessFindings.Add([pscustomobject][ordered]@{
            category = 'direct_user_assignment'
            assignment_id = [string] $assignment.id
            principal_id = [string] $assignment.principal_id
            role_definition_id = [string] $assignment.role_definition_id
            scope = [string] $assignment.scope
        })
    }
}
}

```

The comparison key includes principal, role definition, assignment scope, condition version, and condition. A broader scope produces a different key. Removing a condition does too.

Condition text is compared exactly. Harmless formatting can therefore open a finding. This cautious rule will not hide a meaningful expression change.

Unexpected does not mean hostile. It means the active assignment lacks approval in this baseline. Review must find its owner, purpose, and next action.

Missing assignments matter too. A deployment identity without needed access can block delivery or recovery. Absence is not always safer.

The intent can forbid direct user assignments. Groups usually provide a clearer ownership and review path. A permitted exception must remain visible.

Deployment identities receive a separate check. Each principal must have the expected object ID and service-principal type. Its assignments must also match the common allowlist.

Managed identities appear as service principals in Azure RBAC evidence. Object IDs, not application IDs, identify assignment principals. Mixing those IDs creates false missing results. Principal type is still a coarse label. It cannot distinguish every application or managed-identity purpose. Approved intent supplies that ownership context outside the command response.

Privileged roles enter intent by stable role definition ID. The summary counts their assignments for review. Role name alone cannot decide whether one is justified.

POWERSHELL

```
foreach ($allowedAssignment in @($Intent.allowed_role_assignment)) {
    $key = Get-RoleAssignmentKey -Assignment $allowedAssignment
    if (-not $actualKeys.Contains($key)) {
        $accessFindings.Add([pscustomobject][ordered]@{
            category = 'missing_assignment'
            assignment_id = $null
            principal_id = ([guid] (
                $allowedAssignment.principal_id
            )).ToString()
            role_definition_id = ([string] (
                $allowedAssignment.role_definition_id
            )).TrimEnd('/').ToLowerInvariant()
            scope = ([string] $allowedAssignment.scope).TrimEnd('/').ToLowerInvariant()
        })
    }
}

$deploymentIdentities = [System.Collections.Generic.List[object]]::new()
foreach ($deploymentId in ($validatedIntent.deployment_ids | Sort-Object)) {
    $deploymentAssignments = @(
        $roleAssignments |
        Where-Object { $_.principal_id -ceq $deploymentId }
    )
    if ($deploymentAssignments.Count -eq 0) {
        $accessFindings.Add([pscustomobject][ordered]@{
            category = 'deployment_principal_without_assignment'
            assignment_id = $null
            principal_id = $deploymentId
            role_definition_id = $null
            scope = $null
        })
    }
    elseif (@(
        $deploymentAssignments |
        Where-Object { $_.principal_type -cne 'ServicePrincipal' }
    ).Count -gt 0) {
        $accessFindings.Add([pscustomobject][ordered]@{
            category = 'deployment_principal_type'
            assignment_id = $null
            principal_id = $deploymentId
            role_definition_id = $null
            scope = $null
        })
    }
}

$deploymentIdentities.Add([pscustomobject][ordered]@{
```

```

        principal_id = $deploymentId
        assignment_count = $deploymentAssignments.Count
        assignment_ids = @($deploymentAssignments.id)
    })
}

$privilegedAssignments = @(
    $roleAssignments |
    Where-Object {
        $validatedIntent.privileged_ids.Contains(
            [string] $_.role_definition_id
        )
    }
)

$controlStatus = if (
    $hierarchyFindings.Count -eq 0 -and
    $accessFindings.Count -eq 0
) { 'pass' } else { 'review' }

[pscustomobject][ordered]@{
    schema_version = '1.0'
    started_utc = $startedUtc
    completed_utc = (Get-Date).ToUniversalTime().ToString('o')
    collection_status = 'collected'
    control_status = $controlStatus
    intent = $Intent
    hierarchy = $hierarchy
    role_assignments = $roleAssignments
    deployment_identities = $deploymentIdentities.ToArray()
    findings = [ordered]@{
        hierarchy = $hierarchyFindings.ToArray()
        access = $accessFindings.ToArray()
    }
    summary = [ordered]@{
        hierarchy_node_count = $hierarchy.Count
        role_assignment_count = $roleAssignments.Count
        privileged_assignment_count = $privilegedAssignments.Count
        deployment_principal_count = $deploymentIdentities.Count
        hierarchy_finding_count = $hierarchyFindings.Count
        access_finding_count = $accessFindings.Count
    }
}

```

Collection status and control status stay separate. A valid file can contain findings. A query failure can never become a passing control result.

Azure RBAC is additive. Group membership can add transitive grants, and assignments can overlap. This inventory records assignment objects instead of computing every action.

Conditions can narrow a grant, while deny assignments can block allowed actions. Both facts shape effective authorization. The baseline stores conditions but does not evaluate deny assignments.

Eligible Privileged Identity Management assignments use separate schedule data. Azure Lighthouse grants are absent from normal role-assignment lists. Both need companion checks when used.

Assignments from a parent outside the managed subtree can still inherit downward. Their original scope remains in each normalized tuple. Intent must allow them explicitly or report them.

Data-plane permissions can follow other models. Storage keys, database grants, and application roles remain outside this collector. A clean Azure RBAC baseline cannot certify them.

These exclusions keep the claim honest. The artifact covers active Azure RBAC assignments at declared control-plane scopes. It is not a universal entitlement report.

The invocation reuses the fail-closed session preflight. It loads protected JSON intent instead of embedding tenant and principal IDs. Empty or invalid intent stops before Azure queries.

```
POWERSHELL
. ../06_01/Test-AzureCliSession.ps1
. ../Test-AzureHierarchyAccess.ps1

$session = Test-AzureCliSession `
    -ExpectedTenantId $env:EXPECTED_TENANT_ID `
    -ExpectedSubscriptionId $env:MANAGEMENT_SUBSCRIPTION_ID `
    -ExpectedIdentityName $env:EXPECTED_IDENTITY_NAME `
    -RequiredExtensions @{}

if ($session.status -cne 'ready') {
    throw 'The Azure CLI session preflight did not pass.'
}

$intentText = Get-Content -Raw -LiteralPath $env:ACCESS_INTENT_PATH
if ([string]::IsNullOrEmpty($intentText)) {
    throw 'The hierarchy and access intent file is empty.'
}

try {
    $intent = $intentText | ConvertFrom-Json -Depth 100
}
catch {
    throw 'The hierarchy and access intent file is invalid JSON.'
}

$runStamp = (Get-Date).ToUniversalTime().ToString('yyyyMMddTHH:mm:ssfffZ')
$outputDirectory = Join-Path './evidence' "hierarchy-access-$runStamp"
Export-AzureHierarchyAccessEvidence -Intent $intent `
    -OutputDirectory $outputDirectory |
    ConvertTo-Json -Depth 5
```

The JSON stores intent, observations, assignments, deployment identities, findings, and counts. Principal IDs and access scopes make it sensitive evidence. Storage and retention must reflect that risk.

A pass means the visible hierarchy and active assignments matched a reviewed baseline. It does not prove that baseline grants least privilege. Approval quality remains a governance responsibility.

Baselines also age. A valid assignment can outlive its project, owner, or delivery method. Review cadence and expiry triggers keep equality from becoming permanent approval.

Corrections return through the owning control path. Terraform handles persistent hierarchy and assignments it owns. Identity governance handles PIM, groups, reviews, and exceptions.

Empty findings close only the declared comparison. Which authorization paths can still alter the platform without appearing in this lens? That open question keeps a clean baseline from becoming false certainty.

Chapter 6: Observe and Test Azure with CLI and PowerShell

Chapter 6.4: Test Policy, Connectivity, and Monitoring

A resource can match its declared configuration and still fail as a service. Azure can show a policy result, route, or diagnostic setting. End-to-end behavior can still be wrong. A useful test set must name the layer its evidence covers.

This section tests the platform's declared configuration layer. It checks the inherited policy baseline and hub network. It also checks the route table, diagnostic settings, and central Log Analytics workspace. It does not claim that packets arrive or logs can be queried.

That boundary matters. Each surface has a different delay and failure path. Azure Policy reports the result of evaluation cycles. Network resources describe intended paths, while Azure Monitor settings describe collection routes.

The protected intent file is the comparison boundary. It names stable resource IDs, prefixes, subnet associations, and routes. It also names required log selectors and the workspace destination. A portal view or successful deployment cannot silently redefine those expectations. Input validation runs before the first Azure CLI call. It rejects the wrong schema version or missing required collections. It also rejects duplicate IDs or names, negative thresholds, and unsupported log-selector kinds. A diagnostic target cannot name another workspace. Invalid intent therefore fails collection instead of creating a misleading difference against Azure.

The capstone keeps that input small. It names the three inherited assignments introduced in Chapter 5.3, one logical hub virtual network, one route table, and one central workspace. Real platforms can add entries without changing the comparison method.

Each expected policy assignment has a maximum count of noncompliant resources. The capstone uses zero for its baseline, but an approved transition could use another threshold for a limited period. That threshold is a review decision, not a default produced by the script.

Policy state is observed at the Platform Foundation management-group scope. The summary contains scope-wide totals and results for individual assignments. Each assignment's resource ID links its result to the protected intent.

Azure Policy data is not instantaneous. Resource changes can appear only after the related evaluation completes. The standard reevaluation cycle also runs on its own cadence. A result therefore needs collection time and evaluation context before it supports a decision.

The collector does not trigger a scan. A scan starts asynchronous policy evaluation. It does not alter resources, but it changes the process being observed. This section remains read-only and records the latest completed state available to the observer.

The policy excerpt rejects an unexpected response shape before comparison. It treats an absent declared assignment as a finding and excessive noncompliance as a different finding. Those categories send reviewers toward different causes.

POWERSHELL

```
$document = Invoke-AzJson -Arguments @(
    'policy', 'state', 'summarize',
    '--management-group', [string] $PolicyIntent.management_group_id
)
if ('value' -notin $document.PSObject.Properties.Name -or
    @($document.value).Count -ne 1) {
    throw 'Azure CLI returned an unexpected policy summary shape.'
}

$summary = @($document.value)[0]
foreach ($property in @('results', 'policyAssignments')) {
    if ($property -notin $summary.PSObject.Properties.Name) {
        throw "Policy summary is missing '$property'."
    }
}
foreach ($property in @(
    'nonCompliantPolicies', 'nonCompliantResources'
)) {
    if ($property -notin $summary.results.PSObject.Properties.Name) {
        throw "Policy summary results are missing '$property'."
    }
}

$assignmentById = @{}
foreach ($assignment in @($summary.policyAssignments)) {
    foreach ($property in @('policyAssignmentId', 'results')) {
        if ($property -notin $assignment.PSObject.Properties.Name) {
            throw "Policy assignment summary is missing '$property'."
        }
    }
    foreach ($property in @(
        'nonCompliantPolicies', 'nonCompliantResources'
    )) {
        if ($property -notin $assignment.results.PSObject.Properties.Name) {
            throw "Policy assignment results are missing '$property'."
        }
    }
}

$assignmentId = Get-NormalizedResourceId `
    -ResourceId ([string] $assignment.policyAssignmentId)
if ($assignmentById.ContainsKey($assignmentId)) {
    throw "Policy summary contains duplicate assignment '$assignmentId'."
}
$assignmentById[$assignmentId] = [pscustomobject][ordered]@{
    resource_id = $assignmentId
    non_compliant_policies = [int] (
        $assignment.results.nonCompliantPolicies
    )
    non_compliant_resources = [int] (
        $assignment.results.nonCompliantResources
    )
}

foreach ($expected in @($PolicyIntent.assignment)) {
```

```

$expectedId = Get-NormalizedResourceId `
    -ResourceId ([string] $expected.resource_id)
if (-not $assignmentById.ContainsKey($expectedId)) {
    $Finding.Add([pscustomobject][ordered]@{
        category = 'missing_policy_summary'
        resource_id = $expectedId
        expected = 'assignment in compliance summary'
        actual = $null
    })
    continue
}

$actual = $assignmentById[$expectedId]
$maximum = [int] $expected.maximum_non_compliant_resources
if ($actual.non_compliant_resources -gt $maximum) {
    $Finding.Add([pscustomobject][ordered]@{
        category = 'policy_non_compliance_threshold'
        resource_id = $expectedId
        expected = $maximum
        actual = $actual.non_compliant_resources
    })
}
}

```

A missing summary is not equivalent to compliance. The assignment might be absent, or evaluation might not have started. The observer might also lack visibility, or the response shape might have changed. The resulting finding preserves uncertainty instead of converting it into zero.

Noncompliance also needs investigation beyond the total. The summary can show that a threshold was crossed, but it does not explain the failing resource property. Detailed policy-state evidence belongs in a focused follow-up.

Network checks begin with the declared virtual-network resource ID. The JMESPath projection returns a stable response envelope with address prefixes and subnet arrays. It preserves an empty subnet list as valid JSON instead of letting it disappear in PowerShell output.

The comparison treats prefix arrays as sets. It normalizes case and order before testing equality.

A different prefix remains visible even when Azure returns values in another sequence.

```

POWERSHELL
$virtualNetworks = [System.Collections.Generic.List[object]]::new()
foreach ($expectedVnet in @($Intent.virtual_network)) {
    $expectedVnetId = Get-NormalizedResourceId `
        -ResourceId ([string] $expectedVnet.resource_id)
    $actualVnet = Invoke-AzJson -Arguments @(
        'network', 'vnet', 'show',
        '--ids', $expectedVnetId,
        '--query',
        '{id:id,location:location,' +
        'address_prefixes:addressSpace.addressPrefixes,' +
        'subnets:subnets[. {name:name,address_prefix:addressPrefix,' +
        'address_prefixes:addressPrefixes,route_table_id:routeTable.id}}'
    )
    foreach ($property in @('id', 'location', 'address_prefixes', 'subnets')) {
        if ($property -notin $actualVnet.PSObject.Properties.Name) {
            throw "Virtual-network observation is missing '$property'."
        }
    }
    $actualVnetId = Get-NormalizedResourceId `
        -ResourceId ([string] $actualVnet.id)
    if ($actualVnetId -cne $expectedVnetId) {

```

```

        throw "Azure CLI returned another virtual network '$actualVnetId'."
    }

    $expectedPrefixes = @(Get-NormalizedStringSet `
        -Value @($expectedVnet.address_prefix))
    $actualPrefixes = @(Get-NormalizedStringSet `
        -Value @($actualVnet.address_prefixes))
    if (-not (Test-StringSetEqual -Expected $expectedPrefixes `
        -Actual $actualPrefixes)) {
        $Finding.Add([pscustomobject][ordered]@{
            category = 'virtual_network_address_prefix'
            resource_id = $expectedVnetId
            expected = $expectedPrefixes
            actual = $actualPrefixes
        })
    }
}

```

Within a virtual network, the subnet name is its comparison key. The script checks each declared subnet's address prefixes. It also checks the route-table resource ID. Missing, changed, and unexpected subnets receive separate findings.

The association check is important because a correct route table can exist without being attached to the intended subnet. Resource presence alone would miss that broken relationship. The evidence retains both sides of the expected association.

Routes receive an exact comparison by route name. The contract includes the destination prefix and next-hop type. It also includes the next-hop address. An extra route is a finding because it can change the shared path even when every required route remains present.

The network observation also retains resource locations and the route table's BGP propagation flag. Those fields are evidence, but this schema does not declare them as controls. They therefore cannot generate findings. Promoting either value into the control contract requires an intentional schema change and matching test. It cannot rest on an ad hoc reviewer assumption.

POWERSHELL

```

$routeTables = [System.Collections.Generic.List[object]]::new()
foreach ($expectedTable in @($Intent.route_table)) {
    $expectedTableId = Get-NormalizedResourceId `
        -ResourceId ([string] $expectedTable.resource_id)
    $actualTable = Invoke-AzJson -Arguments @(
        'network', 'route-table', 'show',
        '--ids', $expectedTableId,
        '--query',
        '{id:id,location:location,' +
        'disable_bgp_route_propagation:disableBgpRoutePropagation,' +
        'routes:routes[].{name:name,address_prefix:addressPrefix,' +
        'next_hop_type:nextHopType,' +
        'next_hop_ip_address:nextHopIpAddress}}'
    )
    foreach ($property in @(
        'id', 'location', 'disable_bgp_route_propagation', 'routes'
    )) {
        if ($property -notin $actualTable.PSObject.Properties.Name) {
            throw "Route-table observation is missing '$property'."
        }
    }
    $actualTableId = Get-NormalizedResourceId `
        -ResourceId ([string] $actualTable.id)
    if ($actualTableId -cne $expectedTableId) {
        throw "Azure CLI returned another route table '$actualTableId'."
    }
}

```

```

$routeByName = @{}
foreach ($route in @($actualTable.routes)) {
    foreach ($property in @(
        'name',
        'address_prefix',
        'next_hop_type',
        'next_hop_ip_address'
    )) {
        if ($property -notin $route.PSObject.Properties.Name) {
            throw "Route observation is missing '$property'."
        }
    }
    $name = ([string] $route.name).Trim().ToLowerInvariant()
    if ($routeByName.ContainsKey($name)) {
        throw "Route table contains duplicate route '$name'."
    }
    $routeByName[$name] = [pscustomobject][ordered]@{
        name = $name
        address_prefix = ([string] $route.address_prefix).
            ToLowerInvariant()
        next_hop_type = [string] $route.next_hop_type
        next_hop_ip_address = [string] $route.next_hop_ip_address
    }
}

$expectedRouteNames = [System.Collections.Generic.HashSet[string]]::new(
    [System.StringComparer]::OrdinalIgnoreCase
)
foreach ($expectedRoute in @($expectedTable.route)) {
    $name = ([string] $expectedRoute.name).Trim().ToLowerInvariant()
    $null = $expectedRouteNames.Add($name)
    if (-not $routeByName.ContainsKey($name)) {
        $Finding.Add([pscustomobject][ordered]@{
            category = 'missing_route'
            resource_id = "$expectedTableId/routes/$name"
            expected = 'present'
            actual = $null
        })
        continue
    }
    $actualRoute = $routeByName[$name]
    foreach ($property in @(
        'address_prefix',
        'next_hop_type',
        'next_hop_ip_address'
    )) {
        $expectedValue = [string] $expectedRoute.$property
        $actualValue = [string] $actualRoute.$property
        if ($actualValue -ine $expectedValue) {
            $Finding.Add([pscustomobject][ordered]@{
                category = "route_$property"
                resource_id = "$expectedTableId/routes/$name"
                expected = $expectedValue
                actual = $actualValue
            })
        }
    }
}
foreach ($actualName in $routeByName.Keys) {
    if (-not $expectedRouteNames.Contains($actualName)) {
        $Finding.Add([pscustomobject][ordered]@{
            category = 'unexpected_route'
            resource_id = "$expectedTableId/routes/$actualName"
            expected = $null
            actual = 'present'
        })
    }
}
}

```

These checks still describe configuration, not traffic. Azure can add system routes and propagate other routes. Azure then selects an effective route in context. Network security groups, firewalls, DNS, peerings, and endpoint state can also change reachability.

A route-table match therefore closes one question only. It proves that the declared user-defined routes and associations were observed. Packet probes and effective-route inspection need a deployed endpoint and an approved operational test.

Monitoring starts at the destination. The script asks for the exact Log Analytics workspace by resource ID. It checks the location, provisioning state, and minimum retention. This prevents a valid diagnostic setting from pointing at an unintended workspace with a similar name.

Retention is part of intent because it affects evidence availability and cost. The chosen number must follow operational and regulatory needs. The script only compares the approved minimum; it does not decide that need.

The same collector asks each declared resource for active diagnostic settings. Diagnostic settings are resource scoped, and each setting selects log or metric categories plus destinations. The capstone requires a named setting and a Log Analytics destination.

POWERSHELL

```
$expectedWorkspaceId = Get-NormalizedResourceId `
    -ResourceId ([string] $Intent.workspace.resource_id)
$workspace = Invoke-AzJson -Arguments @(
    'monitor', 'log-analytics', 'workspace', 'show',
    '--ids', $expectedWorkspaceId,
    '--query',
    '{id:id,location:location,provisioning_state:provisioningState,' +
    'retention_in_days:retentionInDays,sku_name:sku.name}'
)
foreach ($property in @(
    'id',
    'location',
    'provisioning_state',
    'retention_in_days',
    'sku_name'
)) {
    if ($property -notin $workspace.PSObject.Properties.Name) {
        throw "Workspace observation is missing '$property'."
    }
}
$workspaceId = Get-NormalizedResourceId `
    -ResourceId ([string] $workspace.id)
if ($workspaceId -cne $expectedWorkspaceId) {
    throw "Azure CLI returned another workspace '$workspaceId'."
}
if ([string] $workspace.location -ine [string] $Intent.workspace.location) {
    $Finding.Add([pscustomobject][ordered]@{
        category = 'workspace_location'
        resource_id = $workspaceId
        expected = [string] $Intent.workspace.location
        actual = [string] $workspace.location
    })
}
if ([string] $workspace.provisioning_state -cne 'Succeeded') {
    $Finding.Add([pscustomobject][ordered]@{
        category = 'workspace_provisioning_state'
        resource_id = $workspaceId
        expected = 'Succeeded'
    })
}
```

```

        actual = [string] $workspace.provisioning_state
    })
}
if ([int] $workspace.retention_in_days -lt
    [int] $Intent.workspace.minimum_retention_days) {
    $Finding.Add([pscustomobject][ordered]@{
        category = 'workspace_retention'
        resource_id = $workspaceId
        expected = [int] $Intent.workspace.minimum_retention_days
        actual = [int] $workspace.retention_in_days
    })
}

$diagnosticTargets = [System.Collections.Generic.List[object]]::new()
foreach ($target in @($Intent.diagnostic_target)) {
    $targetId = Get-NormalizedResourceId `
        -ResourceId ([string] $target.resource_id)
    $document = Invoke-AzJson -Arguments @(
        'monitor', 'diagnostic-settings', 'list',
        '--resource', $targetId
    )
    $settings = @(Get-DiagnosticSettingArray -Document $document)
}

```

Log selectors can name an individual category or a category group. A category group is a Microsoft-maintained set such as `allLogs` or `audit`. Its membership can change as the resource provider adds supported categories.

That behavior reduces maintenance, but it also changes the comparison meaning. The intent must use a group only when the target resource supports it and the wider collection is acceptable. Otherwise, it should list the required categories directly.

The script considers only enabled selectors. It normalizes each selector into a stable key. One example is `category_group|alllogs`. The script then compares every required key. A disabled selector remains a finding even when the setting itself exists.

The monitoring contract is equally selective. It inspects enabled log selectors. It does not inspect metric selectors, storage accounts, event hubs, or partner destinations. A platform that needs those destinations must declare and test them. An unexamined setting cannot be treated as acceptable.

```

POWERSHELL
$expectedName = ([string] $target.setting_name).
    Trim().ToLowerInvariant()
if (-not $settingByName.ContainsKey($expectedName)) {
    $Finding.Add([pscustomobject][ordered]@{
        category = 'missing_diagnostic_setting'
        resource_id = $targetId
        expected = $expectedName
        actual = $null
    })
}
else {
    $actualSetting = $settingByName[$expectedName]
    $targetWorkspaceId = Get-NormalizedResourceId `
        -ResourceId ([string] $target.workspace_resource_id)
    if ([string] $actualSetting.workspace_resource_id -cne
        $targetWorkspaceId) {
        $Finding.Add([pscustomobject][ordered]@{
            category = 'diagnostic_workspace_destination'
            resource_id = $targetId
            expected = $targetWorkspaceId
        })
    }
}

```



```

        actual = $actualSetting.workspace_resource_id
    })
}
foreach ($requiredLog in @($target.required_log)) {
    $key = @(
        ([string] $requiredLog.kind).ToLowerInvariant(),
        ([string] $requiredLog.name).ToLowerInvariant()
    ) -join '|'
    if ($key -notin $actualSetting.enabled_logs) {
        $Finding.Add([pscustomobject][ordered]@{
            category = 'diagnostic_log_selector'
            resource_id = $targetId
            expected = $key
            actual = $actualSetting.enabled_logs
        })
    }
}
}
}

```

A correct setting does not prove that logs were generated. It does not prove ingestion, table creation, or query access. It also says nothing about alert rules or human response. Azure Monitor notes that data can take time to become available after the setting is configured.

Operational effectiveness therefore needs a time-bounded log query and a known event. Alert delivery and response need their own tests as well. Chapter 10 will develop that wider monitoring loop.

The command wrapper from Chapter 6.1 still controls native failures and JSON parsing. The runner first verifies tenant, active subscription, and identity, then loads the protected intent. No command corrects the configuration it observes.

```

POWERSHELL
. ../06_01/Test-AzureCliSession.ps1
. ./Test-AzurePlatformConfiguration.ps1

$session = Test-AzureCliSession `
    -ExpectedTenantId $env:EXPECTED_TENANT_ID `
    -ExpectedSubscriptionId $env:MANAGEMENT_SUBSCRIPTION_ID `
    -ExpectedIdentityName $env:EXPECTED_IDENTITY_NAME `
    -RequiredExtensions @{}

if ($session.status -cne 'ready') {
    throw 'The Azure CLI session preflight did not pass.'
}

$intentText = Get-Content -Raw -LiteralPath $env:CONFIGURATION_INTENT_PATH
if ([string]::IsNullOrEmpty($intentText)) {
    throw 'The platform configuration intent file is empty.'
}

try {
    $intent = $intentText | ConvertFrom-Json -Depth 100
}
catch {
    throw 'The platform configuration intent file is invalid JSON.'
}

$runStamp = (Get-Date).ToUniversalTime().ToString('yyyyMMddTHHmssfffZ')
$outputDirectory = Join-Path './evidence' "platform-configuration-$runStamp"
Export-AzurePlatformConfigurationEvidence -Intent $intent `
    -OutputDirectory $outputDirectory |
    ConvertTo-Json -Depth 5

```

Collection status and control status remain separate. A malformed response or failed command stops collection, so no control verdict is issued. A complete collection with differences returns `review` and keeps every observation.

A pass means the declared comparisons produced no findings. It does not mean the whole platform is compliant, reachable, monitored, or healthy. The result carries that claim boundary beside the counts.

Findings should return to the owner of persistent state. An approved drift correction belongs in Terraform or its policy library. A changed requirement belongs in the decision register. The observer must not become a second configuration controller.

One JSON document is enough for this check set. It holds intent, observations, findings, status, timestamps, and claim limits. Chapter 6.5 will decide how several bounded results become a reusable evidence pack.

The strongest result is not an empty finding list alone. It is an empty list paired with a precise statement of what was compared and what remains untested. That boundary turns platform inspection into useful evidence instead of broad reassurance.

Chapter 6: Observe and Test Azure with CLI and PowerShell

Chapter 6.5: Build a Reusable Evidence Pack

Separate evidence files are easy to collect and hard to review as one result. Each can be valid while the set has missing inputs, mixed scopes, or incompatible meanings. A reusable pack must make the set itself testable.

An evidence pack is a bounded review unit, not a folder of convenient exports. It names the platform subject, declared scopes, source files, collection times, result meanings, and claim limits. Reviewers can then tell what belongs together before they interpret any status.

The capstone assembles four sources. Session context records the tools and Azure context verified at preflight. Inventory contributes observed counts and metadata signals. Hierarchy, access, policy, network, and monitoring checks contribute control results and findings.

A protected manifest defines that assembly. Its schema version fixes the pack contract, while each input has a stable ID, a supported kind, and a relative path. The declared subject and scope keep files from unrelated runs out of the same review unit.

The runner validates the whole input set before it creates output. Many input defects stop the pack. They include missing files, invalid schemas, duplicate IDs or paths, unsupported kinds, and impossible collection windows. Successful checks therefore cannot make a partial set look complete.

Each source is read once as raw bytes. The runner rejects an empty file and invalid UTF-8 JSON, then computes SHA-256 over those same bytes. Parsing and hashing the same content avoids a second read that could observe another file version.

POWERSHELL

```
function Read-JsonEvidenceSource {  
    [CmdletBinding()]  
    param(  
        [Parameter(Mandatory)]  
        [ValidateNotNullOrEmpty()]  
        [string] $LiteralPath  
    )  
  
    if (-not (Test-Path -LiteralPath $LiteralPath -PathType Leaf)) {  
        throw "Evidence source '$LiteralPath' does not exist."  
    }  
  
    $bytes = [IO.File]::ReadAllBytes((Resolve-Path -LiteralPath $LiteralPath))  
    if ($bytes.Count -eq 0) {  
        throw "Evidence source '$LiteralPath' is empty."  
    }  
  
    try {  
        $text = [Text.UTF8Encoding]::new(  
            $false,  
            $true
```

```

        ).GetString($bytes)
        $document = $text | ConvertFrom-Json -Depth 100
    }
    catch {
        throw "Evidence source '$LiteralPath' is not valid UTF-8 JSON."
    }

    [pscustomobject][ordered]@{
        document = $document
        sha256 = [Convert]::ToHexString(
            [Security.Cryptography.SHA256]::HashData($bytes)
        ).ToLowerInvariant()
    }
}

```

A source hash detects later content changes, but it does not prove who created the file. A copied or fabricated source can still have a valid hash. Provenance needs protected storage, identity, pipeline records, or a signature outside this local pack.

Input paths are relative to the manifest directory. The resolver rejects rooted paths. It also rejects any normalized path that escapes that directory. A reviewed manifest therefore cannot silently collect arbitrary files elsewhere on the host.

Normalization is the central design problem. The four sources do not share one native schema, and forcing them into identical meanings would erase useful distinctions. Small adapters convert each known schema into a common record without changing the claim behind it.

Session context receives an assessment status of **verified** only after several checks. Preflight must be ready, dynamic extension installation must be disabled, and required IDs and tool versions must exist. The result identifies the tools and Azure context verified before collection. It does not prove that authentication remained valid after preflight.

Inventory is intentionally observational. Its counts can be complete for the declared collection method. Metadata gaps still remain signals for review. They do not become control findings merely because the pack needs one row per input.

```

POWERSHELL
function ConvertFrom-InventorySummary {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [string] $CheckId,

        [Parameter(Mandatory)]
        [string] $SourceFile,

        [Parameter(Mandatory)]
        [string] $SourceSha256,

        [Parameter(Mandatory)]
        [object] $Document
    )

    Assert-ObjectProperty -InputObject $Document -Name @(
        'schema_version',
        'completed_utc',
        'subscription_count',

```

```

        'resource_count',
        'selected_gaps'
    ) -Context 'Inventory summary'
    if ([string] $Document.schema_version -cne '1.0') {
        throw 'Inventory summary schema version must be 1.0.'
    }
    Assert-ObjectProperty -InputObject $Document.selected_gaps -Name @(
        'missing_owner',
        'missing_managed_by'
    ) -Context 'Inventory selected gaps'

    $subscriptionCount = [int] $Document.subscription_count
    $resourceCount = [int] $Document.resource_count
    $missingOwner = [int] $Document.selected_gaps.missing_owner
    $missingManagedBy = [int] $Document.selected_gaps.missing_managed_by
    if (@(
        $subscriptionCount,
        $resourceCount,
        $missingOwner,
        $missingManagedBy
    ) | Where-Object { $_ -lt 0 }) {
        throw 'Inventory summary counts cannot be negative.'
    }

    [pscustomobject][ordered]@{
        check_id = $CheckId
        kind = 'inventory_summary'
        source_file = $SourceFile
        source_sha256 = $SourceSha256
        started_utc = $null
        completed_utc = ConvertTo-UtcTimestamp `
            -Value ([string] $Document.completed_utc) `
            -Context 'Inventory completion time'
        collection_status = 'collected'
        assessment_status = 'observed'
        finding_count = $null
        signal_count = $missingOwner + $missingManagedBy
        claim = 'Scoped inventory counts and selected metadata gaps.'
        excluded = @(
            'Point-in-time consistency across the complete collection window',
            'A control-compliance verdict'
        )
        summary = [ordered]@{
            subscription_count = $subscriptionCount
            resource_count = $resourceCount
            missing_owner = $missingOwner
            missing_managed_by = $missingManagedBy
        }
        findings = @()
    }
}

```

That adapter gives inventory an assessment status of **observed**. Its finding count stays null, while selected gaps become a separate signal count. A pack with inventory signals can therefore remain assembled without claiming that the signals passed a control.

The control adapters use a stricter rule. They require **collected** input, accept only **pass** or **review**, flatten named finding arrays, and recompute the total. A declared status that disagrees with those arrays stops assembly.

Claims and exclusions survive normalization. Hierarchy and access retain their active-assignment boundary. Platform configuration retains its policy, network, monitoring, and

operational exclusions. The common envelope adds comparability without broadening either result.

Adapter code is part of the evidence contract. A source schema change must produce a new supported adapter or a deliberate upgrade. Silently accepting unknown fields or versions would make old and new packs look comparable when they are not.

```
POWERSHELL
function ConvertTo-EvidenceRecord {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [string] $CheckId,

        [Parameter(Mandatory)]
        [ValidateSet(
            'session_context',
            'inventory_summary',
            'hierarchy_access',
            'platform_configuration'
        )]
        [string] $Kind,

        [Parameter(Mandatory)]
        [string] $SourceFile,

        [Parameter(Mandatory)]
        [string] $SourceSha256,

        [Parameter(Mandatory)]
        [object] $Document
    )

    $parameters = @{
        CheckId = $CheckId
        SourceFile = $SourceFile
        SourceSha256 = $SourceSha256
        Document = $Document
    }
    switch ($Kind) {
        'session_context' {
            ConvertFrom-SessionContext @parameters
        }
        'inventory_summary' {
            ConvertFrom-InventorySummary @parameters
        }
        'hierarchy_access' {
            ConvertFrom-HierarchyAccessEvidence @parameters
        }
        'platform_configuration' {
            ConvertFrom-PlatformConfigurationEvidence @parameters
        }
    }
}
```

Every normalized record uses the same outer fields. Those fields cover identity, source hash, time, statuses, counts, claim limits, summary, and findings. Records sort by stable check ID before export. Directory enumeration order cannot change the review sequence.

Collection timestamps remain attached to their original checks. The runner does not rewrite separate observations into one point-in-time snapshot. The pack generation time says when assembly occurred, not when Azure was observed.

The manifest can name results from different collection windows. That choice stays visible. Review policy can reject windows that are too far apart for a particular decision. The generic runner avoids inventing one universal freshness limit.

The main function validates the manifest, scope, IDs, kinds, paths, and source documents before writing. It also refuses an output directory with the same timestamped pack ID. A rerun cannot overwrite the earlier review unit.

POWERSHELL

```
$manifestFullPath = (Resolve-Path -LiteralPath $ManifestPath).Path
$manifestDirectory = Split-Path -Parent $manifestFullPath
$manifestSource = Read-JsonEvidenceSource -LiteralPath $manifestFullPath
$manifest = $manifestSource.document
Assert-ObjectProperty -InputObject $manifest -Name @(
    'schema_version',
    'subject',
    'scope',
    'input'
) -Context 'Evidence-pack manifest'
if ([string] $manifest.schema_version -cne '1.0') {
    throw 'Evidence-pack manifest schema version must be 1.0.'
}
if ([string]::IsNullOrEmpty([string] $manifest.subject)) {
    throw 'Evidence-pack subject is required.'
}

$scope = @(
    $manifest.scope |
    ForEach-Object { ([string] $_).Trim() } |
    Where-Object { -not [string]::IsNullOrEmpty($_) } |
    Sort-Object -Unique
)
if ($scope.Count -eq 0 -or $scope.Count -ne @($manifest.scope).Count) {
    throw 'Evidence-pack scope must contain unique, nonempty values.'
}
if (@($manifest.input).Count -eq 0) {
    throw 'Evidence-pack manifest requires at least one input.'
}

$ids = [System.Collections.Generic.HashSet[string]]::new(
    [StringComparer]::Ordinal
)
$paths = [System.Collections.Generic.HashSet[string]]::new(
    [StringComparer]::OrdinalIgnoreCase
)
$records = [System.Collections.Generic.List[object]]::new()
foreach ($entry in @($manifest.input | Sort-Object id)) {
    Assert-ObjectProperty -InputObject $entry -Name @(
        'id',
        'kind',
        'path'
    ) -Context 'Evidence-pack input'
    $id = [string] $entry.id
    if ($id -cnotmatch '^[a-z][a-z0-9_]{2,63}$' -or
        -not $ids.Add($id)) {
        throw "Invalid or duplicate evidence input ID '$id'."
    }
    $kind = [string] $entry.kind
    if ($kind -cnotin @('
```

```

        'session_context',
        'inventory_summary',
        'hierarchy_access',
        'platform_configuration'
    )) {
        throw "Unsupported evidence input kind '$kind'."
    }

    $relativePath = ([string] $entry.path).Replace('\', '/')
    $resolvedPath = Resolve-EvidenceInputPath `
        -ManifestDirectory $manifestDirectory `
        -RelativePath $relativePath
    if (-not $paths.Add($resolvedPath)) {
        throw "Duplicate evidence input path '$relativePath'."
    }
    $source = Read-JsonEvidenceSource -LiteralPath $resolvedPath
    $record = ConvertTo-EvidenceRecord -CheckId $id -Kind $kind `
        -SourceFile $relativePath -SourceSha256 $source.sha256 `
        -Document $source.document
    $records.Add($record)
}

$generated = $GeneratedUtc.ToUniversalTime()
$stamp = $generated.ToString('yyyyMMddTHHmssfffZ')
$packId = "platform-evidence-$stamp"
$outputDirectory = Join-Path $OutputRoot $packId
if (Test-Path -LiteralPath $outputDirectory) {
    throw "Evidence-pack output '$outputDirectory' already exists."
}

$findingCount = @(
    $records | Where-Object { $null -ne $_.finding_count } |
    ForEach-Object { [int] $_.finding_count }
) | Measure-Object -Sum
$signalCount = @(
    $records | ForEach-Object { [int] $_.signal_count }
) | Measure-Object -Sum
$reviewCount = @(
    $records | Where-Object { $_.assessment_status -ceq 'review' }
).Count
$pack = [pscustomobject][ordered]@{
    schema_version = '1.0'
    pack_id = $packId
    generated_utc = $generated.ToString('o')
    subject = [string] $manifest.subject
    scope = $scope
    pack_status = if ($reviewCount -eq 0) { 'assembled' } else { 'review' }
    source_manifest_sha256 = $manifestSource.sha256
    summary = [ordered]@{
        check_count = $records.Count
        review_check_count = $reviewCount
        observed_check_count = @(
            $records |
            Where-Object { $_.assessment_status -ceq 'observed' }
        ).Count
        finding_count = [int] $findingCount.Sum
        signal_count = [int] $signalCount.Sum
    }
    records = $records.ToArray()
}

if (-not $PSCmdlet.ShouldProcess(
    $outputDirectory,
    'Create Azure evidence pack'
)) {
    return
}

```


Pack status has only two meanings. **review** means that at least one normalized control has findings. **assembled** means that declared inputs were packaged without control findings. Neither value means the Azure platform is healthy, compliant, secure, or effective.

Signals stay visible beside findings. This matters because an inventory gap can require backlog work without failing a declared control. It also prevents a zero finding total from hiding an unowned or untagged resource count.

The JSON output is the complete machine-readable pack. It preserves normalized summaries and findings. It also retains source hashes, claims, and exclusions. Automation can inspect it without reopening every collector-specific document.

The CSV output is a deliberately narrow index. It has one row per check. That shape supports filters, comparisons, and review queues without flattening nested findings into fragile columns. A selected property list fixes the header order.

The Markdown output is for human review. It shows scope, statuses, counts, claims, exclusions, and source hashes, followed by the ownership boundary. It does not reproduce every raw observation or sensitive assignment record.

A fourth file records SHA-256 hashes for the JSON, CSV, and Markdown outputs. Those hashes can confirm that the delivered files still match the assembled set. The hash manifest is not self-hashed, so it must travel through the same protected delivery channel.

POWERSHELL

```
$null = New-Item -ItemType Directory -Force -Path $OutputRoot
$null = New-Item -ItemType Directory -Path $outputDirectory
$jsonPath = Join-Path $outputDirectory 'evidence-pack.json'
$csvPath = Join-Path $outputDirectory 'evidence-summary.csv'
$markdownPath = Join-Path $outputDirectory 'evidence-summary.md'
$manifestOutputPath = Join-Path $outputDirectory 'file-manifest.json'

$pack | ConvertTo-Json -Depth 100 |
    Set-Content -LiteralPath $jsonPath -Encoding utf8NoBOM
$records | Select-Object @(
    'check_id',
    'kind',
    'completed_utc',
    'collection_status',
    'assessment_status',
    'finding_count',
    'signal_count',
    'source_sha256',
    'claim'
) | ConvertTo-Csv -NoTypeInfoation |
    Set-Content -LiteralPath $csvPath -Encoding utf8NoBOM
ConvertTo-EvidencePackMarkdown -Pack $pack |
    Set-Content -LiteralPath $markdownPath -Encoding utf8NoBOM

$fileManifest = [ordered]@{
    schema_version = '1.0'
    pack_id = $packId
    files = @(
        foreach ($path in @($jsonPath, $csvPath, $markdownPath)) {
            [ordered]@{
                name = Split-Path -Leaf $path
```

```

        sha256 = (
            Get-FileHash -Algorithm SHA256 -LiteralPath $path
        ).Hash.ToLowerInvariant()
    }
}
)
}
$fileManifest | ConvertTo-Json -Depth 10 |
    Set-Content -LiteralPath $manifestOutputPath -Encoding utf8NoBOM

[pscustomobject][ordered]@{
    pack_id = $packId
    pack_status = $pack.pack_status
    check_count = $records.Count
    finding_count = [int] $findingCount.Sum
    signal_count = [int] $signalCount.Sum
    output_directory = (Resolve-Path -LiteralPath $outputDirectory).Path
}
}

```

The runner uses a UTC timestamp with millisecond precision in the pack ID. Stable record ordering makes content comparison practical, while the timestamp preserves each run as a separate event. Two packs should not be byte-identical merely because their findings match.

`SupportsShouldProcess` adds a local preview boundary. `-WhatIf` validates inputs and describes the target without creating the pack directory. This does not protect Azure because the runner issues no Azure command at all.

The invocation file accepts only a manifest path and output root. Upstream collectors already performed the approved reads. They produced their own bounded results. The pack step never gains their Azure identities or turns observation into correction.

```

POWERSHELL
. ./New-AzureEvidencePack.ps1

$manifestPath = [Environment]::GetEnvironmentVariable(
    'EVIDENCE_PACK_MANIFEST'
)
$outputRoot = [Environment]::GetEnvironmentVariable(
    'EVIDENCE_PACK_OUTPUT_ROOT'
)
if ([string]::IsNullOrEmpty($manifestPath) -or
    [string]::IsNullOrEmpty($outputRoot)) {
    throw 'Evidence-pack manifest and output-root variables are required.'
}

New-AzureEvidencePack -ManifestPath $manifestPath -OutputRoot $outputRoot |
    ConvertTo-Json -Depth 5

```

That separation makes collection failures clear. If an upstream collector throws, its evidence file is absent. Assembly then fails. A previous successful file must not be substituted unless the manifest and review policy accept its age.

Evidence can contain sensitive platform details even when it has no secrets. Resource IDs, scope structure, role assignments, network routes, findings, and control gaps can aid an attacker. Storage access, retention, transmission, and deletion rules must match that exposure.

The pack copies summaries and findings, not full raw observations. Source hashes let reviewers compare retained originals with the files used during assembly. A deeper investigation can retrieve those originals through the protected evidence store.

Observed gaps return through an owning workflow. A persistent resource difference owned by Terraform becomes a reviewed code change, plan, and apply. The pack itself never calls an imperative Azure update command.

Other gaps need other owners. PIM eligibility and group membership can belong to identity governance. Exemptions or operational response can belong to governance, security, or operations. The finding category and claim boundary keep every difference from being sent to Terraform.

A changed requirement is not drift. It belongs in the decision register or platform backlog before desired state changes. Treating a new expectation as an emergency correction would bypass the design and approval path.

Follow-up records need an owner, decision, due date, and link back to the pack and source finding. Accepted exceptions also need an expiry or review trigger. Without that return path, evidence accumulates while the platform remains unchanged.

Packs become comparable only within compatible contracts. Trend analysis needs the schema, tool versions, scopes, and collection methods. Those details reveal material changes to the evidence contract. A lower finding count can otherwise reflect a narrower query instead of an improved platform.

Reviewers should sample both clean and review packs. A clean case proves that normal inputs assemble, while fault cases prove that missing files, mismatched counts, path escapes, invalid timestamps, and overwrite attempts fail closed. The fixture covers both paths without Azure access.

Audit value comes from repeatability and clear limits, not file quantity. JSON supports automation, while CSV supports triage. Markdown supports human reading, and hashes support change detection. None replaces the source systems, approvals, or judgment behind a decision. An assembled pack closes Chapter 6's observation workflow. The chapter began with protected session context. It then expanded through inventory, hierarchy, access, policy, connectivity, and monitoring evidence. The final artifact carries those bounded observations into a stable review process.

The next movement is adoption. A platform that can explain its state and route its gaps can offer clearer capabilities to workload teams. Evidence then becomes an input to the platform service, not an archive that waits for the next audit.

Chapter 7: Adopt — Build the Platform-to-Workload Interface

Chapter 7.1: Treat the Platform as a Product

Adopt — Build the Platform-to-Workload Interface begins with a maintained offer. A deployed foundation is only the starting point. Workload teams need useful capabilities, clear choices, and dependable support. Resource delivery alone cannot meet that need.

A project optimizes for a defined completion. Its team can disband after the approved scope is delivered. A platform product carries responsibility beyond the launch date.

Product responsibility does not require external sales or profit. It does require named customers and valued outcomes. Durable ownership and evidence keep the offer useful. The platform team must learn after the infrastructure works.

The primary customer is the workload team. That team must deliver and operate a service. Engineers, delivery leads, and operators can experience the same platform differently. A generic developer persona hides those distinct jobs.

Security, governance, finance, and architecture teams are important stakeholders. They supply shared requirements. They also judge effects across the system. Calling every stakeholder a customer can hide whose daily work needs help.

Customer definitions need evidence. Interviews and delivery observations expose real work. Request histories, support records, and abandoned paths add other views. Assumed needs remain guesses until behavior or accountable review supports them.

The value proposition connects those needs to a platform outcome. The capstone offers a governed Azure foundation. Teams can understand, request, and operate it. The promise is not access to a pile of subscriptions and policies.

A product statement should name the customer problem it removes. Repeated setup decisions, unclear handoffs, and late control integration are valid problems. A favored tool or portal is not a problem statement.

That distinction limits solution bias. A new portal may add another interface without reducing effort. Discovery can improve an existing tool or document first. A new surface needs evidence that it will remove a real burden.

The product canvas makes these choices reviewable. It records customers, capabilities, and service expectations. It also records constraints, feedback, lifecycle rules, measures, and ownership. Each field turns a broad claim into a maintained decision.

A platform capability is an outcome that customers can use. People, process, automation, and infrastructure support it. Foundation, connectivity, identity, governance, and management can each be capabilities. A virtual network or policy assignment alone is only a component.

Capabilities belong in a service catalog. Each entry names the outcome, customers, interface, and owner. It also gives the current status, documentation, service expectations, and feedback routes. Discovery then starts with known value instead of organizational memory.

The catalog is not a static inventory. Active entries need current documents. Their interfaces must work as published. Deprecated entries need visible alternatives and dates.

Catalog scope also protects focus. The capstone offers platform and application landing-zone foundations. It does not deploy workload resources. That boundary preserves workload architecture and runtime ownership.

Constraints must be published beside value. Region and network limits can change whether a capability fits. Inherited controls and evidence limits can do the same. Hidden constraints turn predictable choices into late queue failures.

An exception entrypoint does not promise approval. It makes a nonstandard need visible and routes the decision to the right authority. Chapter 7.3 will define the detailed exception workflow.

Service expectations turn words such as fast and supported into visible behavior. A useful target has clear clock boundaries. It also has a time window, evidence source, exclusions, and escalation. Without those fields, teams can report opposite results from one request.

The canvas labels its numerical expectations as targets. A target states the intended behavior. Enough operating data does not yet exist. The target must not be reported as an achieved service level.

Clock boundaries prevent easy measurement games. Fulfillment starts when the stated inputs and approvals exist. Any excluded approval period must be explicit. Completion waits until the customer receives usable outputs and evidence.

Exclusions require the same discipline. Custom connectivity can sit outside a standard target. So can an external procurement delay. Repeated exclusions still need review because they may reveal a weak offer.

Support response is different from issue resolution. A qualified response confirms scope and ownership. It also gives the next diagnostic action. Resolution time depends on cause, authority, dependencies, and workload participation.

Ownership spans several decisions. The product owner ranks outcomes and roadmap work. Engineering and operations leads own technical and service readiness. A sponsor funds the offer and approves material retirement.

Those roles cannot collapse into a team name. Decision rights name who can change scope or accept a release. They also cover service expectations and retirement. Clear authority helps the product survive staff movement.

Feedback also needs more than one route. A support queue restores or unblocks known capabilities. Discovery sessions test problems before a roadmap commitment.

A customer pulse measures effort, trust, and repeated friction. A contribution flow serves another purpose. It lets workload and specialist teams improve documents, templates, and capabilities. Separate channels keep support urgency from becoming the only product signal. Every channel needs an owner and response expectation. Collecting comments is not a feedback loop. Customers stop contributing when decisions stay hidden. A closed loop records the learning, decision, owner, and follow-up evidence.

Feedback is not feature voting. A common request can have several meanings. It may show a shared problem, local taste, or weak documentation. Discovery tests the underlying job before selecting a solution.

Measures should connect platform work to outcomes. Lead time and first-time readiness show speed and quality. Customer effort, capability use, and exception rework add other views. One number cannot represent product health.

Adoption counts need a denominator. They also need an eligibility rule. Raw use can rise because the platform is mandatory or the estate grew. Low use is a discovery signal, not proof of customer resistance.

Experience measures need context too. A median effort score can hide a narrow sample. Response count and role mix expose that weakness. Interviews help explain the pattern before roadmap action.

Guardrails keep speed from becoming the only success condition. Faster fulfillment paired with more exceptions or platform-owned rework is a trade-off, not a clean gain. Product review must show both sides.

The canvas keeps baselines in a collecting state. Synthetic targets can test the contract. They cannot prove actual performance. Evidence turns an aspiration into a measured service statement.

The complete capstone canvas follows. JSON makes ownership, references, and lifecycle dates testable. A separate schema checks the structure. Semantic tests check identity and reference rules.

```
JSON
{
  "schema_version": "1.0",
  "product": {
    "id": "azure_platform_foundation",
    "name": "Azure Platform Foundation",
    "status": "active",
    "value_proposition": "Give workload teams a governed Azure foundation they can understand, request, and operate without rebuilding shared identity, connectivity, management, and governance controls.",
    "problems_addressed": [
      "Workload teams repeat platform decisions and integrations.",
      "Manual handoffs obscure request state and ownership.",
      "Shared controls arrive late or vary between subscriptions."
    ],
    "ownership": {
      "product_owner_role": "platform product owner",
      "engineering_owner_role": "platform engineering lead",
      "operations_owner_role": "platform operations lead",
      "sponsor_role": "cloud steering group",
      "decision_rights": [
        {
          "decision": "Prioritize capability outcomes and roadmap work.",
          "accountable_role": "platform product owner"
        },
        {
          "decision": "Approve architecture and release readiness.",
          "accountable_role": "platform engineering lead"
        },
        {
          "decision": "Approve service expectations and operational escalation.",
          "accountable_role": "platform operations lead"
        },
        {
          "decision": "Approve material investment and product retirement.",
          "accountable_role": "cloud steering group"
        }
      ]
    },
    "catalog_entrypoint": "Internal engineering catalog entry for Azure Platform Foundation",
    "support_entrypoint": "Platform support queue"
  },
  "customers": [
    {
      "id": "workload_delivery",
      "kind": "primary_customer",
      "name": "Workload delivery team",
      "jobs": [
        "Obtain an approved Azure environment for a workload.",
        "Deploy through supported interfaces with clear inherited controls."
      ],
      "needs": [
        "A discoverable capability catalog.",
        "Predictable request progress and completion evidence.",
        "A clear boundary between platform and workload work."
      ],
      "evidence": [
        "Synthetic capstone walkthroughs show repeated setup and handoff questions.",
        "The Chapter 6 evidence pack identifies platform controls that can be offered consistently."
      ]
    }
  ]
}
```

```

    "id": "workload_operations",
    "kind": "secondary_customer",
    "name": "Workload operations team",
    "jobs": [
      "Operate a workload within known platform boundaries.",
      "Find the correct support and escalation path."
    ],
    "needs": [
      "Published service expectations and exclusions.",
      "Visible change, deprecation, and retirement notices."
    ],
    "evidence": [
      "Capstone responsibility reviews require a durable platform-to-workload operating
boundary."
    ]
  },
  {
    "id": "risk_stakeholders",
    "kind": "stakeholder",
    "name": "Security, governance, finance, and architecture stakeholders",
    "jobs": [
      "Express reusable requirements without joining every workload delivery path.",
      "Review evidence that shared controls remain effective."
    ],
    "needs": [
      "Named decision rights and control ownership.",
      "Measures that pair adoption outcomes with guardrails."
    ],
    "evidence": [
      "Capstone governance and access checks produce reusable findings and explicit
exclusions."
    ]
  }
],
"capabilities": [
  {
    "id": "subscription_foundation",
    "name": "Application landing-zone foundation",
    "lifecycle_status": "active",
    "customer_ids": [
      "workload_delivery",
      "workload_operations",
      "risk_stakeholders"
    ],
    "customer_outcome": "A workload receives an approved subscription placement with
inherited platform controls and a reviewable handoff.",
    "interface_summary": "Catalog request followed by validated inputs, automated
foundation delivery, and completion evidence.",
    "owner_role": "platform engineering lead",
    "service_expectation_ids": [
      "request_acknowledgement",
      "standard_fulfillment"
    ],
    "feedback_channel_ids": [
      "support_queue",
      "product_discovery",
      "quarterly_pulse"
    ],
    "documentation": "Application landing-zone foundation catalog page"
  },
  {
    "id": "identity_access_foundation",
    "name": "Identity and access foundation",
    "lifecycle_status": "active",
    "customer_ids": [
      "workload_delivery",
      "workload_operations",
      "risk_stakeholders"
    ],
    "customer_outcome": "A workload starts with approved deployment access, scoped
operational access, and visible ownership boundaries.",

```



```

    "interface_summary": "Declared workload roles map to reviewed identity groups and
deployment-principal access.",
    "owner_role": "identity platform owner",
    "service_expectation_ids": [
        "request_acknowledgement",
        "support_response"
    ],
    "feedback_channel_ids": [
        "support_queue",
        "product_discovery",
        "contribution_flow"
    ],
    "documentation": "Identity and access foundation catalog page"
},
{
    "id": "network_integration",
    "name": "Shared network integration",
    "lifecycle_status": "active",
    "customer_ids": [
        "workload_delivery",
        "workload_operations"
    ],
    "customer_outcome": "A workload can use a documented shared connectivity pattern
without redesigning the hub network.",
    "interface_summary": "Supported connectivity choice, address requirements, DNS
needs, and route evidence enter the platform workflow.",
    "owner_role": "connectivity platform owner",
    "service_expectation_ids": [
        "request_acknowledgement",
        "standard_fulfillment",
        "support_response"
    ],
    "feedback_channel_ids": [
        "support_queue",
        "product_discovery"
    ],
    "documentation": "Shared network integration catalog page"
},
{
    "id": "governance_baseline",
    "name": "Governance baseline",
    "lifecycle_status": "active",
    "customer_ids": [
        "workload_delivery",
        "workload_operations",
        "risk_stakeholders"
    ],
    "customer_outcome": "A workload inherits known policy and metadata controls before
resource delivery begins.",
    "interface_summary": "The platform publishes inherited controls, evidence, and the
entrypoint for a reviewed exception.",
    "owner_role": "governance platform owner",
    "service_expectation_ids": [
        "request_acknowledgement",
        "support_response"
    ],
    "feedback_channel_ids": [
        "support_queue",
        "product_discovery",
        "contribution_flow"
    ],
    "documentation": "Governance baseline catalog page"
},
{
    "id": "management_baseline",
    "name": "Management and evidence baseline",
    "lifecycle_status": "active",
    "customer_ids": [
        "workload_delivery",
        "workload_operations",
        "risk_stakeholders"
    ]
}

```

```

    ],
    "customer_outcome": "A workload receives documented monitoring destinations,
evidence boundaries, and platform support routes.",
    "interface_summary": "Foundation delivery attaches diagnostic configuration,
evidence outputs, support routes, and ownership notes.",
    "owner_role": "platform operations lead",
    "service_expectation_ids": [
        "standard_fulfillment",
        "support_response"
    ],
    "feedback_channel_ids": [
        "support_queue",
        "quarterly_pulse",
        "contribution_flow"
    ],
    "documentation": "Management and evidence baseline catalog page"
}
],
"service_expectations": [
    {
        "id": "request_acknowledgement",
        "name": "Standard request acknowledgement",
        "commitment_type": "target",
        "target": "At least 95 percent of complete standard requests are acknowledged within
one business day.",
        "clock_start": "The request passes automated completeness checks.",
        "clock_stop": "The requester receives an owner, state, and next action.",
        "measurement_window_days": 90,
        "evidence_source": "Timestamped request workflow events",
        "exclusions": [
            "Requests that fail completeness checks",
            "Approved maintenance freezes"
        ],
        "escalation": "Platform product owner after one missed business day"
    },
    {
        "id": "standard_fulfillment",
        "name": "Standard foundation fulfillment",
        "commitment_type": "target",
        "target": "At least 90 percent of approved standard foundations are handed off
within three business days.",
        "clock_start": "All required approvals and conflict-free inputs are present.",
        "clock_stop": "The requester receives usable foundation outputs and completion
evidence.",
        "measurement_window_days": 90,
        "evidence_source": "Timestamped workflow, deployment, and handoff records",
        "exclusions": [
            "Nonstandard connectivity",
            "External quota or procurement delays",
            "Approved maintenance freezes"
        ],
        "escalation": "Platform engineering lead when the forecast exceeds three business
days"
    },
    {
        "id": "support_response",
        "name": "Platform support response",
        "commitment_type": "target",
        "target": "Priority-two platform support requests receive a qualified response
within four business hours.",
        "clock_start": "The support queue accepts a correctly classified request.",
        "clock_stop": "A responder confirms scope, ownership, and the next diagnostic
action.",
        "measurement_window_days": 90,
        "evidence_source": "Support queue state transitions",
        "exclusions": [
            "Workload code defects",
            "Unapproved service use",
            "Priority-one incident procedures"
        ],
        "escalation": "Platform operations lead after two business hours without assignment"
    }
]

```

```

    }
  ],
  "constraints": [
    {
      "id": "platform_scope_only",
      "statement": "The product supplies platform and application landing-zone foundations but does not deploy workload resources.",
      "reason": "Workload teams retain application architecture and runtime ownership.",
      "exception_entrypoint": "Platform exception backlog"
    },
    {
      "id": "approved_regions",
      "statement": "The standard offer supports westeurope and northeurope.",
      "reason": "The capstone has reviewed identity, network, governance, and management dependencies only for these regions.",
      "exception_entrypoint": "Architecture decision intake"
    },
    {
      "id": "declared_connectivity",
      "statement": "Standard fulfillment uses the published shared-network patterns.",
      "reason": "Custom routing or hybrid dependencies need separate design and validation.",
      "exception_entrypoint": "Connectivity design intake"
    },
    {
      "id": "inherited_controls",
      "statement": "Workload teams cannot remove inherited platform policy or diagnostic settings through the standard interface.",
      "reason": "Shared guardrails and evidence remain platform-owned controls.",
      "exception_entrypoint": "Time-limited control exception intake"
    },
    {
      "id": "measured_targets",
      "statement": "Service targets remain provisional until a 90-day baseline is available.",
      "reason": "A synthetic canvas must not present unmeasured targets as achieved service levels.",
      "exception_entrypoint": "Platform product review"
    }
  ],
  "feedback_channels": [
    {
      "id": "support_queue",
      "kind": "support",
      "purpose": "Restore a blocked supported capability or route an operational question.",
      "channel": "Platform support queue",
      "audience": [
        "workload delivery teams",
        "workload operations teams"
      ],
      "owner_role": "platform operations lead",
      "response_expectation": "Queue priority and service target are visible to the requester.",
      "review_cadence_days": 30
    },
    {
      "id": "product_discovery",
      "kind": "discovery",
      "purpose": "Test customer problems and proposed outcomes before roadmap commitment.",
      "channel": "Monthly workload-team discovery sessions",
      "audience": [
        "workload delivery teams",
        "workload operations teams"
      ],
      "owner_role": "platform product owner",
      "response_expectation": "Learnings and resulting decisions are published within five business days.",
      "review_cadence_days": 30
    }
  ],

```

```

{
  "id": "quarterly_pulse",
  "kind": "feedback",
  "purpose": "Measure customer effort, trust, and the most costly points of
friction.",
  "channel": "Quarterly platform customer pulse",
  "audience": [
    "active platform customers"
  ],
  "owner_role": "platform product owner",
  "response_expectation": "Themes, sample size, and chosen actions appear in the
quarterly product review.",
  "review_cadence_days": 90
},
{
  "id": "contribution_flow",
  "kind": "contribution",
  "purpose": "Accept reviewed documentation, template, and capability improvements
from customer teams.",
  "channel": "Platform repository contribution workflow",
  "audience": [
    "workload engineers",
    "security and governance specialists"
  ],
  "owner_role": "platform engineering lead",
  "response_expectation": "A maintainer triages a complete contribution within three
business days.",
  "review_cadence_days": 30
}
],
"lifecycle": {
  "current_version": "1.0.0",
  "last_reviewed_on": "2026-07-20",
  "next_review_on": "2026-10-18",
  "review_cadence_days": 90,
  "change_rules": {
    "minimum_notice_days": 30,
    "breaking_change_requires_version_change": true,
    "communication_channels": [
      "Engineering catalog notice",
      "Platform change feed",
      "Direct notice to recorded capability owners"
    ]
  },
  "retirement_rules": {
    "minimum_deprecation_notice_days": 90,
    "active_consumers_require_exit_plan": true,
    "successor_or_exception_required": true,
    "approval_role": "cloud steering group",
    "retirement_criteria": [
      "Every active consumer has migrated, accepted a time-bound exception, or has an
approved exit plan.",
      "Support, documentation, automation, and ownership records name the retirement
date.",
      "Completion evidence confirms that the retired interface no longer accepts new
requests."
    ]
  }
},
"measures": [
  {
    "id": "usable_foundation_lead_time",
    "kind": "outcome",
    "question": "How long does a complete standard request take to become a usable
foundation?",
    "calculation": "Median and 85th percentile business time from validated input to
accepted handoff.",
    "evidence_source": "Request workflow and handoff timestamps",
    "review_cadence_days": 30,
    "baseline_status": "collecting",

```

```

    "decision_rule": "Investigate delay stages before adding fulfillment capacity or
automation."
  },
  {
    "id": "first_time_ready_rate",
    "kind": "quality",
    "question": "How often can a team use the delivered foundation without platform
rework?",
    "calculation": "Accepted handoffs without platform-owned rework divided by all
accepted handoffs.",
    "evidence_source": "Handoff acceptance and linked rework records",
    "review_cadence_days": 30,
    "baseline_status": "collecting",
    "decision_rule": "Prioritize the recurring platform-owned cause instead of coaching
around it."
  },
  {
    "id": "customer_effort",
    "kind": "experience",
    "question": "How much effort do customer teams spend finding, requesting, and
adopting a capability?",
    "calculation": "Median customer-effort response with response count and role
distribution.",
    "evidence_source": "Quarterly platform customer pulse",
    "review_cadence_days": 90,
    "baseline_status": "collecting",
    "decision_rule": "Pair a worsening score with interviews before selecting a
solution."
  },
  {
    "id": "supported_capability_use",
    "kind": "adoption",
    "question": "Do eligible workload teams choose supported platform capabilities?",
    "calculation": "Eligible active workload teams using at least one supported
capability divided by all eligible active workload teams.",
    "evidence_source": "Catalog, request, and portfolio records",
    "review_cadence_days": 90,
    "baseline_status": "collecting",
    "decision_rule": "Treat low use as a discovery signal, not proof that customers are
resisting change."
  },
  {
    "id": "exception_rework_rate",
    "kind": "guardrail",
    "question": "Does faster adoption create more exceptions or platform-owned rework?",
    "calculation": "Requests with a new exception or platform-owned rework divided by
completed requests.",
    "evidence_source": "Exception backlog and linked rework records",
    "review_cadence_days": 30,
    "baseline_status": "collecting",
    "decision_rule": "Do not celebrate faster fulfillment when guardrail cost rises
without an approved trade-off."
  }
]
}

```

Schema validation does not prove that the product is valuable. It proves that the canvas has the required shape and allowed values. Customer evidence and operating measures must support the claims.

The semantic validator covers rules beyond basic field checks. It rejects duplicate IDs and unknown references. Unused customers, expectations, and channels also fail. Support, customer-learning, outcome, experience, and guardrail coverage are required.

Those failures matter because a polished document can still be disconnected. A service target that no capability uses is catalog debris. A named customer with no offered outcome is not served by the product.

Lifecycle begins before retirement. The canvas has a product version and review page. It records the last and next reviews. Change-notice rules make maintenance part of the original offer.

Versioning gives customer teams a stable reference. A breaking interface change needs a new version. It also needs a notice. Silent changes transfer platform ease into workload risk.

Deprecation is a supported transition state. It is not abandonment. New use can stop while current consumers receive an alternative and time to move. Support and communication remain explicit until retirement.

Retirement ends availability and support. Active consumers need a safe exit. That can be a migration, time-bound exception, or approved plan. A successor or exception keeps retirement from becoming an unowned outage.

Retirement evidence must prove more than an announcement. Documents and support routes must name the date. Automation and catalog state must agree. The retired interface must stop accepting new requests.

A product review joins all canvas sections. It examines service evidence, customer learning, and measures. Constraints, lifecycle, investment, and risk complete the view. Roadmap choices then follow the whole product picture.

The maintained catalog is the public result of that review. A capability can start in discovery and run as a pilot. It can then become active, deprecated, or retired. Status must reflect actual support rather than roadmap optimism.

This approach does not require a big-bang internal developer portal. A thinnest viable platform can start smaller. It can connect existing tools around one valuable journey. Cohesion and ownership matter more than surface count.

The platform team need not build every shared service. Specialist teams can own identity, connectivity, security, or operations capabilities. Consistent catalog entries join those offers. The product owner keeps the customer journey coherent.

A product boundary does not erase the operating model. Platform teams own the shared foundation and its interfaces. Workload teams remain accountable for their applications within those guardrails.

The detailed landing-zone request contract remains outside this section. Chapter 7.2 will define its inputs and outputs. It will also define inherited controls and handoff evidence. This canvas only names the capability and expected service behavior.

Paved-road choices also remain later work. Chapter 7.3 will balance supported paths and autonomy. It will also define exceptions. The canvas supplies the customer and lifecycle context for those choices.

Operational service design continues in Chapter 10. This section states support and fulfillment expectations. It does not design every alert, incident role, or reliability objective. Product ownership creates the duty that later operations must fulfill.

The anchor is simple. A platform becomes a product when someone owns customer value after delivery. Infrastructure is the implementation. Maintained value is the product.

Chapter 7: Adopt — Build the Platform-to-Workload Interface

Chapter 7.2: Design the Application Landing-Zone Contract

An application landing-zone contract turns a platform offer into a clear exchange. The workload team declares facts about its service. The platform returns a governed foundation and evidence about that result.

This exchange is the working edge of subscription vending. One front door collects the facts needed to create and configure subscriptions. Reviewed automation then applies the platform's decisions in a repeatable way.

The interface has four distinct stages. Validation accepts or rejects declared intent, and mapping translates accepted fields. Fulfillment changes Azure, while handoff returns the result and its evidence.

The contract is not an order form for workload resources. It records intent that affects a reusable foundation. Application services, data stores, and runtime settings stay outside its scope.

That boundary divides authority as well as work. The workload team knows its purpose, environment, owners, data, regions, and access needs. The platform team owns placement, shared integration, inherited controls, and fulfillment evidence.

A good request asks only for facts that change an approved platform action. Every field needs a named consumer or control. Unused questions create friction without improving the result.

The request also avoids choices owned by the platform. A team can propose a subscription alias and display name. It cannot omit the governance baseline or select arbitrary placement.

The capstone has one routine workload branch named `landing-zones`. Wider estates can derive placement from network, governance, or data needs. That business logic belongs behind the request interface.

Stable identity begins the request. The request ID correlates validation, fulfillment, and handoff evidence. The workload ID links environment subscriptions to one service record.

Purpose explains why the foundation is needed. Environment changes the commercial and resilience mapping. Criticality and data classification preserve risk context for later design work.

Business and technical ownership are both required. A business owner represents accountable service value. A technical owner maintains the request and receives operational handoff.

References are safer than copied identity objects. The owner resolves to a user or service principal. Groups carry team access, while a managed identity supports deployment without a stored secret.

Dates need precise meanings. `submitted_utc` records when declared input entered the interface. `required_by` states demand, but does not promise fulfillment.

Environment supplies billing intent in this capstone. Production maps to a production route. Development and test map to DevTest, where the commercial agreement permits it.

The workload does not submit that route directly. PowerShell derives it from environment after validation. Fulfillment must still resolve the actual billing scope and available offer.

Region choices are also bounded. The capstone accepts West Europe and North Europe. A production request needs a distinct secondary region.

The secondary region is not proof of resilience. It only preserves a platform input for later workload design. The workload team still owns failover behavior, recovery targets, and application testing.

This capstone publishes two connectivity profiles. Connected uses platform private DNS and requested private address space. Isolated uses Azure default DNS and no hub-routed address allocation.

Address space is an early coordination need. Local validation checks RFC 1918 scope, prefix alignment, and overlap inside one request. It does not reserve space or compare the wider network estate.

Platform-wide IP address management belongs to fulfillment. It must compare approved allocations before reserving a range. A locally valid CIDR can still conflict with another subscription.

Hybrid connectivity is valid only with the connected profile. Public inbound exposure also changes risk. The schema therefore requires an exception reference for public exposure.

An exception reference is not an approval. It links the request to a separate decision record. Chapter 7.3 will define the detailed path, expiry rules, and approval authority.

Metadata carries the contract across tools. A service-catalog ID connects the request to the maintained offer. Standard tags remain governed, while bounded custom tags carry local context.

Budget intent belongs in intake too. The request supplies a monthly estimate, currency, and notification contact. It does not make the budget a spending limit.

Platform-owned keys cannot be replaced through custom metadata. Business unit, cost center, classification, environment, owner, and workload ID come from governed fields. Chapter 8.5 retains the detailed cost-control design.

JSON Schema provides the public validation layer. It checks required objects, types, patterns, enumerations, and conditional combinations. Unknown properties fail because every accepted value needs defined behavior.

The connectivity fragment shows that principle. The selected profile changes the valid address and DNS combination. This catches conflicting choices before a platform queue receives them.

```
JSON
{
  "connectivity": {
    "type": "object",
    "required": [
      "profile",
      "address_spaces",
      "dns_profile",
      "hybrid_connectivity",
      "inbound_exposure"
    ],
    "properties": {
      "profile": { "enum": ["connected", "isolated"] },
      "address_spaces": {
        "type": "array",
        "maxItems": 4,
        "uniqueItems": true,
        "items": {
          "type": "string",
          "pattern": "^(?:[0-9]{1,3}\\\\.){3}[0-9]{1,3}/(?:[8-9]|1[0-9]|2[0-9])$"
        }
      },
      "dns_profile": {
        "enum": ["platform_private_dns", "azure_default"]
      },
      "hybrid_connectivity": { "type": "boolean" },
      "inbound_exposure": { "enum": ["none", "private", "public"] }
    },
    "allOf": [
      {
        "if": {
          "properties": { "profile": { "const": "connected" } },
          "required": ["profile"]
        },
        "then": {
          "properties": {
            "address_spaces": { "minItems": 1 },
            "dns_profile": { "const": "platform_private_dns" }
          }
        }
      },
      {
        "if": {
          "properties": { "profile": { "const": "isolated" } },
          "required": ["profile"]
        },
        "then": {
          "properties": {
            "address_spaces": { "maxItems": 0 },
            "dns_profile": { "const": "azure_default" },
            "hybrid_connectivity": { "const": false }
          }
        }
      }
    ]
  },
}
```

```
"additionalProperties": false
},
```

The schema also applies cross-field rules. Production must include a secondary region. Public inbound exposure must carry at least one exception reference.

A concrete request makes the interface easier to inspect. This synthetic order API needs a connected production foundation. Its identifiers are stable references, not real directory or subscription identifiers.

JSON

```
{
  "schema_version": "1.0",
  "request_id": "lzn-order-api-prod",
  "submitted_utc": "2026-07-20T10:00:00Z",
  "required_by": "2026-08-14",
  "workload": {
    "id": "wl-order-api",
    "name": "Order API",
    "purpose": "Process customer orders for the commerce business unit.",
    "environment": "production",
    "criticality": "business-critical",
    "data_classification": "confidential",
    "business_unit": "commerce",
    "cost_center": "cc-commerce-1042",
    "business_owner_ref": "role-commerce-owner",
    "technical_owner_ref": "team-order-api"
  },
  "subscription": {
    "alias": "order-api-prod",
    "display_name": "Order API Production"
  },
  "regions": {
    "primary": "westeurope",
    "secondary": "northeurope"
  },
  "connectivity": {
    "profile": "connected",
    "address_spaces": [
      "10.42.0.0/23"
    ],
    "dns_profile": "platform_private_dns",
    "hybrid_connectivity": true,
    "inbound_exposure": "private"
  },
  "access": {
    "subscription_owner_ref": "usr-order-api-owner",
    "workload_operator_group_ref": "grp-order-api-operators",
    "workload_reader_group_ref": "grp-order-api-readers",
    "deployment_principal_ref": "mi-order-api-deploy"
  },
  "metadata": {
    "service_catalog_id": "svc-order-api",
    "custom_tags": {
      "regulatory-scope": "pci",
      "service-tier": "gold"
    }
  },
  "budget": {
    "monthly_amount": 5000,
    "currency": "EUR",
    "contact_ref": "grp-commerce-finops"
  },
  "exception_refs": []
}
```

Schema validity is necessary but not sufficient. Some relationships need procedural checks with clearer errors. The second layer checks dates, identity separation, reserved tags, CIDR alignment, and overlap.

Validation errors are part of the product interface. A rejection should identify the field and the violated rule. Workload teams should not need platform access to diagnose ordinary input faults.

Accepted input also needs durable correlation. The request ID and normalized UTC time link workflow records. The source hash identifies the exact bytes that entered mapping.

The validator reads one exact byte sequence. It decodes strict UTF-8, parses JSON, and hashes those same bytes. The resulting digest can identify the request that fulfillment used.

```
POWERSHELL
function Read-ContractJson {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [ValidateNotNullOrEmpty()]
        [string] $LiteralPath
    )

    if (-not (Test-Path -LiteralPath $LiteralPath -PathType Leaf)) {
        throw "Contract file '$LiteralPath' does not exist."
    }
    $fullPath = (Resolve-Path -LiteralPath $LiteralPath).Path
    $bytes = [IO.File]::ReadAllBytes($fullPath)
    if ($bytes.Count -eq 0) {
        throw "Contract file '$LiteralPath' is empty."
    }

    try {
        $text = [Text.UTF8Encoding]::new($false, $true).GetString($bytes)
        $document = $text | ConvertFrom-Json -Depth 100
    }
    catch {
        throw "Contract file '$LiteralPath' is not valid UTF-8 JSON."
    }

    [pscustomobject][ordered]@{
        full_path = $fullPath
        text = $text
        document = $document
        sha256 = [Convert]::ToHexString(
            [Security.Cryptography.SHA256]::HashData($bytes)
        ).ToLowerInvariant()
    }
}
```

PowerShell passes the original JSON text to `Test-Json`. The schema file defines the public shape. A failed structural check stops processing before any output directory exists.

Semantic checks fail closed too. The example below compares dates and role references. It also blocks custom values from taking over standard tag keys.

```
POWERSHELL
function Assert-LandingZoneRequest {
    [CmdletBinding()]
    param(
```

```

        [Parameter(Mandatory)]
        [object] $Request
    )

    $submitted = [datetimeoffset]::Parse(
        [string] $Request.submitted_utc,
        [Globalization.CultureInfo]::InvariantCulture
    )
    $requiredBy = [datetime]::ParseExact(
        [string] $Request.required_by,
        'yyyy-MM-dd',
        [Globalization.CultureInfo]::InvariantCulture
    )
    if ($requiredBy.Date -lt $submitted.UtcDateTime.Date) {
        throw 'required_by cannot precede the submission date.'
    }
    if ($null -ne $Request.regions.secondary -and
        [string] $Request.regions.primary -ceq
        [string] $Request.regions.secondary) {
        throw 'Primary and secondary regions must differ.'
    }
    if ([string] $Request.access.workload_operator_group_ref -ceq
        [string] $Request.access.workload_reader_group_ref) {
        throw 'Operator and reader group references must differ.'
    }

    $reservedTags = @(
        'business-unit',
        'cost-center',
        'data-classification',
        'environment',
        'technical-owner',
        'workload-id'
    )
    foreach ($property in $Request.metadata.custom_tags.PSObject.Properties) {
        if ([string] $property.Name -cin $reservedTags) {
            throw "Custom tag '$($property.Name)' is reserved by the platform mapping."
        }
    }

    $ranges = @(
        foreach ($cidr in @($Request.connectivity.address_spaces)) {
            Get-Ipv4CidrRange -Cidr ([string] $cidr)
        }
    )
    for ($left = 0; $left -lt $ranges.Count; $left++) {
        for ($right = $left + 1; $right -lt $ranges.Count; $right++) {
            if ($ranges[$left].start -le $ranges[$right].end -and
                $ranges[$right].start -le $ranges[$left].end) {
                throw "Address spaces '$($ranges[$left].cidr)' and
'$($ranges[$right].cidr)' overlap."
            }
        }
    }
}

```

Validation and mapping are separate acts. Validation proves declared intent meets the public contract. Mapping adds platform decisions and forms a private automation input.

The capstone maps routine application subscriptions beneath [landing-zones](#). Billing intent is derived from environment. Standard tags are built from governed request fields.

Terraform supplies a second typed boundary. Its object type rejects unexpected mapped shapes.

Variable rules preserve placement, regions, commercial mapping, connectivity, and standard tags.

HCL / TERRAFORM

```
variable "landing_zone_request" {
  description = "Validated application landing-zone request mapped by PowerShell."
  type = object({
    schema_version = string
    request_id      = string
    submitted_utc   = string
    required_by     = string
    workload = object({
      id              = string
      name            = string
      purpose         = string
      environment     = string
      criticality     = string
      data_classification = string
      business_owner_ref = string
      technical_owner_ref = string
    })
    subscription = object({
      alias              = string
      display_name       = string
      management_group_id = string
      billing_offer      = string
    })
    regions = object({
      primary = string
      secondary = string
    })
    connectivity = object({
      profile              = string
      address_spaces       = list(string)
      dns_profile          = string
      hybrid_connectivity = bool
      inbound_exposure    = string
    })
    access = object({
      subscription_owner_ref      = string
      workload_operator_group_ref = string
      workload_reader_group_ref  = string
      deployment_principal_ref    = string
    })
    metadata = object({
      service_catalog_id = string
    })
    budget = object({
      monthly_amount = number
      currency       = string
      contact_ref    = string
    })
    tags          = map(string)
    exception_refs = set(string)
  })

  validation {
    condition = (
      var.landing_zone_request.schema_version == "1.0" &&
      can(regex("^lzh-[a-z0-9-]{3,60}$", var.landing_zone_request.request_id))
    )
    error_message = "The mapped request must use schema 1.0 and a valid request ID."
  }

  validation {
    condition = (
      var.landing_zone_request.subscription.management_group_id == "landing-zones" &&
      contains(
        ["development", "test", "production"],
        var.landing_zone_request.workload.environment
      ) &&
      contains(
        ["westeurope", "northeurope"],
        var.landing_zone_request.regions.primary
      )
    )
  }
}
```

```

    ) &&
    (
        var.landing_zone_request.regions.secondary == null ||
        contains(
            ["westeurope", "northeurope"],
            var.landing_zone_request.regions.secondary
        )
    ) &&
    var.landing_zone_request.regions.primary !=
var.landing_zone_request.regions.secondary
    )
    error_message = "Placement and regions must stay inside the capstone contract."
}

validation {
    condition = (
        (
            var.landing_zone_request.workload.environment == "production" &&
            var.landing_zone_request.subscription.billing_offer == "production" &&
            var.landing_zone_request.regions.secondary != null
        ) ||
        (
            var.landing_zone_request.workload.environment != "production" &&
            var.landing_zone_request.subscription.billing_offer == "devtest"
        )
    )
    error_message = "Environment, billing offer, and production region mapping disagree."
}

validation {
    condition = (
        (
            var.landing_zone_request.connectivity.profile == "connected" &&
            length(var.landing_zone_request.connectivity.address_spaces) > 0 &&
            var.landing_zone_request.connectivity.dns_profile == "platform_private_dns"
        ) ||
        (
            var.landing_zone_request.connectivity.profile == "isolated" &&
            length(var.landing_zone_request.connectivity.address_spaces) == 0 &&
            var.landing_zone_request.connectivity.dns_profile == "azure_default" &&
            !var.landing_zone_request.connectivity.hybrid_connectivity
        )
    )
    error_message = "Connectivity fields do not match the selected profile."
}

validation {
    condition = (
        var.landing_zone_request.budget.monthly_amount > 0 &&
        contains(["EUR", "USD"], var.landing_zone_request.budget.currency) &&
        can(regex(
            "(usr|grp)-[a-z0-9-]{3,60}$",
            var.landing_zone_request.budget.contact_ref
        ))
    )
    error_message = "Budget amount, currency, and contact must satisfy the contract."
}

validation {
    condition = alltrue([
        for key in [
            "business-unit",
            "cost-center",
            "data-classification",
            "environment",
            "technical-owner",
            "workload-id"
        ] : contains(keys(var.landing_zone_request.tags), key)
    ])
    error_message = "The mapped request is missing a platform-owned standard tag."
}

```

```
}  
}
```

Typed values are grouped by their next consumer. Subscription vending receives placement and billing intent. Connectivity, access, budget, and handoff processes receive narrower objects.

```
HCL / TERRAFORM  
locals {  
  subscription_vending_input = {  
    subscription_alias      = var.landing_zone_request.subscription.alias  
    subscription_display_name = var.landing_zone_request.subscription.display_name  
    target_management_group_id = var.landing_zone_request.subscription.management_group_id  
    billing_offer           = var.landing_zone_request.subscription.billing_offer  
    subscription_tags       = var.landing_zone_request.tags  
  }  
  
  connectivity_input = {  
    profile           = var.landing_zone_request.connectivity.profile  
    primary_region    = var.landing_zone_request.regions.primary  
    secondary_region  = var.landing_zone_request.regions.secondary  
    address_spaces    = var.landing_zone_request.connectivity.address_spaces  
    dns_profile       = var.landing_zone_request.connectivity.dns_profile  
    hybrid_connectivity = var.landing_zone_request.connectivity.hybrid_connectivity  
    inbound_exposure  = var.landing_zone_request.connectivity.inbound_exposure  
  }  
  
  access_input = {  
    subscription_owner_ref = var.landing_zone_request.access.subscription_owner_ref  
    workload_operator_group_ref =  
var.landing_zone_request.access.workload_operator_group_ref  
    workload_reader_group_ref =  
var.landing_zone_request.access.workload_reader_group_ref  
    deployment_principal_ref = var.landing_zone_request.access.deployment_principal_ref  
  }  
  
  budget_input = {  
    monthly_amount = var.landing_zone_request.budget.monthly_amount  
    currency        = var.landing_zone_request.budget.currency  
    budget_contact_ref = var.landing_zone_request.budget.contact_ref  
  }  
  
  handoff_metadata = {  
    request_id      = var.landing_zone_request.request_id  
    submitted_utc   = var.landing_zone_request.submitted_utc  
    workload_id     = var.landing_zone_request.workload.id  
    workload_environment = var.landing_zone_request.workload.environment  
    required_by     = var.landing_zone_request.required_by  
    business_owner_ref = var.landing_zone_request.workload.business_owner_ref  
    technical_owner_ref = var.landing_zone_request.workload.technical_owner_ref  
    service_catalog_id = var.landing_zone_request.metadata.service_catalog_id  
    exception_refs   = var.landing_zone_request.exception_refs  
  }  
}
```

These locals contain no resources or providers. They adapt the public request for later platform modules. An offline plan can test the mapping without contacting Azure.

Validation does not mean fulfillment. The generated contract uses the stage `validated_request`. It confirms accepted input, not a created subscription.

A useful handoff defines what the platform must eventually return. It includes identity, placement, tags, access, connectivity, budget, and deployment evidence. Support routes belong with those outputs.

Inherited controls must be visible in that handoff. The capstone expects `Audit-ResourceRGLocation`, `Deny-Classic-Resources`, and `Deny-UnmanagedDisk`. They remain platform-owned even though the workload experiences their effects.

The workload team receives more than access. It needs evidence about the foundation it now operates within. Hidden policy inheritance makes later denials look arbitrary.

The handoff must also state what it does not prove. A created subscription does not contain a deployed workload. An accepted exception reference does not grant approval.

Operational readiness stays with the workload team. This contract cannot prove recovery, support readiness, or safe application access. Chapter 7.4 will test those duties against the Well-Architected pillars.

The local handoff template keeps these limits explicit. It names both sides' duties and required completion evidence. Its negative list stops validation from looking like fulfillment.

POWERSHELL

```
$handoffDocument = [ordered]@{
    schema_version = '1.0'
    contract_stage = 'validated_request'
    generated_utc = $GeneratedUtc.ToString(
        'yyyy-MM-ddTHH:mm:ss.fffZ',
        [Globalization.CultureInfo]::InvariantCulture
    )
    request_id = [string] $request.request_id
    request_sha256 = $requestSource.sha256
    request_context = [ordered]@{
        submitted_utc = $submittedUtcText
        required_by = [string] $request.required_by
        service_catalog_id = [string] $request.metadata.service_catalog_id
        budget = [ordered]@{
            monthly_amount = [decimal] $request.budget.monthly_amount
            currency = [string] $request.budget.currency
            contact_ref = [string] $request.budget.contact_ref
        }
    }
    required_platform_outputs = [ordered]@{
        target_management_group_id = 'landing-zones'
        inherited_controls = @(
            'Audit-ResourceRGLocation',
            'Deny-Classic-Resources',
            'Deny-UnmanagedDisk'
        )
        completion_evidence = @(
            'subscription identifier and management-group placement',
            'applied standard tags',
            'configured access references',
            'connectivity result for the selected profile',
            'configured budget and notification target',
            'validation and deployment record links'
        )
    }
    responsibilities = [ordered]@{
        platform = @(
            'validate and retain the approved request',
            'map approved inputs to reviewed platform automation',
            'place the subscription and preserve inherited controls',
            'return completion evidence and support routes'
        )
        workload = @(
            'deploy and operate workload resources',
            'maintain workload architecture and application access',
```

```

        'manage workload cost forecasts and operational readiness',
        'keep ownership and metadata current'
    )
}
not_provided = @(
    'a created Azure subscription',
    'workload resources or application configuration',
    'approval for an exception reference',
    'proof of operational readiness'
)
}

```

The tested implementation writes only local JSON files. It refuses to replace an existing request directory. `WhatIf` can show the intended write without creating output.

This separation prepares a safer delivery pipeline. Validation and mapping remain quick, local, and deterministic. Fulfillment later adds identity, billing, allocation, approval, and Azure deployment controls.

The request contract still needs lifecycle care. Schema version, catalog documentation, validators, and platform modules must change together. A silent field change can corrupt fulfillment and audit meaning.

The strongest interface leaves little room for accidental ownership. Workload teams declare the service they intend to run. Platform teams return inspectable choices, controls, evidence, and limits.

That clarity changes the meaning of self-service. Speed does not come from hiding the platform boundary. It comes from making that boundary stable enough to trust.

Chapter 7: Adopt — Build the Platform-to-Workload Interface

Chapter 7.3: Create Paved Roads Without Creating Bottlenecks

Unlimited platform choice can resemble autonomy. It transfers research, integration, and support work to each workload team. Every team must rediscover which choices are safe and supported.

Rigid standardization creates the opposite failure. One mandatory design can ignore valid workload needs. Teams then wait, conceal the difference, or build a path around the platform. A paved road balances these pressures. Microsoft guidance calls such routes paved paths. This chapter uses road for a named, supported catalog route.

Paving can be incremental. A partial path removes some toil and makes one safe choice easier. A golden path covers the journey with mature automation, feedback, and support.

The word supported carries obligations. A road needs current documents, maintained automation, known guardrails, and a support route. A template without those services is only reusable code.

A paved road is also not a hidden mandate. Customers can see what the platform provides and what remains theirs. They can also see where the road ends.

The platform should pave a few frequent journeys. More options increase search, testing, maintenance, and support. A long menu can recreate the blank canvas.

Start with observed demand and shared risk. Repeated workload needs justify centralized learning and automation. Rare needs can remain visible without becoming permanent products. The capstone begins with two roads. One supports workloads that need hub-routed connectivity or platform private DNS. The other supports workloads that can operate without those dependencies.

Connected and isolated are local catalog labels. They map directly to the profiles defined in Chapter 7.2. Other platforms can publish different product lines from their own demand and risk.

These roads extend the request contract from Chapter 7.2. They add no workload resources. They map workload needs to approved request values and platform capabilities.

Selection begins with a few workload questions. Does the service need private routing through the shared hub? Does it need platform-managed private DNS?

A third question tests the alternative. Can endpoints or published interfaces replace hub routing? Evidence must support every answer.

Hub routing or platform private DNS selects the connected road. The fulfillment process then allocates nonoverlapping private space and prepares shared integration. The workload still owns its internal application architecture.

The connected road does not centralize every network choice. Workload teams design subnets inside the allocation. They choose application flows within published route and security limits. Workloads without those shared dependencies can select the isolated road. Its request keeps Azure DNS defaults and omits shared-hub address allocation. It still returns a governed application landing-zone foundation.

Isolated does not mean uncontrolled. The subscription retains the same placement, policy, identity, metadata, budget, and ownership guardrails. Only the supported connectivity outcome changes.

Both roads preserve workload choices. Teams select Azure services, resource architecture, deployment methods, and operating design. Platform policy can bound those choices without taking ownership of them.

A guardrail defines an acceptable boundary. Policy, automation, review, or evidence can implement it. The mechanism is not the risk outcome itself.

Guardrails work best when their effects appear early. A fast local check is better than a late production veto. Clear errors let teams repair input without joining a manual queue.

Some controls still require human judgment. The guardrail should name the needed evidence and decision owner. Sending every question to one council converts safety into delay.

Self-service requires discoverability as well as automation. A team must find the road, understand its fit, and predict its outputs. A portal cannot compensate for unclear choices.

Defaults should carry most routine decisions. Customers should provide facts about their workload, not reproduce platform design. The platform can then map those facts to reviewed options.

Each road needs a lifecycle. Its interface, automation, documents, and support knowledge change together. Catalog versions protect customer assumptions from silent changes.

Deprecation also needs an exit. Existing consumers need a successor, transition period, or bounded exception. New requests should not enter a retired interface.

Usage alone cannot prove that a road is useful. Mandated adoption can hide high effort and local repair. Support demand, abandonment, exception patterns, and direct feedback add context.

The path boundary becomes credible when it handles mismatch honestly. An exception requests an authorized deviation from a published boundary. It is neither hidden access nor vague special treatment.

Ordinary support should not enter that workflow. Misunderstood documents, broken templates, and delayed requests need repair. Calling every problem an exception hides defects and inflates risk queues.

The entry test is therefore strict. A material need must not fit any supported road. Meeting it inside a road would break that road's outcome or guardrails.

The request must carry decision-ready context. It covers need, scope, risk, owners, evidence, compensation, expiry, and exit. Missing context returns before risk review.

Completeness does not prove merit. It protects scarce judgment from avoidable clarification. Automated checks can perform that work quickly and consistently.

Triage classifies the request before seeking risk approval. Support faults return to support, and path gaps reach product discovery. Only a real deviation moves into risk review.

Risk review begins with the original control intent. It compares alternatives, residual risk, compensation, duration, and exit feasibility. Only affected risk domains should join the review.

Decision authority must be explicit. The risk owner accepts residual exposure, while the control owner maintains the guardrail. The requester cannot approve the same request.

An approval needs more than a yes. It records rationale, authority, time, expiry, cadence, and residual-risk ownership. It also activates compensating controls and exit work.

A denial also produces a useful result. The response gives a reason and a safe next option. Silent rejection only pushes the same need into another channel.

Every approval is time-limited. The capstone maximum is ninety days. The actual need and exit work should set a shorter period.

Expiry changes authorization, not history. Once the time is reached, the deviation is no longer allowed. The decision record remains available as evidence.

Renewal is a new risk decision. Current evidence, controls, need, and exit progress must be reviewed before the earlier expiry. Copying an old approval would turn a temporary exception into policy by neglect.

Closure needs evidence too. The standard road or an approved replacement must be restored. Remaining exposure must be visible if the exception expires before repair.

An Azure Policy exemption can implement part of a later control decision. Azure supports Waiver and Mitigated categories plus an optional expiration. Chapter 8.4 owns that implementation detail.

The generic workflow stays broader than Azure Policy. A deviation may concern connectivity, process, support, or another platform boundary. The decision record should not depend on one enforcement tool.

Exception handling can become its own bottleneck. Automated completeness checks keep basic faults out of review. Risk tiers keep unrelated specialists away from the decision.

Named primary and backup roles protect flow when one owner is absent. Visible clocks distinguish active review from a documented pause. Queue age must not disappear inside a meeting schedule.

Parallel input can reduce waiting. Affected specialists can review their domains at the same time. One accountable decision owner still assembles the result.

Emergency access does not create silent approval. An urgent operating procedure may act first within its authority. It must still create evidence and enter the required follow-up path.

Exceptions are also product evidence. Each record can show a one-off need, candidate road, path defect, or documentation gap. Repeated patterns deserve platform product review.

The capstone triggers review after three similar exceptions within ninety days. That value is a local product rule, not an industry benchmark. Chapter 7.5 will aggregate broader adoption-friction evidence.

The complete catalog follows. Its selection rules serve as the decision table. Its state transitions, expiry rules, roles, and evidence fields define the exception workflow.

```
JSON
{
  "schema_version": "1.0",
  "catalog": {
    "id": "alz_paved_roads",
    "version": "1.0.0",
    "status": "active",
    "owner_role": "platform product owner",
    "reviewed_on": "2026-07-20",
    "next_review_on": "2026-10-20",
    "customer": "workload delivery teams",
    "entry_interface": "landing-zone-request/1.0",
    "not_in_scope": [
      "Workload resources or application architecture",
      "Approval of a deviation through the standard request",
      "Detailed Azure Policy exemption deployment"
    ]
  },
  "decision_inputs": [
    {
      "id": "needs_hub_routing",
      "question": "Must the workload use private routed connectivity to shared services, other landing zones, on-premises sites, or another cloud?",
      "evidence": "Approved workload context and dependency records"
    },
    {
      "id": "needs_platform_private_dns",
      "question": "Must the workload resolve names through the platform-managed private DNS service?",
      "evidence": "Approved name-resolution and private-endpoint requirements"
    }
  ]
}
```

```

{
  "id": "can_use_endpoint_integration",
  "question": "Can the workload integrate without hub routing by using public
endpoints, private endpoints, or published application interfaces?",
  "evidence": "Reviewed workload communication and exposure requirements"
}
],
"common_guardrails": [
  {
    "id": "application_landing_zone_placement",
    "intent": "Keep routine workload subscriptions under the governed landing-zones
branch.",
    "implementation_owner_role": "platform engineering lead"
  },
  {
    "id": "inherited_policy_baseline",
    "intent": "Preserve inherited platform policy assignments and make their effects
visible at handoff.",
    "implementation_owner_role": "governance platform owner"
  },
  {
    "id": "stable_identity_references",
    "intent": "Use reviewed owner, group, and deployment-identity references without
request-time credentials.",
    "implementation_owner_role": "identity platform owner"
  },
  {
    "id": "owned_metadata_and_budget",
    "intent": "Retain governed workload metadata, cost ownership, budget intent, and
support contacts.",
    "implementation_owner_role": "platform product owner"
  },
  {
    "id": "platform_workload_boundary",
    "intent": "Return a usable foundation and evidence without deploying workload
resources.",
    "implementation_owner_role": "platform engineering lead"
  }
],
"paths": [
  {
    "id": "connected_landing_zone",
    "name": "Connected application landing zone",
    "status": "supported",
    "customer_outcome": "The workload receives a governed foundation prepared for
platform private DNS and approved routed connectivity.",
    "request_mapping": {
      "connectivity_profile": "connected",
      "dns_profile": "platform_private_dns"
    },
    "platform_provides": [
      "Conflict-free private address allocation through the fulfillment IPAM process",
      "The reviewed connection to shared connectivity for the selected region",
      "Platform private DNS integration and handoff evidence",
      "The common application landing-zone guardrails"
    ],
    "workload_choices": [
      "Workload resource architecture inside the fulfilled subscription",
      "Subnet design inside the allocated address space",
      "Application traffic flows within published routing and security limits",
      "Workload deployment and operating methods"
    ],
    "guardrail_refs": [
      "application_landing_zone_placement",
      "inherited_policy_baseline",
      "stable_identity_references",
      "owned_metadata_and_budget",
      "platform_workload_boundary"
    ],
    "service_expectation_refs": [
      "request_acknowledgement",

```

```

        "standard_fulfillment",
        "support_response"
    ],
    "documentation": "Connected application landing-zone catalog page",
    "feedback_routes": [
        "Platform support queue",
        "Platform product discovery"
    ]
},
{
    "id": "isolated_landing_zone",
    "name": "Isolated application landing zone",
    "status": "supported",
    "customer_outcome": "The workload receives a governed foundation without hub-routed connectivity or platform private DNS dependency.",
    "request_mapping": {
        "connectivity_profile": "isolated",
        "dns_profile": "azure_default"
    },
    "platform_provides": [
        "A subscription foundation without hub peering or private address allocation",
        "Azure default DNS as the landing-zone request setting",
        "Published exposure and integration constraints",
        "The common application landing-zone guardrails"
    ],
    "workload_choices": [
        "Workload resource architecture inside the fulfilled subscription",
        "Endpoint and application-interface design within published constraints",
        "Workload network resources that do not depend on shared hub routing",
        "Workload deployment and operating methods"
    ],
    "guardrail_refs": [
        "application_landing_zone_placement",
        "inherited_policy_baseline",
        "stable_identity_references",
        "owned_metadata_and_budget",
        "platform_workload_boundary"
    ],
    "service_expectation_refs": [
        "request_acknowledgement",
        "standard_fulfillment",
        "support_response"
    ],
    "documentation": "Isolated application landing-zone catalog page",
    "feedback_routes": [
        "Platform support queue",
        "Platform product discovery"
    ]
}
],
"selection_rules": [
    {
        "id": "select_connected",
        "priority": 1,
        "match": {
            "any_of": [
                {
                    "input_id": "needs_hub_routing",
                    "equals": true
                },
                {
                    "input_id": "needs_platform_private_dns",
                    "equals": true
                }
            ]
        },
        "outcome": "connected_landing_zone"
    },
    {
        "id": "select_isolated",
        "priority": 2,

```



```

    "match": {
      "all_of": [
        {
          "input_id": "needs_hub_routing",
          "equals": false
        },
        {
          "input_id": "needs_platform_private_dns",
          "equals": false
        },
        {
          "input_id": "can_use_endpoint_integration",
          "equals": true
        }
      ]
    },
    "outcome": "isolated_landing_zone"
  },
  {
    "id": "route_exception",
    "priority": 3,
    "match": {
      "otherwise": true
    },
    "outcome": "platform_path_exception"
  }
],
"exception_workflow": {
  "id": "platform_path_exception",
  "entry_condition": "A material workload need cannot fit a supported path without violating its published outcome or guardrails.",
  "request_fields": [
    "exception_id",
    "landing_zone_request_id",
    "requested_deviation",
    "business_need",
    "affected_scope",
    "risk_statement",
    "risk_owner_ref",
    "control_owner_ref",
    "compensating_controls",
    "evidence_refs",
    "requested_expiry_utc",
    "exit_plan"
  ],
  "approval_fields": [
    "decision",
    "decision_rationale",
    "approved_by_ref",
    "approved_utc",
    "expires_utc",
    "review_cadence",
    "residual_risk_owner_ref"
  ],
  "states": [
    {
      "id": "submitted",
      "owner_role": "workload technical owner",
      "terminal": false,
      "exit_evidence": [
        "Required request fields pass completeness checks"
      ]
    },
    {
      "id": "triage",
      "owner_role": "platform product owner",
      "terminal": false,
      "exit_evidence": [
        "The request is classified as support, path gap, or risk decision",
        "Required specialist and decision roles are named"
      ]
    }
  ]
}

```

```

    },
    {
      "id": "risk_review",
      "owner_role": "governance decision owner",
      "terminal": false,
      "exit_evidence": [
        "Risk, alternatives, compensating controls, and exit plan are reviewed",
        "A reasoned approval or denial is recorded"
      ]
    },
    {
      "id": "active",
      "owner_role": "control owner",
      "terminal": false,
      "exit_evidence": [
        "The deviation is removed or its expiry is reached",
        "Closure evidence is linked"
      ]
    },
    {
      "id": "redirected",
      "owner_role": "platform product owner",
      "terminal": true,
      "exit_evidence": [
        "The supported path or support route is named"
      ]
    },
    {
      "id": "denied",
      "owner_role": "governance decision owner",
      "terminal": true,
      "exit_evidence": [
        "Decision rationale and a safe next option are returned"
      ]
    },
    {
      "id": "expired",
      "owner_role": "control owner",
      "terminal": true,
      "exit_evidence": [
        "The deviation is no longer authorized",
        "Expiry and any remaining exposure are recorded"
      ]
    },
    {
      "id": "closed",
      "owner_role": "control owner",
      "terminal": true,
      "exit_evidence": [
        "The standard path or approved replacement is restored",
        "Closure evidence is recorded"
      ]
    },
    {
      "id": "withdrawn",
      "owner_role": "workload technical owner",
      "terminal": true,
      "exit_evidence": [
        "The requester records why the exception is no longer needed"
      ]
    }
  ],
  "allowed_transitions": [
    {
      "from": "submitted",
      "to": "triage",
      "condition": "The request passes completeness checks."
    },
    {
      "from": "submitted",
      "to": "withdrawn",

```

```

        "condition": "The requester removes the request before a decision."
    },
    {
        "from": "triage",
        "to": "submitted",
        "condition": "Material information is missing and the clock pause is recorded."
    },
    {
        "from": "triage",
        "to": "redirected",
        "condition": "A supported path or normal support action already fits the need."
    },
    {
        "from": "triage",
        "to": "risk_review",
        "condition": "A real deviation needs an authorized risk decision."
    },
    {
        "from": "risk_review",
        "to": "active",
        "condition": "Approval fields, compensating controls, and a bounded expiry are
recorded."
    },
    {
        "from": "risk_review",
        "to": "denied",
        "condition": "The accountable owner rejects the residual risk or proposed
controls."
    },
    {
        "from": "active",
        "to": "closed",
        "condition": "The deviation ends before expiry and closure evidence is accepted."
    },
    {
        "from": "active",
        "to": "expired",
        "condition": "The approved expiry is reached without accepted closure."
    }
],
"time_limit": {
    "maximum_days": 90,
    "starts_at": "approved_utc",
    "expires_at": "expires_utc",
    "expiry_effect": "The deviation is no longer authorized, even when its decision
record remains available for evidence.",
    "renewal_requires": [
        "A new exception record",
        "Current evidence and compensating controls",
        "A new risk decision before the prior expiry"
    ]
},
"requester_cannot_approve": true,
"flow_controls": [
    "Run automated completeness checks before human triage",
    "Route only the affected risk domains to review",
    "Name primary and backup decision roles",
    "Record clock starts, pauses, decisions, and expiry",
    "Do not let emergency access create silent approval"
],
"implementation_boundary": "This workflow records a platform-path decision. Chapter
8.4 owns any Azure Policy exemption implementation."
},
"learning": {
    "classifications": [
        "one_off_need",
        "candidate_supported_path",
        "path_defect",
        "documentation_gap"
    ]
},
"review_trigger_count": 3,

```

```
"review_window_days": 90,  
"owner_role": "platform product owner",  
"feedback_routes": [  
  "Platform product discovery",  
  "Chapter 7.5 adoption-friction review"  
]  
}
```

The catalog metadata names ownership and review dates. It also fixes the request interface at version 1.0. Neither road can silently redefine the landing-zone contract.

Common guardrails are referenced instead of copied into each road. Semantic validation rejects missing or unused references. This protects consistent meaning across the catalog.

The rules have a strict order. Connected fit is tested first, and isolated fit follows. A single final catch-all routes unmatched needs to the exception workflow.

That catch-all is not automatic approval. It only opens the decision route. Approval still requires risk evidence, authority, compensation, and bounded time.

Workflow states make ownership visible. Submitted, triage, risk review, and active states each have an owner and exit evidence. Terminal states cannot start another transition.

Active exceptions can end only by closure or expiry. Renewal cannot loop the existing state back to approval. A fresh record preserves the new evidence and decision.

The validator checks identifiers, references, mappings, rules, transitions, dates, and duration. Twelve tests exercise valid and invalid catalogs. No Azure context is needed.

This artifact proves the shape and internal consistency of the local model. It cannot prove customer demand, review capacity, or control effectiveness. Those claims require operating evidence.

A useful paved road makes the routine journey calmer. A useful exception route keeps uncommon needs visible without normalizing them. The open question is which repeated deviation should reshape the road.

Chapter 7: Adopt — Build the Platform-to-Workload Interface

Chapter 7.4: Connect CAF to Well-Architected Workloads

A governed landing zone provides a foundation. That foundation includes identity, connectivity, policy, and management. It cannot make the workload inside that foundation reliable or efficient. That distinction defines the next platform-to-workload boundary.

The Cloud Adoption Framework organizes adoption across an estate. The Azure Well-Architected Framework improves workload quality. It focuses on a particular workload. The frameworks meet at the application landing zone without becoming interchangeable.

CAF helps central teams build shared foundations. It governs them at scale. Well-Architected guidance helps workload teams balance design choices against business needs. One creates useful conditions while the other tests the workload built within them.

A workload includes application resources, code, and data. It also includes supporting infrastructure. Its team remains responsible for the business outcome those parts serve. Central services can support that duty, but they cannot absorb it silently.

The platform landing zone hosts shared services. Identity and connectivity are examples. An application landing zone hosts one workload. Its resources can span several environments. Platform controls continue to govern that space after workload authority begins.

Shared responsibility does not mean vague collective ownership. Each obligation still needs an accountable role. It also needs an observable output and decision boundary. Collaboration connects those duties rather than blending them.

Some shared components sit outside the workload. They act as dependencies. Hub routing, private DNS, central logging, and identity services are examples. Their availability, capacity, support, and change limits shape workload design.

An unclear platform dependency must be treated as a workload risk. The workload might change its requirement. The platform might change its offer. Silence cannot serve as a compatibility decision.

A responsibility matrix makes that conversation concrete. Each row names one shared outcome through a Well-Architected pillar. It separates platform and workload obligations. It also separates evidence from the joint fit decision.

The platform side describes a service. The platform must truly operate it. It publishes constraints, objectives, control scope, support, and change behavior. A broad promise such as secure by default is not precise enough.

The workload side describes local duties. Those duties cover design and operations. They cover application behavior, data, user flows, deployment, recovery, and ownership. A landing-zone receipt cannot satisfy those obligations.

The fit check joins the two sides at a named point. It tests the platform offer. That offer must support the declared workload need. An accepted answer records rationale instead of assuming compatibility.

Readiness evidence makes the answer inspectable. A reference identifies a reviewed record and its owner. It also records status and useful lifetime. The matrix points to evidence rather than copying every design document.

Evidence must match the decision stage. Before handoff, a design can describe intended work. A test plan can do the same. It must not be mislabeled as a successful production test.

Observed results become necessary when the workload reaches later stages. Load tests and recovery exercises cannot exist before deployment. Neither can incident data or cost history. A next review trigger keeps those future checks attached.

The capstone has four evidence states. They are accepted, conditional, blocked, and not applicable. Conditional evidence needs an explicit action and owner. Not applicable needs a reason, because an empty field proves nothing.

The handoff decision is deliberately narrow. Ready requires evidence for all five responsibility rows. That evidence must support the landing-zone boundary. It does not claim production readiness. It also does not claim a fully Well-Architected workload.

Reliability begins with promised user outcomes. The workload needs critical flows and service objectives. It also needs recovery objectives and failure assumptions. Architecture can then protect what the business actually values.

The platform publishes shared connectivity and DNS boundaries. Those boundaries cover reliability. They also include availability, recovery, capacity, support, and configuration evidence. They describe a dependency rather than the workload's end-to-end reliability.

The workload owns redundancy, data protection, and degraded behavior. It also owns recovery design. Workload tests must challenge each of those choices. Central infrastructure cannot recover application state it does not understand.

The joint check compares dependency behavior with workload targets. Both sides can meet their own standards. A mismatch can still create risk. Reliability evidence needs requirements and dependency profiles. Recovery design and later test results complete the set.

Security begins with the workload's data and threats. Inherited policy and shared controls provide a baseline. They do not decide application authorization or data handling. They also do not decide acceptable residual risk.

The platform publishes inherited policy, identity, and connectivity boundaries. It also publishes logging and escalation boundaries. It operates the shared controls named in the handoff. Evidence must state both their coverage and their limits.

The workload classifies its data. It also records compliance and exposure assumptions. It designs application access, workload identities, secret handling, and security response. These duties remain even when the platform denies unsafe configurations.

The joint check looks for gaps and conflicting assumptions. A private route can reduce exposure. It can also add DNS and support dependencies. Security evidence must connect controls to threats. It must also show operational consequences.

Cost Optimization begins with value and financial constraints. A low-cost design is not automatically a good design. Reliability, security, performance, and operability all create legitimate cost.

The platform publishes shared-service costs. It also publishes their allocation method. It preserves ownership metadata, budget contacts, and notification behavior. It cannot own the workload's demand or product economics.

The workload owns its cost model and forecast. It also owns cost drivers and growth assumptions. It responds when actual spending diverges from the plan. A budget alert provides feedback, not an architectural spending limit.

The joint check combines platform charges with workload demand. It tests whether the estimate still supports the business outcome. Evidence should expose assumptions and tradeoffs. One unexplained total is not enough.

Operational Excellence begins with clear ownership and repeatable change. The workload must remain supportable while it evolves. One working deployment proves little. It is not an operating model.

The platform publishes support hours and escalation routes. It also publishes change notices and telemetry boundaries. It operates its service and communicates material changes. Hidden platform change can invalidate a workload runbook.

The workload owns application observability, incident response, and delivery. It also owns rollback and learning. Named people must act on those processes. Shared logs do not replace workload signals or response authority.

The joint check follows an event across the boundary. Detection, routing, context, and ownership must survive that crossing. Evidence should prevent incidents from bouncing between queues. A decision owner must remain visible.

Performance Efficiency begins with user-flow targets and changing demand. Capacity is useful only in relation to those targets. Static overprovisioning can hide a weak scaling strategy. It can also increase cost.

The platform publishes supported regions, network limits, and quotas. Material latency constraints are also published. Capacity escalation and change routes remain visible. Those facts bound the workload's feasible design space.

The workload defines performance targets and demand assumptions. It also defines scaling behavior. It plans representative load tests and later measures real use. Platform headroom cannot compensate for inefficient application design forever.

The joint check compares targets with the selected region. It also tests the paved road. Early acceptance is provisional until workload tests create observations. The evidence record distinguishes an accepted plan from a measured result.

The five pillars cannot be reviewed as independent scores. Redundancy raises cost. Tighter security can add operating complexity. Capacity headroom changes efficiency. A useful handoff preserves those tensions.

A tradeoff record names the affected pillars and decision. It also records rationale, ownership, and evidence. It turns compromise into an explicit design choice. Later evidence can then confirm or challenge the assumption.

Priority also belongs to the workload context. Business criticality and compliance can change the work order. Time and funding can do the same. The matrix covers every pillar. It does not claim that every concern has equal weight.

The matrix extends the existing landing-zone contract. It references the request and selected paved road. It adds no intake fields. Chapter 7.2 remains the source of request truth.

The matrix also leaves exception approval untouched. A blocked fit can return for design work. It can also use the established deviation route. Readiness review cannot approve its own bypass. The capstone example records five responsibility rows. Each row links platform and workload evidence. It also links a fit decision and relevant tradeoffs. The structure makes missing ownership and unsupported claims testable.

JSON

```
{
  "schema_version": "1.0",
  "matrix": {
    "id": "alz_workload_readiness",
    "version": "1.0.0",
```



```

    "workload_ref": "orders-api",
    "landing_zone_request_ref": "lzreq-orders-production",
    "paved_road_ref": "connected_landing_zone",
    "assessment_boundary": "application landing-zone handoff",
    "reviewed_on": "2026-07-21",
    "next_review_on": "2026-08-21",
    "platform_owner_role": "platform product owner",
    "workload_owner_role": "workload product owner",
    "decision_owner_role": "workload product owner",
    "scope_limits": [
      "The matrix does not certify that the workload is Well-Architected.",
      "The matrix does not approve production traffic.",
      "The matrix does not transfer workload design or operating responsibility to the platform team."
    ]
  },
  "responsibility_rows": [
    {
      "id": "reliability_boundary",
      "pillar": "reliability",
      "shared_outcome": "Platform dependencies and workload recovery choices can support the workload's stated service objectives.",
      "platform": {
        "owner_role": "connectivity platform owner",
        "obligations": [
          "Publish availability, recovery, capacity, and support boundaries for shared connectivity and private DNS.",
          "Return the selected road's dependency and configuration evidence at handoff."
        ],
        "evidence_refs": [
          "platform-shared-services-profile-v3",
          "landing-zone-handoff-lzreq-orders-production"
        ]
      },
      "workload": {
        "owner_role": "workload solution architect",
        "obligations": [
          "Define critical user flows, service objectives, dependency tolerances, and recovery objectives.",
          "Design and plan tests for workload redundancy, data recovery, and degraded operation."
        ],
        "evidence_refs": [
          "orders-reliability-requirements-v1",
          "orders-recovery-design-v1"
        ]
      },
      "fit_check": {
        "question": "Can the connected road's shared-service boundaries support the workload's declared reliability targets?",
        "decision": "accepted",
        "rationale": "The dependency objectives meet the declared targets, and workload-owned recovery remains explicit.",
        "condition": null
      },
      "tradeoff_refs": [
        "reliability-cost-capacity"
      ]
    },
    {
      "id": "security_boundary",
      "pillar": "security",
      "shared_outcome": "Inherited controls and workload-specific controls form an understood defense without hiding residual risk.",
      "platform": {
        "owner_role": "security platform owner",
        "obligations": [
          "Publish inherited policy, identity, connectivity, logging, and incident-escalation boundaries.",
          "Operate the platform controls and shared security services named in the handoff."
        ]
      }
    }
  ]
}

```

```

    ],
    "evidence_refs": [
      "platform-control-baseline-v4",
      "landing-zone-handoff-lzreq-orders-production"
    ]
  },
  "workload": {
    "owner_role": "workload security lead",
    "obligations": [
      "Classify workload data and document compliance, exposure, and threat assumptions.",
      "Design application authorization, workload identities, secret handling, and security response."
    ],
    "evidence_refs": [
      "orders-data-classification-v2",
      "orders-threat-and-access-model-v2"
    ]
  },
  "fit_check": {
    "question": "Do platform controls cover their declared scope without being mistaken for complete workload security?",
    "decision": "accepted",
    "rationale": "The inherited baseline is mapped, and the workload design owns the application and data threats beyond it.",
    "condition": null
  },
  "tradeoff_refs": [
    "security-operability-private-connectivity"
  ]
},
{
  "id": "cost_optimization_boundary",
  "pillar": "cost_optimization",
  "shared_outcome": "Platform charges and workload demand form a cost model that can support the intended business outcome.",
  "platform": {
    "owner_role": "platform product owner",
    "obligations": [
      "Publish shared-service allocation, showback or chargeback, and budget-notification behavior.",
      "Preserve workload ownership metadata and the approved budget contact in the landing zone."
    ],
    "evidence_refs": [
      "platform-shared-cost-model-2026",
      "landing-zone-handoff-lzreq-orders-production"
    ]
  },
  "workload": {
    "owner_role": "workload product owner",
    "obligations": [
      "Estimate workload demand, total operating cost, cost drivers, and growth assumptions.",
      "Own forecasts, budget responses, and cost tradeoffs across the workload lifecycle."
    ],
    "evidence_refs": [
      "orders-cost-model-v1",
      "orders-budget-ownership-v1"
    ]
  },
  "fit_check": {
    "question": "Does the combined platform and workload estimate fit the approved financial boundary?",
    "decision": "accepted",
    "rationale": "Shared-service costs and workload demand are visible inside the approved initial estimate.",
    "condition": null
  },
  "tradeoff_refs": [

```

```

        "reliability-cost-capacity",
        "performance-cost-headroom"
    ]
},
{
    "id": "operational_excellence_boundary",
    "pillar": "operational_excellence",
    "shared_outcome": "Platform and workload teams can detect, route, change, and learn from operational events without an ownership gap.",
    "platform": {
        "owner_role": "platform product owner",
        "obligations": [
            "Publish support hours, escalation routes, change notices, and shared telemetry boundaries.",
            "Operate and improve the platform service without silently changing workload-facing assumptions."
        ],
        "evidence_refs": [
            "platform-support-change-profile-v2",
            "landing-zone-handoff-lzreq-orders-production"
        ]
    },
    "workload": {
        "owner_role": "workload service owner",
        "obligations": [
            "Define workload support ownership, observability, incident response, and operational learning.",
            "Maintain an automated delivery path with tested rollback and controlled workload change."
        ],
        "evidence_refs": [
            "orders-operations-plan-v1",
            "orders-release-rollback-plan-v1"
        ]
    },
    "fit_check": {
        "question": "Can an event cross the platform-to-workload boundary without losing ownership, evidence, or escalation context?",
        "decision": "accepted",
        "rationale": "Support and change routes name both sides, while workload runbooks retain application operations.",
        "condition": null
    },
    "tradeoff_refs": [
        "security-operability-private-connectivity"
    ]
},
{
    "id": "performance_efficiency_boundary",
    "pillar": "performance_efficiency",
    "shared_outcome": "Platform constraints and workload scaling choices can meet declared user-flow targets without unmanaged capacity assumptions.",
    "platform": {
        "owner_role": "connectivity platform owner",
        "obligations": [
            "Publish supported regions, shared-network capacity, quotas, latency boundaries, and escalation routes.",
            "Make material capacity or regional changes visible to affected workload teams."
        ],
        "evidence_refs": [
            "platform-regional-capacity-profile-v2",
            "platform-support-change-profile-v2"
        ]
    },
    "workload": {
        "owner_role": "workload solution architect",
        "obligations": [
            "Define user-flow performance targets, demand assumptions, and scaling behavior.",
            "Plan representative load tests and post-deployment measurement against those targets."
        ]
    }
}

```

```

    ],
    "evidence_refs": [
        "orders-performance-targets-v1",
        "orders-load-test-plan-v1"
    ]
},
"fit_check": {
    "question": "Can the supported region and connected road satisfy the initial
performance targets and test plan?",
    "decision": "accepted",
    "rationale": "Published latency and capacity boundaries fit the initial targets,
subject to workload load testing.",
    "condition": null
},
"tradeoff_refs": [
    "performance-cost-headroom"
]
}
],
"evidence": [
    {
        "id": "platform-shared-services-profile-v3",
        "pillar": "reliability",
        "owner_role": "connectivity platform owner",
        "kind": "platform service profile",
        "status": "accepted",
        "required_for_handoff": true,
        "location": "evidence://platform/shared-services/profile/v3",
        "reviewed_on": "2026-07-18",
        "expires_on": "2026-10-18",
        "condition": null
    },
    {
        "id": "landing-zone-handoff-lzreq-orders-production",
        "pillar": "cross_pillar",
        "owner_role": "platform product owner",
        "kind": "landing-zone handoff",
        "status": "accepted",
        "required_for_handoff": true,
        "location": "evidence://platform/handoff/lzreq-orders-production",
        "reviewed_on": "2026-07-21",
        "expires_on": null,
        "condition": null
    },
    {
        "id": "orders-reliability-requirements-v1",
        "pillar": "reliability",
        "owner_role": "workload solution architect",
        "kind": "requirements record",
        "status": "accepted",
        "required_for_handoff": true,
        "location": "evidence://orders/reliability/requirements/v1",
        "reviewed_on": "2026-07-17",
        "expires_on": null,
        "condition": null
    },
    {
        "id": "orders-recovery-design-v1",
        "pillar": "reliability",
        "owner_role": "workload solution architect",
        "kind": "recovery design",
        "status": "accepted",
        "required_for_handoff": true,
        "location": "evidence://orders/reliability/recovery/v1",
        "reviewed_on": "2026-07-17",
        "expires_on": null,
        "condition": null
    },
    {
        "id": "platform-control-baseline-v4",
        "pillar": "security",

```

```

    "owner_role": "security platform owner",
    "kind": "control baseline",
    "status": "accepted",
    "required_for_handoff": true,
    "location": "evidence://platform/security/baseline/v4",
    "reviewed_on": "2026-07-18",
    "expires_on": "2026-10-18",
    "condition": null
  },
  {
    "id": "orders-data-classification-v2",
    "pillar": "security",
    "owner_role": "workload security lead",
    "kind": "data classification",
    "status": "accepted",
    "required_for_handoff": true,
    "location": "evidence://orders/security/data-classification/v2",
    "reviewed_on": "2026-07-16",
    "expires_on": null,
    "condition": null
  },
  {
    "id": "orders-threat-and-access-model-v2",
    "pillar": "security",
    "owner_role": "workload security lead",
    "kind": "threat and access model",
    "status": "accepted",
    "required_for_handoff": true,
    "location": "evidence://orders/security/threat-access/v2",
    "reviewed_on": "2026-07-19",
    "expires_on": null,
    "condition": null
  },
  {
    "id": "platform-shared-cost-model-2026",
    "pillar": "cost_optimization",
    "owner_role": "platform product owner",
    "kind": "shared cost model",
    "status": "accepted",
    "required_for_handoff": true,
    "location": "evidence://platform/cost/shared-model/2026",
    "reviewed_on": "2026-07-15",
    "expires_on": "2026-12-31",
    "condition": null
  },
  {
    "id": "orders-cost-model-v1",
    "pillar": "cost_optimization",
    "owner_role": "workload product owner",
    "kind": "workload cost model",
    "status": "accepted",
    "required_for_handoff": true,
    "location": "evidence://orders/cost/model/v1",
    "reviewed_on": "2026-07-17",
    "expires_on": null,
    "condition": null
  },
  {
    "id": "orders-budget-ownership-v1",
    "pillar": "cost_optimization",
    "owner_role": "workload product owner",
    "kind": "budget ownership",
    "status": "accepted",
    "required_for_handoff": true,
    "location": "evidence://orders/cost/budget-ownership/v1",
    "reviewed_on": "2026-07-17",
    "expires_on": null,
    "condition": null
  },
  {
    "id": "platform-support-change-profile-v2",

```

```

    "pillar": "cross_pillar",
    "owner_role": "platform product owner",
    "kind": "support and change profile",
    "status": "accepted",
    "required_for_handoff": true,
    "location": "evidence://platform/operations/support-change/v2",
    "reviewed_on": "2026-07-18",
    "expires_on": "2026-10-18",
    "condition": null
  },
  {
    "id": "orders-operations-plan-v1",
    "pillar": "operational_excellence",
    "owner_role": "workload service owner",
    "kind": "operations plan",
    "status": "accepted",
    "required_for_handoff": true,
    "location": "evidence://orders/operations/plan/v1",
    "reviewed_on": "2026-07-19",
    "expires_on": null,
    "condition": null
  },
  {
    "id": "orders-release-rollback-plan-v1",
    "pillar": "operational_excellence",
    "owner_role": "workload service owner",
    "kind": "delivery and rollback plan",
    "status": "accepted",
    "required_for_handoff": true,
    "location": "evidence://orders/operations/release-rollback/v1",
    "reviewed_on": "2026-07-19",
    "expires_on": null,
    "condition": null
  },
  {
    "id": "platform-regional-capacity-profile-v2",
    "pillar": "performance_efficiency",
    "owner_role": "connectivity platform owner",
    "kind": "regional capacity profile",
    "status": "accepted",
    "required_for_handoff": true,
    "location": "evidence://platform/performance/regional-capacity/v2",
    "reviewed_on": "2026-07-18",
    "expires_on": "2026-10-18",
    "condition": null
  },
  {
    "id": "orders-performance-targets-v1",
    "pillar": "performance_efficiency",
    "owner_role": "workload solution architect",
    "kind": "performance targets",
    "status": "accepted",
    "required_for_handoff": true,
    "location": "evidence://orders/performance/targets/v1",
    "reviewed_on": "2026-07-17",
    "expires_on": null,
    "condition": null
  },
  {
    "id": "orders-load-test-plan-v1",
    "pillar": "performance_efficiency",
    "owner_role": "workload solution architect",
    "kind": "validation plan",
    "status": "accepted",
    "required_for_handoff": true,
    "location": "evidence://orders/performance/load-test-plan/v1",
    "reviewed_on": "2026-07-20",
    "expires_on": null,
    "condition": null
  }
],

```

```

"tradeoff_records": [
  {
    "id": "reliability-cost-capacity",
    "pillars": [
      "reliability",
      "cost_optimization"
    ],
    "decision": "Retain capacity headroom for one declared demand surge instead of sizing only for average demand.",
    "rationale": "The accepted reliability target justifies the modeled incremental cost.",
    "owner_role": "workload product owner",
    "evidence_refs": [
      "orders-reliability-requirements-v1",
      "orders-cost-model-v1"
    ]
  },
  {
    "id": "security-operability-private-connectivity",
    "pillars": [
      "security",
      "operational_excellence"
    ],
    "decision": "Use platform private DNS and routed connectivity with named troubleshooting ownership.",
    "rationale": "The reduced exposure is accepted with the added dependency and support complexity.",
    "owner_role": "workload security lead",
    "evidence_refs": [
      "platform-support-change-profile-v2",
      "orders-threat-and-access-model-v2",
      "orders-operations-plan-v1"
    ]
  },
  {
    "id": "performance-cost-headroom",
    "pillars": [
      "performance_efficiency",
      "cost_optimization"
    ],
    "decision": "Begin with bounded autoscaling headroom and refine the limit from load-test and production evidence.",
    "rationale": "The design protects the initial user-flow target without treating peak forecast as permanent capacity.",
    "owner_role": "workload solution architect",
    "evidence_refs": [
      "orders-cost-model-v1",
      "orders-performance-targets-v1",
      "orders-load-test-plan-v1"
    ]
  }
],
"handoff_decision": {
  "status": "ready_for_landing_zone_handoff",
  "decided_on": "2026-07-21",
  "decision_owner_role": "workload product owner",
  "basis": [
    "All five Well-Architected pillar rows have an accepted platform-to-workload fit decision.",
    "Every evidence item required for this handoff is accepted and linked.",
    "Cross-pillar tradeoffs have named owners and evidence."
  ],
  "conditions": [],
  "not_proven": [
    "The workload is Well-Architected.",
    "The workload is ready for production traffic.",
    "The planned controls and tests will remain effective after deployment."
  ],
  "next_review_trigger": "Before the first production release, after a material workload or platform change, or by the next review date."
}

```

|}

The example reaches ready for landing-zone handoff. All required evidence is accepted. Every fit decision is also accepted. The record states the review date. Three explicit limits prevent that result from becoming a certification claim.

A conditional result would retain the action required before fulfillment. A blocked result would stop handoff. Evidence or platform fit must change before it resumes. The underlying rows determine the decision status. It cannot be selected for convenience.

The evidence locations are synthetic references. Their dates and ownership fields make staleness and accountability visible. A real organization would link approved repositories. It would also link assessment exports and operating records.

The Azure Well-Architected Review remains useful after this handoff. It is a collaborative self-assessment. Its recommendations enter normal prioritization and improvement work. The local matrix does not replace that deeper review.

The review must also repeat as the system changes. A workload requirement can change. The new requirement might exceed a platform service boundary. A platform change can invalidate an earlier workload assumption.

The example includes a next review date. It also includes event triggers. Production entry, material change, and elapsed time can all reopen the fit decision. Accepted evidence is a maintained claim, not permanent truth.

Local schema tests can verify structure, references, and pillar coverage. Semantic tests can verify status derivation and date order. They cannot prove that designs will meet targets. They also cannot prove that teams will perform their duties. Those claims require authorized review and observed workload behavior.

A landing zone can make sound workload design easier. It cannot make the workload's design decisions. Those decisions determine workload quality. The matrix keeps that boundary visible, owned, and connected to evidence.

Chapter 7: Adopt — Build the Platform-to-Workload Interface

Chapter 7.5: Measure Adoption Friction

Fulfillment is an event; adoption is an experience. A request may close while a team still carries the cost of unclear fields, repeated evidence, silent waiting, or a missing capability. Those costs are adoption friction. Measuring them turns the onboarding path into a source of product learning.

Friction is not a synonym for control. A review of identity, cost ownership, or network evidence may be essential. The question is whether the control is delivered at the right time, with clear criteria and the least avoidable effort. This distinction protects risk intent while exposing poor implementation.

A credible scorecard starts by fixing the population and measurement window. The example includes every request submitted during one month, with an inclusive UTC start and an exclusive UTC end. It retains fulfilled, withdrawn, and pending outcomes. That population rule stops completed work from presenting an incomplete success story.

Provisioning lead time starts at submission and ends at fulfillment. Only fulfilled requests enter that distribution because a pending request has no final duration. The other outcomes stay visible in the population counts. Any service target should state the same start, end, denominator, and exclusion rules.

The scorecard reports p50 and p90 rather than leaning on one average. The p50 describes the middle fulfilled request after durations are ordered. The p90 makes the slower tail visible. A wide gap between them invites inspection, but neither percentile explains why the tail exists. For that explanation, each request carries a continuous timeline. One dimension records the current owner category: platform active, workload wait, or external wait. Every observed hour must belong to exactly one category. The result describes where work or waiting sat, not who deserves blame.

A second dimension classifies the same interval as standard work, necessary control, customer rework, platform defect, or external dependency. Cause and current owner often differ. A platform defect may leave a workload team waiting for corrected guidance. Combining the dimensions would erase that distinction and encourage false attribution.

Timeline quality is enforced before aggregation. Intervals begin at submission, meet without a gap or overlap, and reach the observed end. That end is completion for fulfilled or withdrawn

work and the window boundary for pending work. A broken timeline is rejected instead of being turned into a polished but incomplete number.

Validation failures form another signal. The scorecard counts affected requests, failure events, and stable reason codes. A quick rejection can be a useful guardrail when it prevents costly downstream work. Repeated failures against the same field, however, may expose unclear language, poor examples, or validation that arrives too late.

Exception demand remains separate from exception control. The aggregator reads the learning classification produced by the governed path, such as a one-off need, documentation gap, candidate supported path, or path defect. It does not approve or close anything. Product measurement observes the pattern while the workflow remains authoritative.

Support contacts reveal effort that lead time can miss. Reason codes distinguish help with validation, path selection, status, documentation, or an external dependency. Contacts per submitted request add a stable denominator. Volume must still be read beside adoption growth because more customers can create more contacts even when the experience improves.

Satisfaction provides a direct customer view, but the average needs guardrails of its own. Response count and response rate travel with the result. The average is suppressed below a configured sample floor. This makes weak evidence visible instead of disguising it with decimal precision.

Capability gaps describe needs that no current path supports. They differ from a defect, where a promised capability fails, and from an exception, where a governed decision handles a deviation. Repeated gaps can justify discovery work. Repetition alone does not prove broad demand or create a delivery promise.

The scorecard therefore carries no composite result. Weighting speed, support, exceptions, rework, and satisfaction into one number would hide tradeoffs. It would also reward teams for moving the easiest input while another part of the experience gets worse. Separate measures keep the review anchored in customer outcomes and evidence quality.

The following script implements this contract against synthetic JSON records. It requires PowerShell 7.5 or later because ConvertFrom-Json must preserve timestamp text for exact UTC validation. The command reads local data and returns or writes local JSON. It reads no Azure account context and makes no cloud change.

```
POWERSHELL
[CmdletBinding()]
param(
    [Parameter()]
    [string] $InputPath,

    [Parameter()]
    [string] $OutputPath,
```

```

        [Parameter()]
        [ValidateRange(1, 1000)]
        [int] $MinimumSatisfactionResponses = 5,

        [Parameter()]
        [ValidateRange(1, 1000)]
        [int] $MinimumSignalRepetition = 2
    )

Set-StrictMode -Version Latest
$ErrorActionPreference = 'Stop'

function ConvertTo-UtcInstant {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [string] $Value,

        [Parameter(Mandatory)]
        [string] $FieldName
    )

    if ($Value -notmatch '^\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}Z$') {
        throw "$FieldName must use UTC in yyyy-MM-ddTHH:mm:ssZ format."
    }

    try {
        return [DateTimeOffset]::ParseExact(
            $Value,
            "yyyy-MM-dd'T'HH:mm:ss'Z'",
            [Globalization.CultureInfo]::InvariantCulture,
            [Globalization.DateTimeStyles]::AssumeUniversal -bor
            [Globalization.DateTimeStyles]::AdjustToUniversal
        )
    }
    catch {
        throw "$FieldName is not a valid UTC timestamp."
    }
}

function Get-NearestRankPercentile {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [double[]] $Values,

        [Parameter(Mandatory)]
        [ValidateRange(0.01, 1.0)]
        [double] $Percentile
    )

    if ($Values.Count -eq 0) {
        return $null
    }

    $ordered = @($Values | Sort-Object)
    $rank = [Math]::Ceiling($Percentile * $ordered.Count)
    return [Math]::Round(
        [double] $ordered[$rank - 1],
        2,
        [MidpointRounding]::AwayFromZero
    )
}

function Get-Distribution {
    [CmdletBinding()]

```

```

    param(
        [Parameter(Mandatory)]
        [AllowEmptyCollection()]
        [double[]] $Values
    )

    return [ordered]@{
        population = $Values.Count
        p50 = Get-NearestRankPercentile -Values $Values -Percentile 0.50
        p90 = Get-NearestRankPercentile -Values $Values -Percentile 0.90
    }
}

function Get-CountObject {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [AllowEmptyCollection()]
        [string[]] $Values
    )

    $result = [ordered]@{}
    $groups = @(
        $Values |
        Where-Object { -not [string]::IsNullOrEmpty($_) } |
        Group-Object |
        Sort-Object -Property @(
            @{ Expression = 'Count'; Descending = $true },
            @{ Expression = 'Name'; Ascending = $true }
        )
    )
    foreach ($group in $groups) {
        $result[$group.Name] = [int] $group.Count
    }
    return [pscustomobject] $result
}

function Get-AdoptionFrictionScorecard {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [pscustomobject] $Document,

        [Parameter()]
        [ValidateRange(1, 1000)]
        [int] $MinimumSatisfactionResponses = 5,

        [Parameter()]
        [ValidateRange(1, 1000)]
        [int] $MinimumSignalRepetition = 2
    )

    if ($Document.schema_version -ne '1.0') {
        throw 'schema_version must be 1.0.'
    }

    $windowStart = ConvertTo-UtcInstant `
        -Value ([string] $Document.window.start_utc) `
        -FieldName 'window.start_utc'
    $windowEnd = ConvertTo-UtcInstant `
        -Value ([string] $Document.window.end_utc) `
        -FieldName 'window.end_utc'
    if ($windowEnd -le $windowStart) {
        throw 'window.end_utc must follow window.start_utc.'
    }

    $requests = @($Document.requests)
    if ($requests.Count -eq 0) {

```

```

        throw 'At least one request is required.'
    }

    $duplicate = @(
        $requests |
        Group-Object -Property request_id |
        Where-Object Count -gt 1
    )
    if ($duplicate.Count -gt 0) {
        throw 'request_id values must be unique.'
    }

    $allowedPaths = @('connected_landing_zone', 'isolated_landing_zone')
    $allowedOutcomes = @('fulfilled', 'withdrawn', 'pending')
    $allowedOwners = @('platform_active', 'workload_wait', 'external_wait')
    $allowedClasses = @(
        'standard_work',
        'necessary_control',
        'customer_rework',
        'platform_defect',
        'external_dependency'
    )
    $allowedExceptionClasses = @(
        'none',
        'one_off_need',
        'candidate_supported_path',
        'path_defect',
        'documentation_gap'
    )

    $normalized = [Collections.Generic.List[object]]::new()
    foreach ($request in $requests) {
        if ([string]::IsNullOrEmpty($request.request_id)) {
            throw 'Every request requires request_id.'
        }
        if ([string] $request.path_id -notin $allowedPaths) {
            throw "Request $($request.request_id) has an unsupported path_id."
        }
        if ([string] $request.outcome -notin $allowedOutcomes) {
            throw "Request $($request.request_id) has an unsupported outcome."
        }
        if (
            [string] $request.exception_learning_classification -notin
            $allowedExceptionClasses
        ) {
            throw "Request $($request.request_id) has an invalid exception
classification."
        }

        $submitted = ConvertTo-UtcInstant `
            -Value ([string] $request.submitted_utc) `
            -FieldName "$($request.request_id).submitted_utc"
        if ($submitted -lt $windowStart -or $submitted -ge $windowEnd) {
            throw "Request $($request.request_id) was submitted outside the window."
        }

        if ([string] $request.outcome -eq 'pending') {
            if ($null -ne $request.completed_utc) {
                throw "Pending request $($request.request_id) cannot have completed_utc."
            }
            $expectedEnd = $windowEnd
        }
        else {
            if ($null -eq $request.completed_utc) {
                throw "Completed request $($request.request_id) requires completed_utc."
            }
            $expectedEnd = ConvertTo-UtcInstant `
                -Value ([string] $request.completed_utc) `
                -FieldName "$($request.request_id).completed_utc"
        }
    }

```

```

    }
    if ($expectedEnd -le $submitted -or $expectedEnd -gt $windowEnd) {
        throw "Request $($request.request_id) has an invalid observed end."
    }

    $intervals = @($request.intervals)
    if ($intervals.Count -eq 0) {
        throw "Request $($request.request_id) requires intervals."
    }

    $cursor = $submitted
    $ownerHours = [ordered]@{
        platform_active = 0.0
        workload_wait = 0.0
        external_wait = 0.0
    }
    $classHours = [ordered]@{
        standard_work = 0.0
        necessary_control = 0.0
        customer_rework = 0.0
        platform_defect = 0.0
        external_dependency = 0.0
    }
    foreach ($interval in $intervals) {
        if ([string] $interval.owner_category -notin $allowedOwners) {
            throw "Request $($request.request_id) has an invalid owner category."
        }
        if ([string] $interval.friction_class -notin $allowedClasses) {
            throw "Request $($request.request_id) has an invalid friction class."
        }
        $intervalStart = ConvertTo-UtcInstant `
            -Value ([string] $interval.start_utc) `
            -FieldName "$($request.request_id).interval.start_utc"
        $intervalEnd = ConvertTo-UtcInstant `
            -Value ([string] $interval.end_utc) `
            -FieldName "$($request.request_id).interval.end_utc"
        if ($intervalStart -ne $cursor) {
            throw "Request $($request.request_id) intervals contain a gap or overlap."
        }
        if ($intervalEnd -le $intervalStart -or $intervalEnd -gt $expectedEnd) {
            throw "Request $($request.request_id) has an invalid interval end."
        }
        $hours = ($intervalEnd - $intervalStart).TotalHours
        $ownerHours[[string] $interval.owner_category] += $hours
        $classHours[[string] $interval.friction_class] += $hours
        $cursor = $intervalEnd
    }
    if ($cursor -ne $expectedEnd) {
        throw "Request $($request.request_id) intervals do not reach the observed
end."
    }

    $satisfaction = $request.satisfaction_score
    if ($null -ne $satisfaction -and ([int] $satisfaction -lt 1 -or [int]
$satisfaction -gt 5)) {
        throw "Request $($request.request_id) satisfaction_score must be 1 through 5."
    }

    $normalized.Add([pscustomobject]@{
        request_id = [string] $request.request_id
        path_id = [string] $request.path_id
        outcome = [string] $request.outcome
        total_hours = ($expectedEnd - $submitted).TotalHours
        owner_hours = [pscustomobject] $ownerHours
        class_hours = [pscustomobject] $classHours
        validation_failure_codes = @($request.validation_failure_codes)
        exception_learning_classification = [string]
$request.exception_learning_classification
        support_contact_codes = @($request.support_contact_codes)
        satisfaction_score = $satisfaction
    })

```

```

        capability_gap_codes = @($request.capability_gap_codes)
    })
}

$fulfilled = @($normalized | Where-Object outcome -eq 'fulfilled')
$leadTime = [double[]] @($fulfilled | ForEach-Object total_hours)
$platformTime = [double[]] @(
    $fulfilled | ForEach-Object { $_.owner_hours.platform_active }
)
$workloadWait = [double[]] @(
    $fulfilled | ForEach-Object { $_.owner_hours.workload_wait }
)
$externalWait = [double[]] @(
    $fulfilled | ForEach-Object { $_.owner_hours.external_wait }
)

$validationCodes = [string[]] @(
    $normalized | ForEach-Object validation_failure_codes
)
$requestsWithValidationFailure = @(
    $normalized | Where-Object { $_.validation_failure_codes.Count -gt 0 }
).Count
$exceptionClasses = [string[]] @(
    $normalized |
        Where-Object exception_learning_classification -ne 'none' |
        ForEach-Object exception_learning_classification
)
$supportCodes = [string[]] @(
    $normalized | ForEach-Object support_contact_codes
)
$gapCodes = [string[]] @(
    $normalized | ForEach-Object capability_gap_codes
)
$requestsWithGap = @(
    $normalized | Where-Object { $_.capability_gap_codes.Count -gt 0 }
).Count
$satisfactionScores = [double[]] @(
    $normalized |
        Where-Object { $null -ne $_.satisfaction_score } |
        ForEach-Object { [double] $_.satisfaction_score }
)

if ($satisfactionScores.Count -ge $MinimumSatisfactionResponses) {
    $satisfactionState = 'reported'
    $satisfactionAverage = [Math]::Round(
        ($satisfactionScores | Measure-Object -Average).Average,
        2,
        [MidpointRounding]::AwayFromZero
    )
}
else {
    $satisfactionState = 'insufficient_sample'
    $satisfactionAverage = $null
}

$frictionBreakdown = [Collections.Generic.List[object]]::new()
$observedHours = ($normalized | Measure-Object total_hours -Sum).Sum
foreach ($class in $allowedClasses) {
    $hours = (
        $normalized |
            ForEach-Object { $_.class_hours.$class } |
            Measure-Object -Sum
    ).Sum
    $frictionBreakdown.Add([ordered]@{
        friction_class = $class
        total_hours = [Math]::Round($hours, 2)
        percent_of_observed_hours = [Math]::Round(
            100 * $hours / $observedHours,
            2,
            [MidpointRounding]::AwayFromZero

```

```

    )
  })
}

$pathSegments = [Collections.Generic.List[object]]::new()
foreach ($path in $allowedPaths) {
    $pathRequests = @($normalized | Where-Object path_id -eq $path)
    $pathFulfilled = @($pathRequests | Where-Object outcome -eq 'fulfilled')
    $pathLeadTime = [double[]] @($pathFulfilled | ForEach-Object total_hours)
    $distribution = Get-Distribution -Values $pathLeadTime
    $pathSegments.Add([ordered]@{
        path_id = $path
        request_count = $pathRequests.Count
        fulfilled_count = $pathFulfilled.Count
        lead_time_hours = $distribution
    })
}

$signalRows = [Collections.Generic.List[object]]::new()
$signalSources = [ordered]@{
    validation_failure = $validationCodes
    exception_learning = $exceptionClasses
    support_contact = $supportCodes
    capability_gap = $gapCodes
}
foreach ($source in $signalSources.Keys) {
    $groups = @($signalSources[$source] | Group-Object)
    foreach ($group in $groups) {
        if ($group.Count -ge $MinimumSignalRepetition) {
            $signalRows.Add([ordered]@{
                source = $source
                code = $group.Name
                count = [int] $group.Count
            })
        }
    }
}

$orderedSignals = @(
    $signalRows |
    Sort-Object -Property @(
        @{ Expression = 'count'; Descending = $true },
        @{ Expression = 'source'; Ascending = $true },
        @{ Expression = 'code'; Ascending = $true }
    )
)

return [ordered]@{
    schema_version = '1.0'
    scorecard = [ordered]@{
        id = 'adoption_friction_baseline'
        window_start_utc = $Document.window.start_utc
        window_end_utc = $Document.window.end_utc
        status = 'baseline'
        composite_score = $null
    }
    population = [ordered]@{
        request_count = $normalized.Count
        fulfilled_count = $fulfilled.Count
        withdrawn_count = @($normalized | Where-Object outcome -eq 'withdrawn').Count
        pending_count = @($normalized | Where-Object outcome -eq 'pending').Count
    }
    measures = [ordered]@{
        provisioning_lead_time_hours = Get-Distribution -Values $leadTime
        platform_active_time_hours = Get-Distribution -Values $platformTime
        workload_wait_time_hours = Get-Distribution -Values $workloadWait
        external_wait_time_hours = Get-Distribution -Values $externalWait
        validation_rework = [ordered]@{
            requests_with_failure = $requestsWithValidationFailure
            failure_events = $validationCodes.Count
            request_rate_percent = [Math]::Round(

```



```

        100 * $requestsWithValidationFailure / $normalized.Count,
        2
    )
    codes = Get-CountObject -Values $validationCodes
}
exception_demand = [ordered]@{
    request_count = $exceptionClasses.Count
    request_rate_percent = [Math]::Round(
        100 * $exceptionClasses.Count / $normalized.Count,
        2
    )
    classifications = Get-CountObject -Values $exceptionClasses
}
support_demand = [ordered]@{
    contact_count = $supportCodes.Count
    contacts_per_request = [Math]::Round(
        $supportCodes.Count / $normalized.Count,
        2,
        [MidpointRounding]::AwayFromZero
    )
    codes = Get-CountObject -Values $supportCodes
}
satisfaction = [ordered]@{
    response_count = $satisfactionScores.Count
    response_rate_percent = [Math]::Round(
        100 * $satisfactionScores.Count / $normalized.Count,
        2
    )
    minimum_responses = $MinimumSatisfactionResponses
    state = $satisfactionState
    average_1_to_5 = $satisfactionAverage
}
capability_gaps = [ordered]@{
    requests_with_gap = $requestsWithGap
    codes = Get-CountObject -Values $gapCodes
}
}
friction_hours_by_class = @($frictionBreakdown)
path_segments = @($pathSegments)
backlog_signals = [ordered]@{
    minimum_repetition = $MinimumSignalRepetition
    signals = $orderedSignals
}
measurement_limits = @(
    'The records are synthetic and establish a baseline only.',
    'Onboarding duration does not measure workload delivery performance.',
    'Survey averages are hidden below the minimum response count.',
    'An association does not prove cause or justify removing a control.',
    'Friction classifications require evidence and accountable review.'
)
}
}

if (-not [string]::IsNullOrEmpty($InputPath)) {
    $resolvedInput = (Resolve-Path -LiteralPath $InputPath).Path
    $document = Get-Content -LiteralPath $resolvedInput -Raw |
        ConvertFrom-Json -Depth 50 -DateKind String
    $scorecard = Get-AdoptionFrictionScorecard `
        -Document $document `
        -MinimumSatisfactionResponses $MinimumSatisfactionResponses `
        -MinimumSignalRepetition $MinimumSignalRepetition
    $json = $scorecard | ConvertTo-Json -Depth 20

    if ([string]::IsNullOrEmpty($OutputPath)) {
        $json
    }
    else {
        $resolvedOutput =
$ExecutionContext.SessionState.Path.GetUnresolvedProviderPathFromPSPath(
        $OutputPath
    )

```

```

        [IO.File]::WriteAllText(
            $resolvedOutput,
            $json + [Environment]::NewLine,
            [Text.UTF8Encoding]::new($false)
        )
    }
}

```

The generated baseline has eight submitted requests. Six are fulfilled, one is withdrawn, and one is pending at the window boundary. Fulfilled lead time has a p50 of 18 hours and a p90 of 48 hours. These values teach the method; they are not platform targets.

The owner clocks add shape to those totals. For fulfilled requests, platform-active time has a p50 of 10 hours, workload wait has a p50 of 2 hours, and external wait has a p50 of 0 hours. Their p90 values expose less common delay. A product review can open the affected records rather than guessing from total duration.

The full observed population contains 220 hours. External dependency represents 108 hours, or 49.09 percent, while necessary control represents 36 hours, or 16.36 percent. Those labels support investigation but do not establish cause or fault. Accountable owners must review the evidence behind each classification.

Five of the eight requests have a validation failure. Four have an exception-learning classification, and nine support contacts equal 1.13 contacts per request. Six survey responses exceed the minimum of five and yield an average of 3.5 out of 5. Every headline value is paired with its denominator or sample condition.

The repeated-signal threshold is two occurrences in this small fixture. Documentation questions, status updates, validation help, and regional IP address management capacity meet it. The threshold is a local prompt for triage. It is not a benchmark, severity rating, or backlog rank.

```

JSON
{
  "schema_version": "1.0",
  "scorecard": {
    "id": "adoption_friction_baseline",
    "window_start_utc": "2026-06-01T00:00:00Z",
    "window_end_utc": "2026-07-01T00:00:00Z",
    "status": "baseline",
    "composite_score": null
  },
  "population": {
    "request_count": 8,
    "fulfilled_count": 6,
    "withdrawn_count": 1,
    "pending_count": 1
  },
  "measures": {
    "provisioning_lead_time_hours": {
      "population": 6,
      "p50": 18.0,
      "p90": 48.0
    },
    "platform_active_time_hours": {

```

```

    "population": 6,
    "p50": 10.0,
    "p90": 16.0
  },
  "workload_wait_time_hours": {
    "population": 6,
    "p50": 2.0,
    "p90": 12.0
  },
  "external_wait_time_hours": {
    "population": 6,
    "p50": 0.0,
    "p90": 24.0
  },
  "validation_rework": {
    "requests_with_failure": 5,
    "failure_events": 5,
    "request_rate_percent": 62.5,
    "codes": {
      "invalid_environment": 1,
      "invalid_owner_ref": 1,
      "missing_budget_contact": 1,
      "missing_network_evidence": 1,
      "unsupported_region": 1
    }
  },
  "exception_demand": {
    "request_count": 4,
    "request_rate_percent": 50.0,
    "classifications": {
      "candidate_supported_path": 1,
      "documentation_gap": 1,
      "one_off_need": 1,
      "path_defect": 1
    }
  },
  "support_demand": {
    "contact_count": 9,
    "contacts_per_request": 1.13,
    "codes": {
      "documentation_question": 2,
      "status_update": 2,
      "validation_help": 2,
      "exception_status_question": 1,
      "network_dependency_question": 1,
      "path_selection_help": 1
    }
  },
  "satisfaction": {
    "response_count": 6,
    "response_rate_percent": 75.0,
    "minimum_responses": 5,
    "state": "reported",
    "average_1_to_5": 3.5
  },
  "capability_gaps": {
    "requests_with_gap": 5,
    "codes": {
      "regional_ipam_capacity": 2,
      "network_dependency_documentation": 1,
      "ownership_reference_discovery": 1,
      "regional_product_line": 1
    }
  }
},
"friction_hours_by_class": [
  {
    "friction_class": "standard_work",
    "total_hours": 40.0,
    "percent_of_observed_hours": 18.18
  },

```

```

{
  "friction_class": "necessary_control",
  "total_hours": 36.0,
  "percent_of_observed_hours": 16.36
},
{
  "friction_class": "customer_rework",
  "total_hours": 12.0,
  "percent_of_observed_hours": 5.45
},
{
  "friction_class": "platform_defect",
  "total_hours": 24.0,
  "percent_of_observed_hours": 10.91
},
{
  "friction_class": "external_dependency",
  "total_hours": 108.0,
  "percent_of_observed_hours": 49.09
}
],
"path_segments": [
  {
    "path_id": "connected_landing_zone",
    "request_count": 5,
    "fulfilled_count": 4,
    "lead_time_hours": {
      "population": 4,
      "p50": 24.0,
      "p90": 48.0
    }
  },
  {
    "path_id": "isolated_landing_zone",
    "request_count": 3,
    "fulfilled_count": 2,
    "lead_time_hours": {
      "population": 2,
      "p50": 6.0,
      "p90": 18.0
    }
  }
],
"backlog_signals": {
  "minimum_repetition": 2,
  "signals": [
    {
      "source": "support_contact",
      "code": "documentation_question",
      "count": 2
    },
    {
      "source": "support_contact",
      "code": "status_update",
      "count": 2
    },
    {
      "source": "support_contact",
      "code": "validation_help",
      "count": 2
    },
    {
      "source": "capability_gap",
      "code": "regional_ipam_capacity",
      "count": 2
    }
  ]
},
"measurement_limits": [
  "The records are synthetic and establish a baseline only.",
  "Onboarding duration does not measure workload delivery performance.",

```

```
"Survey averages are hidden below the minimum response count.",  
"An association does not prove cause or justify removing a control.",  
"Friction classifications require evidence and accountable review."  
]  
}
```

Path segmentation prevents one route from hiding another. Connected and isolated landing zones retain separate request counts, fulfilled counts, and lead-time percentiles. The samples are too small to support a fair comparison. The output presents the facts without claiming that either route performs better.

Production segments should be stable, relevant, and large enough to interpret. Customer group, request class, or service tier may help if privacy and access rules allow them. A changing segment definition can create a false trend. The definition and its effective date should therefore live beside the measurement contract.

Scorecard review begins with evidence quality. The platform product owner checks window coverage, outcomes, interval continuity, code definitions, and survey participation. Service owners then examine the records behind repeated or extreme signals. Customer representatives add context that event data cannot supply.

Only a supported interpretation should become backlog work. A useful item states a hypothesis, links the evidence, names an accountable owner, and predicts a customer outcome. It also names a later measurement window. This structure turns “requests are slow” into a claim that can be tested.

Suppose regional address capacity repeatedly creates external wait. A hypothesis might propose an earlier capacity check and clearer alternatives before the request enters fulfillment. The next window can test lead time, support contacts, validation rework, and satisfaction together. Improvement is credible only when the intended gain does not produce hidden harm elsewhere. The same discipline applies to necessary control. A slow evidence review may need reusable proof, earlier collection, automated checks, or clearer decision criteria. Duration by itself cannot justify removing the review. The control owner remains accountable for showing that any revised path still meets the risk intent.

Trend comparisons require versioned definitions. A changed request schema, paved road, fulfillment boundary, or reason-code set can break continuity with the prior baseline. A material change should trigger a new baseline or a clearly marked series break. Operational history belongs with the result so later reviewers do not invent a trend from incompatible periods.

Balanced measures also limit gaming. Lead time can fall when difficult work is rejected, support demand can fall when help is hard to reach, and survey scores can rise through selective invitations. Outcome counts, access to support, response rate, rework, exceptions, and

satisfaction should move through the same review. No local incentive should favor one number at the cost of the service.

This measurement loop completes the adoption system. The platform now has a product definition, request contract, supported paths, exception route, and responsibility boundary, plus evidence from actual use. The scorecard sends recurring problems and unmet needs back to product discovery. It does not replace judgment; it makes judgment more accountable.

The evidence also changes the starting point for governance. Recurring defects, manual reviews, late controls, and capability gaps show where guardrails need clearer intent or better delivery. Chapter 8 can use those signals to govern change around explicit risk and observable outcomes. Adoption friction becomes the feedback that helps guardrails protect without becoming needless gates.

Chapter 8: Govern — Turn Risk into Useful Guardrails

Chapter 8.1: Translate Risk into Cloud Policy

Govern — Turn Risk into Useful Guardrails begins with a question about harm. What unwanted outcome could cloud use create for the organization? A public data path, unsupported region, or ownerless cost is meaningful because of its business effect. The service used to enforce a rule comes later.

Risk is not a synonym for noncompliance. Risk is an uncertain unwanted outcome, while an issue already exists. Noncompliance is an observed failure to meet a stated rule. A clear governance system connects these ideas without collapsing them.

The risk statement names the outcome and its cause or entry point. It also defines the affected scope, probability, impact, priority, treatment, and owner. Those fields create a decision record rather than a vague concern. They also allow evidence to revise the assessment later.

Risk treatment may accept, avoid, mitigate, or transfer exposure. Each choice needs a rationale. A high rating can create urgency, but it does not decide the response on its own. Control error and operational harm belong in the decision as well.

Business tolerance turns the assessment into a boundary. The boundary may reject any unapproved occurrence, reject only new noncompliance, or allow a measured threshold. It states the conditions under which exposure is accepted. An authorized business or governance owner must approve that choice.

Technical teams can prepare evidence and response options. They should not create tolerance by selecting a convenient default. Code that blocks a change makes a business choice even when no decision record exists. Explicit approval makes that authority visible.

A cloud governance policy expresses the approved boundary as required behavior. Lowercase policy means the organizational rule; Azure Policy means the Azure service that can implement some rules. The distinction prevents a product from becoming the source of intent. Many governance rules use other technical or procedural controls.

A clear statement has a subject, requirement, and condition. It uses direct terms such as must or must not. It focuses on the outcome instead of a list of current settings. The implementation can then evolve without changing the rule's purpose.

Traceability keeps that purpose intact. Each statement has an ID, category, linked risk, and rationale. A reviewer can see why it exists and whether the source risk changed. A rule with no living risk link can survive long after its value disappears.

Scope supplies the edge of the promise. It identifies environments, asset classes, lifecycle points, and other conditions that decide applicability. “All resources” is rarely enough. Precision reduces surprise for both control owners and affected teams.

An exclusion and an exception are not the same. An exclusion is outside the designed scope from the start. An exception is a reviewed departure from a rule that otherwise applies. Mixing them creates hidden permission and weakens the compliance denominator.

The exception route needs an owner, evidence, and an end. Business need, risk decision, compensating measures, and expiry make the departure reviewable. When expiry arrives, authorization ends even if correction remains open. A recurring need should trigger policy or product review rather than endless renewal.

Ownership then moves from the risk to the operating system. The risk owner coordinates treatment and escalation, while the governance owner maintains policy intent. A control owner implements the approved response. Evidence and remediation owners keep the rule observable and correctable.

These roles can sit in different teams. Governance remains accountable without operating every control. Platform engineering can deliver inherited guardrails without owning the business consequences that set tolerance. Workload teams retain correction duties inside their boundary. A policy is incomplete until adherence can be measured. The record defines a numerator, denominator, target, cadence, evidence sources, owner, and limitation. The denominator names every eligible item in scope. Otherwise, missing records can improve the rate by disappearing. Evidence limits are part of the result. Resource inventory cannot prove data classification, and request records cannot prove all later placement. Billing data can trail recent deployment. Naming those limits protects a measure from being read as a broader assurance claim.

Compliance therefore answers a narrow question. It shows whether the observed population met the stated rule under the declared method. It does not prove that the underlying risk cannot occur. Risk review still needs incidents, tests, feedback, and changes in the business context.

The enforcement decision follows the evidence. This map uses observe, notify, and prevent as governance stages, not Azure Policy effects. Observe collects results without contacting the affected team. Notify adds an actionable correction route, while prevent rejects a new or changed item in scope.

A stronger response needs more than a severe risk. Scope precision must be tested, evidence must be dependable, remediation must be available, and the exception route must work. Weak input can make prevention both disruptive and easy to evade. The control would look strict while producing little assurance.

Monitor-first supports learning when priority is lower or uncertainty is high. It can expose false positives, unclear scope, and hidden dependencies. It is not a ritual that every rule must follow for the same period. A narrow and tested high-priority boundary can justify prevention at once. Proportionality weighs two kinds of harm. One is the harm created by the unwanted outcome. The other is the harm caused by a wrong, opaque, or unusable control. Useful governance reduces the first without ignoring the second.

The synthetic map below makes those choices explicit for the capstone platform. Three mappings link risk, tolerance, policy, scope, roles, evidence, enforcement, and review. The artifact is a decision record, not an Azure Policy payload. It reads no Azure state and creates no resource.

```
JSON
{
  "schema_version": "1.0",
  "map_id": "risk-to-control-capstone-2026q3",
  "title": "Capstone platform risk-to-control map",
  "as_of_date": "2026-07-21",
  "review_cadence_days": 90,
  "scope_context": {
    "organization": "Contoso",
    "platform_boundary": "Azure application landing zones",
    "decision_scope": "Governance intent before Azure Policy implementation",
    "excluded_decisions": [
      "Azure Policy definition logic",
      "initiative composition",
      "assignment hierarchy",
      "effect selection",
      "remediation deployment"
    ]
  },
  "stage_catalog": [
    {
      "id": "observe",
      "meaning": "Collect and review evidence without contacting or blocking the affected team."
    },
    {
      "id": "notify",
      "meaning": "Collect evidence and route an actionable notice without blocking the change."
    },
    {
      "id": "prevent",
      "meaning": "Reject a new or changed item inside the declared scope when it violates the policy statement."
    }
  ],
  "roles": [
    {
      "id": "cloud_governance_owner",
      "name": "Cloud governance lead",
      "accountability": "Maintains governance policy intent, scope, review, and cross-domain decisions."
    },
    {
      "id": "security_risk_owner",
      "name": "Cloud security risk lead",
      "accountability": "Owns treatment and escalation for public-exposure risk."
    },
    {
      "id": "resource_risk_owner",
```

```

    "name": "Cloud architecture risk lead",
    "accountability": "Owns treatment and escalation for unsupported-region risk."
  },
  {
    "id": "finops_risk_owner",
    "name": "FinOps lead",
    "accountability": "Owns treatment and escalation for unallocated cloud cost."
  },
  {
    "id": "platform_control_owner",
    "name": "Platform engineering lead",
    "accountability": "Designs and maintains platform enforcement controls after policy
approval."
  },
  {
    "id": "platform_evidence_owner",
    "name": "Platform operations lead",
    "accountability": "Produces complete, timely, and reviewable governance evidence."
  },
  {
    "id": "workload_remediation_owner",
    "name": "Workload product owner",
    "accountability": "Corrects workload-owned violations and supplies missing workload
context."
  },
  {
    "id": "exception_process_owner",
    "name": "Governance exception lead",
    "accountability": "Maintains authorized, time-limited exception decisions and expiry
evidence."
  }
],
"evidence_sources": [
  {
    "id": "source_resource_inventory",
    "name": "Normalized Azure resource inventory",
    "owner_role_id": "platform_evidence_owner",
    "freshness_days": 1,
    "known_limit": "Resource properties do not prove business data classification."
  },
  {
    "id": "source_data_classification",
    "name": "Workload data-classification register",
    "owner_role_id": "security_risk_owner",
    "freshness_days": 30,
    "known_limit": "Current classification coverage is 82 percent of production data
services."
  },
  {
    "id": "source_landing_zone_requests",
    "name": "Validated landing-zone request records",
    "owner_role_id": "platform_evidence_owner",
    "freshness_days": 1,
    "known_limit": "Records show declared and mapped regions, not later workload-
resource placement."
  },
  {
    "id": "source_ownership_directory",
    "name": "Workload ownership directory",
    "owner_role_id": "workload_remediation_owner",
    "freshness_days": 7,
    "known_limit": "Legacy workload records can contain inactive contacts."
  },
  {
    "id": "source_cost_allocation",
    "name": "Monthly cloud cost-allocation report",
    "owner_role_id": "finops_risk_owner",
    "freshness_days": 35,
    "known_limit": "Recent resources can appear before the next completed billing
period."
  }
],

```

```

{
  "id": "source_exception_register",
  "name": "Governance exception register",
  "owner_role_id": "exception_process_owner",
  "freshness_days": 1,
  "known_limit": "A recorded exception proves authorization, not continuing control effectiveness."
},
],
"mappings": [
  {
    "risk": {
      "id": "RISK-SEC-001",
      "category": "security",
      "status": "open",
      "unwanted_outcome": "A production service exposes regulated data through an unauthorized public network path.",
      "cause_or_entry_point": "Public network access is enabled while the service holds regulated data.",
      "probability": "medium",
      "impact": "critical",
      "priority": "critical",
      "treatment": "mitigate",
      "treatment_rationale": "The potential breach impact exceeds the accepted business tolerance.",
      "owner_role_id": "security_risk_owner"
    },
    "tolerance": {
      "boundary": "No production data service classified as regulated may expose a public network endpoint without an approved and unexpired exception.",
      "boundary_type": "zero_tolerance_with_exception",
      "approved_by_role_id": "cloud_governance_owner",
      "rationale": "The organization accepts controlled exception decisions but not unreviewed regulated-data exposure."
    },
    "policy": {
      "id": "POL-SEC-001",
      "category": "security",
      "risk_id": "RISK-SEC-001",
      "purpose": "Prevent unauthorized public paths to regulated production data.",
      "statement": "Production data services classified as regulated must not expose public network endpoints unless an approved, linked, and unexpired exception applies.",
      "behavior": {
        "subject": "Production data services classified as regulated",
        "requirement": "must not expose public network endpoints",
        "condition": "when the service is in the application landing-zone boundary",
        "verification": "Compare inventory network exposure, workload classification, and active exception evidence."
      },
      "scope": {
        "included_environments": [
          "production"
        ],
        "included_asset_classes": [
          "regulated_data_service"
        ],
        "excluded_asset_classes": [],
        "exception_rule": {
          "required": true,
          "owner_role_id": "exception_process_owner",
          "must_expire": true,
          "required_evidence": [
            "business need",
            "risk decision",
            "compensating controls",
            "expiry date"
          ]
        }
      },
      "owners": {
        "accountable_role_id": "cloud_governance_owner",

```

```

        "implementation_role_id": "platform_control_owner",
        "evidence_role_id": "platform_evidence_owner",
        "remediation_role_id": "workload_remediation_owner"
    }
},
"measure": {
    "id": "MEASURE-SEC-001",
    "name": "regulated_data_private_access_rate",
    "numerator": "Eligible regulated production data services without public exposure
or with an active linked exception.",
    "denominator": "All eligible regulated production data services in the declared
scope.",
    "target_percent": 100,
    "cadence_days": 7,
    "source_ids": [
        "source_resource_inventory",
        "source_data_classification",
        "source_exception_register"
    ],
    "owner_role_id": "platform_evidence_owner",
    "limitation": "Incomplete classification can hide an otherwise eligible service
from the denominator."
},
"enforcement": {
    "stage": "observe",
    "decision_date": "2026-07-21",
    "proportionality_rationale": "Critical impact supports strong control, but
incomplete classification makes safe prevention premature.",
    "readiness": {
        "scope_precision_tested": true,
        "evidence_reliable": false,
        "remediation_available": true,
        "exception_route_ready": true
    },
    "blocking_conditions": [
        "Production data-classification coverage is below 100 percent."
    ],
    "next_stage_criteria": [
        "Classification coverage reaches 100 percent for two review windows.",
        "Scope testing finds no false classification exclusions.",
        "Security and workload owners approve the evidence route."
    ]
},
"review": {
    "next_review_date": "2026-10-19",
    "owner_role_id": "cloud_governance_owner",
    "triggers": [
        "regulated-data incident",
        "classification-model change",
        "material false-positive finding",
        "business tolerance change"
    ]
}
},
{
    "risk": {
        "id": "RISK-RES-001",
        "category": "resource_management",
        "status": "open",
        "unwanted_outcome": "A new application landing zone is provisioned in a region the
organization cannot support or govern.",
        "cause_or_entry_point": "A request selects a region outside the maintained
platform region catalog.",
        "probability": "medium",
        "impact": "high",
        "priority": "high",
        "treatment": "avoid",
        "treatment_rationale": "Unsupported placement creates material service,
compliance, and recovery uncertainty.",
        "owner_role_id": "resource_risk_owner"
    }
},

```

```

    "tolerance": {
      "boundary": "Every new application landing-zone request must use regions approved
for its environment and connectivity profile.",
      "boundary_type": "no_new_noncompliance",
      "approved_by_role_id": "cloud_governance_owner",
      "rationale": "New requests can be checked before provisioning, so the organization
accepts no unmanaged regional placement."
    },
    "policy": {
      "id": "POL-RES-001",
      "category": "resource_management",
      "risk_id": "RISK-RES-001",
      "purpose": "Keep new landing zones inside regions with published platform
support.",
      "statement": "New application landing-zone requests must select only regions
approved for the declared environment and connectivity profile before fulfillment
begins.",
      "behavior": {
        "subject": "New application landing-zone requests",
        "requirement": "must select only regions approved for the declared environment
and connectivity profile",
        "condition": "before the request enters fulfillment",
        "verification": "Compare mapped request regions with the approved platform
region catalog."
      },
      "scope": {
        "included_environments": [
          "development",
          "test",
          "production"
        ],
        "included_asset_classes": [
          "application_landing_zone_request"
        ],
        "excluded_asset_classes": [
          "existing_landing_zone"
        ],
        "exception_rule": {
          "required": true,
          "owner_role_id": "exception_process_owner",
          "must_expire": true,
          "required_evidence": [
            "business need",
            "service-impact review",
            "risk decision",
            "expiry date"
          ]
        }
      },
      "owners": {
        "accountable_role_id": "cloud_governance_owner",
        "implementation_role_id": "platform_control_owner",
        "evidence_role_id": "platform_evidence_owner",
        "remediation_role_id": "workload_remediation_owner"
      }
    },
    "measure": {
      "id": "MEASURE-RES-001",
      "name": "approved_region_request_rate",
      "numerator": "Eligible requests whose mapped regions are approved for the declared
environment and connectivity profile.",
      "denominator": "All eligible new application landing-zone requests submitted in
the measurement window.",
      "target_percent": 100,
      "cadence_days": 7,
      "source_ids": [
        "source_landing_zone_requests",
        "source_exception_register"
      ],
      "owner_role_id": "platform_evidence_owner",

```

```

        "limitation": "The measure covers request mapping and does not prove later
workload-resource placement."
    },
    "enforcement": {
        "stage": "prevent",
        "decision_date": "2026-07-21",
        "proportionality_rationale": "The risk is high, the control applies only to new
requests, and a tested supported path and exception route already exist.",
        "readiness": {
            "scope_precision_tested": true,
            "evidence_reliable": true,
            "remediation_available": true,
            "exception_route_ready": true
        },
        "blocking_conditions": [],
        "next_stage_criteria": []
    },
    "review": {
        "next_review_date": "2026-10-19",
        "owner_role_id": "cloud_governance_owner",
        "triggers": [
            "platform region catalog change",
            "regional service withdrawal",
            "material customer demand",
            "business tolerance change"
        ]
    }
},
{
    "risk": {
        "id": "RISK-COST-001",
        "category": "cost",
        "status": "open",
        "unwanted_outcome": "Cloud cost cannot be assigned to an accountable workload and
business owner.",
        "cause_or_entry_point": "Required ownership references are missing, stale, or
invalid.",
        "probability": "high",
        "impact": "medium",
        "priority": "medium",
        "treatment": "mitigate",
        "treatment_rationale": "Unallocated cost weakens decisions, but abrupt blocking
could disrupt legacy workload maintenance.",
        "owner_role_id": "finops_risk_owner"
    },
    "tolerance": {
        "boundary": "At least 98 percent of monthly application landing-zone cost must
resolve to an active workload and business owner.",
        "boundary_type": "measured_threshold",
        "approved_by_role_id": "cloud_governance_owner",
        "rationale": "A small reconciliation window is accepted while missing ownership
remains visible and assigned."
    },
    "policy": {
        "id": "POL-COST-001",
        "category": "cost",
        "risk_id": "RISK-COST-001",
        "purpose": "Keep platform and workload cost linked to active accountability.",
        "statement": "Application landing zones must carry resolvable workload, business-
owner, technical-owner, and environment references throughout their active lifecycle.",
        "behavior": {
            "subject": "Active application landing zones",
            "requirement": "must carry resolvable workload, business-owner, technical-owner,
and environment references",
            "condition": "throughout the active landing-zone lifecycle",
            "verification": "Compare landing-zone metadata with the ownership directory and
monthly cost-allocation report."
        },
        "scope": {
            "included_environments": [
                "development",

```

```

        "test",
        "production"
    ],
    "included_asset_classes": [
        "application_landing_zone"
    ],
    "excluded_asset_classes": [
        "retired_landing_zone"
    ],
    "exception_rule": {
        "required": true,
        "owner_role_id": "exception_process_owner",
        "must_expire": true,
        "required_evidence": [
            "temporary accountable owner",
            "correction plan",
            "expiry date"
        ]
    }
},
"owners": {
    "accountable_role_id": "finops_risk_owner",
    "implementation_role_id": "platform_control_owner",
    "evidence_role_id": "platform_evidence_owner",
    "remediation_role_id": "workload_remediation_owner"
},
"measure": {
    "id": "MEASURE-COST-001",
    "name": "allocated_landing_zone_cost_rate",
    "numerator": "Eligible monthly landing-zone cost linked to active workload and
business-owner records.",
    "denominator": "All eligible application landing-zone cost in the completed
billing period.",
    "target_percent": 98,
    "cadence_days": 35,
    "source_ids": [
        "source_ownership_directory",
        "source_cost_allocation",
        "source_exception_register"
    ],
    "owner_role_id": "finops_risk_owner",
    "limitation": "New resources can precede the completed billing period and require
later reconciliation."
},
"enforcement": {
    "stage": "notify",
    "decision_date": "2026-07-21",
    "proportionality_rationale": "The current risk is material but below the threshold
for disrupting legacy workload changes.",
    "readiness": {
        "scope_precision_tested": true,
        "evidence_reliable": true,
        "remediation_available": true,
        "exception_route_ready": true
    },
    "blocking_conditions": [
        "Legacy ownership records still require contact reconciliation."
    ],
    "next_stage_criteria": [
        "Ownership resolution remains at or above 98 percent for three completed billing
periods.",
        "Notification evidence shows correction within the agreed service expectation.",
        "FinOps and workload representatives approve any stronger response."
    ]
},
"review": {
    "next_review_date": "2026-10-19",
    "owner_role_id": "finops_risk_owner",
    "triggers": [
        "allocation rate below 95 percent",

```

```

        "material ownership-directory change",
        "cost-accountability dispute",
        "business tolerance change"
    ]
}
},
],
"decision_limits": [
    "The map records governance intent and does not deploy or assign an Azure Policy object.",
    "Risk rankings and tolerance boundaries are synthetic and require authorized business approval.",
    "A compliance measure tests the stated rule but does not prove that the underlying risk is eliminated.",
    "An enforcement stage can change only through a reviewed decision with current evidence.",
    "Exceptions remain governed records and must not become hidden exclusions."
]
}

```

The first record addresses unauthorized public access to regulated production data. Its impact and priority are critical, and its desired compliance rate is 100 percent. Yet the chosen stage is observe. Only 82 percent of production data services have current classification evidence.

That gap makes immediate prevention unsafe. An unclassified service might escape the eligible population, while a wrong class could stop valid work. The record does not lower the tolerance boundary. It names the evidence defect and sets criteria for a new stage decision.

The second record covers unsupported regions for new application landing-zone requests. The request path already maps approved regions by environment and connectivity profile. Invalid requests can select another region or enter a time-limited exception path. Every readiness condition is true.

Prevention fits that narrow boundary. It stops new unsupported placement before fulfillment while leaving existing landing zones outside the decision. The measure proves only request mapping. It does not claim that later workload deployments remain in approved regions.

The third record concerns cloud cost without a current workload and business owner. The tolerance requires at least 98 percent of monthly landing-zone cost to resolve to active ownership. Legacy contacts still need reconciliation. Notification keeps correction active without blocking useful maintenance.

None of these stages is a maturity score. Observation is responsible when evidence cannot support a safe block. Prevention is responsible when impact, scope, data, remedy, and exceptions align. Notification is responsible when a material gap has a workable correction path but not enough cause for rejection.

Review makes each decision temporary in the right way. The map gives every record a next date and an owner. Incidents, false positives, catalog changes, disputes, and revised tolerance can reopen it sooner. Fresh evidence can support a stronger response, a weaker response, or a changed rule.

This is also where Chapter 8.1 stops. The map defines no Azure Policy condition, initiative, assignment, scope hierarchy, effect, or remediation task. Chapter 8.2 will explain those objects and their evaluation path. Chapter 8.3 will test how enforcement changes without surprising the estate.

Chapter 8.4 will place implementation under version control and govern exemptions. Chapter 8.5 will apply the method to cost and resource consistency. Those mechanisms need the decision chain built here. Risk gives the later code a reason to exist.

The closing test is not how many actions a platform can deny. It is whether an affected team can trace a guardrail back to a real outcome, approved tolerance, fair scope, named owner, honest measure, and reviewable response. If that chain breaks, stricter enforcement will not repair the missing logic. It will only make the gap harder to ignore.

Chapter 8: Govern — Turn Risk into Useful Guardrails

Chapter 8.2: Understand Azure Policy as a Hierarchical System

A policy definition can be valid and still govern nothing. Azure Policy turns stored logic into behavior through a chain of objects. A definition describes a test and its effect, while an initiative groups definitions into a control set. An assignment binds that logic to a scope and supplies local values.

This separation creates both reuse and confusion. The definition's location limits where it may be assigned; it does not name the resources to evaluate. The assignment creates that population. Storage location grants potential reach, while assignment scope uses it.

A policy definition contains the rule Azure evaluates. Its `if` clause selects matching resources, and its `then` clause names the effect. Parameters leave context-sensitive choices open until a later object supplies them. Metadata adds classification and traceability without changing evaluation.

Definition location is a design choice. A custom definition stored at a management group can be assigned at that group or below it. It cannot be assigned to a parent or an unrelated branch. A rule intended for many landing zones belongs high enough to be shared, but no higher than its true ownership boundary.

An initiative, represented as a policy set definition in the Azure API and Terraform provider, collects related definitions. It gives the group one assignment surface and one shared control identity. Each included definition has a reference ID so messages and results can identify the member rule. Initiative parameters can also feed the parameters of one or more included definitions.

An initiative is a composition object, not an extra enforcement layer. The member definition still performs the resource test and supplies the effect. The initiative controls how rules are grouped, tuned, assigned, and reported. It simplifies management without hiding the identity of each member.

Each link answers a different question. The rule asks what must be tested, and the initiative asks which tests belong together. The assignment decides where the set applies. Its parameter values tune the shared logic for that scope.

An assignment turns stored logic into applicable governance. It points to one definition or initiative, declares a scope, and binds parameter values. Optional exclusions can remove narrower scopes from evaluation. The assignment can also provide messages that explain what an affected team should do.

Scope follows Azure Resource Manager hierarchy. A management-group assignment includes that management group, its child management groups, and subscriptions and resources below them. A subscription assignment reaches its resource groups and resources. A resource-group assignment reaches the resources inside that group.

Consider one storage account in a landing-zone subscription. The assignment sits above it at the landing-zone management group, and the initiative points to the location rule. The assignment passes `westeurope` and `northeurope` into that rule. A storage account in `eastus` matches the audit condition.

Inheritance flows downward, and applicable assignments accumulate. A child assignment does not replace an inherited parent assignment. A resource may therefore be evaluated by a platform baseline, a branch-specific initiative, and a workload control at the same time. The effective system is the set of every applicable assignment, not the nearest item alone.

A hierarchy change can alter that set without changing the resource. Moving the subscription to another branch may remove one inherited assignment and add another. The definition files can stay fixed during that move. The result still changes because its parent changed.

Scope errors cut both ways. An assignment placed too high can reach unrelated resources, while one placed too low can miss key assets. Moving a subscription can cross either boundary. The management tree is therefore a live control surface.

This distinction separates platform baselines from workload-specific controls. The platform team can publish a reusable initiative at a shared management group and assign it to the landing-zone branch. A regulated workload can receive additional rules at its subscription or resource group. Lower placement narrows applicability; it does not erase the platform promise.

The layers should express different needs. A baseline covers rules every landing zone shares, while a workload rule addresses one service trait or risk class. Both can apply to the same resource. Azure does not assume the lower rule wins when they conflict.

Assignment review therefore starts with ancestry. The workload repository gives only the local view, and the parent baseline misses local rules. The resource joins both views at evaluation time. Its ancestry is part of its effective configuration.

The first Terraform file pins the tested toolchain. Pinning protects the example from silent provider changes. The provider block stays minimal because local contract tests need no live plan or apply. Authentication remains an environment concern, never a value in this configuration.

```
HCL / TERRAFORM
terraform {
  required_version = "~> 1.15.0"
```

```

    required_providers {
      azurerm = {
        source = "hashicorp/azurerm"
        version = "~> 4.81.0"
      }
    }
  }

  provider "azurerm" {
    features {}
  }
}

```

The inputs expose two distinct scopes. The parent management group owns the reusable definition and initiative, while the child receives the assignment. The example assumes that this relationship already exists. String validation proves resource ID shape, not ancestry or deployment rights.

The approved locations are assignment data rather than copied rule logic. A list validation rejects duplicates, mixed case, and the special `global` location. That convention makes the later JSON payload stable. Real location choices still require service availability, compliance, recovery, and cost review.

HCL / TERRAFORM

```

variable "definition_management_group_id" {
  description = "Resource ID of the management group that owns the reusable policy definition and initiative."
  type        = string
  default     = "/providers/Microsoft.Management/managementGroups/mg-platform"

  validation {
    condition = can(regex("^/providers/Microsoft\\.Management/managementGroups/[A-Za-z0-9._()-]+$", var.definition_management_group_id))
    error_message = "definition_management_group_id must be a full management-group resource ID."
  }
}

variable "assignment_management_group_id" {
  description = "Resource ID of the child management group where the initiative is assigned."
  type        = string
  default     = "/providers/Microsoft.Management/managementGroups/mg-landing-zones"

  validation {
    condition = can(regex("^/providers/Microsoft\\.Management/managementGroups/[A-Za-z0-9._()-]+$", var.assignment_management_group_id))
    error_message = "assignment_management_group_id must be a full management-group resource ID."
  }
}

variable "allowed_locations" {
  description = "Lowercase Azure locations accepted by the platform baseline assignment."
  type        = list(string)
  default     = ["northeurope", "westeurope"]

  validation {
    condition = (
      length(var.allowed_locations) > 0 &&
      length(distinct(var.allowed_locations)) == length(var.allowed_locations) &&
      alltrue([
        for location in var.allowed_locations :

```

```

        location == lower(trimSpace(location)) && location != "global"
    ])
)
error_message = "allowed_locations must contain distinct, nonempty lowercase locations
and must not include global."
}
}

```

The configuration creates the three-object chain. The definition audits regional resources whose `location` is absent from `allowedLocations`. A separate `global` condition avoids treating nonregional resources as violations. `Indexed` mode limits the rule to resource types that support tags and location, while metadata keeps the risk link.

The initiative exposes `initiativeAllowedLocations` and maps it into the definition's `allowedLocations` parameter. The names differ on purpose so the handoff is visible. A stable reference ID links the member definition to its noncompliance message. Adding another location rule later would not require a second assignment surface.

The assignment sits at the landing-zone management group. It binds the approved locations and points to the initiative ID. Its `enforce = true` setting selects the default enforcement mode, but the definition still uses `audit`. A mismatch is evaluated and recorded rather than rejected.

```

HCL / TERRAFORM
locals {
  definition_parameters = {
    allowedLocations = {
      type = "Array"
      metadata = {
        displayName = "Allowed locations"
        description = "Azure locations approved by the assignment owner."
        strongType = "location"
      }
    }
  }
}

policy_rule = {
  if = {
    allOf = [
      {
        field = "location"
        notIn = "[parameters('allowedLocations')]"
      },
      {
        field      = "location"
        notEquals = "global"
      }
    ]
  }
  then = {
    effect = "audit"
  }
}

initiative_parameters = {
  initiativeAllowedLocations = {
    type = "Array"
    metadata = {
      displayName = "Allowed locations"
      description = "Locations passed to every location policy in this initiative."
      strongType = "location"
    }
  }
}

```

```

    }
  }

resource "azurerms_policy_definition" "allowed_locations" {
  name           = "audit-approved-locations"
  display_name   = "Audit resources outside approved locations"
  description     = "Records noncompliance when an Azure resource uses an unapproved regional location."
  policy_type    = "Custom"
  mode           = "Indexed"
  management_group_id = var.definition_management_group_id

  metadata = jsonencode({ category = "Platform Governance", riskId = "GOV-LOCATION-001" })
  parameters = jsonencode(local.definition_parameters)
  policy_rule = jsonencode(local.policy_rule)
}

resource "azurerms_management_group_policy_set_definition" "platform_baseline" {
  name           = "platform-location-baseline"
  display_name   = "Platform location baseline"
  description     = "Groups the platform's location rules behind one assignment contract."
  policy_type    = "Custom"
  management_group_id = var.definition_management_group_id

  metadata = jsonencode({ category = "Platform Governance", controlFamily = "Resource placement" })
  parameters = jsonencode(local.initiative_parameters)

  policy_definition_reference {
    policy_definition_id = azurerms_policy_definition.allowed_locations.id
    reference_id         = "auditAllowedLocations"
    parameter_values = jsonencode({
      allowedLocations = {
        value = "[parameters('initiativeAllowedLocations')]"
      }
    })
  }
}

resource "azurerms_management_group_policy_assignment" "landing_zone_baseline" {
  name           = "platform-loc-baseline"
  display_name   = "Platform location baseline"
  description     = "Evaluates the landing-zone management group and its descendants against approved locations."
  management_group_id = var.assignment_management_group_id
  policy_definition_id = azurerms_management_group_policy_set_definition.platform_baseline.id
  enforce          = true

  metadata = jsonencode({
    assignedBy = "platform-governance"
    riskId     = "GOV-LOCATION-001"
  })

  parameters = jsonencode({
    initiativeAllowedLocations = {
      value = var.allowed_locations
    }
  })

  non_compliance_message {
    content = "Use a platform-approved Azure location or obtain a documented exception."
    policy_definition_reference_id = "auditAllowedLocations"
  }
}

```

Effects answer what Azure should do when a rule matches. `audit` records noncompliance without blocking the request, while `deny` can reject it. Effects such as `modify` and `deployIfNotExists` can change or add settings; they also raise identity and remediation concerns. The next section develops the evidence needed for those stronger choices.

The effect belongs to the rule, while enforcement mode and scope belong to the assignment. Compliance state comes from a later evaluation. These facts interact, but they are not the same. Keeping them separate makes diagnosis far easier.

Assignment enforcement mode is a separate control. `DoNotEnforce` allows evaluation while suppressing enforcement actions, which makes it useful for testing an assignment's reach. A `disabled` effect prevents that definition from being evaluated. Confusing the two can make a test appear quiet while removing the evidence it was supposed to collect.

Azure evaluates each applicable assignment-definition-resource relationship. For an applicable resource, a true `audit` condition produces a noncompliant state and a false condition produces a compliant state. Azure first tests applicability, so a resource the definition does not cover does not enter that result. An exemption remains visible as exempt, unlike an excluded scope that is removed from evaluation.

Each state answers a narrow question: how did one resource relate to one rule during one evaluation? It does not prove that all risk is gone or that the service works as intended. Other tests and business evidence still matter. Compliance is one signal in the governance system.

An initiative rolls up its member results. One noncompliant applicable member makes the resource noncompliant for the initiative. Rules that clash can also yield a conflicting state. A rollup helps review, but the cause sits in the member reference and its bound value.

Compliance is delayed evidence, not instant proof of safety. Assignment and resource changes take time to appear in results. Errors, unregistered providers, and resources not yet assessed deserve review rather than a success label. A governance review should preserve that doubt.

Delay matters during a scope change. A moved subscription may still show old results for a time. A new assignment may not have assessed each resource yet. The absence of a violation is not instant proof. Reviewers need the assignment time and the last evaluation time.

Evaluation boundaries require two further distinctions. A `notScopes` entry excludes a subtree or resource from the assignment, so Azure does not evaluate it for that assignment. A policy exemption keeps the resource in governance reporting with an exempt state and supporting context. The code leaves exclusions empty and defers governed exemption implementation to the later policy-as-code section.

The outputs publish a review trace rather than a compliance claim. They expose the definition location, initiative reference, assignment scope, parameter path, and effect. A real deployment must resolve resource IDs and confirm the assumed parent-child link. The local test checks code links without contacting Azure.

```
HCL / TERRAFORM
output "evaluation_trace" {
  description = "The definition-to-resource path reviewers should follow when diagnosing
policy behavior."
  value = {
    definition_location      = var.definition_management_group_id
    definition_id            = azurerm_policy_definition.allowed_locations.id
    initiative_id            =
azurerm_management_group_policy_set_definition.platform_baseline.id
    assignment_scope        = var.assignment_management_group_id
    assignment_id           =
azurerm_management_group_policy_assignment.landing_zone_baseline.id
    included_reference       = "auditAllowedLocations"
    assignment_parameter     = var.allowed_locations
    definition_parameter     = "allowedLocations"
    effect                   = "audit"
    enforcement_mode        = "Default"
    inherited_by_descendants  = true
    excluded_scopes         = []
  }
}

output "expected_evaluation_sequence" {
  description = "A compact statement of how the hierarchy turns a stored rule into
compliance data."
  value = [
    "The parent management group stores the reusable definition.",
    "The initiative references the definition and maps its parameter.",
    "The child management-group assignment binds scope and approved locations.",
    "Resources below that assignment inherit it unless the evaluation boundary removes
them.",
    "A location mismatch triggers audit and produces a noncompliant result without
blocking deployment."
  ]
}
```

A useful investigation can now follow one direction. Start with the resource and list assignments at its scope and ancestors. Inspect each initiative member, resolve both parameter mappings, and then read the rule and effect. Finally, account for exclusions, exemptions, enforcement mode, and evaluation time.

That trace replaces “Is policy enabled?” with a testable question: why did this resource receive this state? Skipping any link invites the wrong diagnosis. A portal percentage cannot supply the missing path. The object chain can.

The hierarchy is the control. A definition supplies logic, an initiative supplies composition, and an assignment supplies scope and local values. Inheritance carries applicable rules down; compliance carries evaluated results back to governance owners. Azure Policy behavior is the whole path, never the definition alone.

Chapter 8: Govern — Turn Risk into Useful Guardrails

Chapter 8.3: Move from Monitoring to Enforcement Carefully

An inherited audit assignment can expose unwanted behavior without stopping work. Visibility is useful, but it does not prove that the rule is ready to deny a request. Enforcement changes both the platform's response and the workload team's options. The move must be supported by evidence.

Audit-first is a learning stage, not weak governance. It tests whether the rule finds the intended resources and whether its aliases expose the needed properties. It also reveals which teams, tools, and older designs depend on the behavior. A severe risk may justify speed, but urgency does not repair a false positive.

The first baseline should preserve counts, resource identities, definition references, timestamps, and effect. A percentage alone cannot show whether one faulty rule found many resources or many rules found one resource. It can also hide items that are not started, not registered, excluded, or exempt. Promotion needs the population behind the score.

The collection function below queries one management-group assignment and one initiative member. It verifies the active tenant before reading policy state. The state query omits `--all`, so each resource contributes its latest state rather than a trail of historical records. A separate summary query preserves Azure's rollup for comparison.

```
POWERSHELL
Set-StrictMode -Version Latest
$ErrorActionPreference = 'Stop'

function Invoke-AzureCliJson {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [string[]] $Arguments
    )

    $raw = & az @Arguments 2>&1
    if ($LASTEXITCODE -ne 0) {
        throw "Azure CLI failed: $($raw -join [Environment]::NewLine)"
    }

    try {
        return $raw | ConvertFrom-Json -Depth 50 -DateKind String
    }
    catch {
        throw 'Azure CLI returned invalid JSON.'
    }
}

function Export-PolicyCompliance {
    [CmdletBinding(SupportsShouldProcess)]
    param(
        [Parameter(Mandatory)]
        [guid] $ExpectedTenantId,
```

```

[Parameter(Mandatory)]
[ValidatePattern('^[A-Za-z0-9._-]+$')]
[string] $ManagementGroupId,

[Parameter(Mandatory)]
[ValidatePattern('^[A-Za-z0-9._-]{1,24}$')]
[string] $PolicyAssignmentName,

[Parameter(Mandatory)]
[ValidatePattern('^[A-Za-z0-9._-]+$')]
[string] $PolicyDefinitionReferenceId,

[Parameter(Mandatory)]
[ValidateNotNullOrEmpty()]
[string] $OutputPath,

[Parameter()]
[switch] $Force
)

$account = Invoke-AzureCliJson -Arguments @(
    'account', 'show',
    '--output', 'json',
    '--only-show-errors'
)
if ([guid] $account.tenantId -ne $ExpectedTenantId) {
    throw "Active tenant '$($account.tenantId)' does not match the expected tenant."
}

$resolvedOutput =
$ExecutionContext.SessionState.Path.GetUnresolvedProviderPathFromPSPath(
    $OutputPath
)
$outputParent = Split-Path -Parent $resolvedOutput
if (-not (Test-Path -LiteralPath $outputParent -PathType Container)) {
    throw "Output directory '$outputParent' does not exist."
}
if ((Test-Path -LiteralPath $resolvedOutput -PathType Leaf) -and -not $Force) {
    throw "Output file '$resolvedOutput' already exists. Use -Force to replace it."
}

$assignmentId = (
    "/providers/Microsoft.Management/managementGroups/$ManagementGroupId" +
    "/providers/Microsoft.Authorization/policyAssignments/$PolicyAssignmentName"
)
$states = @(Invoke-AzureCliJson -Arguments @(
    'policy', 'state', 'list',
    '--management-group', $ManagementGroupId,
    '--policy-assignment', $PolicyAssignmentName,
    '--filter', "policyDefinitionReferenceId eq '$PolicyDefinitionReferenceId'",
    '--select', (
        'timestamp,resourceId,resourceType,resourceLocation,' +
        'complianceState,policyAssignmentId,' +
        'policyDefinitionReferenceId,policyDefinitionAction'
    ),
    '--output', 'json',
    '--only-show-errors'
))
$summary = Invoke-AzureCliJson -Arguments @(
    'policy', 'state', 'summarize',
    '--management-group', $ManagementGroupId,
    '--policy-assignment', $PolicyAssignmentName,
    '--filter', "policyDefinitionReferenceId eq '$PolicyDefinitionReferenceId'",
    '--output', 'json',
    '--only-show-errors'
)

```

```

$document = [ordered]@{
    schema_version = '1.0'
    collected_utc = [DateTimeOffset]::UtcNow.ToString(
        'yyyy-MM-ddTHH:mm:ssZ',
        [Globalization.CultureInfo]::InvariantCulture
    )
    collection = [ordered]@{
        tenant_id = $ExpectedTenantId.ToString()
        management_group_id = $ManagementGroupId
        assignment_name = $PolicyAssignmentName
        assignment_id = $assignmentId
        policy_definition_reference_id = $PolicyDefinitionReferenceId
        latest_only = $true
    }
    provider_summary = $summary
    states = @(
        $states |
        Sort-Object -Property resourceId |
        ForEach-Object {
            [ordered]@{
                timestamp = [string] $_.timestamp
                resource_id = [string] $_.resourceId
                resource_type = [string] $_.resourceType
                resource_location = [string] $_.resourceLocation
                compliance_state = [string] $_.complianceState
                policy_assignment_id = [string] $_.policyAssignmentId
                policy_definition_reference_id = [string]
                $_.policyDefinitionReferenceId
                policy_definition_action = [string] $_.policyDefinitionAction
            }
        }
    )
}

if ($PSCmdlet.ShouldProcess($resolvedOutput, 'Write policy compliance evidence')) {
    $json = $document | ConvertTo-Json -Depth 20
    [IO.File]::WriteAllText(
        $resolvedOutput,
        $json + [Environment]::NewLine,
        [Text.UTF8Encoding]::new($false)
    )
}

return [pscustomobject] $document
}

```

The function fails closed on a tenant mismatch and refuses to replace an evidence file without `-Force`. Its selected fields keep assignment, member reference, action, resource, state, and evaluation time together. `ShouldProcess` supports a dry run of the local write. No command in the function triggers a scan or changes an Azure resource.

Latest state and historical activity answer different questions. Latest state describes the current compliance population. Historical records help explain when a change appeared, but counting them as current resources inflates the denominator. A promotion review should name which view it uses.

The provider summary is a cross-check, not a verdict. A raw count can fall because a resource was fixed, deleted, moved, excluded, or no longer evaluated. Each cause has a different meaning for risk. Review must reach the affected resources and the reasons for their state.

Every noncompliant item then needs a review class. A confirmed violation means the observed resource breaks the approved statement. A false positive means the rule caught behavior that the statement allows. An unresolved item remains unknown and cannot safely count as success or failure.

False positives reveal more than bad syntax. They can expose missing business context, an alias that differs by API version, or a scope that joins unlike workloads. Fixing the rule changes the experiment. Observation should restart long enough to test the changed logic against both old and new requests.

Confirmed violations need an owned correction path. The path states what changes, who can perform it, how service impact is limited, and how success is checked. A team that cannot correct an existing item has little room when a future request is denied. Enforcement readiness therefore includes correction capacity.

Time matters as well. Azure Policy evaluates after several triggers and on a recurring cycle, so a recent assignment or resource change may not yet appear. The decision record uses a declared observation window rather than the age of the oldest resource. Reviewers can then judge whether it covered normal delivery patterns.

The local decision function turns those facts into explicit gates. It accepts only latest compliant and noncompliant states, rejects duplicate resources, and keeps the three review classes separate. Corrected and blocked items must come from the confirmed set. The function reports eligibility for a limited canary only when every gate passes.

```
POWERSHELL
[CmdletBinding()]
param(
    [Parameter()]
    [string] $PolicyStatesPath,

    [Parameter()]
    [string] $ReviewPath,

    [Parameter()]
    [string] $OutputPath
)

Set-StrictMode -Version Latest
$ErrorActionPreference = 'Stop'

function ConvertTo-UtcInstant {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [string] $Value,

        [Parameter(Mandatory)]
        [string] $FieldName
    )

    if ($Value -notmatch '^\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}Z$') {
```

```

        throw "$FieldName must use UTC in yyyy-MM-ddTHH:mm:ssZ format."
    }

    return [DateTimeOffset]::ParseExact(
        $Value,
        "yyyy-MM-dd'T'HH:mm:ss'Z'",
        [Globalization.CultureInfo]::InvariantCulture,
        [Globalization.DateTimeStyles]::AssumeUniversal -bor
            [Globalization.DateTimeStyles]::AdjustToUniversal
    )
}

function Get-NormalizedSet {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [AllowEmptyCollection()]
        [object[]] $Values
    )

    return @(
        $Values |
        ForEach-Object { ([string] $_).Trim().ToLowerInvariant() } |
        Where-Object { -not [string]::IsNullOrEmpty($_) } |
        Sort-Object -Unique
    )
}

function Get-Percent {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [int] $Numerator,

        [Parameter(Mandatory)]
        [int] $Denominator
    )

    if ($Denominator -eq 0) {
        return 0.0
    }
    return [Math]::Round(
        100.0 * $Numerator / $Denominator,
        2,
        [MidpointRounding]::AwayFromZero
    )
}

function Assert-KnownSubset {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [string] $Name,

        [Parameter(Mandatory)]
        [AllowEmptyCollection()]
        [string[]] $Values,

        [Parameter(Mandatory)]
        [AllowEmptyCollection()]
        [string[]] $Allowed
    )

    $unknown = @($Values | Where-Object { $_ -notin $Allowed })
    if ($unknown.Count -gt 0) {
        throw "$Name contains resources outside the current noncompliant population."
    }
}

```

```

}

function Get-PolicyEnforcementDecision {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [pscustomobject] $PolicyStates,

        [Parameter(Mandatory)]
        [pscustomobject] $Review
    )

    if ([string] $PolicyStates.schema_version -cne '1.0') {
        throw 'Policy state evidence must use schema_version 1.0.'
    }
    if ([string] $Review.schema_version -cne '1.0') {
        throw 'Review evidence must use schema_version 1.0.'
    }
    if ([string] $PolicyStates.collection.assignment_id -cne
        [string] $Review.assignment_id) {
        throw 'The policy state and review assignment IDs must match exactly.'
    }
    if (-not [bool] $PolicyStates.collection.latest_only) {
        throw 'Policy state evidence must contain latest states only.'
    }

    $windowStart = ConvertTo-UtcInstant `
        -Value ([string] $Review.observation_window.start_utc) `
        -FieldName 'observation_window.start_utc'
    $windowEnd = ConvertTo-UtcInstant `
        -Value ([string] $Review.observation_window.end_utc) `
        -FieldName 'observation_window.end_utc'
    if ($windowEnd -le $windowStart) {
        throw 'The observation window end must be after its start.'
    }
    $observationDays = [Math]::Floor(($windowEnd - $windowStart).TotalDays)

    $states = @($PolicyStates.states)
    $resourceIds = @(Get-NormalizedSet -Values @($states.resource_id))
    if ($resourceIds.Count -ne $states.Count) {
        throw 'Policy state resource IDs must be nonempty and unique.'
    }
    $unsupportedStates = @(
        $states.compliance_state |
        Where-Object { $_ -notin @('Compliant', 'NonCompliant') } |
        Sort-Object -Unique
    )
    if ($unsupportedStates.Count -gt 0) {
        throw 'Decision input must contain only Compliant and NonCompliant states.'
    }

    $noncompliant = @(Get-NormalizedSet -Values @(
        $states |
        Where-Object compliance_state -eq 'NonCompliant' |
        ForEach-Object resource_id
    ))
    $falsePositive = @(Get-NormalizedSet -Values @(
        $Review.classification.false_positive_resource_ids
    ))
    $confirmed = @(Get-NormalizedSet -Values @(
        $Review.classification.confirmed_violation_resource_ids
    ))
    $unresolved = @(Get-NormalizedSet -Values @(
        $Review.classification.unresolved_resource_ids
    ))
    foreach ($entry in @(
        @{ Name = 'false_positive_resource_ids'; Values = $falsePositive },
        @{ Name = 'confirmed_violation_resource_ids'; Values = $confirmed },
        @{ Name = 'unresolved_resource_ids'; Values = $unresolved }
    ))

```

```

    )) {
        Assert-KnownSubset -Name $entry.Name -Values $entry.Values -Allowed $noncompliant
    }
    $classified = @($falsePositive + $confirmed + $unresolved)
    if (@(Get-NormalizedSet -Values $classified).Count -ne $classified.Count) {
        throw 'Classification groups must not overlap.'
    }

    $corrected = @(Get-NormalizedSet -Values @(
        $Review.remediation.corrected_resource_ids
    ))
    $blocked = @(Get-NormalizedSet -Values @(
        $Review.remediation.blocked_resource_ids
    ))
    Assert-KnownSubset -Name 'corrected_resource_ids' `
        -Values $corrected -Allowed $confirmed
    Assert-KnownSubset -Name 'blocked_resource_ids' `
        -Values $blocked -Allowed $confirmed
    if (@(Get-NormalizedSet -Values @($corrected + $blocked)).Count -ne
        ($corrected.Count + $blocked.Count)) {
        throw 'Corrected and blocked remediation groups must not overlap.'
    }

    $reviewCoverage = Get-Percent `
        -Numerator $classified.Count `
        -Denominator $noncompliant.Count
    $falsePositiveRate = Get-Percent `
        -Numerator $falsePositive.Count `
        -Denominator $classified.Count

    $gateResults = [ordered]@{
        observation_window = $observationDays -ge
            [int] $Review.gates.minimum_observation_days
        review_coverage = $reviewCoverage -ge
            [double] $Review.gates.minimum_review_coverage_percent
        false_positive_rate = $falsePositiveRate -le
            [double] $Review.gates.maximum_false_positive_rate_percent
        canary_outcomes = [int] $Review.canary.unexpected_outcomes -le
            [int] $Review.gates.maximum_unexpected_canary_outcomes
        unresolved_reviews = (-not [bool] $Review.gates.require_zero_unresolved_reviews) -
or
            $unresolved.Count -eq 0
        blocked_remediation = (-not [bool]
$Review.gates.require_zero_blocked_remediations) -or
            $blocked.Count -eq 0
        exception_process = (-not [bool] $Review.gates.require_exception_process_ready) -
or
            [bool] $Review.exception_process_ready
    }

    $failedGates = @(
        $gateResults.GetEnumerator() |
        Where-Object { -not $_.Value } |
        ForEach-Object Key
    )
    $eligible = $failedGates.Count -eq 0

    $criteria = [ordered]@{
        observation_window = "Observe at least $($Review.gates.minimum_observation_days)
days."
        review_coverage = "Classify at least
$($Review.gates.minimum_review_coverage_percent) percent of noncompliant resources."
        false_positive_rate = "Reduce reviewed false positives to
$($Review.gates.maximum_false_positive_rate_percent) percent or less."
        canary_outcomes = "Reduce unexpected canary outcomes to
$($Review.gates.maximum_unexpected_canary_outcomes) or fewer."
        unresolved_reviews = 'Resolve every sampled compliance dispute.'
        blocked_remediation = 'Clear every blocked remediation path in the reviewed
population.'
    }

```

```

        exception_process = 'Confirm that the governed exception process is ready before
deny testing.'
    }

    return [pscustomobject] [ordered]@{
        schema_version = '1.0'
        decision_id = [string] $Review.decision_id
        decision_date = [string] $Review.decision_date
        assignment_id = [string] $Review.assignment_id
        policy_definition_reference_id = [string] (
            $PolicyStates.collection.policy_definition_reference_id
        )
        current_stage = 'audit'
        candidate_next_stage = 'limited_deny_canary'
        decision = if ($eligible) {
            'eligible_for_limited_deny_canary'
        }
        else {
            'hold_audit'
        }
        decision_owner = [string] $Review.decision_owner
        evidence = [ordered]@{
            observation_days = [int] $observationDays
            total_resources = $states.Count
            compliant_resources = @(
                $states | Where-Object compliance_state -eq 'Compliant'
            ).Count
            noncompliant_resources = $noncompliant.Count
            reviewed_noncompliant_resources = $classified.Count
            review_coverage_percent = $reviewCoverage
            false_positive_resources = $falsePositive.Count
            false_positive_rate_percent = $falsePositiveRate
            confirmed_violation_resources = $confirmed.Count
            unresolved_resources = $unresolved.Count
            corrected_resources = $corrected.Count
            blocked_remediation_resources = $blocked.Count
            canary_sample_size = [int] $Review.canary.sample_size
            unexpected_canary_outcomes = [int] $Review.canary.unexpected_outcomes
            exception_process_ready = [bool] $Review.exception_process_ready
        }
        gates = [pscustomobject] $gateResults
        failed_gates = $failedGates
        promotion_criteria = @($failedGates | ForEach-Object { $criteria[$_] })
        authority_limit = 'This record supplies evidence for an accountable decision; it
does not change a policy assignment or authorize enforcement.'
        next_review_date = [string] $Review.next_review_date
    }
}

if ($PolicyStatesPath -or $ReviewPath -or $OutputPath) {
    if (-not ($PolicyStatesPath -and $ReviewPath -and $OutputPath)) {
        throw 'PolicyStatesPath, ReviewPath, and OutputPath must be supplied together.'
    }
    $statesDocument = Get-Content -LiteralPath $PolicyStatesPath -Raw |
        ConvertFrom-Json -Depth 50 -DateKind String
    $reviewDocument = Get-Content -LiteralPath $ReviewPath -Raw |
        ConvertFrom-Json -Depth 50 -DateKind String
    $decisionDocument = Get-PolicyEnforcementDecision `
        -PolicyStates $statesDocument `
        -Review $reviewDocument
    $json = $decisionDocument | ConvertTo-Json -Depth 30
    [IO.File]::WriteAllText(
        $OutputPath,
        $json + [Environment]::NewLine,
        [Text.UTF8Encoding]::new($false)
    )
}

```

The thresholds are decision inputs, not Azure defaults. This synthetic review requires fourteen observed days, full classification of noncompliant resources, no more than five percent false

positives, and no unexpected canary result. It also requires no unresolved review, no blocked correction, and a ready exception process. A different risk can justify other thresholds, but the accountable owner must record why.

The review input makes those expectations visible before a result is calculated. It also names the decision owner and next review date. The exception-process flag checks readiness only; Chapter 8.4 retains the design and code for Azure Policy exemptions. This section creates no exemption.

JSON

```
{
  "schema_version": "1.0",
  "decision_id": "policy-enforcement-review-2026-07-21",
  "decision_date": "2026-07-21",
  "assignment_id": "/providers/Microsoft.Management/managementGroups/mg-landing-zones/providers/Microsoft.Authorization/policyAssignments/platform-loc-baseline",
  "decision_owner": "cloud-governance-lead",
  "observation_window": {
    "start_utc": "2026-06-30T08:00:00Z",
    "end_utc": "2026-07-21T08:00:00Z"
  },
  "classification": {
    "false_positive_resource_ids": [
      "/subscriptions/sub-app-005/resourceGroups/rg-e/providers/Microsoft.Storage/storageAccounts/stappreview01"
    ],
    "confirmed_violation_resource_ids": [
      "/subscriptions/sub-app-006/resourceGroups/rg-f/providers/Microsoft.Storage/storageAccounts/stappfix01",
      "/subscriptions/sub-app-007/resourceGroups/rg-g/providers/Microsoft.KeyVault/vaults/kvappblocked01"
    ],
    "unresolved_resource_ids": [
      "/subscriptions/sub-app-008/resourceGroups/rg-h/providers/Microsoft.Network/virtualNetworks/vnet-app-review-01"
    ]
  },
  "remediation": {
    "corrected_resource_ids": [
      "/subscriptions/sub-app-006/resourceGroups/rg-f/providers/Microsoft.Storage/storageAccounts/stappfix01"
    ],
    "blocked_resource_ids": [
      "/subscriptions/sub-app-007/resourceGroups/rg-g/providers/Microsoft.KeyVault/vaults/kvappblocked01"
    ]
  },
  "canary": {
    "sample_size": 6,
    "unexpected_outcomes": 1
  },
  "exception_process_ready": false,
  "gates": {
    "minimum_observation_days": 14,
    "minimum_review_coverage_percent": 100,
    "maximum_false_positive_rate_percent": 5,
    "maximum_unexpected_canary_outcomes": 0,
    "require_zero_unresolved_reviews": true,
    "require_zero_blocked_remediations": true,
    "require_exception_process_ready": true
  },
  "next_review_date": "2026-08-04"
}
```

The synthetic state fixture contains twelve latest resource results: eight compliant and four noncompliant. All four noncompliant items were classified, so review coverage reached one hundred percent. Yet one was false, one remained unresolved, and one confirmed correction was blocked. The canary evidence also contained one unexpected outcome.

The calculated record therefore holds the assignment at audit. Passing duration and review coverage cannot cancel failures in scope accuracy or correction readiness. Each failed gate becomes a concrete promotion condition. The record informs an owner; it does not authorize or perform an assignment change.

```
JSON
{
  "schema_version": "1.0",
  "decision_id": "policy-enforcement-review-2026-07-21",
  "decision_date": "2026-07-21",
  "assignment_id": "/providers/Microsoft.Management/managementGroups/mg-landing-zones/providers/Microsoft.Authorization/policyAssignments/platform-loc-baseline",
  "policy_definition_reference_id": "auditAllowedLocations",
  "current_stage": "audit",
  "candidate_next_stage": "limited_deny_canary",
  "decision": "hold_audit",
  "decision_owner": "cloud-governance-lead",
  "evidence": {
    "observation_days": 21,
    "total_resources": 12,
    "compliant_resources": 8,
    "noncompliant_resources": 4,
    "reviewed_noncompliant_resources": 4,
    "review_coverage_percent": 100.0,
    "false_positive_resources": 1,
    "false_positive_rate_percent": 25.0,
    "confirmed_violation_resources": 2,
    "unresolved_resources": 1,
    "corrected_resources": 1,
    "blocked_remediation_resources": 1,
    "canary_sample_size": 6,
    "unexpected_canary_outcomes": 1,
    "exception_process_ready": false
  },
  "gates": {
    "observation_window": true,
    "review_coverage": true,
    "false_positive_rate": false,
    "canary_outcomes": false,
    "unresolved_reviews": false,
    "blocked_remediation": false,
    "exception_process": false
  },
  "failed_gates": [
    "false_positive_rate",
    "canary_outcomes",
    "unresolved_reviews",
    "blocked_remediation",
    "exception_process"
  ],
  "promotion_criteria": [
    "Reduce reviewed false positives to 5 percent or less.",
    "Reduce unexpected canary outcomes to 0 or fewer.",
    "Resolve every sampled compliance dispute.",
    "Clear every blocked remediation path in the reviewed population.",
    "Confirm that the governed exception process is ready before deny testing."
  ],
  "authority_limit": "This record supplies evidence for an accountable decision; it does not change a policy assignment or authorize enforcement.",
}
```

```
} "next_review_date": "2026-08-04"
```

Remediation has two meanings that should not be mixed. A workload owner may correct a resource through its normal delivery path. Azure Policy can run a remediation task for a definition that uses `modify` or `deployIfNotExists`. Both paths change resources and need tests, permissions, limits, and recovery plans.

Managed effects add another control path. Assignments using `modify` or `deployIfNotExists` need a managed identity with rights to the target operation. A remediation task applies the defined change to existing noncompliant resources. A wrong target set, role, or template can spread harm at scale.

A `deny` effect works differently. It can reject a matching create or update request, but changing the effect does not repair existing resources. Existing debt needs a separate plan before, during, or after rollout. A low noncompliance rate is not that plan.

These stages are not a maturity score. One precise, high-impact rule may move quickly, while another stays in audit because its scope remains unclear. Returning from a canary to audit is feedback, not failure. The correct stage is the safest response supported by current evidence.

Safe progression narrows the first enforced population. A limited deny canary can use a child scope or another controlled assignment boundary after the rule passes audit. Expected allow and deny cases should be submitted through the normal delivery route. Any unexpected result stops expansion and returns the rule to review.

The canary must test more than successful denial. It should test valid requests, automation, deployment identities, error messages, owner routing, and the correction route. A rule that blocks the wrong request is unsafe. A rule that blocks correctly but leaves teams unable to recover is incomplete.

Enforcement creates new feedback. Denied requests, pipeline failures, support demand, exception demand, lead time, and repeated workarounds show how the control affects the wider system. Some increase is expected when a real boundary becomes firm. A sharp or lasting shift can reveal poor scope, weak guidance, or missing platform capability.

Compliance can improve for the wrong reason after enforcement. Teams may stop deploying, choose an unmonitored path, or seek broad exclusions. The dashboard then looks better while risk moves elsewhere. Governance owners need workload feedback and delivery evidence beside the policy score.

A rollback signal should be decided before expansion. Examples include an unexpected denial, a failed critical deployment, an unowned remediation queue, or a surge in bypass requests.

Rollback does not erase the policy decision. It returns the control to a safer stage while evidence and design are repaired.

The staged record does not turn governance into a formula. Gates make missing evidence hard to ignore, but thresholds cannot grant business authority. The decision owner still weighs harm from the risk against harm from the guardrail. The record keeps that judgment visible and reviewable.

The tool should not promote its own assignment. An automated effect change would let the measurement system alter the boundary it judges. The output stops at eligibility for a limited canary. Policy delivery, approval, version review, and governed exceptions remain separate work.

Monitoring becomes useful enforcement only when it survives contact with real resources and teams. Which false positive, blocked correction, or unexpected request would still be acceptable after denial begins? What evidence would make that answer safe enough to act on? Until then, audit is doing the work the system needs.

Chapter 8: Govern — Turn Risk into Useful Guardrails

Chapter 8.4: Manage Policy as Code and Govern Exceptions

A portal edit can change policy behavior and still weaken governance. The intent, reviewer, test evidence, and exact release may vanish behind the current Azure object. A later operator sees what exists but cannot tell why it was approved. Policy as code makes that chain part of the control.

The delivery unit includes more than a policy rule. It includes definitions, initiatives, assignments, parameters, and the approved exemptions that depend on them. Each change moves through source control, review, validation, and a central deployment path. The repository holds reviewed intent, while Azure exposes deployed and evaluated state.

A small change set is easier to judge. It should state the control, scope, release, and reason in plain terms. Tests should cover both the wanted result and unsafe inputs. A plan should show only the objects that reviewers expect.

This separation prevents a common mistake. A merged file proves that reviewers accepted text, not that Azure evaluated the rule correctly. Nonproduction evaluation and promotion evidence still govern the change. Policy as code adds custody of the objects that carry that decision.

The toolchain is pinned before the governance objects are described. Version constraints help local checks and delivery runs interpret the same configuration. The lock file records the provider build chosen during initialization. An upgrade becomes a reviewed delivery change instead of an unseen workstation difference.

```
HCL / TERRAFORM
terraform {
  required_version = "~> 1.15.0"

  required_providers {
    azurerm = {
      source = "hashicorp/azurerm"
      version = "~> 4.81.0"
    }
  }
}

provider "azurerm" {
  features {}
}
```

A release needs an identity that survives beyond one plan. The example requires a semantic label, source revision, change record, and reviewing authority. The label versions this repository bundle; it is not an Azure policy-definition version. These fields aid investigation but do not replace protected repository and pipeline evidence.

The initiative definition is intentionally an input. Its full identifier must point to a management-group policy set, but its business rule is not recreated here. This keeps the delivery pattern

independent of any one control. Detailed tag, location, quota, and cost rules remain a later concern.

The exemption is a typed input instead of an unstructured note. It names one Azure resource and one member reference inside the assigned initiative. It carries an owner, separate approver, three dates, reason, evidence location, and compensating control. Missing context fails before Terraform can propose an Azure write.

HCL / TERRAFORM

```
variable "management_group_id" {
  description = "Full resource ID of the management group that receives the initiative assignment."
  type        = string

  validation {
    condition = can(regex(
      "^/providers/Microsoft\\.Management/managementGroups/[A-Za-z0-9._()-]+$",
      var.management_group_id
    ))
    error_message = "management_group_id must be a full management-group resource ID."
  }
}

variable "assignment_name" {
  description = "Stable Azure Policy assignment name."
  type        = string

  validation {
    condition = (
      length(var.assignment_name) >= 1 &&
      length(var.assignment_name) <= 24 &&
      can(regex("[A-Za-z0-9._()-]+$", var.assignment_name))
    )
    error_message = "assignment_name must use 1-24 letters, numbers, periods, underscores, or hyphens."
  }
}

variable "policy_definition_id" {
  description = "Full resource ID of the reviewed initiative definition."
  type        = string

  validation {
    condition = can(regex(
      "^/providers/Microsoft\\.Management/managementGroups/[A-Za-z0-9._()-]+/providers/Microsoft\\.Authorization/policySetDefinitions/[A-Za-z0-9._()-]+$",
      var.policy_definition_id
    ))
    error_message = "policy_definition_id must identify a management-group initiative definition."
  }
}

variable "validation_time_utc" {
  description = "Trusted UTC time supplied by the delivery run for deterministic expiry checks."
  type        = string

  validation {
    condition = (
      can(regex("^\\d{4}-\\d{2}-\\d{2}T\\d{2}:\\d{2}:\\d{2}Z$", var.validation_time_utc))
      &&
      can(timecmp(var.validation_time_utc, var.validation_time_utc))
    )
  }
}
```

```

    )
    error_message = "validation_time_utc must be a valid UTC timestamp in yyyy-MM-
ddTHH:mm:ssZ form."
  }
}

variable "release" {
  description = "Reviewed policy release evidence attached to the assignment."
  type = object({
    id           = string
    source_revision = string
    change_record  = string
    reviewed_by    = string
  })

  validation {
    condition = (
      can(regex("^governance-baseline-v[0-9]+\\. [0-9]+\\. [0-9]+$", var.release.id)) &&
      can(regex("[0-9a-f]{7,40}$", var.release.source_revision)) &&
      can(regex("^CHG-[0-9]{4}-[0-9]{4}$", var.release.change_record)) &&
      length(trimspaces(var.release.reviewed_by)) > 0
    )
    error_message = "release must identify a semantic release, source revision, change
record, and reviewer."
  }
}

variable "exemption" {
  description = "One approved, bounded exception to one member of the assigned
initiative."
  type = object({
    name                     = string
    resource_id              = string
    policy_definition_reference_id = string
    category                 = string
    owner                    = string
    approved_by              = string
    approved_at_utc          = string
    review_due_utc           = string
    expires_on_utc           = string
    evidence_uri             = string
    reason                   = string
    compensating_control      = string
  })

  validation {
    condition = (
      length(var.exemption.name) >= 1 &&
      length(var.exemption.name) <= 64 &&
      can(regex("[A-Za-z0-9._-]+$", var.exemption.name)) &&
      can(regex("^/subscriptions/[^/]+/resourceGroups/[^/]+/providers/[^/]+/.+$",
var.exemption.resource_id)) &&
      can(regex("[A-Za-z0-9._-]+$", var.exemption.policy_definition_reference_id)) &&
      contains(["Mitigated", "Waiver"], var.exemption.category) &&
      length(trimspaces(var.exemption.owner)) > 0 &&
      length(trimspaces(var.exemption.approved_by)) > 0 &&
      can(regex("^https://", var.exemption.evidence_uri)) &&
      length(trimspaces(var.exemption.reason)) >= 20 &&
      length(var.exemption.reason) <= 512 &&
      length(trimspaces(var.exemption.compensating_control)) >= 20
    )
    error_message = "exemption identity, scope, category, evidence, reason, or
compensating control is invalid."
  }
}

validation {
  condition = alltrue([
    for value in [
      var.exemption.approved_at_utc,

```

```

        var.exemption.review_due_utc,
        var.exemption.expires_on_utc
    ] : (
        can(regex("^\\d{4}-\\d{2}-\\d{2}T\\d{2}:\\d{2}:\\d{2}Z$", value)) &&
        can(timecmp(value, value))
    )
  })
  error_message = "Exemption dates must be valid UTC timestamps in yyyy-MM-ddTHH:mm:ssZ
form."
}
}
}

```

An Azure Policy exclusion and exemption carry different meanings. An excluded scope leaves the assignment's applicable population. An exemption remains tied to an otherwise applicable assignment and receives the exempt compliance state. That visible record suits a reviewed exception better than a quiet scope escape.

Azure provides two exemption categories. **Mitigated** means another method meets the policy intent. **Waiver** means the noncompliance is temporarily accepted. The label explains the decision basis; it does not grade the risk or grant approval by itself.

Scope is just as important as category. Exempting a subscription can silence evaluation for a large resource hierarchy. This object covers one resource and lists one initiative member reference. It also leaves Azure's default assignment-scope validation in place.

Member references are stable contracts between an initiative and its exemptions. A renamed or removed reference can break that relationship even when the underlying definition still exists. Initiative review must therefore search dependent exemption files before changing a reference. The dependency belongs in the same change review as the policy set.

The assignment records its release, revision, change record, and reviewer as JSON metadata. The exemption repeats the release facts beside its approval evidence. The two Azure objects answer different questions. One identifies the governing release; the other explains a bounded departure from one member.

The assignment stays nonblocking with **enforce = false**. Promotion to a stronger stage still requires its approved evidence path. The code also omits **not_scopes**, so a broad hidden exclusion cannot replace the reviewable exemption. A later enforcement change should arrive as its own reviewed release.

```

HCL / TERRAFORM
locals {
  assignment_metadata = {
    schema_version = "1.0"
    policy_release = var.release.id
    source_revision = var.release.source_revision
    change_record  = var.release.change_record
    reviewed_by    = var.release.reviewed_by
  }

  exemption_metadata = {
    schema_version = "1.0"
    exception_id   = var.exemption.name
  }
}

```



```

    owner                = var.exemption.owner
    approved_by          = var.exemption.approved_by
    approved_at_utc      = var.exemption.approved_at_utc
    review_due_utc       = var.exemption.review_due_utc
    expires_on_utc       = var.exemption.expires_on_utc
    evidence_uri         = var.exemption.evidence_uri
    reason               = var.exemption.reason
    compensating_control = var.exemption.compensating_control
    policy_release       = var.release.id
    source_revision      = var.release.source_revision
  }
}

resource "azurerm_management_group_policy_assignment" "governance" {
  name                = var.assignment_name
  display_name        = "Reviewed governance baseline"
  description          = "Versioned initiative assignment delivered from the governance repository."
  management_group_id = var.management_group_id
  policy_definition_id = var.policy_definition_id
  enforce             = false
  metadata            = jsonencode(local.assignment_metadata)
}

resource "azurerm_resource_policy_exemption" "temporary" {
  name                = var.exemption.name
  display_name        = "Approved temporary policy exception"
  description          = var.exemption.reason
  resource_id         = var.exemption.resource_id
  policy_assignment_id =
    azurerm_management_group_policy_assignment.governance.id
  policy_definition_reference_ids = [var.exemption.policy_definition_reference_id]
  exemption_category   = var.exemption.category
  expires_on           = var.exemption.expires_on_utc
  metadata            = jsonencode(local.exemption_metadata)

  lifecycle {
    precondition {
      condition = lower(var.exemption.owner) != lower(var.exemption.approved_by)
      error_message = "The exception owner and approver must be different identities."
    }

    precondition {
      condition = timecmp(
        var.validation_time_utc,
        var.exemption.approved_at_utc
      ) >= 0
      error_message = "The delivery validation time cannot precede approval."
    }

    precondition {
      condition = timecmp(
        var.exemption.expires_on_utc,
        var.validation_time_utc
      ) > 0
      error_message = "The exemption must still be active when the delivery run validates it."
    }

    precondition {
      condition = timecmp(
        var.exemption.review_due_utc,
        var.validation_time_utc
      ) >= 0
      error_message = "The exemption review date cannot already be overdue."
    }

    precondition {
      condition = (

```

```

        timecmp(
            var.exemption.review_due_utc,
            var.exemption.expires_on_utc
        ) <= 0 &&
        timecmp(
            var.exemption.review_due_utc,
            var.exemption.approved_at_utc
        ) > 0
    )
    error_message = "The review date must follow approval and cannot follow expiration."
}

precondition {
    condition = timecmp(
        var.exemption.expires_on_utc,
        timeadd(var.exemption.approved_at_utc, "2160h")
    ) <= 0
    error_message = "The exemption authorization cannot exceed 90 days."
}
}
}

```

Expiration deserves a plan-time gate because Azure gives it precise behavior. When the time arrives, Azure preserves the exemption object for history but stops honoring it. The object is not deleted by expiry. The repository must decide when its retention need has ended.

Expiry is not the same as cleanup. The old object can help explain a past exempt state. It can also clutter a live view if nobody owns retention. A scheduled review should sort records that must stay from records that may go.

The first time check compares approval with the delivery run's validation time. The next requires both review and expiry to remain ahead. Another places review after approval and no later than expiry. The last caps authorization at 2,160 hours, or 90 days.

The injected validation time is a trust boundary. A protected pipeline should create it once and prevent a pull request from choosing a convenient past value. Deterministic local tests can supply a fixed time without contacting Azure. Passing comparisons prove date relationships, not the clock source's honesty.

Owner and approver must also differ. This separation stops a workload team from authorizing its own exception in the example contract. It does not prove that either identity has the proper organizational role. Branch protection, reviewer groups, and the deployment identity must enforce that authority outside Terraform.

The evidence URI is a pointer, not proof. Reviewers still need the decision, affected risk, control results, and recovery plan stored there. An inaccessible or mutable page weakens the record even when its URI passes validation. Retention should span the policy and exception audit period.

The sample values show one reviewed release and one narrow exception. The resource type and initiative member are synthetic, so the example teaches the contract without prescribing a

business rule. Its **Mitigated** category matches the recorded compensating control. A waiver would use the other category and still require the same owner, dates, and evidence.

```
HCL / TERRAFORM
management_group_id = "/providers/Microsoft.Management/managementGroups/mg-landing-zones"
assignment_name      = "gov-baseline-audit"
policy_definition_id = "/providers/Microsoft.Management/managementGroups/mg-
platform/providers/Microsoft.Authorization/policySetDefinitions/platform-governance-
baseline"
validation_time_utc  = "2026-07-22T00:00:00Z"

release = {
  id              = "governance-baseline-v1.4.0"
  source_revision = "9d4c1e0"
  change_record   = "CHG-2026-0719"
  reviewed_by     = "governance-review-board"
}

exemption = {
  name              = "EXC-2026-0042"
  resource_id       = "/subscriptions/sub-app-007/resourceGroups/rg-app-
governance/providers/Contoso.Platform/widgets/widget-07"
  policy_definition_reference_id = "auditGenericControl"
  category          = "Mitigated"
  owner             = "workload-team-seven"
  approved_by       = "risk-approver"
  approved_at_utc   = "2026-07-21T00:00:00Z"
  review_due_utc    = "2026-09-15T00:00:00Z"
  expires_on_utc    = "2026-10-19T00:00:00Z"
  evidence_uri      = "https://governance.example.invalid/exceptions/EXC-
2026-0042"
  reason            = "A migration dependency prevents the standard
implementation during this release."
  compensating_control = "The workload team verifies the protected condition
each day and retains review evidence."
}
```

Pull-request review should read the assignment and exemption as one change. Reviewers compare scope, member reference, category, dates, evidence, and the plan against the approved record. Automated tests reject an expired authorization, an overdue review, a duration beyond 90 days, self-approval, and an unversioned release label. A positive test also proves that a bounded waiver retains its declared category.

The failed cases matter as much as the valid case. They show which facts stop a plan before any remote action. Their messages should tell an author what must change. A vague test failure can turn a useful guard into a bypass target.

Good gates make safe work feel routine. They stop bad input with a clear cause. They leave no need to guess what passed. That plain feedback helps teams fix a change.

Logs should name the release and check. A review can then trace each failed fact. Teams can fix the record or drop the request. No one needs to edit live state by hand.

The plan is necessary but incomplete. Mocked provider tests check resource shape, metadata, and local preconditions without credentials. They cannot confirm that the member exists, the resource lies under the assignment, or the delivery identity has permission. Those facts need an authorized integration check before apply.

Azure places a further permission barrier around exemptions. Creation requires policy-exemption write access on the resource and exemption action access on the target assignment. The central delivery identity should receive those rights only at the needed scopes. Portal access must not become an alternate path around review.

Renewal must not extend a date until the exception becomes permanent. A fresh request needs current evidence, new approval, a new expiry, and preferably a new exception identifier. Repository history can then expose repeated dependence on the same waiver. Terraform cannot infer that pattern from one desired-state snapshot.

An exception inventory should ask more than whether each object is active. It should group repeated owners, resources, member references, reasons, and compensating controls across releases. A growing cluster may reveal an impractical rule, a missing platform capability, or risk that nobody plans to remove. The pattern should feed policy and platform backlogs rather than silent renewals.

Deletion closes the technical object, not the underlying decision. If the resource is now compliant, evidence should show how that result was verified. If the policy changed, the release record should explain why the exception is no longer needed. If risk remains, expiry should restore evaluation rather than erase the uncomfortable signal.

Policy as code cannot make judgment objective. It can make change, authority, duration, and evidence harder to hide. The lasting test is whether an exception still looks temporary when its whole history is read. A permanent waiver is often governance debt wearing a series of valid dates.

Chapter 8: Govern — Turn Risk into Useful Guardrails

Chapter 8.5: Govern Cost and Resource Consistency

Governance fails when consistency becomes its goal. A platform can enforce one region, size, and tag pattern yet leave the system slower or less resilient. Cost control also fails when a lower bill hides a weaker service. Useful guardrails connect each constraint to the purpose of the system.

That connection needs several signals, not one governance score. Policy compliance shows whether resources match declared standards, while cost data shows where money was consumed. A compliant workload can still overspend, and a cheap workload can still break an important rule. Separate signals prevent one favorable number from hiding the other.

Tags form a small contract for resource identity. This baseline requires `CostOwner`, `Environment`, and `ServiceName` so a finding has an accountable group, operating context, and service boundary. Each value should come from a controlled vocabulary or owned service catalog where practical. Free-form tags otherwise create many spellings for the same fact.

Tags are useful metadata, but they cannot safely hold secrets or personal data and are not supported by every resource type. Some services also fail to emit every resource tag into cost records. New tags can take time to appear after cost data refreshes. A blank cost-owner group is therefore an attribution warning, not proof of zero cost.

The toolchain is pinned before any control is defined. Terraform and the AzureRM provider then interpret the policy rule and budget schema consistently across local checks and delivery runs. After initialization, the dependency lock file identifies the chosen provider build. Provider upgrades remain explicit review events.

```
HCL / TERRAFORM
terraform {
  required_version = "~> 1.15.0"

  required_providers {
    azurerm = {
      source = "hashicorp/azurerm"
      version = "~> 4.81.0"
    }
  }
}

provider "azurerm" {
  features {}
}
```

The inputs place both objects at one management group. That common boundary helps reviewers relate policy evaluation to cost, but their time periods still differ. The budget contract requires a positive amount and a bounded contact list. Duplicate or malformed email addresses stop the plan.

Budget dates must be UTC timestamps on the first day of a month. The resource precondition later proves that the end follows the start. These checks protect the shape of the request, not the authority of the contacts or the suitability of the amount. Accountable owners must still approve the financial target.

HCL / TERRAFORM

```
variable "management_group_id" {
  description = "Full resource ID of the management group that owns the policy definition and budget."
  type        = string

  validation {
    condition = can(regex(
      "^/providers/Microsoft\\.Management/managementGroups/[A-Za-z0-9._()-]+$",
      var.management_group_id
    ))
    error_message = "management_group_id must be a full management-group resource ID."
  }
}

variable "budget" {
  description = "Monthly management-group budget and notification contract."
  type = object({
    name          = string
    amount        = number
    start_date_utc = string
    end_date_utc  = string
    contact_emails = list(string)
  })

  validation {
    condition = (
      length(var.budget.name) >= 1 &&
      length(var.budget.name) <= 63 &&
      can(regex("^[A-Za-z0-9._()-]+$", var.budget.name)) &&
      var.budget.amount > 0
    )
    error_message = "The budget needs a valid name and a positive amount."
  }

  validation {
    condition = alltrue([
      for value in [var.budget.start_date_utc, var.budget.end_date_utc] : (
        can(regex("^[0-9]{4}-[0-9]{2}-01T00:00:00Z$", value)) &&
        can(timecmp(value, value))
      )
    ])
    error_message = "Budget dates must be valid UTC timestamps on the first day of a month."
  }

  validation {
    condition = (
      length(var.budget.contact_emails) >= 1 &&
      length(var.budget.contact_emails) <= 5 &&
      length(distinct([for email in var.budget.contact_emails : lower(email)])) ==
      length(var.budget.contact_emails) &&
      alltrue([
        for email in var.budget.contact_emails : can(regex(
          "^[^@\\s]+@[^@\\s]+\\.([^@\\s]+)$",
          email
        ))
      ])
    )
    error_message = "Provide one to five unique, valid budget contact email addresses."
  }
}
```

```
}  
}
```

The tag rule tests both absence and an empty value for all three names. In **Indexed** mode, Azure evaluates only resource types with tag and location support. This reduces misleading results from unsupported objects. The **audit** effect observes drift without rejecting a deployment, while metadata records the exact standard.

A policy definition alone has no effect on resources. It must enter the reviewed initiative and assignment path established earlier in this chapter. That path owns scope, exclusions, exemptions, evaluation, and any later move toward enforcement. This section defines the business control rather than bypassing its delivery process.

The budget has two notification points. A forecast alert at 80 percent gives owners time to investigate before the period closes, while an actual alert at 100 percent reports that the amount has been reached. An Azure budget sends signals but does not stop consumption. Workloads still need a response plan that protects service outcomes.

```
HCL / TERRAFORM  
locals {  
  required_tag_names = [  
    "CostOwner",  
    "Environment",  
    "ServiceName"  
  ]  
  
  required_tag_rule = {  
    "if" = {  
      anyOf = flatten([  
        for tag_name in local.required_tag_names : [  
          {  
            field = "tags['${tag_name}']"  
            exists = "false"  
          },  
          {  
            field = "tags['${tag_name}']"  
            equals = ""  
          }  
        ]  
      ]) )  
    }  
    then = {  
      effect = "audit"  
    }  
  }  
}  
  
resource "azurerms_policy_definition" "required_governance_tags" {  
  name                = "audit-required-governance-tags"  
  display_name        = "Audit missing governance tags"  
  description         = "Audits taggable resources missing CostOwner, Environment, or  
ServiceName."  
  policy_type         = "Custom"  
  mode                = "Indexed"  
  management_group_id = var.management_group_id  
  policy_rule         = jsonencode(local.required_tag_rule)  
  metadata = jsonencode({  
    category          = "Resource consistency"  
    standard           = "governance-tags-v1"  
    required_tag_names = local.required_tag_names  
  })  
}
```

```

}

resource "azurerm_consumption_budget_management_group" "governance" {
  name                 = var.budget.name
  management_group_id = var.management_group_id
  amount               = var.budget.amount
  time_grain           = "Monthly"

  time_period {
    start_date = var.budget.start_date_utc
    end_date   = var.budget.end_date_utc
  }

  notification {
    enabled      = true
    operator      = "GreaterThanOrEqualTo"
    threshold     = 80
    threshold_type = "Forecasted"
    contact_emails = var.budget.contact_emails
  }

  notification {
    enabled      = true
    operator      = "GreaterThanOrEqualTo"
    threshold     = 100
    threshold_type = "Actual"
    contact_emails = var.budget.contact_emails
  }

  lifecycle {
    precondition {
      condition = timecmp(
        var.budget.end_date_utc,
        var.budget.start_date_utc
      ) > 0
      error_message = "The budget end date must follow its start date."
    }
  }
}

```

Allowed regions need the same link to purpose. Residency, latency, service availability, recovery design, and support capability should shape the approved set. Apparent savings alone should not ban a region. A narrow list that breaks recovery or excludes a required service is not useful.

Resource standards should describe supported choices and their reasons. Approved service tiers, virtual-machine families, storage redundancy, and naming patterns can reduce surprise and operational spread. A standard that omits a valid workload need sends teams toward exceptions or shadow infrastructure. Exception trends and support demand should improve the paved road.

Quota sets a different boundary from cost. It limits capacity in a region, subscription, or resource family, while a budget measures or forecasts spend. Extra budget cannot create missing quota during a scale event. Teams should monitor use, request increases early, and test capacity assumptions before demand arrives.

The example values create a one-year monthly budget beginning in August 2026. Those dates are a publication snapshot and must move with a real rollout. The `.invalid` contacts and sample

management group are deliberately synthetic. They demonstrate the contract without addressing a live tenant.

```
HCL / TERRAFORM
management_group_id = "/providers/Microsoft.Management/managementGroups/mg-landing-zones"

budget = {
  name          = "platform-monthly-budget"
  amount        = 25000
  start_date_utc = "2026-08-01T00:00:00Z"
  end_date_utc   = "2027-08-01T00:00:00Z"
  contact_emails = [
    "platform-finops@example.invalid",
    "governance-owners@example.invalid"
  ]
}
```

The controls provide a baseline, but reporting turns it into operating evidence. This example collects one policy summary and cost for the last complete month at the same management group. It never changes Azure state. The output retains the raw policy response and named cost rows for later review.

The collector first checks the active tenant against an expected identifier. A mismatch stops the run before either governance query begins. This check is a read boundary, not proof of authorization. The signed-in identity still needs permission to read the selected policy and cost data.

The policy command names the management group, assignment, and initiative member reference. It intentionally asks for the current summary instead of downloading every policy-state row. That result can answer how the selected control is evaluating at collection time. Detailed investigation may then use a separately approved, bounded query.

Cost Management receives a query through the core `az rest` command. The request sums pretax cost, groups it by resource group and `CostOwner`, and selects `TheLastMonth`. A completed month is easier to compare than a changing current period. It still reflects the cost-data refresh cycle rather than instant consumption.

Cost query rows arrive as arrays paired with column definitions. The function maps each value by the server-provided name instead of assuming a fixed order. It rejects blank, duplicate, or changing columns and rows of the wrong width. Pagination follows each next link, rejects cycles, and stops after 100 pages.

Every Azure CLI call fails closed on a nonzero exit, empty required response, or invalid JSON. The cost endpoint can return an empty response, so that case is handled explicitly. The report records page count and column metadata even when no cost rows exist. Absence remains visible instead of becoming invented zero spend.

```
POWERSHELL
Set-StrictMode -Version Latest
```

```

$ErrorActionPreference = 'Stop'

function Invoke-AzureCliJson {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [string[]] $Arguments,

        [Parameter()]
        [switch] $AllowEmpty
    )

    $raw = & az @Arguments 2>&1
    if ($LASTEXITCODE -ne 0) {
        throw "Azure CLI failed: $($raw -join [Environment]::NewLine)"
    }

    $text = $raw -join [Environment]::NewLine
    if ([string]::IsNullOrEmpty($text)) {
        if ($AllowEmpty) {
            return $null
        }
        throw 'Azure CLI returned an empty response.'
    }

    try {
        return $text | ConvertFrom-Json -Depth 50 -DateKind String
    }
    catch {
        throw 'Azure CLI returned invalid JSON.'
    }
}

function Convert-CostRow {
    [CmdletBinding()]
    [OutputType([object[]])]
    param(
        [Parameter(Mandatory)]
        [object[]] $Columns,

        [Parameter(Mandatory)]
        [AllowEmptyCollection()]
        [object[]] $Rows
    )

    $names = @($Columns | ForEach-Object { [string] $_.name })
    if ($names.Count -eq 0 -or @($names | Where-Object {
        [string]::IsNullOrEmpty($_)
    }).Count -gt 0) {
        throw 'Cost query columns must have names.'
    }
    if (@($names | Sort-Object -Unique).Count -ne $names.Count) {
        throw 'Cost query column names must be unique.'
    }

    return @(
        foreach ($row in $Rows) {
            $values = @($row)
            if ($values.Count -ne $names.Count) {
                throw 'A cost row does not match the returned column count.'
            }

            $record = [ordered]@{}
            for ($index = 0; $index -lt $names.Count; $index++) {
                $record[$names[$index]] = $values[$index]
            }
            [pscustomobject] $record
        }
    )
}

```

```

    }
  )
}

function Get-AzureGovernanceSignal {
  [CmdletBinding()]
  param(
    [Parameter(Mandatory)]
    [guid] $ExpectedTenantId,

    [Parameter(Mandatory)]
    [ValidatePattern('^[A-Za-z0-9._()-]+$')]
    [string] $ManagementGroupId,

    [Parameter(Mandatory)]
    [ValidatePattern('^[A-Za-z0-9._-]{1,24}$')]
    [string] $PolicyAssignmentName,

    [Parameter(Mandatory)]
    [ValidatePattern('^[A-Za-z0-9._-]+$')]
    [string] $PolicyDefinitionReferenceId,

    [Parameter(Mandatory)]
    [ValidatePattern('^[A-Za-z0-9._-]+$')]
    [string] $CostOwnerTagName,

    [Parameter(Mandatory)]
    [ValidatePattern('^\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}Z$')]
    [string] $CollectedUtc
  )

  $account = Invoke-AzureCliJsn -Arguments @(
    'account', 'show',
    '--output', 'json',
    '--only-show-errors'
  )
  if ([guid] $account.tenantId -ne $ExpectedTenantId) {
    throw "Active tenant '$($account.tenantId)' does not match the expected tenant."
  }

  $policySummary = Invoke-AzureCliJsn -Arguments @(
    'policy', 'state', 'summarize',
    '--management-group', $ManagementGroupId,
    '--policy-assignment', $PolicyAssignmentName,
    '--filter', "policyDefinitionReferenceId eq '$PolicyDefinitionReferenceId'",
    '--output', 'json',
    '--only-show-errors'
  )

  $scope = "/providers/Microsoft.Management/managementGroups/$ManagementGroupId"
  $costUrl = (
    'https://management.azure.com' + $scope +
    '/providers/Microsoft.CostManagement/query?api-version=2025-03-01'
  )
  $costRequest = [ordered]@{
    type = 'Usage'
    timeframe = 'TheLastMonth'
    dataset = [ordered]@{
      aggregation = [ordered]@{
        totalCost = [ordered]@{
          name = 'PreTaxCost'
          function = 'Sum'
        }
      }
    }
    granularity = 'None'
    grouping = @(
      [ordered]@{
        name = 'ResourceGroup'

```

```

        type = 'Dimension'
    },
    [ordered]@{
        name = $CostOwnerTagName
        type = 'TagKey'
    }
}
)
}
}
$costBody = $costRequest | ConvertTo-Json -Depth 10 -Compress

$pages = 0
$nextUrl = $costUrl
$seenUrls = [Collections.Generic.HashSet[string]]::new(
    [StringComparer]::OrdinalIgnoreCase
)
$columns = @()
$rows = @()
while ($null -ne $nextUrl) {
    if (-not $seenUrls.Add($nextUrl)) {
        throw 'Cost query pagination returned a repeated next link.'
    }
    if ($pages -ge 100) {
        throw 'Cost query pagination exceeded 100 pages.'
    }

    $page = Invoke-AzureCliJson -AllowEmpty -Arguments @(
        'rest',
        '--method', 'post',
        '--url', $nextUrl,
        '--body', $costBody,
        '--output', 'json',
        '--only-show-errors'
    )
    $pages++
    if ($null -eq $page) {
        $nextUrl = $null
        continue
    }

    $pageColumns = @($page.properties.columns)
    if ($columns.Count -eq 0) {
        $columns = $pageColumns
    }
    elseif (($pageColumns | ConvertTo-Json -Compress) -cne
        ($columns | ConvertTo-Json -Compress)) {
        throw 'Cost query columns changed between pages.'
    }
    $rows += @(Convert-CostRow -Columns $pageColumns -Rows @(
        $page.properties.rows
    ))
    $nextUrl = if ([string]::IsNullOrEmpty(
        [string] $page.properties.nextLink
    )) {
        $null
    }
    else {
        [string] $page.properties.nextLink
    }
}

return [pscustomobject] [ordered]@{
    schema_version = '1.0'
    collected_utc = $CollectedUtc
    scope = [ordered]@{
        tenant_id = $ExpectedTenantId.ToString()
        management_group_id = $ManagementGroupId
        resource_id = $scope
    }
    policy = [ordered]@{

```

```

        assignment_name = $PolicyAssignmentName
        policy_definition_reference_id = $PolicyDefinitionReferenceId
        latest_summary = $true
        response = $policySummary
    }
    cost = [ordered]@{
        api_version = '2025-03-01'
        type = 'Usage'
        timeframe = 'TheLastMonth'
        cost_column = 'PreTaxCost'
        groupings = @('ResourceGroup', $CostOwnerTagName)
        pages = $pages
        columns = $columns
        rows = $rows
    }
    interpretation = [ordered]@{
        combined_status = $false
        note = 'Policy and cost signals require separate owners, thresholds, and
review.'
    }
}
}
}

```

The wrapper supplies the collection time and fixes the cost-owner tag name. It resolves the local output path, requires its parent directory to exist, and refuses to replace an existing file. UTF-8 output avoids a machine-specific encoding choice. These guards preserve earlier evidence when a scheduled run is repeated accidentally.

```

POWERSHELL
[CmdletBinding()]
param(
    [Parameter(Mandatory)]
    [guid] $ExpectedTenantId,

    [Parameter(Mandatory)]
    [string] $ManagementGroupId,

    [Parameter(Mandatory)]
    [string] $PolicyAssignmentName,

    [Parameter(Mandatory)]
    [string] $PolicyDefinitionReferenceId,

    [Parameter(Mandatory)]
    [string] $OutputPath
)

Set-StrictMode -Version Latest
$ErrorActionPreference = 'Stop'
. "$PSScriptRoot/Get-AzureGovernanceSignals.ps1"

$signals = Get-AzureGovernanceSignal `
    -ExpectedTenantId $ExpectedTenantId `
    -ManagementGroupId $ManagementGroupId `
    -PolicyAssignmentName $PolicyAssignmentName `
    -PolicyDefinitionReferenceId $PolicyDefinitionReferenceId `
    -CostOwnerTagName 'CostOwner' `
    -CollectedUtc ([DateTimeOffset]::UtcNow.ToString(
        'yyyy-MM-ddTHH:mm:ssZ',
        [Globalization.CultureInfo]::InvariantCulture
    ))

$resolvedOutput = $ExecutionContext.SessionState.Path.GetUnresolvedProviderPathFromPSPath(
    $OutputPath
)

```

```

$outputParent = Split-Path -Parent $resolvedOutput
if (-not (Test-Path -LiteralPath $outputParent -PathType Container)) {
    throw "Output directory '$outputParent' does not exist."
}
if (Test-Path -LiteralPath $resolvedOutput -PathType Leaf) {
    throw "Output file '$resolvedOutput' already exists."
}

$json = $signals | ConvertTo-Json -Depth 50
[IO.File]::WriteAllText(
    $resolvedOutput,
    $json + [Environment]::NewLine,
    [Text.UTF8Encoding]::new($false)
)

```

A scheduled collection should retain its command version, parameters, identity, and output with the review record. Policy and cost data have different freshness, so one timestamp does not make them simultaneous. Reviewers should compare like periods and note late cost adjustments. A missing run is itself a gap in governance evidence.

Policy findings should reach the team able to correct the resource or the standard. Repeated missing tags may expose a weak deployment module, unclear ownership, or a service that does not support the rule. An exemption should remain bounded and visible when a valid need conflicts with the baseline. A high count alone does not identify the right remedy.

Cost rows need similar context. A rise can come from customer demand, recovery tests, retained evidence, an inefficient design, or abandoned capacity. Owners should relate the change to service units such as requests, users, or completed business transactions. Cost per useful outcome often guides a better decision than cost alone.

Budget alerts need named responses as well. A forecast notice might trigger attribution review, anomaly investigation, or a planned architecture change. Reaching the actual threshold may require leadership review without forcing an unsafe shutdown. The response must reflect the workload's criticality and the cost of interruption.

The joint review is a conversation between signals, not a formula that blends them. Policy evidence reveals consistency gaps, while cost evidence reveals financial movement. Quota and service-health evidence add context when a change could affect resilience. Decisions remain traceable because each signal keeps its source, period, and owner.

Standards should improve as evidence accumulates. Frequent exceptions may show that an allowed-region list is too narrow or an approved tier is missing. Unattributed spend may expose a weak tag contract or platform module. Falling costs with rising incidents may show that optimization has crossed into damage.

Governance earns trust when it makes these tradeoffs visible early. The platform can keep resource choices consistent enough to operate without confusing sameness with safety. Budgets, quotas, policy, and cost records then become a learning system, not a wall of controls.

The next test is whether that governed platform protects identities, data, and workloads against active threats.

Chapter 9: Secure — Protect the Azure System

Chapter 9.1: Security Across Every CAF Methodology

A governed platform is not always a secure one. Policies may be active, budgets watched, and resources compliant while critical access paths remain exposed. Security must shape each choice from the start.

That work spans the whole cloud system and its operating life. Business goals show which assets matter, plans fund protection, and landing zones create shared boundaries. Workload releases add attack paths, governance sets allowed behavior, and daily work reveals drift and threats.

The Cloud Adoption Framework Secure methodology provides this broad view. Secure is one of seven CAF methods, yet Microsoft also applies its guidance through Strategy, Plan, Ready, Adopt, Govern, and Manage. The duty stays with each phase. This overlap is deliberate because a separate security lane cannot repair every earlier gap.

Zero Trust supplies three principles for those phase choices. Verify explicitly checks each request with relevant current signals, while least-privilege access grants only what is needed for a limited time. Assume breach limits harm and prepares teams for controls that fail.

Explicit verification goes beyond one sign-in step. Access may depend on identity, device, location, behavior, asset, and risk; its signals and enforcement points vary by system boundary. The principle stays stable as tools change.

Least privilege also spans the full lifecycle. Strategy funds the work, Plan names owners, Ready creates narrow scopes, and Adopt avoids lasting secrets. Manage removes access when its valid need ends because one role review cannot carry that whole chain.

Assume breach asks what happens when prevention fails. This leads to limited zones, useful logs, protected records, exercises, response, and recovery plans. The stance does not claim that every component is breached; it stops one failed assumption from granting broad trust.

Confidentiality, integrity, and availability provide a second lens. The first limits disclosure, the second preserves trustworthy state, and the third keeps data and services ready when needed. Failure in one can weaken the other two. A sound choice tests all three information-protection goals.

Encryption may protect secrets but cannot make bad backups useful, while a fast restore may bring back a corrupt image. A strict network rule may also block the only tested response path. Each control affects several goals, so the three-part lens exposes those links early.

Shared responsibility adds another boundary. Microsoft secures the physical foundation and managed layers promised by each Azure service. Customers still own data, identities, access, settings, and controlled cloud components, while the split changes across IaaS, PaaS, and SaaS. Teams must check that boundary before design approval.

Shared responsibility does not mean vague joint ownership. Each service needs a recorded split that names provider work, customer work, and any shared task. A managed service shifts some work but not ownership of data or access choices, and an unnamed duty becomes an exposed seam.

Internal roles matter as much as the provider split. Business, security, platform, workload, governance, and operations teams shape different parts, so each needs clear authority and evidence duties. No role can safely infer all the rest. Each also needs a feedback route because giving all protection work to one team only hides dependencies.

The example overlay turns these lenses into a local review artifact. It covers the platform landing zone, not every workload control. The root records the service-model review, three Zero Trust principles, and three information objectives before seven ordered entries cover the CAF cycle.

Each phase asks one question for every information objective and names the Zero Trust principles most relevant there. It also lists at least two responsibilities and one accountable owner. These fields expose missing thought without posing as the full threat model that later design work must supply.

Risk reduction means an observed change in exposure, not a count of installed products. Each record names a risk, desired change, measure, and counter-signal for stalled work, repeat exceptions, or missing coverage. Together they resist success claims based on one favorable number.

Evidence records name a source, producer, reviewer, decision use, and maximum age. Producer and reviewer must differ so one team cannot approve its own claim in silence. Age affects meaning, while feedback links show where the evidence can alter another CAF choice.

JSON

```
{
  "schema_version": "1.0",
  "scope": "platform-landing-zone",
  "responsibility_boundary": {
    "service_model_review_required": true,
    "provider": "Secure the Azure infrastructure and managed layers promised by each selected service.",
    "customer": "Own data, identities, access, configuration, and every cloud component under customer control.",
    "shared": "Record the service-specific division before architecture approval because the boundary changes across IaaS, PaaS, and SaaS."
  },
  "zero_trust_principles": [
```

```

    "verify-explicitly",
    "use-least-privilege",
    "assume-breach"
  ],
  "information_objectives": [
    "confidentiality",
    "integrity",
    "availability"
  ],
  "methodologies": [
    {
      "id": "strategy",
      "methodology": "Strategy",
      "owner": "cloud-strategy-lead",
      "security_questions": {
        "confidentiality": "Which business information would cause material harm if disclosed?",
        "integrity": "Which decisions or records require provable correctness and origin?",
        "availability": "Which outcomes fail when a service or security capability is unavailable?"
      },
      "zero_trust": [
        "verify-explicitly",
        "assume-breach"
      ],
      "responsibilities": [
        {
          "owner": "business-and-security-leaders",
          "action": "Approve priority security outcomes, risk tolerance, and decision authority."
        },
        {
          "owner": "cloud-strategy-team",
          "action": "Connect adoption outcomes to named threats, affected assets, and review triggers."
        }
      ],
      "risk_reduction": {
        "risk": "Security enters after business and architecture commitments are difficult to change.",
        "desired_change": "Priority business scenarios begin with approved security outcomes and owned risk decisions.",
        "measure": "Share of priority scenarios with an approved security outcome and accountable risk owner.",
        "counter_measure": "High-impact risks accepted without a named treatment date."
      },
      "evidence": [
        {
          "id": "strategy-security-outcomes",
          "source": "security-strategy-register",
          "producer": "cloud-strategy-team",
          "reviewer": "security-risk-lead",
          "decision_use": "Approve, revise, or stop an adoption objective.",
          "maximum_age_days": 90
        }
      ],
      "feedback_to": [
        "Strategy",
        "Plan"
      ]
    },
    {
      "id": "plan",
      "methodology": "Plan",
      "owner": "cloud-adoption-lead",
      "security_questions": {
        "confidentiality": "Does the plan include the skills and dependencies needed to classify and protect data?",
        "integrity": "Can the plan preserve trustworthy decisions, code, and evidence through delivery?",

```

```

        "availability": "Does planned capacity cover security work, response roles, and
recovery exercises?"
    },
    "zero_trust": [
        "use-least-privilege",
        "assume-breach"
    ],
    "responsibilities": [
        {
            "owner": "cloud-adoption-team",
            "action": "Sequence security dependencies, skills, capacity, and acceptance work
with adoption delivery."
        },
        {
            "owner": "security-architecture-team",
            "action": "Expose missing threat knowledge, control dependencies, and response
capabilities."
        }
    ],
    "risk_reduction": {
        "risk": "Adoption dates omit security dependencies and create rushed exceptions
near release.",
        "desired_change": "Security work has owners, capacity, dependencies, and exit
evidence in the adoption plan.",
        "measure": "Share of planned releases with funded security work and named
acceptance evidence.",
        "counter_measure": "Security work items aging beyond their agreed decision date."
    },
    "evidence": [
        {
            "id": "plan-security-capacity",
            "source": "cloud-adoption-backlog",
            "producer": "cloud-adoption-team",
            "reviewer": "security-architecture-lead",
            "decision_use": "Change scope, sequence, or capacity before a release
commitment.",
            "maximum_age_days": 30
        }
    ],
    "feedback_to": [
        "Strategy",
        "Ready"
    ]
},
{
    "id": "ready",
    "methodology": "Ready",
    "owner": "platform-engineering-lead",
    "security_questions": {
        "confidentiality": "Do landing-zone boundaries restrict access to sensitive
platform and workload paths?",
        "integrity": "Can trusted desired state and deployment evidence reveal
unauthorized platform change?",
        "availability": "Do shared identity, network, management, and recovery
dependencies avoid single fragile paths?"
    },
    "zero_trust": [
        "verify-explicitly",
        "use-least-privilege",
        "assume-breach"
    ],
    "responsibilities": [
        {
            "owner": "platform-engineering-team",
            "action": "Build secure defaults, explicit trust boundaries, telemetry paths,
and recovery foundations."
        },
        {
            "owner": "security-engineering-team",
            "action": "Test platform threat assumptions and the evidence produced by shared
controls."
        }
    ]
}

```

```

    }
  ],
  "risk_reduction": {
    "risk": "The landing zone creates broad trust, weak evidence, or shared failure paths inherited by workloads.",
    "desired_change": "The platform exposes narrow access, observable boundaries, and tested recovery assumptions.",
    "measure": "Share of approved platform security requirements with automated or exercised evidence.",
    "counter_measure": "Workload exceptions caused by unusable or unavailable platform security capabilities."
  },
  "evidence": [
    {
      "id": "ready-platform-security",
      "source": "landing-zone-validation-pack",
      "producer": "platform-engineering-team",
      "reviewer": "security-engineering-team",
      "decision_use": "Accept the platform boundary or return a failed requirement to engineering.",
      "maximum_age_days": 14
    }
  ],
  "feedback_to": [
    "Plan",
    "Adopt"
  ]
},
{
  "id": "adopt",
  "methodology": "Adopt",
  "owner": "workload-delivery-lead",
  "security_questions": {
    "confidentiality": "Does each workload identify sensitive data and restrict every access path to it?",
    "integrity": "Can the workload prove the origin and approved state of released code and configuration?",
    "availability": "Can the workload meet its recovery and continuity needs when dependencies fail?"
  },
  "zero_trust": [
    "verify-explicitly",
    "use-least-privilege",
    "assume-breach"
  ],
  "responsibilities": [
    {
      "owner": "workload-team",
      "action": "Apply the platform contract and own workload-specific threats, data, code, and recovery."
    },
    {
      "owner": "platform-product-team",
      "action": "Provide usable secure paths and route legitimate gaps to accountable decisions."
    }
  ],
  "risk_reduction": {
    "risk": "Migration or product pressure bypasses security decisions and creates hidden exposure.",
    "desired_change": "Workloads reach release with owned threats, accepted evidence, and bounded exceptions.",
    "measure": "Share of priority workload releases with current security acceptance evidence.",
    "counter_measure": "Days lost to unclear control ownership or unavailable secure platform paths."
  },
  "evidence": [
    {
      "id": "adopt-workload-acceptance",
      "source": "workload-readiness-record",

```

```

        "producer": "workload-team",
        "reviewer": "security-assurance-lead",
        "decision_use": "Release, remediate, or record a time-bounded exception.",
        "maximum_age_days": 30
    }
],
"feedback_to": [
    "Ready",
    "Govern"
]
},
{
    "id": "govern",
    "methodology": "Govern",
    "owner": "cloud-governance-lead",
    "security_questions": {
        "confidentiality": "Do policies express approved boundaries for sensitive data and access?",
        "integrity": "Are policy intent, assignments, exceptions, and evidence protected from hidden change?",
        "availability": "Do guardrails preserve required service and recovery paths while limiting risk?"
    },
    "zero_trust": [
        "verify-explicitly",
        "use-least-privilege"
    ],
    "responsibilities": [
        {
            "owner": "cloud-governance-team",
            "action": "Translate risk boundaries into reviewable policy, measures, and exception decisions."
        },
        {
            "owner": "security-risk-team",
            "action": "Confirm that governance decisions reduce named exposure without hiding operational harm."
        }
    ],
    "risk_reduction": {
        "risk": "Security rules become detached from risk, widely bypassed, or harmful to service outcomes.",
        "desired_change": "Guardrails remain risk-linked, measurable, usable, and adjustable through evidence.",
        "measure": "Share of eligible security findings resolved or explicitly accepted within the target period.",
        "counter_measure": "Repeated exceptions and failed delivery caused by the same guardrail."
    },
    "evidence": [
        {
            "id": "govern-security-policy",
            "source": "policy-and-exception-review",
            "producer": "cloud-governance-team",
            "reviewer": "security-risk-team",
            "decision_use": "Retain, revise, promote, or retire a security guardrail.",
            "maximum_age_days": 30
        }
    ],
    "feedback_to": [
        "Strategy",
        "Secure"
    ]
},
{
    "id": "secure",
    "methodology": "Secure",
    "owner": "security-program-lead",
    "security_questions": {
        "confidentiality": "Are high-value identities, data, and access paths protected and observed?",

```

```

        "integrity": "Can security teams detect tampering and trust the evidence used for
response?",
        "availability": "Can security capabilities and response teams operate through
attack and recovery?"
    },
    "zero_trust": [
        "verify-explicitly",
        "use-least-privilege",
        "assume-breach"
    ],
    "responsibilities": [
        {
            "owner": "security-posture-team",
            "action": "Prioritize exposure reduction and verify that controls remain
effective."
        },
        {
            "owner": "security-operations-team",
            "action": "Prepare, detect, investigate, contain, recover, and preserve incident
learning."
        }
    ],
    "risk_reduction": {
        "risk": "Static controls lose value while threats, assets, identities, and
attacker methods change.",
        "desired_change": "Protection, detection, response, and resilience improve against
current exposure.",
        "measure": "High-exposure findings reduced within the approved risk response
period.",
        "counter_measure": "Unmonitored assets, failed response exercises, and recurring
high-impact findings."
    },
    "evidence": [
        {
            "id": "secure-exposure-review",
            "source": "security-posture-and-response-review",
            "producer": "security-posture-team",
            "reviewer": "security-program-lead",
            "decision_use": "Prioritize protection, detection, response, or resilience
work.",
            "maximum_age_days": 14
        }
    ],
    "feedback_to": [
        "Ready",
        "Manage"
    ]
},
{
    "id": "manage",
    "methodology": "Manage",
    "owner": "platform-operations-lead",
    "security_questions": {
        "confidentiality": "Do operations detect and contain unauthorized access or data
movement?",
        "integrity": "Can operators distinguish trusted change from tampering or drift?",
        "availability": "Can teams restore safe service and validate recovery under attack
conditions?"
    },
    "zero_trust": [
        "verify-explicitly",
        "assume-breach"
    ],
    "responsibilities": [
        {
            "owner": "platform-operations-team",
            "action": "Observe drift, protect evidence, recover shared services, and test
operating procedures."
        },
        {
            "owner": "security-operations-team",

```

```

        "action": "Turn incidents, exercises, and threat intelligence into prioritized
changes."
    },
    ],
    "risk_reduction": {
        "risk": "Operations restore service without correcting the conditions that enabled
an incident.",
        "desired_change": "Operational and incident evidence changes architecture,
controls, plans, and strategy.",
        "measure": "Share of material incidents and exercises with verified corrective
actions by due date.",
        "counter_measure": "Repeat incidents and reopened actions linked to the same
control weakness."
    },
    "evidence": [
        {
            "id": "manage-security-learning",
            "source": "incident-and-exercise-backlog",
            "producer": "platform-operations-team",
            "reviewer": "security-operations-lead",
            "decision_use": "Revise operating procedures, architecture, backlog, or security
outcomes.",
            "maximum_age_days": 30
        }
    ],
    "feedback_to": [
        "Strategy",
        "Plan"
    ]
}
]
}

```

The lifecycle mapping starts in Strategy with business harm rather than tools. It asks which data, records, and outcomes matter, then requires leaders to approve security outcomes and risk owners. Its measure checks whether priority cases have both, while the counter-signal finds high-impact acceptance with no treatment date.

Plan turns Strategy intent into capacity and sequence. Skills, dependencies, evidence, response roles, and recovery exercises need room and must compete honestly with other work. The plan must expose that tradeoff. Aging protection tasks challenge a plan that hides open choices, and the evidence can send work back to Strategy or on to Ready.

Ready turns that planned work into the platform foundation. It asks whether landing-zone boundaries restrict access, desired state reveals unknown change, and shared dependencies avoid fragile paths. Platform engineers produce the evidence, security engineers review it, and workload exceptions from an unusable safe path remain a counter-signal.

Adopt brings workload teams onto that foundation. The platform contract cannot own workload data, code, threats, or recovery needs, so release evidence must show current security acceptance or a bounded exception. Time lost to unclear ownership reveals when the safe path is not usable.

Govern turns repeated risk into reviewable rules. Policy intent, assignments, exceptions, and measures must resist hidden change while staying open to review. Resolution counts are

balanced by repeat exceptions and failed work because a rule that breeds the same bypass needs investigation, not a better headline.

Secure concentrates posture and response work without claiming every security choice. Posture teams rank exposure, while security operations prepares to detect, investigate, contain, and recover. No score can replace that test. Current threats and exercises test whether defenses still work, and their evidence can change platform design and daily priorities.

Manage closes the operating loop. Drift, incidents, recovery exercises, and threat reports expose assumptions that every earlier phase missed. Closing an incident while its root flaw remains is incomplete, so verified fixes must return to Strategy and Plan.

Those feedback links make the overlay more than a checklist. Strategy can change Plan, failed readiness can reshape capacity, workload friction can improve the platform, and incidents can revise the original outcome. Secure and Govern add distinct evidence without becoming isolated ends, and the system learns when a finding changes an earlier choice.

The measures do not prove causality by themselves. A falling count may mean less exposure, missing logs, fewer assets, or a changed classification. A rate without clear scope can still mislead. Reviewers need a denominator, time window, source limits, and counter-signals before the measure can guide a named decision.

The overlay can start a planning or architecture review. Owners update evidence links, check freshness, discuss failed questions, and record the action each signal supports. Passing syntax is not proof of a true claim. Local validation catches missing phases, unknown principles, duplicate evidence, stale contracts, and broken feedback, while human review decides whether the claims are true and sufficient.

This approach also prevents a security theater gate near release. Early problems can change scope, design, capacity, or platform features while those options remain open. A valid exception stays visible and time-bound, and usability evidence matters because unusable controls invite bypass.

The overlay stops before detailed implementation. Chapter 9.2 owns identity and privilege paths, while Chapter 9.3 owns the platform baseline. Chapter 9.4 owns posture and threat signals, and Chapter 9.5 owns delivery-chain protection and incident learning. This section supplies the lifecycle contract that connects them, but it does not certify a platform or workload. Security is not a product list or a late release gate. It is the quality of decisions about trust, responsibility, information, evidence, and recovery across the adoption cycle. That duty remains part of normal engineering work. Protection becomes continuous when every CAF

methodology can change exposure, receive corrective feedback, and cannot hand off or postpone the duty.

Chapter 9: Secure — Protect the Azure System

Chapter 9.2: Protect the Identity and Privilege System

One trusted identity can undo many sound controls because a valid grant can pass through network and policy gates. The harmful act then looks allowed. Identity defense starts with every path from sign-in to power, not with a count of accounts.

A privileged path has six parts: subject, sign-in rules, role, scope, time, and removal check. Each part can widen the harm, and a narrow role can still reach too far when its scope is broad. The full path matters more than any one grant.

Two systems shape most Azure admin paths. Microsoft Entra roles grant directory rights, while Azure resource access is governed by Azure role-based access control, or Azure RBAC. One principal may hold both kinds of grant and cross the boundary between them.

Discovery comes before cleanup. Collect Azure assignments at a clear scope, including grants inherited from above, then collect active and eligible Microsoft Entra directory roles. Add Conditional Access policy data and access-review definitions as separate sets with their own query limits.

Each set answers a different question, and none closes another set's evidence gap. A management-group query omits grants made below that group, while group membership and custom role actions can add unseen rights. Azure Lighthouse grants also sit outside the normal assignment list.

Standing access needs no new activation step, so a stolen identity can offer useful power at once. It can also turn a simple mistake into a large event. High-impact human roles therefore need a clear reason to remain active at all times.

Microsoft Entra Privileged Identity Management, or PIM, can reduce exposure by making a user eligible before a role becomes active. Time limits, stronger sign-in, a reason, and approval can bound the event. Alerts and logs can then support review.

PIM is a control set, not a verdict. A weak activation rule adds little safety, while a long window can resemble a standing grant. Any claim of reduced exposure must compare active roles, eligible roles, activation settings, and actual use.

Conditional Access applies sign-in rules after the first factor and can use the subject, app, device, place, or risk signal. It can block access or require another control. Policy names and states show a design, but sign-in evidence and policy tests must show coverage and results.

Human access starts with a named task, strong sign-in, and a short grant where possible. Its admin path should remain separate from daily work. Shared accounts break the link between an action and one person.

Automation follows a different contract because a workload cannot answer an approval prompt like a person. It needs noninteractive sign-in, narrow reach, a named owner, and a known life span. Its rights should fit one service need, and no person should borrow the identity.

Managed identities help remove app-held secrets because Azure supplies tokens without exposing a password, key, or certificate to code. A system-assigned identity shares the resource life cycle. A user-assigned identity lives on its own and can serve more than one resource.

Managed identity does not mean narrow access; Azure RBAC and data rights still need review. The raw `ServicePrincipal` type marks only a machine-identity candidate because apps also appear as service principals. The collector refuses to guess without approved identity and resource data.

Access reviews address a need that was valid once but may have aged. They can check group access, app access, and role grants on a set schedule, yet a definition proves only that a review series exists. Decisions, removed grants, reviewer quality, and overdue work show whether the review changed access.

The operating loop follows a clear order. Find and rank the paths with the widest harm, reduce standing power, and split human use from software use. Then ask for fresh proof that each high-impact grant remains needed and can be removed.

The read-only collector turns that loop into a repeatable starting point. It checks the expected tenant before any inventory query, stops Azure CLI from looking up principal names, and uses only GET calls against Microsoft Graph v1.0. Its output records facts and limits rather than a security grade.

POWERSHELL

```
Set-StrictMode -Version Latest
$ErrorActionPreference = 'Stop'

function Invoke-AzureCliJson {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [string[]] $Arguments
    )

    $raw = & az @Arguments 2>&1
    if ($LASTEXITCODE -ne 0) {
        throw "Azure CLI failed: $($raw -join [Environment]::NewLine)"
    }

    $text = $raw -join [Environment]::NewLine
    if ([string]::IsNullOrEmpty($text)) {
        throw 'Azure CLI returned an empty response.'
```

```

    }

    try {
        return $text | ConvertFrom-Json -Depth 50 -DateKind String
    }
    catch {
        throw 'Azure CLI returned invalid JSON.'
    }
}

function Test-GraphV1Url {
    [CmdletBinding()]
    [OutputType([bool])]
    param(
        [Parameter(Mandatory)]
        [string] $Url
    )

    $uri = $null
    if (-not [Uri]::TryCreate($Url, [UriKind]::Absolute, [ref] $uri)) {
        return $false
    }

    return (
        $uri.Scheme -ceq 'https' -and
        $uri.Host -ceq 'graph.microsoft.com' -and
        $uri.AbsolutePath.StartsWith('/v1.0/', [StringComparison]::Ordinal)
    )
}

function Get-GraphCollection {
    [CmdletBinding()]
    [OutputType([object])]
    param(
        [Parameter(Mandatory)]
        [string] $InitialUrl
    )

    $nextUrl = $InitialUrl
    $pages = 0
    $items = [Collections.Generic.List[object]]::new()
    $seenUrls = [Collections.Generic.HashSet[string]]::new(
        [StringComparer]::Ordinal
    )

    while ($null -ne $nextUrl) {
        if (-not (Test-GraphV1Url -Url $nextUrl)) {
            throw "Graph URL is outside the allowed v1.0 endpoint: '$nextUrl'."
        }
        if (-not $seenUrls.Add($nextUrl)) {
            throw 'Microsoft Graph pagination returned a repeated next link.'
        }
        if ($pages -ge 100) {
            throw 'Microsoft Graph pagination exceeded 100 pages.'
        }

        $page = Invoke-AzureCliJson -Arguments @(
            'rest',
            '--method', 'get',
            '--url', $nextUrl,
            '--resource', 'https://graph.microsoft.com/',
            '--output', 'json',
            '--only-show-errors'
        )
        $pages++

        if ($null -eq $page -or
            $page.PSObject.Properties.Name -notcontains 'value') {

```

```

        throw 'Microsoft Graph response is missing its value collection.'
    }
    foreach ($item in @($page.value)) {
        if ($null -ne $item) {
            $items.Add($item)
        }
    }

    $candidate = if (
        $page.PSObject.Properties.Name -contains '@odata.nextLink'
    ) {
        [string] $page.'@odata.nextLink'
    }
    else {
        ''
    }
    $nextUrl = if ([string]::IsNullOrEmpty($candidate)) {
        $null
    }
    else {
        $candidate
    }
}

return [pscustomobject] [ordered]@{
    pages = $pages
    items = $items.ToArray()
}
}

function Get-IdentityPrivilegeInventory {
    [CmdletBinding()]
    [OutputType([object])]
    param(
        [Parameter(Mandatory)]
        [guid] $ExpectedTenantId,

        [Parameter(Mandatory)]
        [ValidatePattern('^[A-Za-z0-9._()]+$')]
        [string] $ManagementGroupId,

        [Parameter(Mandatory)]
        [ValidatePattern('^\\d{4}-\\d{2}-\\d{2}T\\d{2}:\\d{2}:\\d{2}Z$')]
        [string] $CollectedUtc
    )

    $account = Invoke-AzureCliJson -Arguments @(
        'account', 'show',
        '--output', 'json',
        '--only-show-errors'
    )
    if ($account.PSObject.Properties.Name -notcontains 'tenantId' -or
        [guid] $account.tenantId -ne $ExpectedTenantId) {
        throw 'The active Azure CLI tenant does not match the expected tenant.'
    }

    $scope = "/providers/Microsoft.Management/managementGroups/$ManagementGroupId"
    $azureAssignments = @(
        Invoke-AzureCliJson -Arguments @(
            'role', 'assignment', 'list',
            '--scope', $scope,
            '--include-inherited',
            '--fill-principal-name', 'false',
            '--output', 'json',
            '--only-show-errors'
        )
    )
}

```

```

$urls = [ordered]@{
    active_directory_roles = (
        'https://graph.microsoft.com/v1.0/roleManagement/directory/' +
        'roleAssignments?$select=id,principalId,roleDefinitionId,' +
        'directoryScopeId,appScopeId'
    )
    eligible_directory_roles = (
        'https://graph.microsoft.com/v1.0/roleManagement/directory/' +
        'roleEligibilityScheduleInstances?$select=id,principalId,' +
        'roleDefinitionId,directoryScopeId,appScopeId,startDateTime,' +
        'endDateTime,memberType,roleEligibilityScheduleId'
    )
    conditional_access_policies = (
        'https://graph.microsoft.com/v1.0/identity/conditionalAccess/' +
        'policies?$select=id,displayName,state,createdDateTime,' +
        'modifiedDateTime'
    )
    access_review_definitions = (
        'https://graph.microsoft.com/v1.0/identityGovernance/' +
        'accessReviews/definitions?$top=100&$skip=0&$select=id,' +
        'displayName,status,' +
        'createdDateTime,lastModifiedDateTime'
    )
}

$activeRoles = Get-GraphCollection -InitialUrl $urls.active_directory_roles
$eligibleRoles = Get-GraphCollection `
    -InitialUrl $urls.eligible_directory_roles
$conditionalAccess = Get-GraphCollection `
    -InitialUrl $urls.conditional_access_policies
$accessReviews = Get-GraphCollection `
    -InitialUrl $urls.access_review_definitions

$servicePrincipalCandidates = @(
    $azureAssignments | Where-Object {
        $_.principalType -ceq 'ServicePrincipal'
    }
)

return [pscustomobject] [ordered]@{
    schema_version = '1.0'
    collected_utc = $CollectedUtc
    environment = 'AzureCloud'
    scope = [ordered]@{
        tenant_id = $ExpectedTenantId.ToString()
        management_group_id = $ManagementGroupId
        resource_id = $scope
    }
    permission_contract = [ordered]@{
        azure_action = 'Microsoft.Authorization/roleAssignments/read'
        graph_least_privileged_permissions = @(
            'RoleManagement.Read.Directory',
            'RoleEligibilitySchedule.Read.Directory',
            'Policy.Read.All',
            'AccessReview.Read.All'
        )
        write_permissions_required = $false
    }
    azure_rbac = [ordered]@{
        query_scope = $scope
        includes_parent_scope_assignments = $true
        includes_child_scope_assignments = $false
        assignments = $azureAssignments
        service_principal_candidates = $servicePrincipalCandidates
    }
    entra_directory_roles = [ordered]@{
        active = $activeRoles
        eligible = $eligibleRoles
    }
    conditional_access = $conditionalAccess
}

```

```

    access_reviews = $accessReviews
    interpretation = [ordered]@{
        effective_access_proven = $false
        least_privilege_proven = $false
        azure_resource_pim_eligibility_included = $false
        managed_identity_classification_complete = $false
        conditional_access_effectiveness_proven = $false
        access_review_execution_proven = $false
        note = (
            'Use this inventory to open review questions; assignment and ' +
            'definition lists do not prove effective access or control outcomes.'
        )
    }
}
}

```

The result preserves the boundary around every evidence set. Azure rows show the query scope and parent inheritance, while stating that child-scope grants are absent. Graph results keep object identifiers plus page counts, which supports a later approved join without requesting names during collection.

The script follows each full Graph next link and accepts an empty page when another link exists. It rejects a repeated link, foreign host, beta path, or more than one hundred pages. Those checks prevent a paging flaw from sending a token to an unexpected endpoint.

The Graph eligibility set covers Microsoft Entra directory roles, not eligible Azure resource-role schedules. ServicePrincipal rows also remain candidates, not proven managed identities. Both limits prevent a narrow inventory from posing as a complete privilege map.

Collection authority needs a reviewable contract. The next artifact names one Azure action and four Graph permissions, using the least-privileged permission in current API guidance. No write permission belongs in this collection job because inventory and change require separate authority.

```

JSON
{
  "schema_version": "1.0",
  "environment": "AzureCloud",
  "azure": {
    "action": "Microsoft.Authorization/roleAssignments/read",
    "scope": "/providers/Microsoft.Management/managementGroups/mg-platform",
    "notes": [
      "Reader includes the required Azure RBAC read action.",
      "The scoped query includes assignments inherited from parents but not assignments created below the management-group scope.",
      "Azure Lighthouse authorized assignments are outside this result."
    ]
  },
  "microsoft_graph": [
    {
      "dataset": "active_directory_roles",
      "least_privileged_permission": "RoleManagement.Read.Directory",
      "method": "GET",
      "endpoint": "/v1.0/roleManagement/directory/roleAssignments"
    },
    {
      "dataset": "eligible_directory_roles",
      "least_privileged_permission": "RoleEligibilitySchedule.Read.Directory",
      "method": "GET",
    }
  ]
}

```

```

    "endpoint": "/v1.0/roleManagement/directory/roleEligibilityScheduleInstances"
  },
  {
    "dataset": "conditional_access_policies",
    "least_privileged_permission": "Policy.Read.All",
    "method": "GET",
    "endpoint": "/v1.0/identity/conditionalAccess/policies"
  },
  {
    "dataset": "access_review_definitions",
    "least_privileged_permission": "AccessReview.Read.All",
    "method": "GET",
    "endpoint": "/v1.0/identityGovernance/accessReviews/definitions"
  }
],
"constraints": {
  "write_permissions_required": false,
  "delegated_role_requirements_may_also_apply": true,
  "effective_access_proven": false,
  "least_privilege_proven": false,
  "azure_resource_pim_eligibility_included": false,
  "managed_identity_classification_complete": false,
  "conditional_access_effectiveness_proven": false,
  "access_review_execution_proven": false
}
}

```

A delegated user may need a supported directory role as well, so an OAuth scope alone may not allow the call. App-only and delegated use also have different review needs. The execution record must state which model ran the job and who approved its access.

The runner adds a UTC time, writes one new file, and will neither make a missing parent folder nor replace old evidence. The tenant ID below is synthetic. An approved run must replace it and use a protected evidence path.

```

POWERSHELL
$inventoryParams = @{
    ExpectedTenantId = '00000000-0000-0000-0000-000000000000'
    ManagementGroupId = 'mg-platform'
    OutputPath = './evidence/identity-privilege-2026-07-21.json'
}
./Invoke-IdentityPrivilegeInventory.ps1 @inventoryParams

```

Partial identity data can create a false safe state, so collection fails closed. A tenant mismatch, bad JSON, missing value list, or unsafe page link stops the run. Failure cannot quietly become an empty inventory or a claim of no exposure.

Six limits travel with every result. The data does not include Azure resource PIM eligibility or prove effective access, least privilege, managed-identity type, working Conditional Access, or completed reviews. A report must retain those limits instead of turning collection success into a control verdict.

Triage starts with blast radius: broad Azure roles at high scopes, active directory roles, and software identities with admin power. Check for a subject that can cross both control planes. Treat emergency access separately, with tight storage, alerts, tests, named rules, and no normal admin use.

For people, measure high-impact roles that are eligible and time-bound rather than always active. Pair that rate with failed activation, long delay, and frequent elevation so a control cannot hide blocked work. Those signals may also show that one broad role covers several distinct tasks.

For software, measure owned identities with a narrow scope and no app-held secret. Also count shared identities, unknown owners, and grants with no current service need. Recent use can inform the review but cannot decide it alone because a rare recovery job may remain valid.

Decision quality also needs proof. Completion can hide weak approval, so sample reasons, removals, self-review, late work, and overrides. A review that never changes access may reflect stable need or a habit with no challenge.

Each finding needs a route back to engineering work. A broad grant can lead to a smaller role or scope, frequent elevation can reveal a missing service feature, and an unowned identity can stop release. The decision owner records the change and its due date.

A Conditional Access gap can reshape the admin path, while weak review data can change its source, reviewer, or schedule. The next inventory tests whether the action altered the path. That feedback makes privilege review part of normal platform work.

This section does not build the full platform baseline. Network, key, log, backup, and resource controls come next, while posture scores and threat signals follow later. This inventory supplies one protected boundary and the questions that test it; it does not certify the landing zone.

Excess access rarely arrives through one loud choice. It gathers through wide scope, long grants, reused software, faded need, and reviews that cease to challenge it. Protection stays alive only while every grant can explain why it exists, when it ends, and who will remove it.

Chapter 9: Secure — Protect the Azure System

Chapter 9.3: Build the Platform Security Baseline

Strong security components can still leave a weak path between them. Private access fails when DNS points elsewhere, while encryption cannot correct overly broad authorization. Logs also add little when no accountable reviewer can use them.

A platform security baseline is the minimum protection approved for a named platform scope. Each row names an owner, dependencies, delivery state, cost class, evidence, limits, and a feedback route. Any lower state needs an authorized, evidenced, and dated exception.

The baseline is not one Azure product checklist for every cloud estate. A regulated service and a private test estate can need different layers above it. The common floor stays stable while extra controls follow risk and context.

Dependencies between controls matter as much as their individual settings. An administrator needs a verified identity before a private network path can help. A private endpoint needs effective routing and private DNS before an approved client can use it.

Identity forms the first boundary. The prior section's privileged-path contract covers people and software, so this section consumes its inventory. Each high-impact action still needs a named subject, narrow authority, bounded duration, and removal route.

Segmentation divides identity, connectivity, management, and workload traffic into clear zones. A reviewed flow contract names the source, destination, protocol, port, and business purpose. Network security groups and routes enforce parts of that contract, while observation tests the effective path.

Secure administration joins identity and network design. A managed administrator source uses an approved private route, and direct public RDP or SSH remains blocked. A team may choose a privileged workstation, VPN, Azure Bastion, or just-in-time access after it reviews fit and cost.

Private access narrows exposure for sensitive platform services. Azure Private Link places a service endpoint in a virtual network, but that endpoint grants no authorization. Microsoft Entra and Azure RBAC still decide who can use the service.

A private endpoint also has resource-hour and data costs. The baseline therefore makes private connectivity conditional on threat, data, availability, and price. It stays off until that choice is approved.

DNS belongs inside the same control. The normal service name must resolve to the private address from every approved client path. Effective routing must also avoid an unintended public data path.

Data protection starts with the chosen service's encryption model. Many Azure services encrypt stored data with platform-managed keys by default and without customer key operations. That simpler model remains the floor unless an approved requirement calls for customer key control. Customer-managed keys are a choice, not a badge. They add authority over a key encryption key, but also add rotation, authorization, availability, recovery, and service dependencies. A recovery exercise must test those duties before the platform relies on that key model.

Secrets need a protected store. The example uses Azure Key Vault with Azure RBAC, soft delete, purge protection, blocked public access, and a private endpoint. Its standard tier has transaction charges, so the usage-metered cost flag remains visible.

Terraform can set up a vault without carrying any secret value. Managed identities can retrieve approved values at runtime without application-held credentials. The safer rule keeps secret values out of both configuration and state.

Platform logging starts with control-plane activity. Azure keeps Activity Log events for ninety days by default. These events cover control-plane changes and actions, but they do not usually record data-plane reads.

Resource logs answer other questions and are not collected by default. A resource-scoped diagnostic setting sends selected service categories to an approved destination. The owning teams must decide volume, reviewer access, ingestion delay, retention, and cost.

Category groups can follow a service's supported audit set as Microsoft changes it. That feature cannot make every event useful or provide a transactionally lossless audit ledger. A safe test event must show that the needed fields arrive.

Defender for Cloud adds another layer without forcing every paid plan into the floor. Foundational Cloud Security Posture Management comes with the enabled service without an added Defender plan charge. Enhanced posture and workload-protection plans remain separate paid choices.

A paid tool can create signals faster than a team can act. Firewall, Sentinel, enhanced Defender, private endpoints, extra key control, and longer log paths may all help. Each one still needs a purpose, accountable owner, operating capacity, exit path, and price decision.

The next JSON block is a short view of the full baseline. Required rows show the accepted floor, while conditional rows show added choices. No row treats a configured setting as proof of control effectiveness.

JSON

```
{
  "schema_version": "1.0",
  "baseline_release": "platform-security-baseline-v1.0.0",
  "scope": "platform-landing-zone",
  "region": "westeurope",
  "controls": [
    {
      "id": "identity-privileged-path",
      "domain": "identity",
      "baseline_state": "required",
      "default_enabled": true,
      "cost_flag": "included",
      "dependencies": [],
      "effectiveness_proven": false
    },
    {
      "id": "network-segmentation",
      "domain": "network",
      "baseline_state": "required",
      "default_enabled": true,
      "cost_flag": "included",
      "dependencies": [
        "identity-privileged-path"
      ],
      "effectiveness_proven": false
    },
    {
      "id": "secure-administration",
      "domain": "administration",
      "baseline_state": "required",
      "default_enabled": true,
      "cost_flag": "included",
      "dependencies": [
        "identity-privileged-path",
        "network-segmentation"
      ],
      "effectiveness_proven": false
    },
    {
      "id": "private-platform-services",
      "domain": "network",
      "baseline_state": "conditional",
      "default_enabled": false,
      "cost_flag": "optional_paid",
      "dependencies": [
        "network-segmentation"
      ],
      "effectiveness_proven": false
    },
    {
      "id": "platform-managed-encryption",
      "domain": "data_protection",
      "baseline_state": "required",
      "default_enabled": true,
      "cost_flag": "included",
      "dependencies": [
        "identity-privileged-path"
      ],
      "effectiveness_proven": false
    },
    {
      "id": "customer-managed-keys",
      "domain": "data_protection",
      "baseline_state": "conditional",
      "default_enabled": false,
      "cost_flag": "optional_paid",
      "dependencies": [
        "platform-managed-encryption"
      ],
      "effectiveness_proven": false
    }
  ]
}
```

```

    },
    {
      "id": "protected-secret-store",
      "domain": "secrets",
      "baseline_state": "required",
      "default_enabled": true,
      "cost_flag": "usage_metered",
      "dependencies": [
        "identity-privileged-path",
        "platform-managed-encryption"
      ],
      "effectiveness_proven": false
    },
    {
      "id": "platform-activity-evidence",
      "domain": "telemetry",
      "baseline_state": "required",
      "default_enabled": true,
      "cost_flag": "included",
      "dependencies": [
        "identity-privileged-path"
      ],
      "effectiveness_proven": false
    },
    {
      "id": "resource-audit-export",
      "domain": "telemetry",
      "baseline_state": "conditional",
      "default_enabled": false,
      "cost_flag": "optional_paid",
      "dependencies": [
        "platform-activity-evidence",
        "protected-secret-store"
      ],
      "effectiveness_proven": false
    },
    {
      "id": "defender-foundational-posture",
      "domain": "threat_protection",
      "baseline_state": "required",
      "default_enabled": true,
      "cost_flag": "included",
      "dependencies": [
        "identity-privileged-path",
        "platform-activity-evidence"
      ],
      "effectiveness_proven": false
    },
    {
      "id": "defender-enhanced-plans",
      "domain": "threat_protection",
      "baseline_state": "conditional",
      "default_enabled": false,
      "cost_flag": "optional_paid",
      "dependencies": [
        "defender-foundational-posture"
      ],
      "effectiveness_proven": false
    }
  ]
}

```

The row order has meaning. A dependency may point only to an earlier control, which prevents a cycle and exposes missing prerequisites. Seven domains span eleven rows because several areas split the floor from paid growth.

Three cost flags avoid a false free-or-paid split. Included needs no separate capability decision here, while usage metered keeps transaction charges visible. Optional paid means conditional, off by default, and blocked without cost approval.

The full record adds owners, proof sources, reviewers, age limits, tests, known gaps, and feedback. Its checker rejects self-review, old proof, unknown links, and missing domains. It also rejects any optional paid choice that silently enables itself.

The Terraform example delivers selected rows. It pins Terraform and AzureRM, uses the sample region, and expects existing network and log targets. All identifiers are synthetic, and no credential or secret value is present.

HCL / TERRAFORM

```
terraform {
  required_version = "= 1.15.8"

  required_providers {
    azurerem = {
      source = "hashicorp/azurerem"
      version = "= 4.81.0"
    }
  }
}

provider "azurerem" {
  features {}
}
```

HCL / TERRAFORM

```
variable "location" {
  description = "Replaceable sample region selected after availability, compliance, and cost review."
  type        = string
  default     = "westeurope"

  validation {
    condition = var.location == lower(var.location) && length(trimspaces(var.location)) > 0
    error_message = "The example region must be a nonempty lowercase Azure location name."
  }
}

variable "resource_group_name" {
  description = "Existing resource group for the selected platform security controls."
  type        = string
  default     = "rg-platform-security-weu"
}

variable "tenant_id" {
  description = "Expected Microsoft Entra tenant for the platform vault."
  type        = string
  default     = "00000000-0000-0000-0000-000000000000"

  validation {
    condition = can(regex("^[0-9a-f]{8}(-[0-9a-f]{4}){3}-[0-9a-f]{12}$", var.tenant_id))
    error_message = "tenant_id must be a lowercase UUID string."
  }
}
```

```

variable "administration_subnet_id" {
  description = "Existing subnet that carries the approved private administration path."
  type        = string
  default     = "/subscriptions/00000000-0000-0000-0000-000000000000/resourceGroups/rg-
platform-connectivity-weu/providers/Microsoft.Network/virtualNetworks/vnet-platform-
weu/subnets/snet-administration"
}

variable "deploy_secret_store" {
  description = "Creates the example Key Vault only after its operating and usage costs
are accepted."
  type        = bool
  default     = false
}

variable "enable_private_endpoint" {
  description = "Creates the optional paid private endpoint for the example vault."
  type        = bool
  default     = false
}

variable "private_endpoint_subnet_id" {
  description = "Existing subnet for the vault private endpoint; required when private
access is enabled."
  type        = string
  default     = null
  nullable    = true
}

variable "private_dns_zone_id" {
  description = "Existing privatelink.vaultcore.azure.net zone; required when private
access is enabled."
  type        = string
  default     = null
  nullable    = true
}

variable "log_analytics_workspace_id" {
  description = "Existing governed workspace for optional vault audit logs."
  type        = string
  default     = null
  nullable    = true
}

variable "enable_vault_audit_export" {
  description = "Routes the vault audit category group after volume, retention, and cost
approval."
  type        = bool
  default     = false
}

variable "enable_defender_servers_plan" {
  description = "Enables the optional paid Defender for Servers plan after a separate
security investment decision."
  type        = bool
  default     = false
}

variable "cost_acceptance" {
  description = "Explicit approvals for every metered or optional paid capability in this
example."
  type = object({
    secret_store      = bool
    private_endpoint  = bool
    vault_audit_export = bool
    defender_for_servers = bool
  })
  default = {

```

```

    secret_store      = false
    private_endpoint  = false
    vault_audit_export = false
    defender_for_servers = false
  }
}

```

HCL / TERRAFORM

```

resource "azurerm_network_security_group" "secure_admin" {
  name                = "nsg-platform-administration"
  location            = var.location
  resource_group_name = var.resource_group_name

  security_rule {
    name                = "Deny-Internet-SSH"
    priority            = 100
    direction           = "Inbound"
    access              = "Deny"
    protocol            = "Tcp"
    source_port_range   = "*"
    destination_port_range = "22"
    source_address_prefix = "Internet"
    destination_address_prefix = "*"
  }

  security_rule {
    name                = "Deny-Internet-RDP"
    priority            = 110
    direction           = "Inbound"
    access              = "Deny"
    protocol            = "Tcp"
    source_port_range   = "*"
    destination_port_range = "3389"
    source_address_prefix = "Internet"
    destination_address_prefix = "*"
  }
}

resource "azurerm_subnet_network_security_group_association" "secure_admin" {
  subnet_id                = var.administration_subnet_id
  network_security_group_id = azurerm_network_security_group.secure_admin.id
}

resource "azurerm_key_vault" "platform" {
  count = var.deploy_secret_store ? 1 : 0

  name                = "kv-platform-security-weu"
  location            = var.location
  resource_group_name = var.resource_group_name
  tenant_id           = var.tenant_id
  sku_name            = "standard"
  rbac_authorization_enabled = true
  public_network_access_enabled = false
  purge_protection_enabled = true
  soft_delete_retention_days = 90

  lifecycle {
    precondition {
      condition = (
        var.cost_acceptance.secret_store &&
        var.enable_private_endpoint &&
        var.cost_acceptance.private_endpoint
      )
      error_message = "The secret store and its disabled public data plane require accepted metered use and an approved private endpoint."
    }
  }
}

```



```

resource "azurerm_private_endpoint" "vault" {
  count = var.deploy_secret_store && var.enable_private_endpoint ? 1 : 0

  name           = "pep-kv-platform-security-weu"
  location       = var.location
  resource_group_name = var.resource_group_name
  subnet_id      = var.private_endpoint_subnet_id

  private_service_connection {
    name                       = "psc-kv-platform-security-weu"
    private_connection_resource_id = azurerm_key_vault.platform[0].id
    is_manual_connection       = false
    subresource_names          = ["vault"]
  }

  private_dns_zone_group {
    name             = "vault-private-dns"
    private_dns_zone_ids = [var.private_dns_zone_id]
  }

  lifecycle {
    precondition {
      condition = (
        var.private_endpoint_subnet_id != null &&
        var.private_dns_zone_id != null &&
        var.cost_acceptance.private_endpoint
      )
      error_message = "Private access requires a subnet, a private DNS zone, and cost acceptance."
    }
  }
}

resource "azurerm_monitor_diagnostic_setting" "vault_audit" {
  count = var.deploy_secret_store && var.enable_vault_audit_export ? 1 : 0

  name                       = "platform-vault-audit"
  target_resource_id         = azurerm_key_vault.platform[0].id
  log_analytics_workspace_id = var.log_analytics_workspace_id

  enabled_log {
    category_group = "audit"
  }

  lifecycle {
    precondition {
      condition = (
        var.log_analytics_workspace_id != null &&
        var.cost_acceptance.vault_audit_export
      )
      error_message = "Vault audit export requires a governed workspace and explicit cost acceptance."
    }
  }
}

resource "azurerm_security_center_subscription_pricing" "servers" {
  count = var.enable_defender_servers_plan ? 1 : 0

  tier           = "Standard"
  resource_type = "VirtualMachines"

  lifecycle {
    precondition {
      condition = var.cost_acceptance.defender_for_servers
    }
  }
}

```

```
        error_message = "The paid Defender for Servers plan requires explicit cost  
acceptance."  
    }  
}  
}
```

The network security group blocks public SSH and RDP on the admin subnet. No allow rule or private route is created. Those flows must come from the approved connectivity design.

The vault is absent from the default plan. Enabling it needs approval for vault use and its private endpoint. The vault then blocks public data-plane access and uses AzureRM's current RBAC authorization field.

Soft delete and purge protection preserve recovery options after deletion. They do not show that every app handles a secret well. An end-to-end test must still check authorized use, denied access, and recovery.

The endpoint needs a subnet and the Key Vault private DNS zone. Its private service connection targets the Key Vault data plane. An authorized integration test must still check DNS, routing, authorization, and rejection of the public path.

Vault audit export is also off by default. It needs a governed Log Analytics workspace and cost approval before the plan adds it. A test event must then show subject, target, result, event time, arrival time, and reviewer access.

The Defender for Servers resource follows the same gate. The enable flag alone makes the plan fail. Cost approval plus that flag selects the paid virtual-machine protection plan.

These defaults are incomplete by design. They leave paid Defender, log export, the vault, and its endpoint off until the right inputs exist. A missing approval causes a clear plan failure instead of an accidental purchase or unreachable service.

Proof must cross resource lines. One test follows an admin from managed source through identity, route, action, and event. Another fetches a test vault value privately and then checks that the public path fails.

Useful security measures follow these end-to-end tests. They can count approved paths that work, key resources with fresh audit proof, and exceptions closed on time. Bypasses, missing event fields, broken private paths, old alert queues, and ownerless cost serve as counter-signals. A reviewed release changes the baseline. New proof may tighten the floor, add a paid layer, or remove one that no longer helps. Every change keeps its source revision, approver, test result, affected owner, and next review date.

An exception cannot erase linked effects. A temporary public path or missing log needs an owner, reason, evidence, compensating action, expiry, and removal decision. If it breaks a need, every later claim must fall with it.

Suppose a team blocks a vault's public access before linking private DNS. The portal shows a strict setting, but delivery can no longer reach the vault. The team may then seek a risky bypass. The baseline exposes that DNS link before the change. It tests the supported path instead of praising one setting. The failure returns to network and service work.

Another team may turn on every Defender plan during a trial. Findings rise while owners, time, and future cost stay unclear. The matrix sends that signal to a security investment review instead of calling plan count a successful outcome.

This section sets the desired floor and shows selected delivery code. The next section tests posture and threat signals with its own measures. A configured floor supplies posture evidence, but it cannot predict detection quality or risk reduction.

Several questions keep the baseline alive. Which link failed first, which public path still works, and which proof cannot support its claim? Which paid control no longer earns a place?

Chapter 9: Secure — Protect the Azure System

Chapter 9.4: Measure Posture and Detect Threats

A security baseline states the protection a platform intends to provide. Posture evidence asks whether that intent appears in the environment now. Detection evidence asks whether harmful behavior can still produce a useful response signal.

The questions belong in one feedback system, but their answers are not interchangeable. A configured control can fail at runtime. A busy alert queue can also hide a broken data path. Useful measurement connects configuration, exposure, telemetry, detection, and reviewed outcomes without blending their meanings.

Posture is the observed state of recommendations and controls. It belongs to a named scope and time. Detection effectiveness starts with an in-scope test or event. That input must produce expected telemetry, an alert, and a routing result. Alert quality describes how useful generated alerts are after enough cases receive consistent review.

Each definition supports a different decision. Posture directs remediation. Detection tests direct engineering, while reviewed outcomes direct tuning. One unexplained security number would hide the reason for action.

Microsoft Defender for Cloud provides recommendations, assessment results, and Secure Score views. Two Secure Score models now exist. The classic score is in the Azure portal.

The risk-based Cloud Secure Score is in the Defender portal. Its calculations and values differ from the classic score. Stored evidence must therefore name its model.

The Azure CLI command in this example reads the classic secure-score surface. The collector labels every result `classic_azure_portal_secure_score`. It also rejects comparison with the newer score. A trend can compare snapshots only under stable conditions. Model, scope, and recommendation coverage must not change.

The classic score helps an engineer find contributing controls. It is not a compromise probability or a certification. It does not prove that a control stopped an attack. Movement can also follow a coverage or recommendation change rather than platform work.

Recommendations need the same caution. The example reads individual security assessment results. It does not rely on legacy grouped recommendation records. An unhealthy result starts a review. It does not settle exposure, exploitability, importance, ownership, or applicability.

Paid Defender CSPM can add Microsoft risk prioritization. Its factors can include exposure, exploitability, and business context. That risk model remains separate from classic Secure Score. A team without the paid capability still needs an explainable review order.

The dashboard supplies a local review band, not a replacement Microsoft risk score. It joins a current unhealthy result to several facts. These include exposure, high-impact paths, business criticality, threat evidence, ownership, and exception state. When required context is absent, the item enters **verify** instead of appearing low risk.

Urgent means current threat evidence intersects a confirmed exposure or privileged path. High means an unhealthy finding intersects exposure, privilege, or high business criticality. Normal means an owner exists without a confirmed high-impact path. Verify holds uncertain scope, state, exposure, ownership, or applicability.

The bands order review rather than estimate breach likelihood. Paid Defender CSPM risk data is preferred when it is available. The dashboard preserves that source for its supported scope. Local inputs stay visible. Every queue position can then be explained.

Collection must not widen authority. The PowerShell example first checks the active tenant and subscription. It then reads classic scores, assessments, and two Log Analytics queries. It uses explicit scopes and JSON output, and it contains no Azure write command.

```
POWERSHELL
[CmdletBinding()]
param(
    [Parameter(Mandatory)] [guid] $ExpectedTenantId,
    [Parameter(Mandatory)] [guid] $ExpectedSubscriptionId,
    [Parameter(Mandatory)] [guid] $WorkspaceId
)

Set-StrictMode -Version Latest
$ErrorActionPreference = 'Stop'

function Invoke-AzJson {
    param([Parameter(Mandatory)] [string[]] $Arguments)

    $raw = & az @Arguments 2>&1
    if ($LASTEXITCODE -ne 0) {
        throw "Azure CLI failed: $($raw -join [Environment]::NewLine)"
    }
    return ($raw -join [Environment]::NewLine) |
        ConvertFrom-Json -Depth 50 -DateKind String
}

$account = Invoke-AzJson @(
    'account', 'show', '--output', 'json', '--only-show-errors'
)
if ([guid] $account.tenantId -ne $ExpectedTenantId -or
    [guid] $account.id -ne $ExpectedSubscriptionId) {
    throw 'The active Azure CLI context is outside the approved scope.'
}

$subscription = $ExpectedSubscriptionId.ToString()
$workspace = $WorkspaceId.ToString()
$alertQuery = Get-Content -Raw "$PSScriptRoot/security-alert-signals.kql"
$incidentQuery = Get-Content -Raw "$PSScriptRoot/security-incident-quality.kql"

$classicScores = Invoke-AzJson @(
    'security', 'secure-scores', 'list',
    '--subscription', $subscription,
    '--output', 'json', '--only-show-errors'
```

```

)
$assessments = Invoke-AzJson @(
    'security', 'assessment', 'list',
    '--subscription', $subscription,
    '--output', 'json', '--only-show-errors'
)
$alertSignals = Invoke-AzJson @(
    'monitor', 'log-analytics', 'query',
    '--workspace', $workspace,
    '--analytics-query', $alertQuery,
    '--timespan', 'P30D',
    '--subscription', $subscription,
    '--output', 'json', '--only-show-errors'
)
$incidentSignals = Invoke-AzJson @(
    'monitor', 'log-analytics', 'query',
    '--workspace', $workspace,
    '--analytics-query', $incidentQuery,
    '--timespan', 'P30D',
    '--subscription', $subscription,
    '--output', 'json', '--only-show-errors'
)

[pscustomobject] [ordered]@{
    secure_score = [ordered]@{
        model = 'classic_azure_portal_secure_score'
        comparable_to_new_cloud_secure_score = $false
        rows = $classicScores
    }
    recommendations = [ordered]@{
        model = 'individual_assessment_results'
        rows = $assessments
    }
    alert_workload_signals = $alertSignals
    reviewed_incident_signals = $incidentSignals
    proof_limits = [ordered]@{
        control_effectiveness_proven = $false
        absence_of_threats_proven = $false
        missed_detections_measured = $false
        alert_volume_is_alert_quality = $false
    }
}

```

The complete tested collector also blocks unexpected query tables. It rejects external-data operators as well. Its runner creates a new evidence file and refuses an overwrite. These controls make scope, time, and query text part of the record.

Microsoft Sentinel remains optional because licensing, ingestion, and operating capacity need approval. When enabled, its [SecurityAlert](#) table stores alerts from connected providers. Fields can differ or remain blank. The first query measures generated workload and entity enrichment. It does not label either result as quality.

```

KUSTO
SecurityAlert
| where TimeGenerated >= ago(30d)
| summarize
    Alerts = count(),
    DistinctAlertTypes = dcount(AlertType),
    AlertsWithEntity = countif(isnotempty(CompromisedEntity)),
    AlertsWithConfidence = countif(
        isnotempty(ConfidenceLevel) or isnotnull(ConfidenceScore)
    )
    by Day = bin(TimeGenerated, 1d), AlertSeverity, ProviderName
| extend EntityCoveragePercent = round(
    100.0 * todouble(AlertsWithEntity) / todouble(Alerts),

```

```

1
)
order by Day asc, AlertSeverity asc

```

Its denominator is all alerts in the same day, provider, and severity segment. Falling entity coverage may expose an enrichment or rule problem. Some alert types can legitimately omit the field. That counter-signal needs review before repair work begins.

Alert counts do not contain analyst conclusions. Sentinel records incident updates in [SecurityIncident](#). The next query keeps the latest observed record per incident number. It counts closure classifications without pretending to calculate universal detection precision.

```

KUSTO
SecurityIncident
| where TimeGenerated >= ago(30d)
| summarize arg_max(TimeGenerated, *) by IncidentNumber
| summarize
    Incidents = count(),
    Closed = countif(Status == 'Closed'),
    ClassifiedClosed = countif(
        Status == 'Closed' and isnotempty(Classification)
    ),
    TruePositive = countif(Classification == 'TruePositive'),
    FalsePositive = countif(Classification == 'FalsePositive'),
    BenignPositive = countif(Classification == 'BenignPositive'),
    Undetermined = countif(
        Classification == 'Undetermined' or
        (Status == 'Closed' and isempty(Classification))
    )
    by Severity
| extend ClassificationCoveragePercent = iff(
    Closed == 0,
    real(null),
    round(100.0 * todouble(ClassifiedClosed) / todouble(Closed), 1)
)

```

Classification coverage comes before false-positive share. An outcome ratio is weak when few closed incidents receive a classification. The design therefore sets a sample gate. It suppresses the ratio until the selected segment contains at least twenty classified closures.

Even a well-reviewed set describes only generated and investigated incidents. It cannot count an attack that produced no telemetry. It also misses an attack that matched no rule or never reached an analyst. Low false-positive share can therefore coexist with poor detection coverage. Governed detection tests supply the needed counterweight. Every current case names a technique, data source, and analytic-rule version. It also names the expected alert and route. A pass proves that bounded path in that window. It does not prove detection of every attack variant.

Telemetry health forms another independent input. Required sources need freshness, expected-event, and route checks. Dependent detection results cannot be trusted before those checks pass. A disabled health feature or absent connector creates unknown coverage, never evidence of zero threats.

The compact dashboard contract preserves those boundaries. Each measure names its sources and accountable owner. It also names a supported decision and proof limit. The full tested form keeps numerators, denominators, segments, counter-signals, freshness, and feedback routes.

```
JSON
{
  "schema_version": "1.0",
  "dashboard_id": "platform-security-feedback",
  "source_boundaries": {
    "secure_score_model": "classic_azure_portal_secure_score",
    "recommendation_model": "individual_assessment_results",
    "alert_volume_is_alert_quality": false,
    "unknown_is_not_zero": true
  },
  "priority_model": {
    "name": "local_review_band",
    "is_microsoft_risk_score": false,
    "paid_defender_cspm_risk_level_preferred_when_available": true,
    "ordered_bands": [
      "urgent",
      "high",
      "normal",
      "verify"
    ]
  },
  "measures": [
    {
      "id": "classic_secure_score_percent",
      "sources": [
        "classic_secure_score_snapshots"
      ],
      "owner": "cloud security posture owner",
      "decision": "Investigate material movement and its contributing controls.",
      "proof_limit": "A higher score can coexist with exploitable or business-critical exposure."
    },
    {
      "id": "exposed_unhealthy_recommendations",
      "sources": [
        "individual_assessments",
        "resource_exposure_register"
      ],
      "owner": "security remediation lead",
      "decision": "Assign, time-box, mitigate, or document a governed exception.",
      "proof_limit": "The local band is a review order, not a Microsoft risk score."
    },
    {
      "id": "alert_entity_coverage_percent",
      "sources": [
        "security_alert_rollup"
      ],
      "owner": "detection engineering lead",
      "decision": "Repair enrichment or tune rules when sustained gaps slow triage.",
      "proof_limit": "Entity presence does not establish alert correctness."
    },
    {
      "id": "incident_classification_coverage_percent",
      "sources": [
        "reviewed_incident_rollup"
      ],
      "owner": "security operations lead",
      "decision": "Correct the review workflow before using outcome ratios.",
      "proof_limit": "Coverage measures completion, not classification accuracy."
    },
    {
      "id": "false_positive_share_percent",
      "sources": [
        "reviewed_incident_rollup"
      ]
    }
  ]
}
```



```

    ],
    "owner": "detection engineering lead",
    "decision": "Review high-volume rules with representative cases before tuning.",
    "proof_limit": "A low share says nothing about missed detections."
  },
  {
    "id": "detection_test_pass_percent",
    "sources": [
      "detection_test_runs",
      "telemetry_health"
    ],
    "owner": "detection validation owner",
    "decision": "Restore the broken data path or detection before expanding coverage.",
    "proof_limit": "Passing scenarios do not prove detection of untested variants."
  },
  {
    "id": "telemetry_health_percent",
    "sources": [
      "telemetry_health"
    ],
    "owner": "security telemetry owner",
    "decision": "Treat missing telemetry as unknown and restore collection first.",
    "proof_limit": "Delivery health does not prove semantic completeness or useful
detection."
  }
],
"feedback_routes": [
  "platform security baseline backlog",
  "detection engineering backlog",
  "platform telemetry backlog",
  "analytics-rule tuning backlog"
]
}

```

The first view places classic score beside maximum points. It also shows a coverage-change warning. The score serves as navigation into contributing controls, never as a release gate. Owners inspect the recommendations behind movement before accepting a posture claim.

The second view joins unhealthy assessments to the governed exposure register. It groups urgent, high, normal, and verify items. Owner and exception state stay visible. Unknown exposure or criticality remains missing evidence instead of improving a denominator.

The third view describes generated alert workload. It splits the count by provider, severity, type, and day. Entity coverage shows whether a common triage dependency is present. A volume change may require capacity review, but it proves neither coverage nor correctness.

The fourth view starts with incident classification coverage. Outcome shares remain hidden until coverage and sample gates pass. Detection engineers can then examine representative cases for one rule. They do not need to chase a fleet-wide ratio.

The fifth view measures due detection tests. Its numerator contains cases that produced expected telemetry, an alert, and the expected route. Failed execution remains a counter-signal. So do unhealthy telemetry and stale rule content.

The sixth view reports health for sources named in an approved detection inventory. A missing source blocks confidence in dependent measures. It also routes work to telemetry engineering. Merely enabled connectors do not enlarge the denominator.

These views operate on different clocks. Posture snapshots may be daily. Alert and incident rollups may run every few hours. Tests may be weekly or release-triggered. Every value therefore carries a collection time, evaluation window, and freshness state.

Ownership also stays divided. Asset and baseline owners remediate posture. Detection engineers tune rules, while telemetry owners restore data paths. Named decision makers retain incident review and risk acceptance. Those duties do not fall to a generic security group.

Cost and licensing remain explicit. Sentinel ingestion and retention need an approved service decision. So do paid Defender plans, test systems, and analyst time. When a source is unavailable, its measure is unknown rather than reassuringly empty.

The feedback routes close the loop without taking over later topics. Exposed findings return to the baseline, and failed tests return to detection engineering. Missing sources return to telemetry engineering. Repeated review patterns enter rule tuning, while full incident learning remains its own lifecycle.

This conversation is more useful than asking whether a score rose. A team can name what is unhealthy and why it matters. It can test a detection path, review incident labels, and mark what remains unknown. Every answer leads to an owner and a bounded decision.

The anchor is simple. Posture shows what appears exposed. Tests show what the security system can currently see. Reviewed outcomes improve generated signals, but unknown coverage stays visible. Security becomes continuous when those observations repeatedly change controls, telemetry, and detections.

Chapter 9: Secure — Protect the Azure System

Chapter 9.5: Secure the Delivery Chain and Learn from Incidents

A private repository does not make delivery trustworthy. An approved Terraform plan cannot do that either. Source, modules, providers, tasks, agents, identities, state, evidence, and protected settings all shape what reaches Azure. These parts form the infrastructure delivery chain.

The chain is part of the platform attack surface. A provider change can alter resource behavior. A stolen secret can bypass sound Terraform. A task update can also change execution without a repository diff.

Security must admit known inputs and boundaries. Trusting a pipeline name is not enough. Each admitted claim needs evidence and a stated limit.

An admission pipeline is narrower than the delivery pipeline built earlier. It proves selected conditions and emits bounded evidence. It does not plan, apply, destroy, import, alter state, or unlock state.

Start with a clean checkout. Shallow history reduces unused content. Disabled submodules and Git LFS avoid undeclared fetch paths. `persistCredentials: false` removes the checkout token from later Git operations.

Those controls do not prove that source is safe. Branch policy still needs review and restricted pipeline changes. Signing or provenance can add evidence. The organization must still define its trusted issuer, subject, and failure response.

Dependencies need equal care. Pin the Terraform CLI and AzureRM provider to exact versions. Commit `.terraform.lock.hcl` and review its selection and hashes. Run `terraform init -lockfile=readonly` during admission.

A version update should arrive as its own reviewed change. Generate hashes from approved upstream packages for each runner platform. Reject a missing platform hash. Treat an unexplained signer change as a separate decision.

The lock file records provider packages. Its hashes do not prove that a provider is safe or approved. The file also excludes remote modules. Use an exact registry version or an immutable version-control commit for each module.

Other inputs stay mutable. `PowerShell@2` and `AzureCLI@2` pin only a task major version. The `ubuntu-24.04` label also changes over time. Evidence should record observed versions and flag unexpected movement.

Agent choice creates another boundary. A hosted agent is replaced after its job, but its image still changes. A self-hosted agent gives more control. It also creates patching, isolation, persistence, and cleanup duties.

Network reach should match the job. Verification may need approved package endpoints. Admission needs neither the Terraform backend nor broad resource access. Private agents and controlled egress can help when their operating cost is justified.

Long-lived credentials add theft and rotation risk. The Azure Resource Manager service connection should use workload identity federation. Use the current Microsoft Entra issuer. The pipeline then exchanges its identity for a short-lived token instead of storing a client secret.

Federation does not create least privilege. This admission identity only needs to read its Azure context. A different identity should own later resource changes. That identity needs its own scope and approval path.

YAML references the service connection by a literal name. It cannot prove the connection scheme, federated subject, tenant, or effective roles. Those facts need protected connection review. They also need an authorized integration test.

Approvals live outside YAML. Resource owners configure checks on environments, connections, variable groups, pools, and secure files. Pipeline authors cannot change those checks through the consuming file. This separation protects the decision point.

Protected resources should authorize named pipelines only. Open access lets unrelated definitions request the same identity or variables. Expected tenant and subscription values can use a protected variable group. Identifiers are still not credentials.

The admission environment should require branch control, nonself approval, and an exclusive lock. `lockBehavior: sequential` orders runs after the lock exists. It does not create the external check. Evidence must keep that limit visible.

External checks can drift without a repository edit. Export their settings after administrator changes and on a review schedule. Compare branch, approver, bypass, and lock rules with the contract. Do this before trusting them again.

The following YAML preserves those boundaries. Verify inspects the contract and Terraform without backend access. Admit reaches the protected environment. It checks only the federated Azure context.

```
YAML
trigger: none
pr: none

name: security-admission-$(Date:yyyyMMdd).$(Rev:r)

variables:
```

```
terraformVersion: '1.15.8'
terraformRoot: 'platform'
securityRoot: 'security'
```

stages:

- stage: Verify
 - displayName: Verify delivery inputs
 - jobs:
 - job: VerifySupplyChain
 - pool:
 - vmImage: ubuntu-24.04
 - workspace:
 - clean: all
 - steps:
 - checkout: self
 - clean: true
 - fetchDepth: 1
 - lfs: false
 - submodules: false
 - persistCredentials: false
- task: PowerShell@2
 - displayName: Validate supply-chain contract
 - inputs:
 - targetType: inline
 - pwsh: true
 - script: |
 - \$ErrorActionPreference = 'Stop'
 - \$directory = Join-Path `
 - '\$(Build.ArtifactStagingDirectory)' `
 - 'security-evidence'
 - New-Item -ItemType Directory `
 - Path \$directory -ErrorAction Stop | Out-Null
 - python '\$(securityRoot)/validate_delivery_security.py' `
 - repo-root '\$(Build.SourcesDirectory)' `
 - contract '\$(securityRoot)/delivery-security-contract.json' `
 - pipeline 'azure-pipelines.security.yml' `
 - incident '\$(securityRoot)/incident-to-backlog.example.json' `
 - output "\$directory/delivery-evidence.json"
 - if (\$LASTEXITCODE -ne 0) {
 - throw 'Delivery-security validation failed.'
 - }
- task: PowerShell@2
 - displayName: Verify Terraform without backend access
 - env:
 - TF_IN_AUTOMATION: 'true'
 - TF_INPUT: 'false'
 - inputs:
 - targetType: inline
 - pwsh: true
 - script: |
 - \$ErrorActionPreference = 'Stop'
 - \$version = terraform version -json |
 - ConvertFrom-Json -Depth 20
 - if (\$version.terraform_version -cne '\$(terraformVersion)') {
 - throw "Unexpected Terraform version."
 - }
 - terraform -chdir='\$(terraformRoot)' fmt -check
 - if (\$LASTEXITCODE -ne 0) { throw 'terraform fmt failed.' }
 - terraform -chdir='\$(terraformRoot)' init `
 - backend=false -input=false -lockfile=readonly
 - if (\$LASTEXITCODE -ne 0) { throw 'terraform init failed.' }
 - terraform -chdir='\$(terraformRoot)' validate
 - if (\$LASTEXITCODE -ne 0) { throw 'terraform validate failed.' }
- task: PublishPipelineArtifact@1
 - displayName: Publish bounded validation evidence
 - inputs:
 - targetPath: '\$(Build.ArtifactStagingDirectory)/security-evidence'

```

        artifact: 'security-evidence-$(Build.BuildId)'
        publishLocation: pipeline

- stage: Admit
  dependsOn: Verify
  condition: >-
    and(succeeded(), eq(variables['Build.SourceBranch'], 'refs/heads/main'))
  lockBehavior: sequential
  jobs:
    - deployment: VerifyFederatedBoundary
      pool:
        vmImage: ubuntu-24.04
      environment: platform-security-admission
      strategy:
        runOnce:
          deploy:
            steps:
              - checkout: none

    - task: AzureCLI@2
      displayName: Verify workload-federated context
      inputs:
        azureSubscription: sc-platform-admission-wif
        scriptType: pscore
        scriptLocation: inlineScript
        powerShellErrorActionPreference: stop
        addSpnToEnvironment: false
        useGlobalConfig: false
        failOnStandardError: false
        powerShellIgnoreLASTEXITCODE: false
        visibleAzLogin: false
        inlineScript: |
          $account = az account show `
            --query '{tenantId:tenantId,subscriptionId:id}' `
            --output json --only-show-errors |
            ConvertFrom-Json -Depth 10
          if ($LASTEXITCODE -ne 0) {
            throw 'Azure context query failed.'
          }
          if ($account.tenantId -cne $env:EXPECTED_TENANT_ID -or
              $account.subscriptionId -cne `
                $env:EXPECTED_SUBSCRIPTION_ID) {
            throw 'Azure context is outside the approved boundary.'
          }
      env:
        EXPECTED_TENANT_ID: $(expectedTenantId)
        EXPECTED_SUBSCRIPTION_ID: $(expectedSubscriptionId)

```

The full fixture emits another JSON file with match booleans and proof limits. It omits tenant and subscription values. Hidden sign-in output reduces disclosure. Logs and generated content still need review.

The context check has limited authority. It can stop a connection to the wrong tenant or subscription. It cannot reveal inherited roles, deny assignments, data-plane rights, or later privilege changes. A match is not a permission proof.

The machine-readable contract binds selected files by SHA-256. It rejects floating tool and module versions. It also records mutable inputs. Further rules reject saved credentials, token variables, service-principal material, state names, and Terraform change commands.

Tests must challenge those rules. The fixture changes checkout settings, identity inputs, environments, locks, dependencies, evidence links, and learning status. Each unsafe change must fail. Only then does a success result carry useful meaning.

Only `delivery-evidence.json` and `azure-boundary.json` may be published. Plans, state, backup state, variable files, CLI profiles, and broad directories are denied. Plan and state data can expose sensitive values. Marking an output sensitive does not remove that risk.

An allowlist controls names, not content. Build each JSON field from known values and refuse overwrite. Review its schema, size, retention, and access path. Complete that review before the artifact becomes production evidence.

This rule does not replace backend protection. The backend still needs access control, encryption, locks, retention, recovery tests, and activity evidence. Admission simply avoids another sensitive copy. State recovery remains a separate procedure.

A dependency decision needs useful context. Show old and new versions, hashes, signer facts, change purpose, tests, and mutable execution inputs. A matching hash proves identity against one trust source. It does not prove approval or safe behavior.

Evidence must survive a failed run. Preserve the source commit, reviewed contract, dependency manifest, external checks, run record, and decision identities. Richer logs need legal, privacy, retention, and access rules. Collection should never become uncontrolled disclosure.

Preservation needs stronger custody than normal build retention. Record an evidence owner, hashes, access history, and purpose. A stored hash can reveal later change. It cannot attest who created the original record.

Incident review asks how system conditions combined. It should not force one root cause. It should never encode a person as blame. Observed and inferred conditions stay distinct, and each claim cites retained evidence.

Timelines need the same care. A pipeline event can cite its run record. Possible excess scope stays inferred until an access test confirms it. Urgency must not turn uncertainty into permanent fact.

Containment and closure do not prove improvement. Each condition needs owned work and a due date. The item also needs a verification. A counter-signal must be able to challenge success. Severity can order response, but it cannot replace evidence. A high label may speed containment and senior review. It should not raise confidence in an inference. It must not waive the learning test.

The tested record uses five targets. Code separates version, hash, and signer evidence. Policy protects checks, and pipeline work exposes mutability. Identity work narrows access, while tests exercise the whole path.

```
JSON
{
  "learning_items": [
    {
      "id": "learn-code-lock-review",
      "condition_ids": ["cond-dependency-review"],
      "change_target": "code",
      "owner_role": "platform module maintainer",
      "due_date": "2026-07-31",
      "change": "Split provider version, checksum, and signer review evidence.",
      "verification": "A synthetic signer change fails admission until separately approved.",
      "counter_signal": "The test passes because signer evidence is absent.",
      "status": "proposed"
    },
    {
      "id": "learn-policy-required-checks",
      "condition_ids": ["cond-approval-context"],
      "change_target": "policy",
      "owner_role": "delivery governance owner",
      "due_date": "2026-08-07",
      "change": "Require branch control, nonself approval, and sequential lock checks.",
      "verification": "A protected-resource export shows all required checks enabled.",
      "counter_signal": "An administrator bypass remains unexplained or unreviewed.",
      "status": "proposed"
    },
    {
      "id": "learn-pipeline-mutability",
      "condition_ids": ["cond-task-mutation", "cond-artifact-boundary"],
      "change_target": "pipeline",
      "owner_role": "pipeline security owner",
      "due_date": "2026-07-24",
      "change": "Capture task and hosted-image mutability in every admission record.",
      "verification": "The evidence names mutable dependencies and the observed image version.",
      "counter_signal": "The record still labels a major task pin as immutable.",
      "status": "proposed"
    },
    {
      "id": "learn-identity-scope",
      "condition_ids": ["cond-identity-scope"],
      "change_target": "identity",
      "owner_role": "platform identity owner",
      "due_date": "2026-07-24",
      "change": "Replace the admission connection with a read-only scoped identity.",
      "verification": "Allowed reads succeed and a representative write is denied.",
      "counter_signal": "Role inheritance or a deny exception changes effective access.",
      "status": "proposed"
    },
    {
      "id": "learn-test-release-admission",
      "condition_ids": [
        "cond-dependency-review",
        "cond-task-mutation",
        "cond-identity-scope",
        "cond-approval-context",
        "cond-artifact-boundary"
      ],
      "change_target": "test",
      "owner_role": "security validation owner",
      "due_date": "2026-08-14",
      "change": "Add a release-admission exercise covering all five conditions.",
      "verification": "The exercise blocks release and creates bounded evidence for each condition.",
    }
  ]
}
```



```
    "counter_signal": "The exercise depends on production authority or exposes plan  
data.",  
    "status": "proposed"  
  }  
]  
}
```

Every example item remains **proposed**. Local validation proves contract shape and links only. It does not prove backlog intake, owner action, completed tests, or lower risk. Those claims need later evidence.

Incident closure and learning closure must stay separate. Containment may close the event while improvement work remains open. A failed test, invalid owner, missed date, or repeated condition should reopen the decision. The closed incident remains part of its history.

The loop crosses team boundaries. Responders preserve facts and uncertainty. Platform, identity, pipeline, test, and governance owners change their part of the system. They also retain the result.

Repeated conditions should alter the work system. Another signer-review failure may justify a required template or isolated agent pool. Another approval bypass may need stronger separation. Administrator exceptions then need their own review.

Review dates keep the mechanism active between incidents. Owners examine overdue items, failed checks, stale dependencies, and recurring conditions together. Incident work can then change delivery design. It no longer ends as a closed archive.

This closes the Secure chapter without calling the platform secure. Baselines, identity limits, posture signals, delivery admission, and incident learning now produce linked evidence. The next chapter organizes the people and capabilities that keep those loops alive.

Chapter 10: Manage — Operate and Improve the Platform

Chapter 10.1: Organize Operations with RAMP

A platform landing zone becomes operational when responsibility survives handoffs and disruption. Deployed resources cannot provide that continuity. People need a common decision model. It must show who acts, what evidence they use, and why the work matters.

The Manage methodology is larger than maintenance. It keeps governance, security, reliability, performance, and cost tied to business goals. This work continues after deployment. It also creates evidence for improvement.

An operations baseline supplies common visibility, compliance, and recovery foundations. Shared platforms may need deeper investment because many workloads depend on them. Workload-specific operation stays with the application team. That team understands its outcome and context.

The Cloud Adoption Framework organizes management through RAMP: Ready, Administer, Monitor, and Protect. Ready establishes the operating foundation. The other stages control the estate, create awareness, and prepare for disruption. Together they structure ongoing work.

RAMP Ready is not the separate CAF Ready methodology. CAF Ready prepares Azure environments and landing zones for adoption. RAMP Ready prepares ownership, support, skills, and procedures. Those elements keep an existing environment operating.

RAMP is not a sequence that ends. Administration changes what monitoring observes. Monitoring exposes protection gaps. Incidents then change procedures, skills, and service boundaries.

An operational capability is a repeatable ability to produce a useful result. It joins an owner, trigger, activity, evidence, output, escalation route, and counter-signal. A team label is not enough. Neither is a product license.

Service ownership gives capabilities a stable home. A platform service needs a purpose, consumers, scope, exclusions, support route, and one owner. That owner holds decision rights. Several teams may still perform the work.

Criticality shapes the service commitment. Owners need the business impact of delay or failure. Only then can they choose coverage and recovery needs. A technical priority alone can protect too much or too little.

The capstone uses shared management. Central owners operate services used across landing zones. Workload owners retain application resources. They also retain application monitoring, security, performance, cost, and recovery.

This split avoids two failures. Full central control can turn the platform team into an application queue. Full local control can duplicate shared tools. It can also leave common dependencies without an owner.

The map names four shared services. Identity covers platform access and shared identity dependencies. Connectivity covers routing and name resolution. Management supplies telemetry and evidence, while Delivery owns platform code and release paths.

These services are not subscription inventories. Identity, Connectivity, and Management subscriptions locate many shared resources. Ownership also follows code and external settings. Dependencies and support routes can sit outside a subscription.

Workload resources stay outside the map. The workload service owner still appears at each boundary. Every handoff states the platform offer and workload duty. It also names evidence and escalation.

The cloud provider boundary remains visible. Microsoft operates physical facilities, hardware, and parts of each managed service. Platform teams own the customer duties left by the service model. The map must not assign provider work internally.

CAF guidance divides central and workload responsibility across nine areas. They include compliance, security, resource management, deployment, and development. Monitoring, cost, reliability, and performance complete the set. The map assigns both sides.

Accountability is more than a task list. A platform owner decides within the shared boundary and owns escalation. A workload owner decides inside the application boundary. A business owner approves outcomes and material tradeoffs.

Both sides can be responsible without doing the same work. The platform may route a common alert. The workload team interprets application impact. Handoff evidence stops this split becoming an argument during an event.

Access is separate from accountability. A role name grants no Azure rights. A live map must connect the role to a person and identity path. Separation and review rules still apply.

Daily activity needs an outcome path. Capabilities link to reliable foundations, controlled change, timely decisions, and recoverable services. Each outcome names evidence. Each also states a proof limit.

These statements are not service-level objectives. Every target remains `requires_business_approval`. Accountable stakeholders must approve tolerances and clocks. They must also approve populations, exclusions, and evidence rules.

Counter-signals test weak success claims. Fast triage can miss harmful conditions. A release can lack evidence. A recovery exercise can omit a critical dependency.

Decision rights need matching escalation. Service owners should resolve normal work within approved limits. Cross-service risk moves to the business owner. So do disputed ownership and tolerance changes.

RAMP Ready contains three local capabilities. They review service ownership, support routes, and skills or guides. The goal is simple. Expose unowned decisions before pressure arrives.

Support coverage follows approved commitments. A critical shared path might need an on-call rotation. Another service might use business-hours support. Consumers still need a clear route in either case.

Skills form part of capacity. A critical action that depends on one engineer creates a bottleneck. Safe practice can reduce that risk. So can usable runbooks and verified access.

Administer covers work that changes or maintains shared services. Identity, connectivity, management evidence, and delivery retain service owners. Their paths need approval and observed results. They also need consumer impact evidence.

Chapter 10.2 keeps the detailed mechanics. Inventory, patching, drift, access, configuration, releases, and emergency changes need distinct controls. This map first gives each path an owner. It also gives the path an outcome and escalation route.

Monitor turns observations into decisions. Health, change, compliance, security, cost, capacity, demand, and evidence gaps enter owned queues. A missing source stays unknown. It never becomes a healthy or zero value.

Chapter 10.3 will design telemetry and health. This section assigns owners to shared signals and decisions. Workload teams retain application meaning. They also retain workload-specific alerts.

Protect joins reliability and security without merging them. It covers response authority, evidence, continuity, recovery, exercises, and learning. Security operations leads security response. Platform operations coordinates shared-service continuity.

Chapter 10.4 keeps recovery objectives and exercise design. This map establishes owners and handoffs only. A component test cannot speak for omitted consumers. It cannot cover omitted dependencies either.

Cadence gives the map working time. Daily triage routes current events and gaps. Weekly review joins health, changes, incidents, cost, capacity, and improvements. Quarterly review tests ownership and investment.

Each review needs a decision record. Attendance and dashboards are inputs only. The output names an owner and action. It also names the evidence need and unresolved tradeoff.

These frequencies are examples, not CAF defaults. A live team should consider criticality, change rate, evidence delay, and decision cost. Event triggers remain necessary. Calendars cannot predict every failure.

The tested JSON shows the compact map. It preserves the central-workload split and the capstone scope. Four examples show the contract used by all twelve capabilities. The full record carries the omitted detail.

```
JSON
{
  "schema_version": "1.0",
  "map_id": "plz-ramp-example",
  "scope": {
    "service_boundary": "platform_landing_zone_shared_services",
    "operating_model": "shared_management",
    "platform_subscriptions": [
      "identity",
      "connectivity",
      "management"
    ],
    "workload_resources_in_scope": false,
    "example_data": true
  },
  "ramp_model": {
    "ordered_stages": [
      "ready",
      "administer",
      "monitor",
      "protect"
    ],
    "definitions": {
      "ready": "Establish the operating model, service boundaries, ownership, support, skills, procedures, and review routes.",
      "administer": "Control routine and event-driven changes to shared platform services and their approved configuration.",
      "monitor": "Observe shared-service health, change, compliance, cost, capacity, and security signals and route decisions.",
      "protect": "Prepare for, respond to, recover from, and learn from reliability and security disruption."
    },
    "ready_is_caf_ready_methodology": false,
    "is_org_chart": false,
    "is_product_selection": false
  },
  "service_ownership": [
    {
      "service_id": "identity_shared_service",
      "service_owner_role_id": "identity_platform_owner",
      "lifecycle_status": "designed_not_proven"
    },
    {
      "service_id": "connectivity_shared_service",
      "service_owner_role_id": "connectivity_platform_owner",
      "lifecycle_status": "designed_not_proven"
    },
    {

```

```

    "service_id": "management_evidence_service",
    "service_owner_role_id": "observability_platform_owner",
    "lifecycle_status": "designed_not_proven"
  },
  {
    "service_id": "platform_delivery_service",
    "service_owner_role_id": "platform_delivery_owner",
    "lifecycle_status": "designed_not_proven"
  }
],
"responsibility_boundaries": [
  {
    "management_area": "compliance",
    "platform_owner_role_id": "platform_operations_owner",
    "workload_owner_role_id": "workload_service_owner"
  },
  {
    "management_area": "security",
    "platform_owner_role_id": "security_operations_partner",
    "workload_owner_role_id": "workload_service_owner"
  },
  {
    "management_area": "resource_management",
    "platform_owner_role_id": "platform_operations_owner",
    "workload_owner_role_id": "workload_service_owner"
  },
  {
    "management_area": "deployment",
    "platform_owner_role_id": "platform_delivery_owner",
    "workload_owner_role_id": "workload_service_owner"
  },
  {
    "management_area": "development",
    "platform_owner_role_id": "platform_delivery_owner",
    "workload_owner_role_id": "workload_service_owner"
  },
  {
    "management_area": "monitoring",
    "platform_owner_role_id": "observability_platform_owner",
    "workload_owner_role_id": "workload_service_owner"
  },
  {
    "management_area": "cost",
    "platform_owner_role_id": "platform_operations_owner",
    "workload_owner_role_id": "workload_service_owner"
  },
  {
    "management_area": "reliability",
    "platform_owner_role_id": "platform_operations_owner",
    "workload_owner_role_id": "workload_service_owner"
  },
  {
    "management_area": "performance",
    "platform_owner_role_id": "platform_operations_owner",
    "workload_owner_role_id": "workload_service_owner"
  }
],
"capability_counts": {
  "ready": 3,
  "administer": 3,
  "monitor": 3,
  "protect": 3
},
"capability_examples": [
  {
    "capability_id": "ready-service-ownership",
    "ramp_stage": "ready",
    "service_ids": [
      "identity_shared_service",
      "connectivity_shared_service",
      "management_evidence_service",

```

```

        "platform_delivery_service"
    ],
    "owner_role_id": "platform_operations_owner",
    "trigger": "quarterly and after a service-boundary change",
    "output": "approved platform service ownership map",
    "business_outcome_id": "timely_operational_decisions",
    "escalation_route": "business_service_owner",
    "counter_signal": "A shared service or decision has no current owner.",
    "status": "designed_not_proven"
},
{
    "capability_id": "administer-management-and-delivery",
    "ramp_stage": "administer",
    "service_ids": [
        "management_evidence_service",
        "platform_delivery_service"
    ],
    "owner_role_id": "platform_delivery_owner",
    "trigger": "approved platform release or evidence-path change",
    "output": "reviewed platform release with evidence-boundary status",
    "business_outcome_id": "controlled_platform_change",
    "escalation_route": "platform_operations_owner",
    "counter_signal": "A release succeeds while required platform evidence is absent or
stale.",
    "status": "designed_not_proven"
},
{
    "capability_id": "monitor-shared-service-health",
    "ramp_stage": "monitor",
    "service_ids": [
        "identity_shared_service",
        "connectivity_shared_service",
        "management_evidence_service",
        "platform_delivery_service"
    ],
    "owner_role_id": "observability_platform_owner",
    "trigger": "continuous collection with daily triage",
    "output": "owned platform health queue with unknown states visible",
    "business_outcome_id": "timely_operational_decisions",
    "escalation_route": "platform_operations_owner",
    "counter_signal": "A missing source appears as a healthy or zero state.",
    "status": "designed_not_proven"
},
{
    "capability_id": "protect-platform-continuity",
    "ramp_stage": "protect",
    "service_ids": [
        "identity_shared_service",
        "connectivity_shared_service",
        "management_evidence_service",
        "platform_delivery_service"
    ],
    "owner_role_id": "platform_operations_owner",
    "trigger": "reliability event or approved exercise",
    "output": "time-ordered recovery decision and gap record",
    "business_outcome_id": "recoverable_shared_services",
    "escalation_route": "business_service_owner",
    "counter_signal": "A component recovery test omits a critical consumer or
dependency.",
    "status": "designed_not_proven"
}
],
"operating_cadences": [
    {
        "cadence_id": "daily-platform-triage",
        "frequency": "business_daily",
        "owner_role_id": "platform_operations_owner",
        "output": "owned daily decision queue",
        "proof_limit": "A completed triage does not prove that every harmful condition
produced a signal."
    }
],

```

```

{
  "cadence_id": "weekly-operations-review",
  "frequency": "weekly",
  "owner_role_id": "platform_operations_owner",
  "output": "reviewed cross-service action record",
  "proof_limit": "Meeting attendance does not prove decision quality or completed
action."
},
{
  "cadence_id": "quarterly-responsibility-review",
  "frequency": "quarterly",
  "owner_role_id": "business_service_owner",
  "output": "approved responsibility and capability revision",
  "proof_limit": "An approved map does not prove that named capabilities operate
effectively."
}
],
"proof_limits": [
  "This synthetic map does not prove that named teams, skills, support coverage, tools,
or procedures exist.",
  "A responsibility assignment does not grant Azure permission or prove that effective
access is least privilege.",
  "A capability marked designed_not_proven has no claim of operational effectiveness.",
  "Business targets remain unapproved until accountable stakeholders set tolerances and
evidence rules.",
  "Platform evidence cannot prove workload behavior outside the declared shared-service
boundary."
]
}

```

The full record contains eight roles and four business outcomes. It also keeps activities, supporting roles, evidence, handoff content, and reopen signals. The validator proves that this summary comes from that record. No manual summary claim can drift unnoticed.

Every service and capability remains `designed_not_proven`. The label records a design without claiming live operation. Route tests and access checks are still required. So are exercises and repeated evidence.

The map can guide an operating-model workshop. Participants begin with services and consumers. They then resolve each boundary. Owners come after decision rights, evidence, escalation, and exclusions are clear.

Blank fields become visible work. An unknown owner is an accountability gap. An untested route is a readiness gap. An unavailable source is a monitoring gap.

Open gaps need one owned backlog. Otherwise, each RAMP stage can create work that vanishes at its boundary. Shared review exposes stalled actions and overloaded owners. It can also reveal repeated handoff failure.

The map should not copy an organization chart. Teams and vendors change over time. Stable service and capability IDs preserve history. Responsibility can move without erasing evidence. It should not become a tool catalog either. Azure services and work systems can support capabilities. Tools cannot approve business tolerance. They cannot decide whether evidence deserves action.

Operating reviews expose effects that one service cannot see. A network saving may increase workload delay. A stricter identity path may lower risk and raise support demand. Fast delivery can also raise review load.

Escalation is therefore part of design. Business owners resolve tradeoffs across services or tolerances. Platform owners must not optimize a local measure by hiding impact. Cost and risk cannot be moved silently to workload teams.

The artifact supplies structure, not certainty. It cannot prove skills, useful alerts, safe changes, or recovery. Later operating evidence must support those claims. The map only makes them testable.

The key question is not whether every box has a team label. It is whether a business condition can reach an accountable decision. The decision must return as verified action. RAMP makes that path visible.

Chapter 10: Manage — Operate and Improve the Platform

Chapter 10.2: Administer Through Controlled Change

Routine work is where an operating model becomes real. That work needs a controlled path. Otherwise, portal edits, private notes, and urgent exceptions outrun the ownership record.

Administration is repeated work that keeps an approved service usable. It includes inventory, access, configuration, patching, drift triage, and documentation. Every activity needs a trigger, owner, evidence, and result.

Not every recurring task is an engineering change. Access review or patch review can follow a proven procedure. A new role design, update strategy, or network topology changes the platform. It belongs in the engineering delivery path.

This distinction protects both paths. A small request should not wait for a redesign project. A structural decision should not hide inside a routine service ticket.

The service model sets the boundary. Azure operates physical infrastructure and parts of each managed service. The platform team owns the customer duties that remain for shared services. Those duties differ across infrastructure, platform, and software services. A virtual machine creates guest operating-system work. A managed platform service might absorb that work. Ownership must follow the service contract, not a generic checklist.

Controlled change begins with a known estate. Inventory supplies IDs, types, locations, grouping, and ownership markers. Its approved scope matters. Inventory does not prove every configuration field is correct.

Inventory also exposes growth. A new resource may be approved, temporary, orphaned, or outside Terraform ownership. Its count and identity start the review. They do not supply the verdict.

Chapter 6.2 built the normalized inventory used here. Collection status and subscription scope remain evidence. A partial or inaccessible collection cannot support a clean result.

The comparison also needs approved intent. Terraform remains authoritative for persistent configuration. A release can publish a small, non-sensitive contract as a named root output.

Other sources still have bounded roles. Azure Policy can report or enforce declared rules. Activity and change records can help explain movement. Neither replaces reviewed Terraform intent for owned resources.

The output is not a state export. It contains only values safe for comparison. Examples include resource IDs, types, locations, groups, and selected tag values. Secrets and unrelated outputs stay outside the artifact.

This limit matters because `terraform output -json` can expose sensitive values in plain text. The delivery pipeline publishes only the reviewed named output. Administration then consumes that protected artifact.

Approval gives the artifact meaning. The record names its release, revision, approval time, and approver role. A workspace copy is not an approved baseline.

Observation comes from a later inventory window. Azure may change while Resource Graph returns pages. The report keeps the timestamp and its not-point-in-time label.

Only declared fields are compared. IDs, types, groups, and locations ignore letter case. Azure APIs can return different casing. The selected `owner` and `managed_by` values use exact comparison because tag values are case-sensitive.

A match means selected values aligned during observation. It does not prove health, access, patch state, policy effect, or full configuration. Omitted resources and fields remain unknown.

A difference also has a narrow meaning. It does not prove an unauthorized edit. Provider behavior or an emergency action can cause it. Stale evidence, delay, or planned change can do the same.

A missing record is narrower still. The resource might be deleted or hidden by access. It might also be absent from the index. The workflow labels it `missing_from_observation`, not confirmed deletion.

The PowerShell workflow enforces those boundaries. It rejects sensitive output and incomplete collection. It also rejects missing scope and duplicate IDs. It produces evidence but contains no repair command.

```
POWERSHELL
[CmdletBinding()]
param(
    [Parameter(Mandatory)]
    [string] $ApprovedOutputFile,

    [Parameter(Mandatory)]
    [string] $ObservedInventoryFile,

    [Parameter(Mandatory)]
    [string] $ReportFile
)

Set-StrictMode -Version Latest
$ErrorActionPreference = 'Stop'

$approved = Get-Content -Raw -LiteralPath $ApprovedOutputFile |
    ConvertFrom-Json -Depth 50 -DateKind String
$observed = Get-Content -Raw -LiteralPath $ObservedInventoryFile |
```

```
ConvertFrom-Json -Depth 50 -DateKind String
```

```
if ([bool] $approved.sensitive) {
    throw 'A sensitive Terraform output cannot enter this evidence path.'
}
if ($observed.inventory_schema -cne $approved.scope.inventory_schema) {
    throw 'The observed inventory schema does not match the contract.'
}
if ($observed.collection_status -cne
    $approved.scope.required_collection_status) {
    throw 'The observed inventory is not a complete collection.'
}

$requiredScope = @(
    $approved.scope.subscriptions |
        ForEach-Object { ([string] $_).ToLowerInvariant() }
)
$observedScope = @(
    $observed.scope |
        ForEach-Object { ([string] $_).ToLowerInvariant() }
)
if (@($requiredScope | Where-Object { $_ -notin $observedScope }).Count) {
    throw 'The observed inventory does not cover every approved subscription.'
}

$observedById = @{}
foreach ($resource in $observed.resources) {
    $id = ([string] $resource.id).TrimEnd('/').ToLowerInvariant()
    if ($observedById.ContainsKey($id)) {
        throw "Observed inventory contains duplicate resource ID '$id'."
    }
    $observedById[$id] = $resource
}

$comparisons = foreach ($expected in $approved.resources) {
    $id = ([string] $expected.resource_id).TrimEnd('/').ToLowerInvariant()
    if (-not $observedById.ContainsKey($id)) {
        [ordered]@{
            resource_id = $id
            service_owner = $expected.service_owner
            status = 'missing_from_observation'
            differences = @([ordered]@{
                field = 'resource_presence'
                approved = 'present'
                observed = 'not_returned'
            })
        }
        continue
    }

    $actual = $observedById[$id]
    $differences = foreach ($field in @(
        'type', 'resource_group', 'location', 'owner', 'managed_by'
    )) {
        $expectedValue = [string] $expected.approved_values.$field
        $actualValue = [string] $actual.$field
        $comparison = if ($field -in @('owner', 'managed_by')) {
            [StringComparison]::Ordinal
        }
        else {
            [StringComparison]::OrdinalIgnoreCase
        }
        if (-not [string]::Equals(
            $expectedValue, $actualValue, $comparison
        )) {
            [ordered]@{
                field = $field
                approved = $expectedValue
                observed = $actualValue
            }
        }
    }
}
```

```

    }
  }
}

[ordered]@{
  resource_id = $id
  service_owner = $expected.service_owner
  status = if (@($differences).Count) {
    'difference_requires_review'
  }
  else {
    'aligned_selected_values'
  }
  differences = @($differences)
}
}

$counts = [ordered]@{
  approved = @($approved.resources).Count
  aligned = @($comparisons | Where-Object {
    $_.status -ceq 'aligned_selected_values'
  }).Count
  different = @($comparisons | Where-Object {
    $_.status -ceq 'difference_requires_review'
  }).Count
  missing = @($comparisons | Where-Object {
    $_.status -ceq 'missing_from_observation'
  }).Count
}
$overallStatus = if ($counts.missing) {
  'incomplete_selected_observation'
}
elseif ($counts.different) {
  'review_required'
}
else {
  'aligned_selected_values'
}

$report = [ordered]@{
  schema_version = '1.0'
  compared_utc = [DateTime]::UtcNow.ToString('yyyy-MM-ddTHH:mm:ssZ')
  overall_status = $overallStatus
  approved_release = $approved.release
  observation = [ordered]@{
    completed_utc = $observed.completed_utc
    consistency = $observed.consistency
  }
  counts = $counts
  comparisons = @($comparisons)
  decision_contract = [ordered]@{
    automatic_change_allowed = $false
    difference_is_unauthorized_change = $false
    missing_is_confirmed_deletion = $false
    next_action = if ($overallStatus -ceq 'aligned_selected_values') {
      'retain evidence and continue normal review'
    }
    else {
      'assign an owner and classify before any corrective change'
    }
  }
}

if (Test-Path -LiteralPath $ReportFile) {
  throw 'The report file already exists.'
}
[System.IO.File]::WriteAllText(
  [System.IO.Path]::GetFullPath($ReportFile),
  ($report | ConvertTo-Json -Depth 50),
  [System.Text.UTF8Encoding]::new($false)
)

```

l)

The sample contract declares three shared resources. They span the connectivity and management subscriptions. The matching fixture returns `aligned_selected_values`. This synthetic result proves only comparison behavior.

Any difference enters triage with its service owner. The owner classifies cause, impact, authorization, and response. The report never chooses that response.

Chapter 5.5 defined refresh-only plans and Terraform drift response. This workflow does not replace them. It signals when a controlled investigation may be useful.

The next step depends on classification. An unauthorized change returns through reviewed code and a new plan. An emergency action must be codified. A stale contract needs approved release evidence.

No difference should trigger an automatic apply. The live value may be correct. Automatic repair could overwrite an incident action, migration, or unfinished release.

Change requests make the decision visible. A ticket, pull request, or form can carry the record. It names type, requester, environment, scope, reason, owner, risk, and evidence. Routine service requests use a separate path.

Risk changes the approval route. High-impact work needs stronger review and active monitoring. It also needs tested rollback and gradual exposure where possible. Lower-risk work earns automation only after its limits are proven.

Deployment is one part of the record. Post-deployment validation compares the approved result with fresh observations. Failure follows the declared rollback or recovery path.

Emergency changes shorten time, not accountability. When conditions allow, the record starts before action. It names authority, scope, reason, effect, and expiry. Reconciliation to code and normal access begins when the service is stable.

Access administration follows the same pattern. A request names task, scope, role, approver, identity, duration, and evidence. Exceptional work should use time-bound access instead of standing privilege.

An approved ticket grants no permission by itself. The identity path must still be correct. Granted, expired, and unused access need review in the owned queue.

Recurring access work leaves its own evidence. The record keeps request, approval, activation, expiry, and removal states separate. A completed request does not prove access ended on time. Patching begins with an inventory of customer-managed operating systems. The capstone does not assume a platform virtual machine exists. No machine means no guest-patch claim, not automatic compliance.

When managed machines exist, the team defines assessment and orchestration. It also defines windows, restarts, health checks, and rollback. Azure Update Manager can assess and schedule updates for supported machines. The owner still decides scope and acceptable disruption.

Patch success needs more than an installed count. A service may fail after a clean installation.

Health checks and consumer impact complete the record. Missing results stay unknown.

Platform services move patch work across the provider boundary. Customers may still own versions, maintenance settings, tests, or upgrades. The record names each remaining duty. It does not mark the whole service patched.

Operational documentation keeps these paths repeatable. A procedure defines policy and decisions. A runbook gives exact steps for one bounded task.

Documentation changes deserve review like code. A pull request can join the guide, script, test, and expected evidence. The release then carries one operating contract.

Each runbook needs a trigger, scope check, access, steps, evidence, and stop rules. It also needs validation, rollback, an owner, and a review date. Keep it beside the code it operates. A detached guide can stay readable after it becomes unsafe.

Documentation review is recurring administration. Releases, incidents, retired services, access changes, and failed exercises can reopen a guide. Stale guidance becomes an owned gap instead of tribal knowledge.

Cadence follows the rate and impact of change. A critical service may need checks after every release. A stable component can use a longer interval if event-driven checks still catch emergency work.

Each run keeps collection and approval times. Fresh inventory against an old contract may expose stale intent. Reviewers need both clocks before correcting either source.

The queue separates maintenance debt from platform improvement. An expired guide, access delay, or missed patch window needs ownership. A repeated pattern can become engineering work with a testable result.

This split closes a common loophole. Operations cannot defer every weakness to engineering. Engineering cannot dismiss repeated manual effort as normal. Evidence routes the issue to the path that can change its cause.

The administration queue now has three valid outcomes. A selected comparison can align, require classification, or remain incomplete. None of those outcomes changes Azure.

That is the anchor for controlled administration: observe a declared scope, compare it with approved intent, assign the decision, and return every accepted change to code and evidence.

Chapter 10: Manage — Operate and Improve the Platform

Chapter 10.3: Monitor the Platform as a System

Platform monitoring begins with behavior, not a dashboard. A landing zone supports workloads, controlled change, timely decisions, and recovery. Its signals should show whether those outcomes remain possible.

Many charts can miss that purpose. CPU, request counts, and event totals describe parts. They do not explain what the platform owes its consumers. A useful design starts with an outcome, names its signals, and records the next action.

This approach also limits scope. Chapter 6.4 checked whether required configuration was declared. It did not prove that logs arrived. Chapter 9.4 handled security detections and their response path. This section watches the shared platform as an operating system.

The outcome contract from Chapter 10.1 supplies the starting point. Reliable workload foundations need service-health evidence. Controlled platform change needs control-plane results. Timely decisions need fresh telemetry. Recoverable shared services need owned alerts and confirmed recovery.

Each outcome becomes a health entity. A health entity is the smallest platform capability with a state and owner. Examples include the change path, shared services, monitoring evidence, and shared-service recovery.

Signals explain why an entity has a state. A signal contract names source, scope, time window, freshness, owner, proof, and counter-signal. The last item matters because missing telemetry can look calm.

Coverage and freshness answer different questions. Coverage asks whether every required subscription contributes evidence. Freshness asks whether the newest usable evidence fits its window. Complete but stale events fail the second test. One current subscription can still fail the first.

Azure Monitor brings the main sources together. The Azure Activity Log records subscription control-plane events. Administrative and Policy events explain platform change. ResourceHealth and ServiceHealth help describe provider and resource conditions.

The Activity Log differs from resource logs. Azure provides it for control-plane activity. Resource logs depend on each service. The Activity Log gives this section a consistent subscription view, but it does not cover workload internals.

Built-in retention supports recent investigation. A central record still needs deliberate routing. A diagnostic setting exports selected categories to a Log Analytics workspace. Every platform subscription receives one setting.

The Terraform resource below declares that routing. Its map must contain identity, connectivity, and management subscription IDs. Variable validation requires the change and health categories used here.

```
HCL / TERRAFORM
resource "azurerm_monitor_diagnostic_setting" "platform_activity" {
  for_each = var.platform_subscription_ids

  name                       = var.diagnostic_setting_name
  target_resource_id         = "/subscriptions/${each.value}"
  log_analytics_workspace_id = var.log_analytics_workspace_resource_id

  dynamic "enabled_log" {
    for_each = var.activity_log_categories

    content {
      category = enabled_log.value
    }
  }
}
```

This declaration proves intended routing only. It does not prove ingestion, freshness, permission, or query success. The setting can exist while the workspace stays silent or inaccessible.

Validation reads both ends. Azure CLI first confirms the expected tenant and management subscription. It then reads the named setting from every required subscription. Each result must use the approved workspace and enable Administrative, Policy, ResourceHealth, and ServiceHealth.

The same run sends bounded queries to the workspace. The query command receives the workspace customer ID. Setting validation receives its resource ID. Keeping them distinct prevents a common integration error.

The example calls a tested collector. Protected execution supplies real IDs. Queries stay in reviewed files, and collection time becomes evidence.

```
POWERSHELL
Set-StrictMode -Version Latest
$ErrorActionPreference = 'Stop'
. "$PSScriptRoot/Get-PlatformMonitoringEvidence.ps1"

$changeQuery = Get-Content -Raw "$PSScriptRoot/platform-change-signals.kql"
$healthQuery = Get-Content -Raw "$PSScriptRoot/platform-health-signals.kql"

$parameters = @{
  ExpectedTenantId = '<tenant-id>'
  ManagementSubscriptionId = '<management-subscription-id>'
  PlatformSubscriptionId = @(
    '<identity-subscription-id>'
    '<connectivity-subscription-id>'
  )
}
```

```

    '<management-subscription-id>'
  )
  WorkspaceCustomerId = '<workspace-customer-id>'
  WorkspaceResourceId = '<workspace-resource-id>'
  DiagnosticSettingName = 'plz-activity-to-central'
  ChangeQuery = $changeQuery
  HealthQuery = $healthQuery
  Timespan = 'P1D'
  CollectedUtc = [DateTime]::UtcNow.ToString('yyyy-MM-ddTHH:mm:ssZ')
}

Get-PlatformMonitoringEvidence @parameters |
  ConvertTo-Json -Depth 50

```

The collector is read-only. Its Azure CLI calls show context, read settings, and run queries. It has no create, update, delete, apply, or repair path.

Query safety belongs in the contract. Both queries begin with `AzureActivity`, include a relative time bound, and use Activity Log categories. The collector rejects external data, `evaluate`, and management operators. This guard keeps collection narrow.

The change query groups successful and failed Administrative and Policy events. It reports event, failure, and affected-resource counts. It also reports the newest event for each time bucket and subscription.

```

KUSTO
AzureActivity
| where TimeGenerated >= ago(1d)
| where CategoryValue in~ ("Administrative", "Policy")
| where ActivityStatusValue in~ ("Succeeded", "Failed")
| summarize
  EventCount = count(),
  FailedCount = countif(ActivityStatusValue =~ "Failed"),
  AffectedResources = dcount(ResourceId),
  LastEventUtc = max(TimeGenerated)
  by bin(TimeGenerated, 15m), SubscriptionId, CategoryValue
| project
  TimeBucket = TimeGenerated,
  SubscriptionId,
  CategoryValue,
  EventCount,
  FailedCount,
  AffectedResources,
  LastEventUtc
| order by TimeBucket desc

```

A failed operation is not automatically an incident. It can be an expected rejection, caller error, or recovered deployment step. The result enters an owned review. Operation, resource, caller, and change window supply context.

The health query uses `ResourceHealth` and `ServiceHealth`. It separates active and resolved events by subscription and category. This evidence can explain service impact, but it does not measure every consumer path.

```

KUSTO
AzureActivity
| where TimeGenerated >= ago(1d)
| where CategoryValue in~ ("ServiceHealth", "ResourceHealth")
| summarize
  EventCount = count(),
  ActiveCount = countif(ActivityStatusValue =~ "Active"),

```

```

    ResolvedCount = countif(ActivityStatusValue =~ "Resolved"),
    LastEventUtc = max(TimeGenerated)
  by bin(TimeGenerated, 15m), SubscriptionId, CategoryValue
| project
  TimeBucket = TimeGenerated,
  SubscriptionId,
  CategoryValue,
  EventCount,
  ActiveCount,
  ResolvedCount,
  LastEventUtc
| order by TimeBucket desc

```

An empty result has an important limit. No matching event may have occurred. Delay, wrong scope, missing permission, broken export, or query failure can look the same. Empty results never create a healthy state.

The health model handles that uncertainty. Its states are healthy, degraded, unhealthy, and unknown. Healthy means every required signal is current, complete, and within approved limits. Degraded and unhealthy mean complete evidence crossed warning or critical limits.

Unknown has first precedence. A failed query, stale signal, or omitted subscription makes the entity unknown. A parent cannot become healthy while a required child is unknown. Telemetry failure cannot hide platform risk.

Rollup rules preserve meaning across levels. One damaged shared service can affect several outcomes. A parent shows the strongest state while retaining the affected child and signal. Operators need a cause, not only a color.

State changes need stability. One successful sample should not close a sustained problem. Recovery requires approved successful windows. A manual override records reason, approver, evidence, and expiry.

Alerts turn state into work. An actionable alert reaches a named owner with enough context for a defined first step. A threshold without ownership is only a notification rule.

The alert contract names signal, condition, severity, owner, first action, scope, deduplication, suppression, and resolution. It also defines proof and a counter-signal. Business owners approve severity and production thresholds.

A failed-change alert groups by subscription, category, operation, and resource. Its first action is correlation with the approved change window. An active-health alert groups around the provider event and affected scope. A monitoring-evidence alert fires when required telemetry cannot be verified.

The third alert protects monitoring itself. A query error is not an absence of incidents. It is lost evidence. The owner first checks the setting, workspace access, query status, and ingestion timing.

Suppression stays narrow. A change alert can reference an approved maintenance window and expiry. A health alert can suppress duplicates for the same event. Neither rule can hide another resource or an unknown evidence path.

Resolution needs a counter-signal. A resolved provider event is not enough when a shared service still fails its check. A recovered query is not enough until every required subscription supplies current evidence across approved recovery windows.

Notification tests complete another link. A deployed definition does not prove owner receipt. Controlled tests verify the action group, routing, context, deduplication, and recovery behavior. Service-level indicators connect monitoring to outcomes. An SLI is a measured ratio or quantity. A service-level objective adds an approved target over time. Code should not invent that target.

Activity Log export coverage divides subscriptions with approved routing and queryable current evidence by all required platform subscriptions. A configured setting alone stays outside the numerator when evidence cannot be queried.

Platform change success divides succeeded changes by succeeded plus failed changes. An empty denominator is unknown, not one hundred percent. Deployment context stays visible because control-plane operations do not carry equal risk.

Health-event acknowledgement divides impacting events acknowledged within the approved interval by all impacting events. Shared-service recovery pairs provider resolution with a successful service-level health check. These indicators measure response and recovery, not raw alert volume.

SLIs require owners, proof, and counter-signals. They also need cost and retention choices. Wider ingestion and longer retention can aid investigation while raising workspace cost. The contract states the needed history before collection expands.

Dashboards now have a clear job. They display outcome health, evidence freshness, active impact, recent change results, and SLI movement. They do not replace the source contract or health rules.

Views should also show scope and collection time. A current green result for two subscriptions must not look equivalent to a complete result for three. Visible evidence age makes that difference hard to miss.

The stored observation remains modest. It proves tenant context, declared routing, bounded queries, and returned rows at one time. It does not prove health, alert delivery, operator action, or future availability.

That limit makes the system trustworthy. A green panel is not a promise. Platform monitoring explains what is known, what remains unknown, and who acts next.

The platform can now describe its operating state. Which worsening or missing signal should change the next decision before a consumer finds the problem?

Chapter 10: Manage — Operate and Improve the Platform

Chapter 10.4: Protect and Recover the Platform

Recovery begins before anything fails. The platform team names what could be lost. It also names the threatened outcome, recovery authority, and evidence that must survive.

A backup is only one recovery input. It says little about authority, dependency order, access, capacity, or consumer checks. An intact copy can still be the wrong recovery point.

Operational continuity is the wider goal. Business continuity keeps critical work possible during disruption. High availability handles expected faults through design. Disaster recovery prepares for rare and severe failure.

These ideas overlap but are not interchangeable. A highly available service can still need disaster recovery. A recovery plan can fail when identity, network, key, or decision paths are unavailable.

The platform boundary makes those links visible. Shared connectivity, DNS, identity, policy, configuration, Terraform state, logging, and administration form one control system. Recovery restores enough for normal work to resume.

Order matters inside that system. A service cannot use a key that approved identities cannot reach. A route cannot help when its name resolves incorrectly. The runbook records each needed link before action.

This section covers four failure assumptions. Terraform state can become unavailable or untrusted. Approved configuration can be lost. A regional platform dependency can fail. Normal privileged access can become unusable.

The list is not a forecast. It is a test frame. Each mode protects an outcome. It names start signals, an authority path, success, and a counter-signal.

Recovery objectives turn business tolerance into time limits. The recovery time objective, or RTO, defines acceptable downtime. The recovery point objective, or RPO, defines acceptable data loss before disruption.

Neither value comes from Azure or Terraform. Zero can be costly or impossible. Technical and business owners approve each objective. They consider impact, cost, residency, dependencies, and remaining risk.

Different capabilities can need different values. Identity access might have no data RPO. Source configuration and Terraform state can have separate recovery points. A regional service path can involve both time and data.

The example keeps every production objective unapproved. `requires_business_approval` is deliberate. A sample interval cannot become an accidental service promise.

Authority matters as much as time. The incident commander declares the event and owns its clock. The recovery authority selects the path. That role approves the exact production action.

A recovery operator executes it.

Those roles stay separate. An evidence custodian preserves records. A security observer can stop unsafe access or data handling. A service owner accepts the platform and consumer path.

Out-of-band communication is required. The normal collaboration path may share the failure.

Recovery authority must remain reachable without the affected identity, region, or platform service.

Evidence preservation comes before mutation. Logs, alerts, provider notices, configuration references, state metadata, access events, and health observations establish the starting point.

Source and collection times stay separate.

The evidence index records custody. It names the collector, protected location, and access time.

A hash can show changed bytes. It cannot prove that an item is complete or true.

Preservation also limits false certainty. An incident starts with incomplete knowledge. The record separates observations, decisions, assumptions, and unknowns. It does not invent a neat cause early.

The runbook uses one control path across all four modes. It declares and times the incident. It preserves evidence, establishes authority, freezes competing change, and classifies the failure boundary.

Only then does it select a recovery point and path. The team rehearses that path in isolation. A separate decision approves or rejects the production step. Execution uses a service-specific procedure outside this common contract.

Validation reaches past the recovered component. It checks platform health, control paths, data, denied paths, and one consumer journey. A provider-ready signal cannot close the event alone.

Return to normal is also a recovery step. Emergency authority ends. Writers resume under normal control. Configuration returns to code. Monitoring continues through an approved window.

The compact summary projects the tested runbook, state evidence, and exercise plan. Its false decision fields are safety results, not missing work.

JSON

```
{
  "schema_version": "1.0",
  "artifact_type": "platform_recovery_manuscript_summary",
  "scope": "shared-platform-control-plane",
  "objectives": {
```

```

    "recovery_time_objective": "requires_business_approval",
    "recovery_point_objective": "requires_business_approval_or_not_applicable"
  },
  "authority": {
    "activation": [
      "incident-commander",
      "recovery-authority"
    ],
    "selection": "recovery-authority",
    "execution": "recovery-operator",
    "evidence": "evidence-custodian",
    "challenge": "security-observer",
    "service_acceptance": "service-owner",
    "approver_and_operator_must_differ": true
  },
  "failure_modes": [
    {
      "id": "terraform-state-unavailable",
      "protected_outcome": "controlled-platform-change",
      "success": "Reviewed planning resumes from one accepted lineage with explainable
actions.",
      "counter_signal": "A write, unlock, or restore occurs before freeze, backup, review,
and rehearsal."
    },
    {
      "id": "approved-configuration-unavailable",
      "protected_outcome": "controlled-platform-change",
      "success": "A protected revision and its dependencies reproduce an explainable
reviewed plan.",
      "counter_signal": "An unverified workstation copy becomes the new source of truth."
    },
    {
      "id": "regional-platform-dependency-unavailable",
      "protected_outcome": "reliable-workload-foundations",
      "success": "The shared-service path and a consumer path meet approved recovery
objectives.",
      "counter_signal": "A provider recovery notice closes the incident while consumers
still fail."
    },
    {
      "id": "privileged-access-path-unavailable",
      "protected_outcome": "recoverable-shared-services",
      "success": "Bounded emergency work is visible and closes when normal access
returns.",
      "counter_signal": "Emergency access remains active or becomes a normal
administration path."
    }
  ],
  "ordered_control_path": [
    "declare-and-timebox",
    "preserve-source-evidence",
    "establish-recovery-authority",
    "freeze-competing-change",
    "classify-failure-boundary",
    "select-recovery-point-and-path",
    "rehearse-in-isolation",
    "authorize-production-action",
    "execute-scenario-procedure",
    "validate-platform-and-consumer-path",
    "return-to-normal-control",
    "close-evidence-and-learning"
  ],
  "terraform_state_candidate_checks": [
    "nonproduction-only",
    "azurerm-backend",
    "writers-frozen",
    "lock-owner-reviewed",
    "current-snapshot-preserved",
    "lineage-matches",
    "candidate-serial-is-older",
    "sha256-values-well-formed",

```



```

    "selection-and-execution-separated",
    "isolated-rehearsal-required"
  ],
  "exercise_scenarios": [
    {
      "id": "terraform-state-unavailable",
      "mode": "local-simulation"
    },
    {
      "id": "approved-configuration-unavailable",
      "mode": "local-simulation"
    },
    {
      "id": "regional-platform-dependency-unavailable",
      "mode": "tabletop-plus-isolated-nonproduction"
    },
    {
      "id": "privileged-access-path-unavailable",
      "mode": "tabletop-plus-isolated-nonproduction"
    }
  ],
  "decision_contract": {
    "production_recovery_authorized": false,
    "production_state_restore_authorized": false,
    "emergency_access_use_authorized": false,
    "regional_failover_authorized": false,
    "external_execution_performed": false,
    "next_action": "obtain approval for a bounded nonproduction exercise"
  },
  "proof_limits": [
    "A valid runbook does not prove that recovery works.",
    "A candidate state can become eligible for rehearsal without becoming correct or authorized.",
    "A tabletop cannot prove identity sign-in, regional capacity, routing, or consumer recovery.",
    "Production recovery requires current objectives, evidence, authority, and service-specific procedures."
  ]
}

```

Terraform state needs special care. It can contain sensitive values and resource identity. Chapter 5.5 owns pull, push, locking, and backup mechanics. This section decides whether a candidate deserves rehearsal.

The AzureRM backend stores state as an Azure blob. Azure Storage supports its lock and consistency checks. Those controls reduce writer conflicts. They do not choose the correct recovery point.

Blob versioning can preserve earlier values after writes. Soft delete can retain lost objects for a set interval. Both add options and cost. Neither covers every account failure or proves that old state matches Azure.

Candidate review starts by freezing writers and checking lock ownership. The current snapshot is preserved first. Candidate lineage must match, and its serial must be older.

A SHA-256 value detects changed bytes. It does not prove correct meaning. Matching lineage and serial cannot show which Azure resources now exist.

Selection and execution remain separate. The result is `eligible_for_isolated_rehearsal`, never `restore_authorized`. No state write or force-unlock command is present.

Configuration loss needs a similar trust boundary. A workstation clone may be current but unreviewed. Recovery uses a protected revision, dependency locks, release evidence, and accepted trust roots.

Rebuilt source first needs local validation. A clean result does not authorize apply. Normal review resumes when revision, dependencies, backend link, and plan are clear.

Regional disruption widens the dependency question. A hub, DNS service, vault, workspace, or other shared part can be regional. Platform subscriptions do not remove that link. Recovery follows service-specific guidance.

Cross-region operation is not automatic. An alternate path needs deployed dependencies and current configuration. It also needs allowed data placement, capacity, routing, names, keys, and access. Cost follows approved need.

Some services use active-active operation. Others use standby, backup and restore, or recovery after a region returns. Approved outcomes choose the pattern. Two regions are not a universal rule.

Regional recovery validates a consumer journey. A healthy secondary part can still have broken DNS, stale data, denied identity, or low capacity. The service owner tests the needed path.

Privileged-access failure can stop every other path. Normal sign-in might depend on federation, authentication, Conditional Access, PIM, or a network route. The emergency path must avoid all shared failure modes it can.

Microsoft recommends multiple cloud-only emergency accounts for tenant lockout. Their methods and network paths should avoid a shared dependency. Every use is rare, bounded, monitored, and urgent.

Availability alone is not enough. The record needs custody, authority, task scope, expiry, and notification. An exercise checks denied excess action. It also checks closure of special access.

The exercise plan stays synthetic and nonproduction. State and source failures use local simulation. Regional and access modes add a tabletop plus an isolated check.

These layers answer different questions. A tabletop exposes weak roles and missing choices. Local simulation tests parsing and selection rules. An isolated service test can reveal access, routing, timing, and cleanup faults. None proves production recovery.

Every scenario has an inject, three steps, evidence, success, a counter-signal, and cleanup. Global stop rules outrank local success. Production IDs, credentials, routes, data, or external writes stop the exercise.

The script validates all three contracts and writes one timestamped local record. It does not run an exercise, read Azure, or change Terraform state.

POWERSHELL

```
[CmdletBinding()]
param(
    [Parameter(Mandatory)]
    [string] $EvidenceDirectory
)

Set-StrictMode -Version Latest
$ErrorActionPreference = 'Stop'

$parameters = @{
    RunbookFile = "$PSScriptRoot/platform-recovery-runbook.example.json"
    StateEvidenceFile = (
        "$PSScriptRoot/terraform-state-recovery-evidence.example.json"
    )
    ExerciseFile = (
        "$PSScriptRoot/nonproduction-recovery-exercise.example.json"
    )
    EvidenceDirectory = $EvidenceDirectory
}

& "$PSScriptRoot/Invoke-PlatformRecoveryAssessment.ps1" @parameters
```

Its result is readiness for an approved nonproduction exercise. Production recovery, state restore, emergency access, and regional failover remain false. Each needs current evidence and separate authority.

Exercise evidence changes the runbook. Delays can expose an unreachable approver, stale dependency list, low capacity, or unsafe cleanup. Each finding becomes owned work.

Cadence follows change and risk. A major identity, network, backend, region, or operating-model change can trigger another exercise. A calendar is only one trigger.

Recovery completes when normal control returns. Emergency rights close. Writers resume.

Approved configuration governs. Monitoring is current, and consumers can use the service.

A recoverable platform does not expect tidy failure. It preserves sound decisions under pressure, keeps action inside a known boundary, and returns special control to normal engineering.

Chapter 10: Manage — Operate and Improve the Platform

Chapter 10.5: Close the Improvement Loop

Continuous improvement is more than collecting lessons. Evidence must change a decision. Once authorized, that decision must change the system. A later measure must test the result. A platform team already has many feedback streams. Incidents expose contributing conditions. Cost records reveal consumption and operating effort. Capacity checks show headroom or missing coverage. Policy data shows denied work and exceptions. Security reviews reveal exposure and control gaps. Adoption feedback shows where paved paths create friction. Monitoring describes platform health. Recovery exercises test whether protected outcomes can be restored.

Each source has a workflow elsewhere in this book. This section does not replace those workflows. It creates a review contract that brings their decisions together.

That boundary matters. A central review can turn into another data store. It can show unrelated counts without meaning. It can become a backlog meeting led by the loudest voice. None of these forms system learning.

The review starts with a stable evidence envelope. Every signal names its source section and artifact. It includes its window, scope, definition version, and owner. It also states the finding, decision use, proof limit, and counter-signal. The source owner confirms what the signal means. These fields prevent false comparison. A monthly monitoring observation cannot quietly become a quarterly trend. A sample of two subscription groups cannot describe an estate. A changed policy definition starts a new series unless the owner supplies a valid bridge.

Unknown is a valid result. The example capacity source excludes two required subscription groups. Its condition remains `unknown`. It cannot be averaged with healthier sources or colored green because no breach was observed.

The review also avoids an opaque composite score. One number would hide the outcome at risk. It would also hide a changed definition or incomplete source. A score of 82 cannot explain an unreachable recovery authority. Nor can it explain a growing low-risk cost variance.

Instead, the review maps evidence to four system outcomes from Chapter 10.1. The first two are reliable workload foundations and controlled platform change. The others are timely operational decisions and recoverable shared services. Each outcome keeps its linked signals and counter-signal.

State follows a simple precedence rule. Unknown source evidence makes the outcome unknown. With complete evidence, an unhealthy source outranks degraded, and degraded outranks healthy. This rule is intentionally conservative and visible.

The rule does not declare causation. It gives reviewers a shared place to ask what needs explanation. The rationale records why the sources matter together. The counter-signal records what could weaken that explanation.

In the synthetic example, repeated name-resolution friction appears in incident learning and adoption feedback. Reliable workload foundations are degraded. An incomplete capacity sample makes timely operational decisions unknown. A failed authority path in a recovery tabletop makes recoverable shared services unhealthy.

The next step is to inspect assumptions. Strategy and architecture depend on statements that were reasonable when they were made. Evidence can support, reduce, contradict, or leave their confidence unknown.

The review keeps the original statement and links the evidence that challenged it. It records confidence before and after. The decision is `retain`, `revise`, `retire`, or `test`. The record also names the owner, revised statement, destination, and review date.

For example, the team assumed its normal collaboration channel could carry recovery authority during a disruption. The tabletop contradicted that assumption. The proposed revision requires an authority path that is independent of affected platform and identity dependencies.

The capacity assumption follows a different route. Evidence coverage is incomplete. The review does not revise a strategy objective or claim a shortage. It turns the assumption into a test. Capacity health stays unknown until all required groups use one definition.

Two other assumptions connect platform engineering to customer experience. Repeated owner exceptions suggest that missing metadata may be a usability problem, not only an enforcement problem. Repeated DNS help suggests that guidance alone may not support self-service.

These changed assumptions identify possible leverage points. A leverage point is a place for a bounded change. That change could improve several interactions or remove a recurring dependency. It is not simply the component with the worst metric.

Total-system benefit is wider than local efficiency. Reviewers ask which protected outcomes could improve. They count the dependencies that a change could unblock. They check if the pattern recurs across sources or windows. They also record effort and possible harm.

Every proposed intervention links to evidence and affected outcomes. It names a leverage target, owner, success measure, countermeasure, effort class, and next review date. Success without a countermeasure can optimize one metric while damaging another part of the system.

Suppose an ownership form reduces policy exceptions by accepting any text. The local exception count improves, but the data becomes useless during an incident. The countermeasure should detect low-quality ownership and pause stronger enforcement.

The review uses priority bands rather than a fabricated return-on-investment value. Incomplete evidence or unknown outcome impact goes to `hold_for_evidence`. Critical, recurring, cross-source evidence goes to `act_first`.

Cross-source, recurring, or dependency-unblocking proposals go to `investigate_next`. Bounded single-source work goes to `schedule_bounded`. Items in the same band are ordered by dependencies unblocked, recurrence, effort class, and stable identifier.

This is a review order, not an authorization. It exposes the assumptions in the sequence and lets decision owners challenge them. A financial case, risk acceptance, architectural decision, or team commitment still follows its normal governance path.

The compact summary below projects the tested review. It preserves the unknown outcome and false authority fields.

JSON

```
{
  "schema_version": "1.0",
  "artifact_type": "continuous_improvement_manuscript_summary",
  "scope": "shared-platform-control-system",
  "source_contract": {
    "required_signal_kinds": [
      "incident_learning",
      "cost",
      "capacity",
      "policy",
      "security_posture",
      "adoption_friction",
      "platform_health",
      "recovery_exercise"
    ],
    "preserve_unknowns": true,
    "compare_only_matching_definitions": true,
    "average_source_states": false,
    "opaque_composite_score_allowed": false
  },
  "outcome_health": [
    {
      "id": "reliable-workload-foundations",
      "state": "degraded",
      "signal_ids": ["incident-learning-q2", "adoption-friction-q2", "security-posture-q2"]
    },
    {
      "id": "controlled-platform-change",
      "state": "degraded",
      "signal_ids": ["policy-q2", "cost-q2", "platform-health-q2"]
    },
    {
      "id": "timely-operational-decisions",
      "state": "unknown",
      "signal_ids": ["capacity-q2", "platform-health-q2"]
    },
    {
      "id": "recoverable-shared-services",
      "state": "unhealthy",
```

```

    "signal_ids": ["recovery-exercise-q2", "security-posture-q2"]
  }
],
"changed_assumptions": [
  {
    "id": "assumption-recovery-authority-reachable",
    "decision": "revise",
    "strategy_target": "hypothesis-register"
  },
  {
    "id": "assumption-capacity-coverage-complete",
    "decision": "test",
    "strategy_target": "hypothesis-register"
  },
  {
    "id": "assumption-owner-exception-temporary",
    "decision": "revise",
    "strategy_target": "plan-backlog"
  },
  {
    "id": "assumption-dns-guidance-sufficient",
    "decision": "revise",
    "strategy_target": "plan-backlog"
  }
],
"priority_rules": [
  "hold incomplete evidence or unknown outcome impact",
  "act first on critical recurring cross-source evidence",
  "investigate cross-source, recurring, or dependency-unblocking work next",
  "schedule bounded single-source work",
  "order equal bands by dependencies unblocked, recurrence, effort class, then stable
ID"
],
"prioritized_interventions": [
  {
    "order": 1,
    "id": "independent-recovery-authority-path",
    "band": "act_first",
    "owner": "recovery-capability-owner",
    "review_date": "2026-07-31"
  },
  {
    "order": 2,
    "id": "private-dns-self-service-validation",
    "band": "investigate_next",
    "owner": "platform-product-owner",
    "review_date": "2026-08-14"
  },
  {
    "order": 3,
    "id": "ownership-metadata-paved-path",
    "band": "investigate_next",
    "owner": "governance-capability-owner",
    "review_date": "2026-08-14"
  },
  {
    "order": 4,
    "id": "monitoring-owner-route-repair",
    "band": "schedule_bounded",
    "owner": "platform-monitoring-owner",
    "review_date": "2026-08-07"
  },
  {
    "order": 5,
    "id": "capacity-evidence-coverage",
    "band": "hold_for_evidence",
    "owner": "capacity-signal-owner",
    "review_date": "2026-08-07"
  }
],
"decision_contract": {

```

```

    "production_change_authorized": false,
    "policy_change_authorized": false,
    "budget_change_authorized": false,
    "backlog_commitment_authorized": false,
    "external_execution_performed": false,
    "next_action": "owners review the proposed interventions and return accepted changes
to Strategy and Plan"
  },
  "proof_limits": [
    "A valid review proves contract consistency, not source truth.",
    "Priority bands do not calculate return on investment or authorize work.",
    "Synthetic evidence does not establish production health.",
    "A later comparable measure must test whether an accepted change improved the system."
  ]
}

```

The first proposal creates an independent recovery-authority path. Its band is `act_first`. The impact is critical, the evidence crosses sources, and the pattern recurs. Its measure requires two independent contacts in the next approved exercise. Its countermeasure stops rollout if the registry creates another shared dependency.

The private DNS self-service validation and ownership metadata path are `investigate_next`. Both connect separate evidence streams and could remove repeated handoffs. They still need bounded discovery because recurrence does not prove the proposed design will work.

The monitoring owner route is smaller. One source supports it, so the team schedules bounded work. Capacity evidence coverage remains on hold. The hold prevents a weak sample from creating a false capacity priority.

The full review is larger than the projection. Its validator resolves every source, outcome, assumption, intervention, and strategy link. It recalculates outcome state and priority band. It rejects false health, changed order, opaque scoring, committed work, and external mutation commands.

The example script runs that validation and writes a timestamped local assessment. It does not read Azure, change Terraform, alter policy, approve budget, or commit backlog work.

```

POWERSHELL
[CmdletBinding()]
param(
    [Parameter(Mandatory)]
    [string] $EvidenceDirectory
)

Set-StrictMode -Version Latest
$ErrorActionPreference = 'Stop'

$parameters = @{
    ReviewFile = (
        "$PSScriptRoot/continuous-improvement-review.example.json"
    )
    EvidenceDirectory = $EvidenceDirectory
}

& "$PSScriptRoot/Invoke-ContinuousImprovementReview.ps1" @parameters

```


The assessment result is `ready_for_owner_review`. All five authority fields remain false. This separation keeps analysis from becoming an accidental change window.

Owners then decide where accepted work belongs. A changed strategic assumption returns to the hypothesis register from Chapter 2.5. A durable architectural tradeoff can require an architecture decision record. An accepted product experiment can enter the Plan backlog. A material change to mission or objectives returns to Strategy.

Returning evidence to Strategy prevents a one-way framework. Cloud adoption methods are often drawn as a sequence. Real adoption is iterative. Business conditions, rules, system behavior, and platform use can change the assumptions behind the original direction.

Not every observation should change strategy. A noisy alert might need a route repair. A repeated cross-system constraint may require a new capability. A contradicted business assumption may change an objective. The review makes those different levels visible.

Cadence combines time and events. A quarterly review can gather comparable windows. A material incident, strategy change, control-plane change, definition change, or recovery finding can trigger an earlier review. The owner records the next date and triggers.

The team also reviews prior interventions. An accepted change needs implementation evidence. It needs a comparable success measure and a countermeasure. If the benefit does not appear, the team revises or retires the work. Sunk effort does not justify more effort.

This step is where a backlog becomes learning. Shipping code does not close the work. A policy change does not close it either. Later evidence must support the intended outcome. It must also show no unacceptable system harm.

The review remains a decision aid. Its validation proves internal consistency, not production truth. Synthetic findings do not establish live health, cost, capacity, security, or recovery capability.

The anchor is simple. Evidence changes an assumption. An authorized decision changes the system. A later comparable measure shows whether the whole system improved.

The Platform That Keeps Learning

The next cloud incident may still arrive with a narrow label. A route is wrong. A policy rejects a deployment. Access is missing, or a Terraform plan contains an unwelcome change.

That resource still matters. Engineers must restore service, correct unsafe state, and protect evidence. System thinking does not replace precise technical work with abstract discussion.

What changes is the work around that repair. The immediate fault is now one event inside a wider system. Repetition calls for more than another ticket. It asks why the same relationships keep producing the same result.

That question marks a shift in the operating model. Reactive Azure work waits for a visible failure and repairs its nearest cause. An adaptive platform also inspects the forces that shaped the event. They include purpose, boundaries, dependencies, authority, information, delays, and incentives.

Purpose comes first because technical success has no value by itself. A landing zone can deploy correctly yet support the wrong business goal. A secure control can block approved teams at the wrong scope. A cheaper design can remove evidence needed during an outage.

Clear outcomes give those choices a test. Strategy states why cloud adoption matters and how success will be recognized. Measures connect investment, risk, delivery, and operations to that intent.

Boundaries make the test honest. The platform landing zone in this book owns shared foundations, not application workloads. Platform teams own common services and controls. Workload teams retain application design, data, reliability, and operating duties within the published interface.

Neither side can succeed through a vague handoff. The application landing-zone contract states what a workload requests and what the platform returns. It names inherited controls and completion evidence. It also records what remains unknown.

Relationships reveal why a resource can look healthy while a service still fails. Identity grants authority through scope. Networks carry traffic through DNS and routes. Policies translate risk into technical effects. Pipelines move reviewed intent toward Azure.

Monitoring carries evidence in the other direction. Ownership connects a signal to a decision. Cost, capacity, and support paths affect whether the design can keep working as demand changes.

These links turn an Azure estate into a sociotechnical system. Resources, code, human choices, operating practices, knowledge, and rewards continually interact. A correction in one part can create pressure elsewhere.

The growing request queue from the Introduction showed that pattern. Manual shortcuts reduced lead time for a few requests. They also increased variation, rework, and support demand. Those effects consumed the capacity needed to reduce the queue.

A system view does not ban a shortcut. It makes the trade visible. The team can bound the exception, record its owner, watch the accumulating stock, and decide whether recurring use points to a missing platform capability.

Feedback gives the team a way to learn from the result. A change produces evidence through deployment records, inventory, policy evaluation, telemetry, cost, adoption experience, incidents, and recovery exercises. That evidence only matters when it reaches someone who can revise a decision.

Delay explains why the effect may arrive later. A policy change can look successful before exception demand grows. A budget action can look efficient before reliability weakens. A new paved path can appear unused before teams finish their current delivery cycles.

Review windows and event triggers keep those delays visible. They prevent a quick result from becoming a final verdict. They also prevent a calendar from delaying action when a serious signal appears.

Incentives complete the picture. Teams respond to delivery targets, security demands, support load, budget ownership, and the effort required by a control. A platform that ignores those forces may encourage the behavior it was designed to prevent.

This does not mean every control should become optional. It means the safe route should be clear, usable, and supported. Necessary friction must be distinguished from avoidable friction. Exceptions should test the platform as well as the requesting team.

The Cloud Adoption Framework gives this system view a practical path. Strategy establishes business intent. Plan turns that intent into an operating model, skills, responsibilities, sequencing, and estimates. Ready prepares the Azure environment and its landing-zone foundation.

Adopt connects workloads to that foundation through migration, modernization, or new development. This capstone stopped at the platform interface. That limit kept workload deployment outside the example while still testing whether the platform could serve real teams. Govern converts risk and business tolerance into policy, standards, exceptions, measures, and learning. Secure carries protection through strategy, design, delivery, operations, incidents, and improvement. Manage establishes the roles, administration, monitoring, protection, and review needed after deployment.

The seven methodologies have an order, but they are not sealed rooms. Plan can expose a strategy assumption that lacks capacity. Ready can reveal an operating duty with no owner. Adoption friction can reveal a weak platform interface.

Governance exceptions can reveal a rule placed at the wrong scope. Security incidents can challenge identity or delivery assumptions. Management evidence can show that the platform no longer supports its intended outcomes.

Those signals return to earlier decisions. Some change a runbook or alert route. Others change a backlog, architecture record, capability boundary, funding choice, or strategic objective. The size of the response should match the evidence.

That return makes CAF a learning cycle. It does not reopen every decision after every event. It keeps evidence from stopping at the phase that produced it.

The same discipline separates desired state from observed state. Terraform expresses persistent platform configuration in reviewed files. Providers translate that configuration into proposed and approved Azure changes. Terraform state links resource addresses with remote objects across runs.

Terraform cannot prove every live condition. A clean plan does not prove that a DNS path works. It cannot prove effective access, log arrival, alert delivery, or usable recovery. Desired state is necessary, but it is not complete operational evidence.

Azure CLI, used through bounded PowerShell workflows here, observes selected parts of actual state. It checks context before collection. It can query hierarchy, inventory, access, policy, monitoring settings, and other review inputs without becoming a second configuration owner. That separation protects both paths. Terraform remains the normal mechanism for persistent change. Observation stays read-only unless an explicit, authorized workflow says otherwise. A difference becomes evidence for review rather than a command to repair automatically.

The distinction also changes how engineers treat drift. Drift is not just an error message. It is a difference between approved intent and observed or recorded state. Its cause might be an emergency change or incomplete inventory. It might also be changed ownership, a stale release, or a second control path.

The response therefore begins with classification and authority. Some differences require repair. Some require acceptance or redesign. Some remain unknown until better evidence arrives.

The platform built across these chapters follows the same logic. Strategy and Plan state outcomes, responsibilities, cost assumptions, and measures. Landing-zone design connects hierarchy, identity, networking, policy, management, and security.

Terraform turns those decisions into reviewed desired state. Pipelines test dependencies, identity, artifacts, and protected approvals. Azure observation then tests selected claims about what exists and how controls behave.

The platform product gives workload teams a catalog, paved paths, contracts, support, and known exceptions. Governance and security convert risk into ongoing controls rather than late gates. RAMP gives operations named responsibilities for readiness, administration, monitoring, and protection.

Incidents, policy data, cost, capacity, security posture, adoption friction, monitoring, and recovery exercises return evidence to the improvement review. Changed assumptions and proposed interventions then return to Strategy and Plan through their authorized routes.

The complete loop can be stated in words. Purpose shapes design. Design becomes desired state. Reviewed change becomes Azure state. Observation produces evidence. Evidence changes assumptions and priorities. The next strategy cycle begins with more knowledge than the last.

No part of that loop proves itself. A diagram does not prove ownership. A Terraform test does not prove production behavior. A dashboard does not prove health. A successful exercise does not guarantee the next recovery.

Confidence grows from several kinds of evidence that agree within clear limits. It also grows when counter-signals remain visible. Unknown is safer than a convenient claim that cannot be supported.

This is why an adaptive platform is never simply finished. Azure services change, and organizations reorganize. Threats evolve, budgets tighten, and workload demand shifts. Definitions and tools change too.

The goal is not endless redesign. Stable interfaces and controls remain valuable. The goal is to make change deliberate when the environment or evidence shows that the current design no longer serves its purpose.

That ability depends on people as much as automation. Someone must own each shared service, review signal, exception, decision, and learning item. Teams need time to examine recurring work instead of only clearing the next queue.

Leaders also have a role. They set incentives, fund capacity, approve tolerances, and decide which outcomes matter. If every team is rewarded for local speed, the platform cannot repair cross-system delay through code alone.

Engineers can still begin with a small practice. When an event repeats, name the outcome that failed. Draw the boundary in words. Trace authority, dependency, information, and cost. Record what is known, inferred, and unknown.

Then choose a bounded intervention. Name its owner, expected measure, countermeasure, and review date. Use the normal path to authorize the change. Later, compare the result under the same definition.

If the system improves, keep the learning and strengthen the path. If the result moves harm elsewhere, revise the intervention. If the evidence is incomplete, improve observation before increasing confidence.

Future incidents will still need technical skill. Each landing-zone change will still need exact configuration, review, and testing. System thinking adds a wider view. It shows why the work exists, who it affects, and what the result should teach.

A platform that keeps learning does not promise an end to failure. It promises that failure, friction, cost, and change can become disciplined evidence. That evidence can shape safer designs and better decisions.

The commitment continues after the last page. Keep the purpose visible. Protect the boundary. Observe the result and challenge the assumption. Then let what the system teaches shape the platform it becomes.